

NORTHWESTERN UNIVERSITY

Generalized t-SNE and Beyond: Probabilistic Methods for Dimensionality
Reduction, Combinatorial Optimization, and Machine Learning

A DISSERTATION

SUBMITTED TO THE GRADUATE SCHOOL
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

DOCTOR OF PHILOSOPHY

Field of Mathematics

By

Yi Gu

EVANSTON, ILLINOIS

June 2025

© Copyright by Yi Gu 2025

All Rights Reserved

Abstract

Generalized t-SNE and Beyond: Probabilistic Methods for Dimensionality Reduction,
Combinatorial Optimization, and Machine Learning

Yi Gu

Probability theory and probabilistic algorithms form the fundamental bedrock of modern data science and machine learning. These mathematical frameworks provide essential tools for tackling contemporary data challenges, notably high dimensionality and intricate dependency structures among data points. This thesis investigates three distinct yet interconnected probabilistic problems. The first addresses dimensionality reduction through the lens of generalized t-SNE. The second confronts complex dependencies by exploring Markov Random Fields in conjunction with the Lovász Local Lemma. Finally, the third problem synthesizes challenges of both dimensionality and dependency via Variational Factor Analysis. To ensure accessibility for a broader audience, an introductory chapter will provide requisite background knowledge on these core concepts and their interplay.

Acknowledgements

To be honest, I would rather finish the acknowledgements after the defense is done because I view it as a means to celebrate, a silent speech to address the long journey of Ph.D. I have partaken. However, the standard practice says otherwise, so I have to write it down now, before sending the thesis to the committee. I reckon this is also a good thing, as it adds more humbleness to the overall tone, something we all need to be reminded of from time to time.

I would first like to express my deepest gratitude to my advisor Antonio (Tuca) Auffinger for his guidance and support throughout these years, as well as the other two committee members Elton Hsu and Sandy Zabell. Such gratitude is much deeper than the ordinary, or dare I say, routine acknowledgements that would usually appear in a Ph.D. thesis. Allow me to elaborate.

During my third year in the graduate program at Northwestern, I suffered from a serious sickness that prevented me from conducting research work for almost two years. What made things worse was that period of time overlapped with the peak of the COVID-19 pandemic. I had to pick up new research problems from scratch after getting out of the hospital at the end of 2021. The rehabilitation process already took most of my energy, and my empty research portfolio told me that my path to academia was no more. I needed time, a lot of time, to put my health, my passion, and my career back together from all the broken pieces scattered across my life. Time was never on my side.

At this time, instead of telling me my time was up with the Ph.D. program, Tuca gave me more time and space to work on my problems, find industry jobs, and most importantly, to think, reflect, and heal. He told me that intelligence and talent were not the problem, but my state of mind was. He supported the worst version of me to become, in my opinion, the current best version of me. Elton also provided me with a lot of help and mental comfort. The math department at Northwestern chose to give me not only all the excellent resources in mathematics but also a sanctuary for lost young folks. This place will forever be a beacon of hope, a place that reminds me of the kindness and intelligence in human beings, and a source of solace during the darkest times.

With the above being said, please bear with me for some extra words for other important people in my life who backed me up along the way.

First, I would like to thank my best friend Xiaoyu Li for his life-long support and for taking care of me during my surgery periods. He generously shared his place with me and looked after me when I needed to constantly go in and out of the hospital due to a lack of patient rooms during the pandemic. Interestingly, he was also the person who drove across the country from Houston to Evanston to move me and my belongings at the beginning of my Ph.D. program. It is my utmost honor and privilege to have such a friend whom I can trust with my life and soul.

Then, I would like to thank my collaborators and dear friends Zichao Wang and Nan Jiang for their inspiration and guidance on the research papers we published. Not only did they offer me their wisdom in computer science, but also their invaluable friendship. Without them, the completion of this thesis would not have been possible.

Next, I would like to express my gratitude to my colleague Chenyao Zhang at Google, who mentored me during my internship. She was a good mentor and friend who not only guided me through my time at Google but also shared her rich industry experience with me. She was the one who pushed for my hiring decision and the reason I had such an amazing team at Google amidst mass layoffs and hiring freezes. Though not knowing me for long, she was willing to trust and support a stranger without hesitation. I will strive to do the same and pass down the kindness to others, just as she did.

Moreover, I would like to thank my good friends Xiaoyini Zhang and Zeyu Li for offering me their places to stay during my internship in the Bay Area. When I received my offer at Google, I was still busy with my rehabilitation process, and relocation across the States was a huge challenge for me. They immediately decided to share their places with me despite their own stressful lives. They, like all the people I have mentioned, are the ones who made me believe in the goodness of mankind—a reminder for me to be kind regardless of the circumstances.

In addition, I would like to thank my violin teacher Addison Teng for his care and inspiration during my hard time of rehabilitation. I met Addison three years ago when I felt that I needed some extra help to not give up the violin, the instrument which I have played for more than 25 years, since I was 4. Addison was one of those teachers who not only excel at what they teach but also care deeply about the well-being of their students. He taught me to reflect on my own internal shortcomings and to face hardship without fear. He taught me to be a better musician and a better human. Playing the violin is an everlasting service to oneself, a procedure that teaches us that we should always strive for

the best and never compromise on our core values and standards. I am fortunate to have him by my side.

There are many, many others to whom I would like to express my appreciation, such as my alumni and friends Yuxin Zhou and Mengxuan Yang for their fruitful discussions regarding my research problems. Others include my old friend Anqi Sun and her partner Ke Jin, who invited me on their travels during my stressful times. The list goes on and on.

Finally, I would like to thank my family, my parents for their unconditional love and support. People say that one's family shapes one's life. There is no doubt that my life was shaped by love, warmth, and wisdom. My parents passed down the most valuable things in our civilization that will shine through the course of history like a burning star. Now it is my turn to uphold their legacy and make my contribution to this world as a mathematician, a scientist, and most importantly, a person with curiosity, hope, and love.

Dedication

This thesis is dedicated to my parents and all the people who have supported me and helped me along the way. I love you all.

“τὰ καλὰ χαλεπά (The beautiful things are difficult).”

- Plato

Table of Contents

Abstract	3
Acknowledgements	4
Dedication	8
Table of Contents	9
List of Tables	12
List of Figures	14
Chapter 1. Introduction	20
1.1. Overview	20
1.2. t-SNE and Dimension Reduction	22
1.3. Markov Random Field and Probabilistic Graphical Models (PGM)	48
1.4. Factor Analysis and Variational Inference	78
Chapter 2. Convergence of t-Distributed Stochastic Neighbor Embedding with Generalized Input and Output Kernels	97
2.1. The Concept of Input and Output Kernels	97
2.2. Setup and Notations	100
2.3. Main Results	104

	10
2.4. The Generality of Conditions	105
2.5. Summary and Structure of Proofs	107
2.6. Existence and Uniqueness of σ^*	108
2.7. Proofs of the Main Theorems	116
Chapter 3. Learning Markov Random Fields for Combinatorial Structures via	
Sampling through Lovász Local Lemma	141
Overview	141
3.1. Introduction	142
3.2. Preliminaries	145
3.3. Sampling Through Lovász Local Lemma	149
3.4. Neural Lovász Sampler	151
3.5. Experiments	156
3.6. Chapter Recap	163
3.7. Probability Distribution of Algorithm 1	164
3.8. Running Time Analysis of Algorithm 1	175
3.9. Constrained MRF Model	182
3.10. Experiment Settings and Configurations	184
Chapter 4. VarFA: A Variational Factor Analysis Framework for Efficient Bayesian	
Learning Analytics	197
Overview	197
4.1. Introduction	198
4.2. Background and Related Work	202

	11
4.3. VarFA: A Variational Inference Factor Analysis Framework	209
4.4. Experiments	215
4.5. Chapter Conclusion and Future Work	225
Bibliography	228

List of Tables

3.1	Sampling efficiency and accuracy for learning K -SAT solutions with preferences. The proposed NELSON is the most efficient (see “Training Time Per Epoch”) and always generates valid assignments (see “Validness”) with a small approximation error (see “Approximation Error of Gradient”) against all baselines. T.O. means time out.	158
3.2	The quality of learning outcomes for learning random K -SAT solutions with preferences. NELSON achieves the best likelihood and MAP@10 scores. T.O. is time out.	159
3.3	Sample efficiency and learning performance of the sink-free orientation task. NELSON is the most efficient (see Training Time Per Epoch) and always generates valid assignments (see Validness), has the smallest error approximating gradients, and has the best learning performance (see MAP@10) among all baselines.	162
3.4	Summary of all the notations used in the theoretical analysis of Algorithm 1.	167
3.5	4 constraints for converting pairwise terms in the potential function into single variable form.	183
4.1	Summary statistics of pre-processed real data sets.	218

- 4.2 Student answer prediction performance comparing VarFA to SPARFA-M on Assistment, Algebra and Bridge data sets. $\uparrow$ and $\downarrow$ denote higher and lower is better, respectively. VarFA performs better than SPARFA-M on all three data sets and evaluation metrics most of the time. Additionally, VarFA's run time is very close to SPARFA-M. 220
- 4.3 Illustration of the estimated latent skills with the their top 3 most strongly associated skill tags in the Assistment data set. The percentage in the parenthesis shows the association probability (summed to 1 for each latent skill). We see that the tagged skills associated with each estimated latent skill form intuitive and interpretable groups. 223

List of Figures

- 1.1 The distribution of distance from points uniformly sampled in unit cubes of different dimensions. The vertical red lines indicate the average distance between points. It is clear that as the dimensionality grows, the distribution of distance becomes more concentrated. 27

- 1.2 The distribution of distance from points uniformly sampled in unit spheres of different dimensions. The vertical red lines indicate the average distance between points. It is clear that as the dimensionality grows, the distribution of distance concentrates towards the length of radius. 28

- 1.3 Part of data processing pipelines introduced in [Stegle et al. \[2015\]](#), with dimension reduction (PCA) being one of the key steps (covariance matrix and visualization). 30

- 1.4 The famous 2-dimensional color wheel, created by MDS. It effectively and intuitively captured the similarity and discrepancies between different colors on the spectrum of visible light. 33

- 1.5 The input Swiss roll (left) and the output of Isomap (right). The Isomap successfully captures the nature of the manifold and “unrolls” the Swiss roll. 37

- 1.6 A failed LLE output based on the same Swiss roll input in figure 1.5. The algorithm tries to search for neighbors in a overly broad area, causing the result to be distorted. 38
- 1.7 A sample of the MNIST dataset (left) and the output of t-SNE (right). t-SNE successfully grouped the handwriting with different digits together, with some minor imperfections (there are two clusters for the digit 0). 44
- 1.8 The running result of different SNE algorithms in Kobak et al. [2020], with (A) being the light-tail t-SNE, (B) the normal t-SNE and (C) the heavy-tailed t-SNE. Heavy tailed variants are producing tighter and finer clusters, while the light-tail version produces results similar to the original SNE. (t -distribution will converge to Gaussian if $\alpha \rightarrow \infty$.) 47
- 1.9 A simple example of image segmentation, by B Palac, from Wikimedia Commons. Different objects like the bus, the road and the buildings in the background can be detected and identified by algorithms. This technology is widely used in critical fields like self-driving cars. 50
- 1.10 A Waymo self-driving taxi, operating in downtown San Francisco, by Dllu, from Wikimedia Commons. One can easily spot the giant central hub for sensors on the top of the car, as well as many other additional cameras and radars the taxi relies on to deliver a smooth transportation experience. Waymo uses a mixture of technologies, with computer vision and image segmentation being one of the key component of its smooth operation. 51

- 1.11 A medical image segmentation that identifies the tumor, by EmmaWiki2020 from Wikimedia Commons, and a 3D image(s) of Heritage Tabo Gumpa by Nevil Zaveri from Flickr. To see the 3D image on the right, focus on the center line of the image pair. A third image will appear, the stereo one. 51
- 1.12 An example of GRN, by Min Oh, Jaegyeon Ahn, and Youngmi Yoon, from Wikimedia Commons. Graph structure is crucial in visualizing complex relationships between genes and different types of cancers. 53
- 1.13 A illustration about HMM modeling multiple sequence alignment, by EMBL-EBI Train Online, from Wikimedia Commons. 55
- 1.14 An intuitive example of path diagrams with edge weights (contributing factors). The daily activities of a person depend on the transportation and food choices, which further depend on the weather, mood and how hungry the person is. Using the diagram, we can conduct analysis such as estimating the impact of weather on the person's daily activities. 58
- 1.15 A path analysis diagram from Wright [1921]. Wright used this diagram to visualize and analyze the relations between two mated individuals and their progeny. 59
- 1.16 Besag used MRF to develop the Iterated Condition Modes (ICM) algorithm that is able to restore dirty images. The figure is from his work in 1986 Besag [1986]. As we can see, ICM performs much better than the old maximum-likelihood classifier. 64

- 1.17 The diagram illustrating belief propagation in [Kschischang et al. \[2001\]](#).
 Belief propagation is a powerful model built upon BN and MRF that
 are broadly used in information theory and other related fields. 65
- 1.18 Figure 3.3 from [Koller and Friedman \[2009\]](#), a simple BN that models
 the letter graph and SAT scores of students. 67
- 1.19 Figure 3.5 from [Koller and Friedman \[2009\]](#) that shows common
 connecting patterns in BN. 68
- 1.20 A pair BNs illustrating the concept of d-separation. 70
- 1.21 Two MRFs with different edges (and thus cliques) from figure 4.4,
[Koller and Friedman \[2009\]](#). 72
- 1.22 An MRF with two indicated independent clusters. The nodes with
 darker grey in the middle are observed, causing the clusters on the left
 and right to be independent. 74
- 1.23 The original handwritten digits from MNIST (top row) and VAE-
 generated digits (bottom row), from [Mak et al. \[2023\]](#). Though some
 outputs are not entirely natural, all are recognizable and there is no
 mixing of two different digits. 79
- 3.1 Running time and the percentage of valid structures sampled uniformly
 at random from solutions of K-SAT problems. Among all the problem
 sizes, NELSON always generate valid solutions and is the most efficient
 sampler. 160

- 3.2 Running time, the percentage of valid solutions generated, and rounds of resampling for weighted sample generation of K-SAT solutions. Among all the problem sizes, NELSON scales the best among all approaches and always generates valid solutions. 160
- 3.3 Frequency histograms for the number of resample and the total time of NELSON method for uniformly sampling visiting paths for vehicle routing problem. 164
- 3.4 Implementation pipeline of the NELSON-CD algorithm with $m = 1$. The proposed NELSON can be efficiently adapted to a Pytorch-based machine learning library and enforces constraint satisfaction during learning. 187
- 3.5 **(a)** An un-directed graph $G(V, E)$ where the vertices are $V = \{v_1, v_2, v_3, v_4\}$ and the un-directed edges are $E = \{e_1 = (v_1, v_2), e_2 = (v_1, v_3), e_3 = (v_2, v_3), e_4 = (v_2, v_4), e_5 = (v_3, v_4)\}$. **(b)** A possible sink-free orientation of the edges in the graph and its matrix representation x , where every vertex has at least one outgoing edge. 190
- 3.6 The distribution of resampling steps in the NELSON and Algorithmic-LLL [Moser and Tardos, 2010]. Both of them get a valid sample within $T_{tryouts}$. NELSON takes much fewer resamples than Algorithmic-LLL because it resamples all the violated clauses at every iteration while Algorithmic-LLL only resamples one of them. 196

- 4.1 Performance of the SPARFA-M, SPARFA-B, and VarFA algorithms on the synthetic data set with different data sizes. Plots from left to right show comparison on accuracy (ACC), area under curve (AUC) and F1 metrics, respectively. Higher is better for all metrics. VarFA performs similarly to SPARFA-M and SPARFA-B. 214
- 4.2 Training run time comparison among VarFA, SPARFA-M, and SPARFA-B on synthetic data sets of varying sizes. VarFA performs approximate Bayesian inference almost 100x faster than SPARFA-B and is close to the run time of SPARFA-M. Thus, VarFA enables practical and scalable Bayesian inference for very large data sets. 217
- 4.3 Violin plot showing the mean and standard deviation of the estimated skill mastery levels on 10 selected students on the 3rd, 4th and 7th latent skills that VarFA computes. In each sub-figure, bottom and top axes respectively shows student IDs and top axis shows the number of questions each student answered. The more questions a student answers, the tighter the credible interval. (Best viewed in color.) 219
- 4.4 Comparison between the estimated skill mastery levels using VarFA's predictions and using empirical observations for student with ID 110. Even though the two curves show different numeric values, they nevertheless demonstrate similar trends, showing that the predictions reasonably match our intuition about student's skill mastery levels. 227

CHAPTER 1

Introduction

1.1. Overview

This thesis is a collection of three different research projects I worked on during my Ph. D study at Northwestern University.

The first project is a preprint about the equilibrium measure of generalized t-SNE, with details in chapter 2. We formulate the generalized form of input and output kernels and illustrate how our formulation maintains generality. Furthermore, we show that under certain general conditions, the input and output kernels of t-SNE can be changed while still proving the algorithm’s convergence towards an equilibrium measure for the output as the sample size tends to infinity.

The second project is a paper, published in AAAI 2023, regarding Lovász Local Lemma (LLL) and its applications in learning combinatorial structures, with details in chapter 3. We developed a novel machine learning framework utilizing LLL that is able to efficiently output valid samples satisfying constraints and learn Markov Random Fields (MRF) for complex combinatorial problems such as K-SAT. We also provide the theoretical foundation of the algorithm, proving its correctness and estimating its expected running time.

The third project is a paper, published in EDM 2020, about combining variational inference and factor analysis to create a more interpretable generative model, with details in chapter 4. We show that the new combination is efficient, accurate and more flexible

when it comes to processing missing entries in the data set. Combining the strengths of both variational autoencoders and factor analysis, we demonstrate significant application value in fields requiring interpretable models, such as education.

In summary, this thesis has an overarching theme in probabilistic algorithms and data science. The three pieces of work we present use different yet correlated methods to tackle problems arising in modern data analysis. Indeed, as we venture further into the 21st century, our world is shaped, built, or dare we say, defined by data ([Hey et al. \[2009\]](#)). From simple information like class rosters, to large scale datasets like the Human Genome Project, we are recording everything, with the realization that there are limitless values to be extracted. Mathematics and algorithms are at the center of this grand picture, as the extent to which we can unlock this value is fundamentally constrained by our mathematical understanding. Indeed, for decades have we collected visual data such as satellite imagery and medical scans, but the studies and utilization of them remained primarily manual. In 2012, the success of ImageNet([Krizhevsky et al. \[2012\]](#)), along with Convolution Neural Network (CNN), made automated analysis of visual data possible. It was moments like this that revealed to us there is so much to be seen and discovered with data.

Among all the mathematical theories and tools that forge the foundation of data science, probability theory and probabilistic algorithms are of unique and paramount importance. With data comes uncertainty and noise, and with forecasting and prediction comes ambiguity. The nature of the tasks data science aims to tackle and the intrinsic problems stemming from all forms of data call for probabilistic approaches. They provide

the mathematical language and framework for us to navigate through randomness with deterministic confidence.

Take the example of CNN we just mentioned. The success of CNN is seen by many as the watershed moment in the history of mathematics and computer science. Though the advancements in semiconductor industry and hardware engineering should never be overlooked, the birth of CNN is the result of a series of brilliant developments in probability since Kholmogorov. The applications of concepts like softmax layers, maximum log-likelihood estimation, etc all have a deep root in probability theory and constitute the core mechanism of CNN(Krizhevsky et al. [2012]). And our stories have just begun. As of today, the development of Large Language Models (LLM), the closest ever creation of humans to Artificial General Intelligence (AGI) is essentially a probabilistic algorithm (Vaswani et al. [2017]).

For the rest of this chapter, we will provide a brief introduction to the probabilistic algorithms we will discuss in this thesis. From basic concepts, to history, and related progress. It is very exciting to see all the mathematical concepts our predecessors have developed, and have been passionate about, are being turned into new technologies that impact and steer the modern world. And may this thesis provides an insignificant, yet meaningful addition to this great painting of our future.

1.2. t-SNE and Dimension Reduction

1.2.1. The Ubiquity and Challenge of High-Dimensional Data

Most data and data sets we deal with on a daily basis are multi-dimensional by nature. For example, a simple class roster can have multiple exam and homework scores. If the total

number of assignments is greater than 3, then this single class roster is already beyond what we can natively visualize (if we want to represent each student as a point). On the other hand, each image can be seen as a high-dimensional vector and the dimensionalities increase rapidly while the resolution we are able to capture advances with technology. The number of variables scale in a breakneck pace as we are more capable of capturing detail-rich data as well as automating the process ([Donoho \[2000\]](#)). As stated in the last section, this process gives us practically unlimited potentials. The objects and processes we would like to study were recorded with utmost details, unlike the past when vital bits of information could be lost forever. However, this also brings caveats and new mathematical challenges.

One of the most important and hardest challenges is the noise that comes with data. Apart from noise caused by engineering factors, the growth of dimensionality also brings noise because many dimensions may be irrelevant for the problems one want to solve. The extra dimensions themselves become the “noise”. In 1996, Robert Tibshirani discovered the famous Least Absolute Shrinkage and Selection Operator (LASSO) for the purpose of variable selection ([Tibshirani \[1996\]](#)). Consider a sample consisting of N cases, each with dimension p . Let y_i be the outcome and x_i be the p -dimensional vector that represents the i -th case. LASSO aims to solve

$$(1.1) \quad \min_{\beta_0, \beta} \sum_{i=1}^N (y_i - \beta_0 - x_i^T \beta)^2,$$

with constraint

$$(1.2) \quad \sum_{j=1}^p |\beta_j| \leq t.$$

LASSO marks the early realization and effort mathematicians and scientists were trying to mitigate the overwhelming information brought by high dimensional data. LASSO was also closely related to Basis Pursuit Denoising (BPDN), introduced in 1994 ([Chen and Donoho \[1994\]](#)). BPDN is a well known NP-hard problem. Before the age of big data comes, mathematicians had been thinking about extracting useful signals from noise/spurious correlations. After the dawn of 21st century and the explosive growth of computer science, problems like BPDN and LASSO become even more important after we add computational cost to the equation ([Fan and Lv \[2010\]](#)).

Apart from feature and variable selection, people also seek to “compress” the data, creating a mathematical summary of the essence. t-SNE is one of such examples. Instead of making a selection, t-SNE aims to find a low-dimensional representation of the data and taking the entire data set into account. Another good example is the Variational Auto Encoder ([Kingma and Welling \[2014\]](#)), a generative model that is able to scale well with large data sets using a low-dimensional “latent space”. We will discuss VAE and our related work on this topic in chapter 4.

In the face of large amount of high dimensional data, digesting it “as is” is more often than not impossible. The above examples show us that we either make a selection, or we perform compression, which are the two most common approaches in terms of getting what we need from the data. We would like to discuss this further because high-dimensional data is not only mathematically and computationally hard to deal with, but

also tends to mask the insights we are looking for. This is somehow a paradoxical nature of high-dimensionality. Faithful recording of data assures us the information and values we seek are within the data sets and within in our reach. But the form and scaling of the high-dimensional data also buries the light we are after. This is the so-called “curse of dimensionality”.

The term “curse of dimensionality” was first coined by Richard Bellman in 1961 (Bellman [1961]). It refers to various phenomena that appears when analyzing data in high-dimensional space that do not occur when dimensionality is low. Qualitatively speaking, the “curse” usually means that as dimensionality increases, the exponential increase of volume makes the same amount of data sparse and harder to work with.

To understand this concept, let us first look at the famous example of hypercube vs hypersphere. Consider a hypercube with edge length $2r$ in d -dimensional Euclidean space and its inscribed sphere with radius r . Denote Γ the gamma function and C_d , S_d the volume of the hypercube and hypersphere respectively. Then we have

$$(1.3) \quad \lim_{d \rightarrow \infty} \frac{S_d}{C_d} = \lim_{d \rightarrow \infty} \frac{\pi^{d/2}}{d 2^{d-1} \Gamma(d/2)} = 0.$$

This means that our low-dimensional intuition fails. The volume of hypercubes are mostly “concentrated” in the corners and the inscribed sphere’s volume becomes negligible. If we want to analyze the data related to the hypercube, a data set that is uniformly distributed in the hypercube will become an increasingly small part of the whole space. In general, as we increase the dimension of our sample space, any finite data set will occupy a vanishingly small part of it. On the other hand, to maintain the roughly same density, one needs to accumulate exponentially more data points to compensate for the increase in dimensions.

Such fact also induces a data mining problem, where it is unrealistic to gather enough amount of data points to guarantee the separability of data generated by two different distributions. It is a well known fact in machine learning that too many features with inadequate data points will lead to overfitting. This discovery is not new, either. In 1968, Hughes showed that adding more features past certain points without providing more data points will cause worse classification performance (Hughes [1968]). Later it becomes known as the famous Hughes phenomenon. Adding features is not a free lunch and the results could be detrimental.

To further illustrate the curse of dimensionality, let us look at two extra examples, one mathematical and one from the field of biology. We just compared the volume of hypercubes and hyperspheres as the dimensionality tends to infinity. This comparison is closely related to another phenomenon called the concentration of distance. It refers to the fact that uniformly sampled points in high-dimensional space tend to be equidistant from each other. This can be seen more intuitively from the two collections of plots below.

For the second plot, we replace the unit cube with the unit sphere as our sample space. On the top of distance concentration, we can see that most points are located towards the surface of the hypersphere, rather than the interior. This aligns with our intuition that the volume of the hypersphere is mostly concentrated near the surface, and the volume of a hypercube is mostly concentrated near the corners.

The two plots reinforce the intuition behind the data mining problem we just brought up. Distance, one of the most fundamental and important feature that distinguishes data points, becomes less and less effective as the dimension of the data increases. It directly

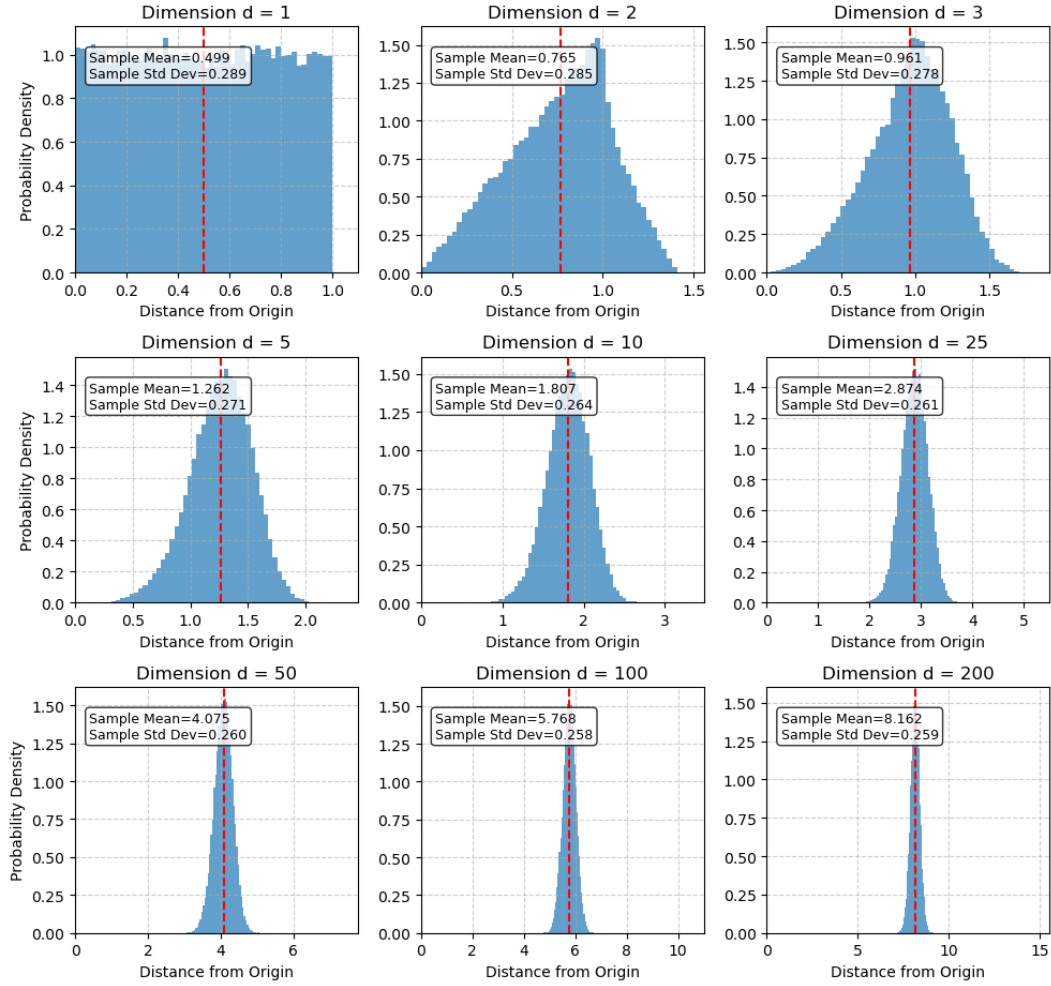


Figure 1.1. The distribution of distance from points uniformly sampled in unit cubes of different dimensions. The vertical red lines indicate the average distance between points. It is clear that as the dimensionality grows, the distribution of distance becomes more concentrated.

leads to the fact that purely distance-based algorithms will perform poorly with high-dimensional data sets. It is crucial to find ways to extract the information we need from high dimensions, then compress, analyze and visualize it in lower-dimensions. The concentration of distance is also one of the reasons that algorithms such as t-SNE are invented.

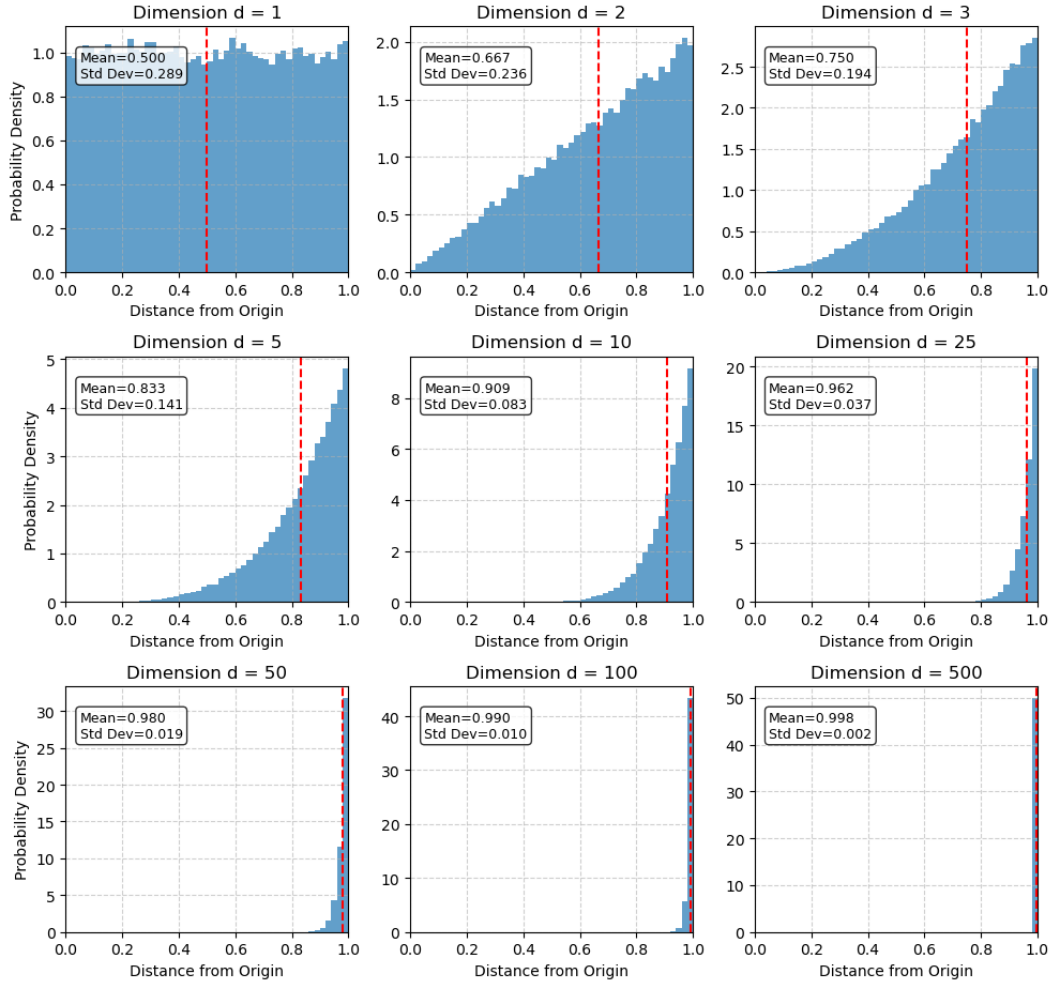


Figure 1.2. The distribution of distance from points uniformly sampled in unit spheres of different dimensions. The vertical red lines indicate the average distance between points. It is clear that as the dimensionality grows, the distribution of distance concentrates towards the length of radius.

As we will see in the coming sections, t-SNE encodes the neighborhood information using probability distributions, rather than plain Euclidean distance.

The last example we would like to share is the RNA sequencing and many genome projects, where the curse of dimensionality manifests itself in real-world scientific research.

Generally speaking, RNA sequencing experiments measure the abundance of tens of thousands of different RNA transcripts on expression level. Such rich information can exist simultaneously within a single biological sample because life itself is tremendously complex. From a data science point of view, all these genes/transcripts are features and each acts as a dimension so the dimensionality is extremely high. For humans, that number is somewhere roughly between 10^4 to 10^5 . For Genome sequencing that number can be even larger, ranging from 10^6 to 10^7 .

This causes all kinds of problems we mentioned about high-dimensional data (Stegle et al. [2015], Kiselev et al. [2019], Visscher et al. [2017]). On one hand, Euclidean distance becomes less meaningful. The Euclidean distance between two patients (data points) may not contain any useful implications for their similarities in genetic traits or tendencies for a specific disease. The occurrence of spurious correlations are much higher, that people often finds significant correlations between features(genes) and outcomes(disease status) that are purely coincidental and not biologically meaningful. On the other hand, Hughes phenomenon is prevalent in model training because it is nearly impossible to gather enough data so that the sample size is significantly larger than the number of features. This results in massive overfitting and poor predictions for critical problems such as tumor types classification. At the same time, genetic sequences are absolute nightmares for hypothesis testing due to their sheer complexity, making false positives much more prone to happen.

For these very reasons, dimension reduction really shines through the field of biology and bioinformatics. Most algorithms introduced in the next section are still widely used today, from the old classic PCA, to methods created in 21st century such as t-SNE.

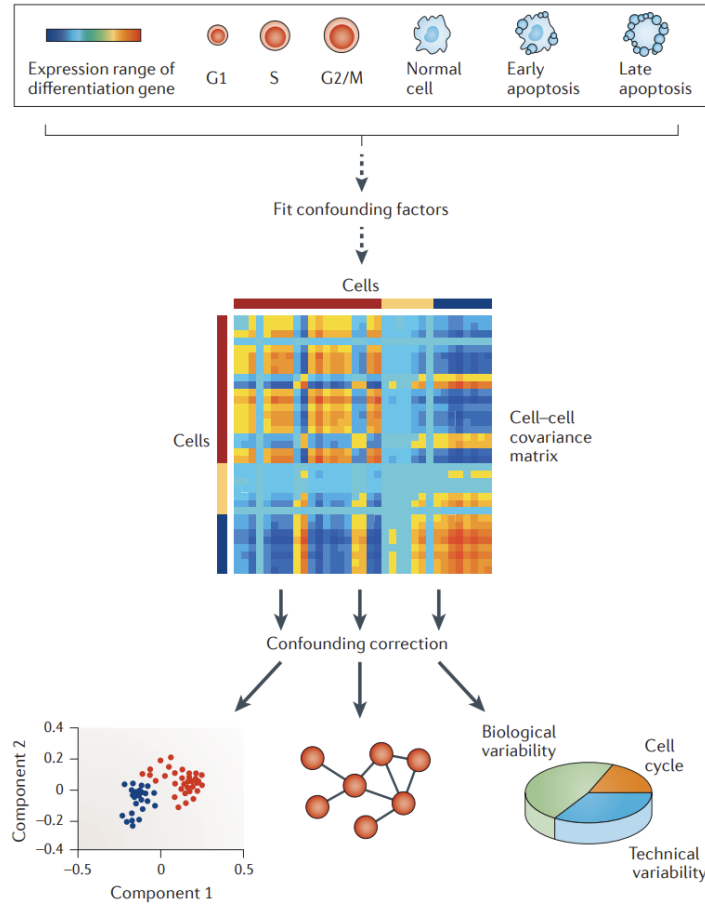


Figure 1.3. Part of data processing pipelines introduced in [Stegle et al. \[2015\]](#), with dimension reduction (PCA) being one of the key steps (covariance matrix and visualization).

Dimension reduction is a crucial toolset that makes continuing research and analysis of genetic data possible, with its impact and usage expanding well beyond the field of biology to almost all scientific subjects related to data science, statistics and machine learning.

With all the above being said, it is of paramount importance to study dimension reduction techniques and algorithms. The knowledge of which aspects of data are important

is obviously not a priori, so it is impossible to select the features/dimensions we need beforehand. The only option for us is to develop the mathematical theory, and reduce the dimensionality to the extent that the information we seek becomes apparent. Now we are almost a quarter into the 21st century, the development of technology resonates with the need for dimension reduction, and puts our mathematical legacy to the test.

1.2.2. A Brief History of Dimensionality Reduction

The theory of dimension reduction can be traced back to the beginning of 20th century. In 1901, Karl Pearson introduced the geometric idea of finding lines and planes of best fit to data points, later known as the famous Principle Component Analysis (PCA)([Karl Pearson \[1901\]](#)). This technique was also independently discovered by Harold Hotelling in 1933 ([Hotelling \[1933\]](#)). The idea behind PCA is mainly linear algebra. It first centers the data set at zero and then tries to find the largest eigenvalue of the covariance matrix. The corresponding eigenvector marks the direction of the most variance in the data, making it the biggest principle component. Then this component gets subtracted and the algorithm repeats to find the next largest eigenvalue. The algorithm can stop at any number of principle component. The corresponding eigenvalues and eigenvectors will then be used to construct a subspace with lower dimensions that represents the original data.

Intuitively speaking, PCA tries to fit a high-dimensional ellipsoid to the data set. Each axis of the ellipsoid represents one principle component. The larger the eigenvalue, the longer the corresponding axis of the ellipsoid. PCA is simple and elegant, so much so that it is still widely used today, but its shortcomings are also apparent. PCA assumes linearity as what it does is primarily finding a new coordinate system where the axes, a.k.a. the

principle components, are linear combinations of the original variables. The mapping from the original high-dimensional space to the lower-dimensional subspace defined by principle components is an orthogonal linear transformation. This means that by analyzing the covariance matrix, PCA can mostly only capture linear dependencies. This approach will fail if the intrinsic structure of the data is non-linear. For example, PCA is likely to give two principle components if asked to analyze a circle, a typical 1-dimensional manifold in 2-dimensional Euclidean space. It is unable to see that the points on a circle are all part of a 1-dimensional space. Similar results with Swiss roll, PCA will very likely be unable to “unroll” the Swiss roll into its natural 2-dimensional representation, but rather cut through it.

PCA is one of the most important and widely used dimension reduction algorithms. Though its assumption on linearity limits its application, it also allows it to be computationally efficient using modern hardware and parallelism. It outperforms many modern algorithms in terms of speed and scalability. For this reason PCA is included in many modern software packages like scikit-learn and PyTorch.

Continuing onward, in the 1950s, Warren Torgerson invented Multidimensional Scaling (MDS) ([Torgerson \[1952\]](#), [Torgerson \[1958\]](#)), with the classical version often known as Torgerson Scaling. MDS is one of the early non-linear dimension reduction algorithms. Given a distance/dissimilarity matrix (where the distance does not have to be Euclidean distance) and a target dimension N , MDS aims to find a set of points in N -dimensional space where the pairwise distances are preserved as much as possible. Often one lets $N = 2$ so the result can be easily visualized. In classical MDS, the final goal is to minimize the

loss function, which is also called strain:

$$(1.4) \quad \text{Strain}(x_1, \dots, x_n) = \left(\frac{\sum_{i,j} (b_{ij} - x_i^T x_j)^2}{\sum_{i,j} b_{i,j}^2} \right).$$

Here, x_i 's are vectors in N -dimensional space that we are trying to find, and b_{ij} are the entries from the Gram matrix that can be computed using the input data. Compared to PCA, MDS no longer assumes linear relationships between data points, and is more intuitive as the distance between points is often a good reflection of their similarity. One of the most famous applications of MDS is the color wheel. One might never hear about MDS, but they must have used the color wheel at some point of their lives.



Figure 1.4. The famous 2-dimensional color wheel, created by MDS. It effectively and intuitively captured the similarity and discrepancies between different colors on the spectrum of visible light.

Being intuitive and flexible, MDS gained a lot of popularity and it was developed into a family of algorithms. Shepard and Kruskal developed non-metric MDS independently in 1962 and 1964 ([Shepard \[1962\]](#), [Kruskal \[1964\]](#)). Their work allowed MDS to handle

Ordinal data, meaning the modified algorithm only requires the rank order of dissimilarities. Such outstanding flexibility is one of the major reasons that it is still widely used today, especially when there is a lack of good distance metrics or quantitative data, such as market research. However, MDS is not without drawbacks. The axes of the output do not have inherent meanings from the original data set and sometimes it is not possible to find a solution with low enough stress values (the stain in the classical MDS). More importantly, MDS is not computationally efficient. Though modern algorithms like t-SNE and UMAP share some of the disadvantages with the MDS family, they often outperform MDS by a fair margin. Nevertheless, the limitations of MDS algorithms do not diminish their importance. MDS is still one of the most important algorithms in the field of dimension reduction.

Fast forwarding to the millennium, the rapid advancement of computer science and applied mathematics created increasingly stronger needs to study data of nonlinear nature. Scientists had been striving to create new tools and algorithms that can handle the emerging complex data sets. Though mentioned in earlier works, many regarded the work of Roweis and Saul in 2000 as the pillar of manifold learning ([Roweis and Saul \[2000\]](#)). Unlike PCA, which tries to project a collection of high dimensional vectors onto a linear subspace, manifold learning projects the high-dimensional data onto a lower-dimensional latent manifold. Since manifold learning was created specifically to tackle problems with nonlinear nature, it is also called Nonlinear Dimension Reduction (NLDR).

In the same paper, Roweis and Saul presented the Local Linear Embedding (LLE), one of the classic algorithms in NLDR. From this point onward, readers who are familiar with SNE and t-SNE may notice the resemblance between the algorithms and how they

draw inspiration from each other. Given original data points $X_i \in \mathbb{R}^D$, the goal of LLE is to find a set of points $Y_i \in \mathbb{R}^d$ usually with $D \gg d$ such that Y_i 's form a good representation(embedding) of X_i 's. This is done in two steps. First, LLE looks at each point X_i and tries to find the closest approximation of X_i using the linear combination of other points. i.e., for each i , LLE computes

$$(1.5) \quad \left\| X_i - \sum_j W_{ij} X_j \right\|^2,$$

where W_{ij} are the weight we assigned to each X_j . W_{ij} is nonzero when X_j is within a certain distance of X_i , or when X_j is considered a “neighbor”. This distance is a hyperparameter set by the user. Summing up through all i 's gives us the reconstruction error:

$$(1.6) \quad E(W) := \sum_{i=1}^N \left\| X_i - \sum_j W_{ij} X_j \right\|^2.$$

While LLE aims to minimize $E(W)$, it is also subject to the restriction

$$(1.7) \quad \sum_j W_{ij} = 1.$$

This ensures that W is not affected by rotation and translation. The first step is finished when LLE successfully finds the weight that minimizes $E(W)$. This is a typical constraint least square problem so we are able to find the global minimum. Now the weight W_{ij} 's are fixed and the second step begins. LLE now tries to find output Y_i 's that minimize the

following cost function:

$$(1.8) \quad C(Y) := \sum_{i=1}^N \left\| Y_i - \sum_j W_{ij} Y_j \right\|^2.$$

The algorithm terminates when the minima of $C(Y)$ is found. Directly from the same paper ([Roweis and Saul \[2000\]](#)), we know that finding the minimum of $C(Y)$ is equivalent to solving a sparse eigenvalue problem and we are able to find the global minimum. This is one of the most appealing mathematical properties of LLE, that both of its core steps yield globally optimal solutions. However, nothing is perfect and LLE does come with its own shortcomings. Let us discuss that after we introduce another NLDR algorithm that was created around the same time, and is just as famous and important: Isomap.

Isomap was first introduced in around 2000 ([Tenenbaum et al. \[2000\]](#)). It is a modern generalization of MDS. Instead of using the Euclidean distance, Isomap generates the so called geodesic distance between each pair data points then apply MDS to get the low-dimensional output. We start with our high-dimensional input $X_i \in \mathbb{R}^D$. The first step of Isomap is to find the neighbors of each X_i . This can be achieved by either considering a ball with radius ϵ around X_i and see which points are inside the ball, or by using the k nearest neighbors. Both ϵ and k are hyperparameters set by the user.

After getting the neighbors for each point, we will construct a weighted graph G based on the information we just obtained. Each vertex of G represents an input data point. We will use the term vertex i when the vertex represents X_i . We draw an edge between vertex i and vertex j if X_j is a neighbor of X_i , which is determined during the previous step of the algorithm. The weight of the edge (i, j) is the Euclidean distance between X_i and X_j . Next up, we perform the Dijkstra algorithm on G , which yields the shortest

graph distance between each pair of vertices i and j . This distance is called the geodesic distance, denoted by $d_G(i, j)$. This concludes the second step of Isomap.

Afterwards, since $d_G(i, j)$ is well defined for each tuple (i, j) , we have a distance matrix as the input of the MDS algorithm. Running the algorithm with the target low dimension d will produce the final result of Isomap.

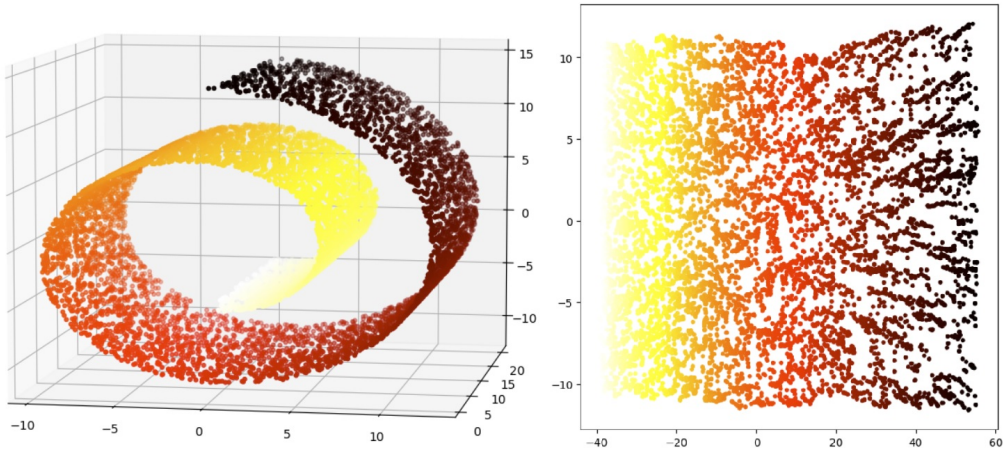


Figure 1.5. The input Swiss roll (left) and the output of Isomap (right). The Isomap successfully captures the nature of the manifold and “unrolls” the Swiss roll.

With the introduction of Isomap, it is easier to explain the drawbacks of LLE. As we can see in the steps of LLE, it emphasizes the local reconstruction of each data point using its neighbors. Because of this, LLE can potentially fail to preserve the global structure of the data, namely the distances on a larger scale. Points that are very far away in the original data set can be mapped close to one another in the output of LLE, especially when the space itself is very “curved”. This shortcoming is addressed by Isomap, as the intermediate step of finding the graph G is specifically designed to preserve global structure. Using MDS subsequently ensures that such preservation is translated into

the low-dimensional output. The graph G represents the grand picture Isomap tries to summarize. Nevertheless, Isomap is not perfect, either. As we already mentioned earlier

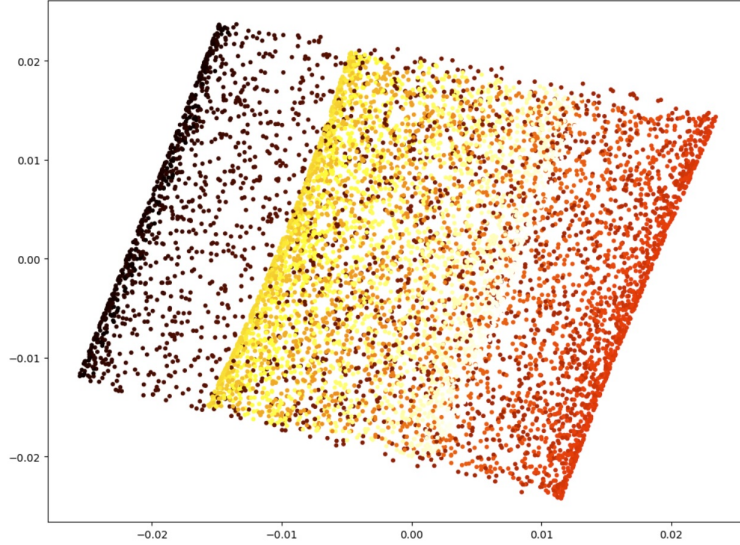


Figure 1.6. A failed LLE output based on the same Swiss roll input in figure 1.5. The algorithm tries to search for neighbors in a overly broad area, causing the result to be distorted.

in this section, MDS is a computationally expensive algorithm. On top of that, given a graph $G = (V, E)$, the Dijkstra algorithm has a time complexity of $O(|E| + |V| \log |V|)$, where $|E|$ and $|V|$ are the number of edges and vertices in the graph respectively (Fredman and Tarjan [1984]). For dense graphs, $|E|$ will scale quadratically with $|V|$. By combining two computationally intense algorithms, Isomap is inevitably slow and becomes a bad choice for large and dense data sets.

Moreover, Isomap is very sensitive to the choice of hyperparameters and outliers. If the number of neighbors or the ball radius is too small, Isomap will sometimes produce a disconnected graph, prevents the algorithm from generating a global structure. But if the hyperparameters are too large, the algorithm might include “short-circuit” edges that

connect points across gaps or folds in the manifold that are close in the ambient Euclidean space, such as distant points on a tightly packed Swiss roll. Then in this case Isomap will contain the worst of both worlds, by being an extraordinarily slow algorithm that also carries the disadvantages of LLE. Besides that, since Isomap is designed to preserve the global structure, it will cover everything, including outliers. Having an outlier in the data set, such as a point far away from everything else, can significantly disrupt the output.

Finally, Isomap carries some implicit assumptions. It assumes that the data on the manifold is well sampled, meaning that the graph constructed in Isomap is a truthful representation of the true geodesic distances on the manifold, from a differential geometry perspective. It also assumes that the intrinsic geometry of the manifold can be isometrically embedded into a lower-dimensional Euclidean space, hence the name. Such assumptions are not always true, and we should treat the output with caution if the stress values (similar to the definition in (1.4)) are high.

Both LLE and Isomap are important milestones in the advancement of dimension reduction algorithms. With different focusing aspects, both provided us with a useful set of tools to analyze high-dimensional data, suited for different scenarios. However, mathematicians and computer scientists did not stop there. All the algorithms up to this point still work primarily with Euclidean distance. What if we want to extract the abstract structure of similarity out of the data sets, rather than the distance itself? What if we encode the information of neighborhood using probability distribution, rather than plain physical proximity? Such ideas inspired the creation of stochastic neighbor embedding (SNE) and the subsequent t-stochastic neighbor embedding (t-SNE). And that is where our story begins.

1.2.3. The Road to t-SNE

SNE was first introduced in 2002 by Hinton and Roweis ([Hinton and Roweis \[2002\]](#), with Roweis also being the creator of LLE). Like we just mentioned at the end of the previous section, SNE uses probability distributions to record the information of neighborhood. With this new framework, we need a new concept to measure the distance between two probability distributions, and that is the famous Kullback-Leibler (KL) divergence.

Definition 1. *Let P and Q be two probability measures with $p(x)$ and $q(x)$ as their respective probability density functions. The KL divergence between P and Q is defined as:*

$$(1.9) \quad D_{KL}(P||Q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx.$$

In particular, if P and Q are discrete probability measures(distributions) defined on the same sample space $\mathcal{X}$, then the KL divergence is defined as:

$$(1.10) \quad D_{KL}(P||Q) = \sum_{x \in \mathcal{X}} p(x) \log \left(\frac{p(x)}{q(x)} \right).$$

Note that the KL divergence is zero if and only if P and Q are the same distribution. At the same time, it is not symmetric and is not a true distance metric. It is also called the relative entropy. Since both SNE and t-SNE will construct probability distributions from the original data set, rather than plain Euclidean distance, KL divergence is used to formulate the loss function we aim to minimize.

SNE starts with defining the following probability mass function for each data point X_i : for each $j \neq i$, let

$$(1.11) \quad p_{j|i} = \frac{\exp(-\|X_i - X_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|X_i - X_k\|^2 / 2\sigma_i^2)}.$$

σ_i is a hyperparameter that is set by the user or by computation. We restrict that for each i , the induced probability distribution has the same entropy for all i . This means for some constant ρ , we have

$$(1.12) \quad \sum_{j \neq i} p_{j|i} \log(p_{j|i}) = \log(N\rho),$$

where N is the number of data points. The quantity $N\rho := \text{Perp}$ is called perplexity. This is a very clever design and it serves as a “slide bar” for each X_i . For example, the majority of the data points are located inside a ball with radius 1 and there is one data point that is more than 1000 units away. Then from that point’s perspective, the rest of the data points are roughly “equally” distant from itself. The induced probability distribution will be very close to a discrete uniform distribution. A lot of information is lost during this process. By introducing perplexity, we allow each induced probability distribution to zoom in and out so that they are “observing” the rest from the data set on the same scale. Later we will see that this setting is not only backed up by clear intuition, but also plays an important role in proving the convergence to equilibrium measure in chapter 2, as well as in the prior work by Auffinger and Fletcher ([Auffinger and Fletcher \[2023\]](#)).

Let Y_i be the low-dimensional output of SNE. We can define a similar probability mass function:

$$(1.13) \quad q_{i|j} = \frac{\exp(-\|Y_i - Y_j\|^2)}{\sum_{k \neq i} \exp(-\|Y_i - Y_k\|^2)}.$$

Next, SNE will then try to minimize the KL divergence between the two (collections of) distributions, i.e.:

$$(1.14) \quad L = \sum_{i=1}^N D_{KL}(P_i || Q_i) = \sum_{i=1}^N \sum_{j \neq i} p_{j|i} \log \left(\frac{p_{j|i}}{q_{j|i}} \right).$$

Since the input X_i 's are given and fixed, the loss function L is a function of the output points Y_i 's. The minimum can be found using gradient descent. Different from LLE, SNE is not guaranteed to find a global minimum.

Though the formulation of SNE resembles LLE, the invention of SNE was still a refreshing evolution. From plain distance, to geodesic distance from constructed graphs, and now to probability distribution, it is good to see that mathematicians are using more flexible concepts to encode the general landscape of data. But people soon discovered that SNE has its own problems. Since the output distribution decays exponentially and it does not have a slide bar like perplexity, SNE is not able to distinguish clusters that are not in close proximity. This is the well-known “crowding problem” of SNE, and it directly inspired the birth the subsequent t-SNE algorithm ([van der Maaten and Hinton \[2008\]](#)).

The t-SNE algorithm shares the same procedure as SNE. However, instead of using the Gaussian function in the output schema, t-SNE uses the much slower decaying student's

t-distribution with 1 degree of freedom, i.e.,

$$(1.15) \quad q_{ij} = \frac{(1 + \|Y_i - Y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|Y_i - Y_k\|^2)^{-1}}.$$

That is also where the letter “t” in the name comes from. The t-distribution greatly mitigates the crowding problem, and it is able to perform very well across different data sets. Also, t-SNE defines the input probability mass p_{ij} as

$$(1.16) \quad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n},$$

which makes the input probability masses symmetric. One of the greatest and most well-known showcase of t-SNE is its implementation on the MNIST data set. The Modified National Institute of Standards and Technology (MNIST) is a large database consisting of handwritten digits. It is one of the mostly commonly used public data set for machine learning and computer vision. Started with USPS database that records handwritten zip codes, today’s MNIST contains 60000 training images and 10000 testing images and serves as a benchmark for many machine learning algorithms. t-SNE is able to effectively capture the structure of data and clearly cluster different digits into separate groups. This algorithm is one of the greatest achievements in the field of dimension reduction and unsupervised learning in the 21st century. Unfortunately, we need to repeat the same statement again: nothing is perfect. t-SNE is also a computationally expensive algorithm. If the data set is large and complex, it is advised to pre-process the data set using lightweight dimension reductions such as PCA. Meanwhile, though with better flexibility and overall performance, t-SNE inherits the same drawback from LLE: poor performance at preserving global structures. The distance between two clusters are often

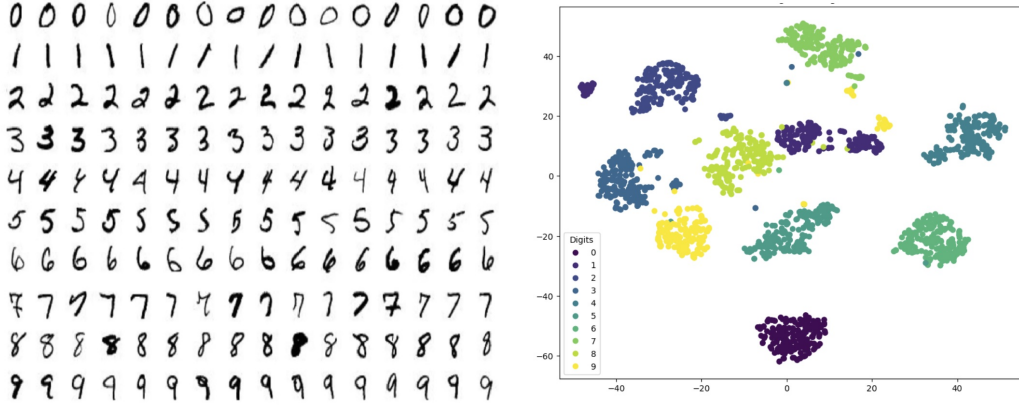


Figure 1.7. A sample of the MNIST dataset (left) and the output of t-SNE (right). t-SNE successfully grouped the handwriting with different digits together, with some minor imperfections (there are two clusters for the digit 0).

meaningless. We only know that the corresponding original data points do not belong to the same group, but little information can be extracted by the distance between the resulting low-dimensional clusters. t-SNE is also non-deterministic by nature: ending results might vary slightly for the same data set and hyperparameters. While the general cluster structures might be similar, the exact positions and orientations can vary, which can be sometimes confusing.

Such traits of t-SNE inspired some discussion regarding the mathematical nature of the algorithm. First, if t-SNE is non-deterministic, can we say something about its convergence and asymptotic behavior? Will the output of t-SNE converge to a certain equilibrium measure as we increase the number of samples to infinity? The answer is yes. We have the following results from [Auffinger and Fletcher \[2023\]](#):

Theorem 1 (Theorem 1.1, [Auffinger and Fletcher \[2023\]](#)). *Assume all input data points of t-SNE are drawn independent from a probability measure μ_X . μ_X has a C^1*

density function $f(x)$ and compact support. Denote X, Y the input and output data set, n the number of data points, ρ the perplexity, and $L_{n,\rho}(X, Y)$ the loss function (KL-divergence) of t -SNE.

Then for any $\rho \in (0, 1)$ given, there exists a function I_ρ and a set of measures $\mathcal{P}_{\mu_X}$, such that

$$(1.17) \quad \lim_{n \rightarrow \infty} \inf_Y L_{n,\rho}(X, Y) = \inf_{\mu \in \mathcal{P}_{\mu_X}} I_\rho(\mu).$$

If we write

$$(1.18) \quad \mu_n = \frac{1}{n} \sum_{i=1}^n \delta_{X_i, Y_i}$$

as the empirical measure of the input and output data set, then the following theorem also holds:

Theorem 2 (Theorem 1.2, [Auffinger and Fletcher \[2023\]](#)). *Assume the same conditions as in theorem 1. Then there exists a measure μ^* and a sub-sequence $\{n_k\}$ such that μ_{n_k} converges weakly to μ^* , and $\mu^* = \inf_{\mu \in \mathcal{P}_{\mu_X}} I_\rho(\mu)$. Moreover, μ^* has compact support.*

The formulation of the theorems are slightly different from the original paper and some technical details are omitted due to the introductory nature of this chapter. The above two theorems send a strong message about the theoretical guarantee of t -SNE. If the input data is drawn independently from the same distribution, t -SNE is able to output a global minimum equilibrium measure (here measure means the distribution of the output data set) as we increase the sample size to infinity, with compact support. These are the first

rigorous mathematical results about this algorithm after it has been used for so many years. And it leads to the next question, the question we are able to address in chapter 2.

For the output distributions, using the Gaussian is not necessary as the successful execution of the algorithm does not rely on the specific properties of Gaussian functions. The two theorems above further verify this observation. This gives rise to the concept of kernels. For a pair of points (x, y) , a kernel is a positive function of $\|x - y\|$ that we can use to determine the normalizing weights that produce the input and output distributions. For input, the kernel is the Gaussian function and for the output, the kernel is the function $f(x) = (1 + x^2)^{-1}$. It is natural to ask:

“Is the student t-distribution not the only kernel that might give us the same theoretical guarantees? If no, then what is the class of output kernels that will give us the same results in [Auffinger and Fletcher \[2023\]](#)? Even more generally speaking, if we swap out both input and output kernels, what conditions do we need for the same theorems to be true?”

These are the exact questions we are able to answer with rigorous proofs in chapter 2. Besides that, people are already swapping out output kernels in practice. Note that in the formula of KL-divergence, for the term $p_{ij} \log(p_{ij}/q_{ij})$ reaches its minimum zero when $p_{ij} = q_{ij}$. This means if we manually increase p_{ij} , the gradient descent process will tend to increase q_{ij} as well. This is achieved by putting the output points Y_i and Y_j closer together, as the output kernel $f(x) = (1 + x^2)^{-1}$ is monotone decreasing. Therefore, in application people often replace p_{ij} with its multiple kp_{ij} for some constant $k > 1$. This is called the early exaggeration, one of the frequently used techniques. Using early exaggeration will result in closer clusters, improving visualization results.

Another known modification of the t-SNE algorithm is the heavy-tailed t-SNE. Recall that the density function of the t-distribution is given by

$$(1.19) \quad f(x) = \left(1 + \frac{x^2}{\alpha}\right)^{-\frac{\alpha+1}{2}},$$

where α is the degree of freedom. Decreasing α will reduce its decaying speed and make it more “heavy-tailed”. This will make q_{ij} relatively larger and the output clusters will be compact. It is also shown in Kobak et al. [2020] that heavy-tailed t-SNE tends to produce finer clusters, such as sub-clusters of the same digit in the MNIST data set. Both early

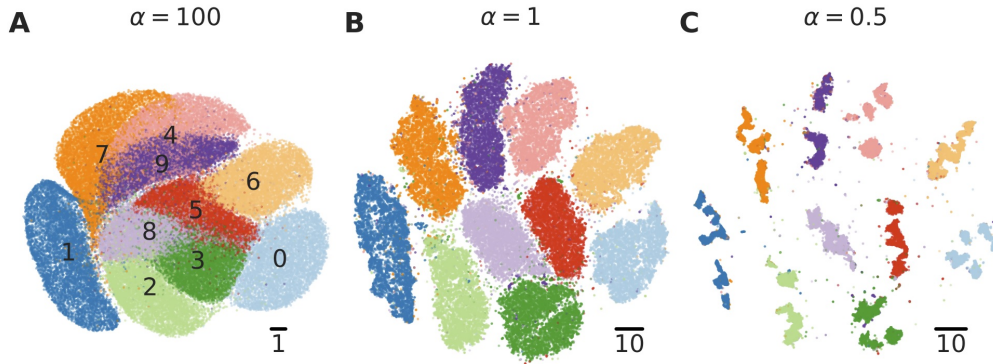


Figure 1.8. The running result of different SNE algorithms in Kobak et al. [2020], with (A) being the light-tail t-SNE, (B) the normal t-SNE and (C) the heavy-tailed t-SNE. Heavy tailed variants are producing tighter and finer clusters, while the light-tail version produces results similar to the original SNE. (t -distribution will converge to Gaussian if $\alpha \rightarrow \infty$.)

exaggeration and heavy-tailed t-SNE can be seen as manipulations of the output kernel. t-SNE works effectively under the modification and produces promising results, besides different characteristics researchers hope to see. Besides exploring different conditions for the convergence to hold and the equilibrium measure to exist, it is also an interesting

problem to see how exactly the output changes with the modification of the output kernel. Some insightful results can be found in [Weinkove \[2024\]](#).

It is fascinating to see the rigorous mathematical theory behind widely used algorithms like t-SNE, which are often used to produce qualitative rather than quantitative results. Getting to the bottom of the convergence behavior will provide valuable insights on the algorithm itself, refining the theoretical landscape of the dimension reduction theory. We will resume our journey in chapter 2.

1.3. Markov Random Field and Probabilistic Graphical Models (PGM)

1.3.1. Dependency: An Unavoidable Problem

We have seen how the theory of dimension reduction has come a long way for the purpose of addressing the challenges from high-dimensional data. Now we turn to another, likely more fundamental problem that probability, statistics and data science need to face: dependency. Though most of probability textbook starts with the concept of independence and theorems assuming it, dependencies are probably one of the oldest and most prevalent challenges we have had to tackle ever since the concept of probability was born. How farmers might change their planting plans based on weather; how gamblers leverage their bets based on the cards in hand; how traders design their investment strategies based on the stock market. All these decisions are based on dependencies and uncertainty. Probabilistic dependency has been with us since the beginning of our time. Even to this day, people might still care about the fate of Schrodinger’s cat, “condition on” the mysterious nature of quantum superposition.

The explosive advancement of computer science and modern industry not only brings high-dimensional data, but also exponentially more complicated dependencies that come with it. Let us take a look at several examples¹ that demonstrate the difficulties and complexity of such problems. The first instance is image segmentation and the famous paper written by Geman and Geman ([Geman and Geman \[1984\]](#)). With the introduction of digital imagery, we receive a grid of many pixels, as well as a grid of intricate dependencies. The color and intensity of any given pixel is highly likely to be similar to that of its immediate neighbors. This is true for almost all real images that include actual content. Pixels from the same object they depict tend to share common characteristics, and that is how our eyes capture and process the world we see through various coherence. If we treat all pixels as independent noises, we will be getting static noise that does not convey any information. Therefore, the authors treat the problem of image segmentation and denoising as a statistical inference problem, with Markov random field being the main tool they deployed to encode the information.

This paper is one of the most famous and important paper in the field of computer vision and pattern analysis. It utilizes a lot of important tools and concepts like Bayesian inference, Gibbs sampling, and Markov random field, some of which will be introduced more systematically in the next several sections. The paper also draws inspiration from statistical physics and today the intersection between statistical physics, math, and computer science has developed into a even larger and more captivating field. It is easy to write a couple of books just based on the keywords from this single paper so we will not

¹All images used in this section are under free or open copyright licenses such as CC BY-SA and CC BY. All images have been properly credited.



Figure 1.9. A simple example of image segmentation, by B Palac, from Wikimedia Commons. Different objects like the bus, the road and the buildings in the background can be detected and identified by algorithms. This technology is widely used in critical fields like self-driving cars.

go into too much nuances. Rather, let us talk about some of the real-life impact this discovery brought.

One of the major application of image segmentation is self-driving cars. In order to identify various objects on the road and navigate through ever-changing traffic, self-driving cars need to have cameras and computer vision technologies to detect and identify objects. This is the aspect that cameras can be much more useful than radars (while radars are better handling positional and distance information). In 2025, we already have commercialized self-driving taxi services by Waymo in several major cities like San Francisco, Los Angeles, and Phoenix, with some known for its notoriously difficult traffic conditions. Not only doing business, Waymo is also active in related research fields, and some of its published results still draw inspiration from Geman and Geman in 1984 (Zhu et al. [2023]).



Figure 1.10. A Waymo self-driving taxi, operating in downtown San Francisco, by Dllu, from Wikimedia Commons. One can easily spot the giant central hub for sensors on the top of the car, as well as many other additional cameras and radars the taxi relies on to deliver a smooth transportation experience. Waymo uses a mixture of technologies, with computer vision and image segmentation being one of the key component of its smooth operation.

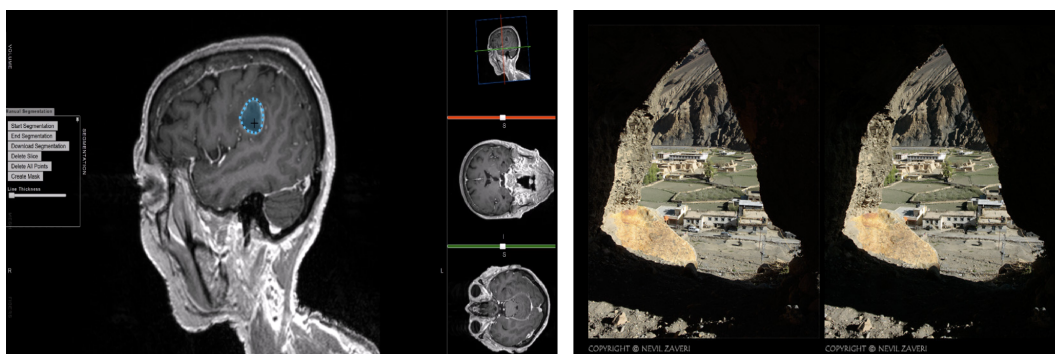


Figure 1.11. A medical image segmentation that identifies the tumor, by EmmaWiki2020 from Wikimedia Commons, and a 3D image(s) of Heritage Tabo Gompa by Nevil Zaveri from Flickr. To see the 3D image on the right, focus on the center line of the image pair. A third image will appear, the stereo one.

Another well-known and important follow-up is the Graph Cuts, emerging in early 2000s ([Boykov and Jolly \[2001\]](#), [Kolmogorov and Zabini \[2004\]](#), [Boykov and Kolmogorov](#)

[2004]). This new image segmentation algorithm was solid step up from Geman and Geman's work and it profoundly influenced the world. Due to its massive improvement in computational efficiency and overall performance, it is widely used in many areas like medical imaging. MRI and CT are able to identify organs, lesions and tissues with the help of segmentation and it is safe to say these devices saved countless lives. Graph Cuts also contributed to the development of stereo vision, which allows the depth of images to be recovered by using two or more cameras, similar to how our eyes work. The narratives and stories above can all be traced back to 1984, where the dependencies problems were effectively modelled and addressed. Vision is the most common sense we use to perceive the world. It contains great obstacles just as it contains great values.

Another good example of dependency research is again, not surprisingly, about genetic sequencing. One of the core reasons that genetic research is extremely difficult is the dependencies between different genes. They do not work in isolation. The expression level of one gene can activate or inhibit the expression of others such as the lac Operon and the trp Operon in *E. coli* (Neville and Gautheret [2010], Jacob and Monod [1961]). For humans a well known genetic piece is the Estrogen Receptor ($ER\alpha$). $ER\alpha$ plays a vital role in the development and function of many tissues, particularly in the female reproduction system, but also in bone, brain and cardiovascular health. $ER\alpha$ affects our body primarily by interacting and binding with other DNA sequences, which ultimately leads to changes in cell behavior. Particularly, $ER\alpha$ dysregulation is a key driver in the majority of breast cancers (Frasor et al. [2004], Heldring et al. [2007]). Drugs that block estrogen binding or $ER\alpha$ activity are critical therapies for breast cancer treatments.

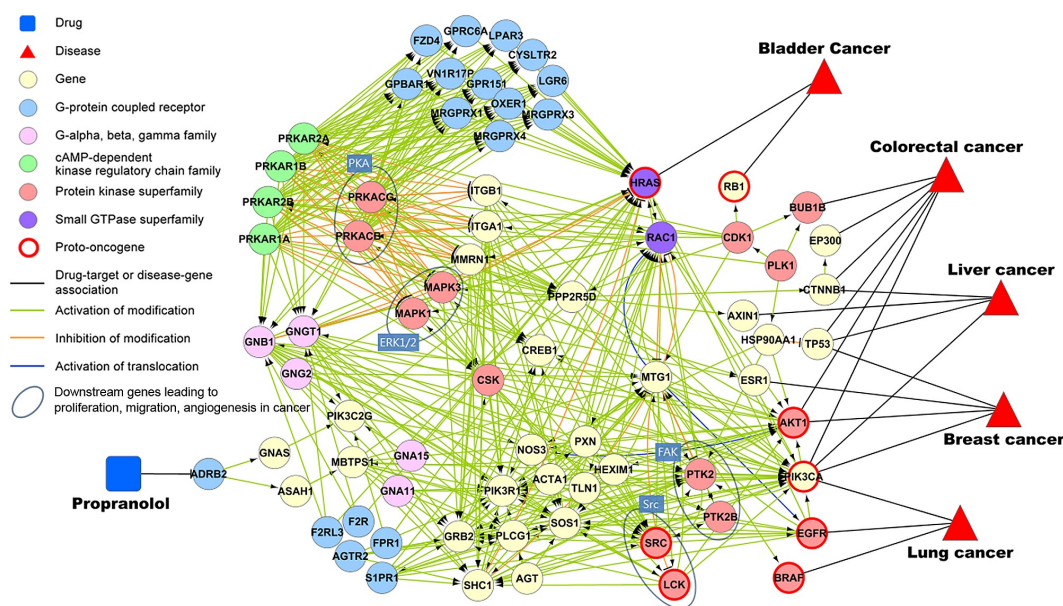


Figure 1.12. An example of GRN, by Min Oh, Jaegyeon Ahn, and Youngmi Yoon, from Wikimedia Commons. Graph structure is crucial in visualizing complex relationships between genes and different types of cancers.

This is just one graph of the vast nest of dependencies in our genetic sequence. Therefore scientists felt it imperative to use mathematical tools to encode, analyze and understand the dependency relationship. Gene Regulatory Networks (GRN) is one of such creations. GRNs use Bayesian Networks (we will cover more details in section 1.3.3.) to model the relations between genetics and cellular function, development and disease. By representing genes as nodes and regulatory influences as directed edges, scientists can infer plausible probabilistic relationships between gene expression levels, even in the presence of noise (Friedman et al. [2000], Sachs et al. [2005]). GRNs are often better understood visually than through textual explanation. Please see figure 1.12 to see how five different types of cancers are related to a large collection of genes via different types of regulatory

influences. Without the help of probability and Bayesian network, such information is impossible to be extracted.

While GRNs map interactions between genes, Hidden Markov Models (HMM) aim to decipher the functional grammar within a single sequence (Krogh [1994-01-31], Durbin et al. [1998]). It is another extremely important tool in genetic research like GRN but with a different focus. One key concept HMM has that is not present in GRN is called the hidden states. Generally speaking, the observed gene sequence is just the tip of the iceberg. There are so many actives and biochemical reactions that happen before a segment of gene is formed and observed. To give an instance, for eukaryotic genes we have concepts of exon and intron. Exons are the coding regions that are present in the final mRNA transcript, while introns are the non-coding regions that get removed during RNA splicing. One of the key functions of HMM is to identify the hidden states, the status “behind the scene”, that connect an intron state to a possible exon state, using a framework similar to the Markov chain, hence the name. For example, if we have a hidden state with some particular intron, HMM will find the transition probability of this state to other hidden states, and ultimately, to different observed sequences.

Contrary to GRNs, whose goal is often the network structure itself, HMMs are used for classification and alignment, such as calculating the probability that a given sequence was generated by a particular collections of hidden states. We call this the emission probabilities. If we see a coding regions with a certain sequence of nucleotide A, C, G, T or some types of amino acids, we are likely able to predict the likelihood of obtaining a specific observable symbol. HMM reveals and decodes the mechanism behind gene

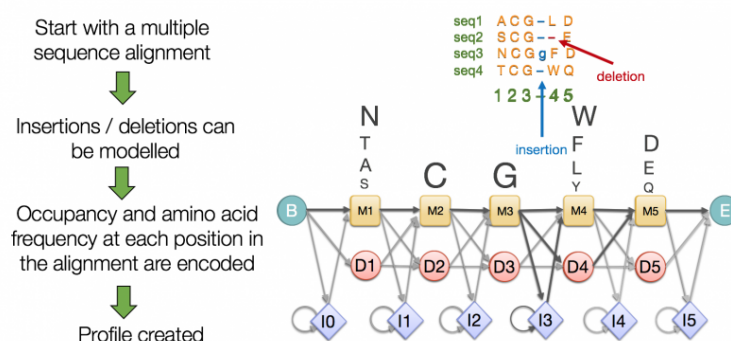


Figure 1.13. A illustration about HMM modeling multiple sequence alignment, by EMBL-EBI Train Online, from Wikimedia Commons.

formation, allowing us to understand the process of gene expression and regulation on a systematic level.

For both examples we just went through (image segmentation and genetic sequencing), we can see the incredible values contained within dependencies, because dependencies encode relations. It is relations that define the world we live in, not isolated objects. All the applications in those examples benefit from properly modelled dependencies. And all those dependencies are transcribed in graph structures. Indeed, probabilistic graphical models are specifically designed to address the dependencies problems, and there have been so many successful advancements and applications of PGMs. As we venture deeper into the finer structure of this universe, both on micro and macro scales, PGMs have become even more important than ever before.

Remark 1. *Though PGMs are the specialized tool to solve dependency problems, we should also be aware that they are not the only tools. As vast as the theories and applications of PGMs have come to be, they are a small part of a much larger overarching theme, combinatorial optimization (CO). This term can refer to any problems that aim*

to find an optimal object from a finite set of objects (Schrijver [2003]), often comes with a set of constraints as well. Probabilistic dependency is only one kind of constraints and PGMs are only one type of tools that record such constraints. The field of CO is extremely diverse and rich. We can start with basic examples like the traveling salesman problem all the way to the major unsolved problems such as $P=NP$. It is easy to go too deep into the rabbit hole. Therefore, it would be appropriate to acknowledge the wider picture while focusing on PGMs in this section.

1.3.2. PGM, a Short History

Both graph theory and probability have a long history. Though the foundation of modern probability is widely regarded as the work by Kolmogorov, probability theory, in general, can date back to the 16th century (LIGHTNER [1991]). Graph theory is often believed to be the brain child of Euler (Euler [1736]). The basic ingredients of PGM have existed long before PGM itself was created. Therefore, the history of PGM can be roughly divided into three parts: the early conceptual seeds (from probability and graph theory), the emergence of mainstream models (Bayesian networks and Markov random fields), and the unification/expansion era that came with the advent of modern computer science.

Prior to the 1980s, there was not a definitive concept of PGM that was widely acknowledged and used among the research community. However, the practices that used graphs to represent probabilistic models started to emerge in the first half of the 20th century. In 1925, Ernst Ising used the lattice graph in his Ph. D thesis to represent the the magnetic dipole moments of atomic “spins” (Ising [1925]). Later this idea was generalized by Lars Onsager in 1944 (Onsager [1944]) and then evolved into the famous

Ising model we know today in statistical physics. Formally speaking, let Λ be the lattice graph(site) and we use positive integers to denote its vertices. For each vertex(site) k , there is a discrete variable σ_k such that $\sigma_k \in \{-1, 1\}$, representing the site's spin. We call the collection of all spins $\sigma := \{\sigma_k\}_{k \in \Lambda}$ the spin configuration. For any two adjacent sites(vertices) $i, j \in \Lambda$, there is an interaction J_{ij} . Finally, let μ be the magnet moment and h_j be the external magnetic field interacting with j . The Ising model is defined by the following random function called the Hamiltonian:

$$(1.20) \quad H(\sigma) = - \sum_{\langle ij \rangle} J_{ij} \sigma_i \sigma_j - \mu \sum_{j \in \Lambda} h_j \sigma_j,$$

where the first summation is over all pairs of vertices/sites i, j that are adjacent on the lattice. Ising model is widely considered the first model in the study of spin glasses. The Hamiltonian effectively captures the interaction, thus dependencies between each spin via a graph structure. Exchanging the lattice graph with other types of graph will give us some of the other famous models, such as the Sherrington-Kirkpatrick model (complete graph) and Edwards-Anderson model (high-dimensional lattice). Though by definition spin glass models can be seen as PGMs, the studies about them do not have a strong relation with tools in graph theory. In fact, it is a drastically different field compared to the topic we are discussing in chapter 3. Rather, we mention Ising model to highlight the novel representation of mathematical objects that bring probability and graph theory together.

Another early example is path analysis introduced by Sewall Wright in 1921 ([Wright \[1921\]](#), [Wright \[1934\]](#)). This method was created to understand and quantify the relationships among a set of correlated random variables, based on a hypothesized causal

structure. Wright represented the hypothesized causal relationship using path diagrams (essentially Directed Acyclic Graphs or DAGs), where nodes represent variables and arrows represent direct causal influences.

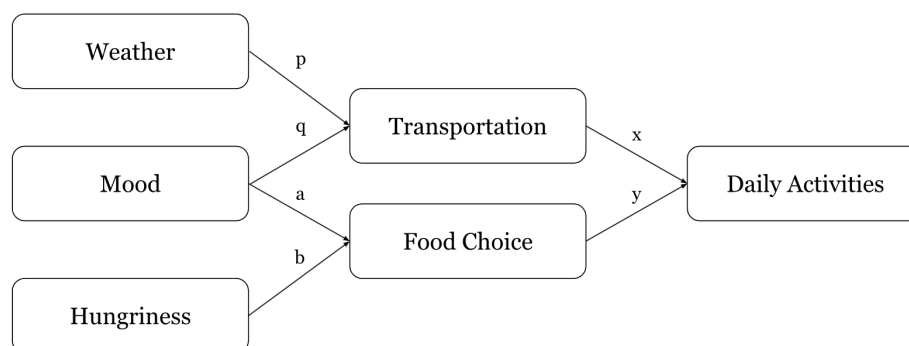


Figure 1.14. An intuitive example of path diagrams with edge weights (contributing factors). The daily activities of a person depend on the transportation and food choices, which further depend on the weather, mood and how hungry the person is. Using the diagram, we can conduct analysis such as estimating the impact of weather on the person's daily activities.

With the help of directed graphs, complicated chain of influence can be decomposed and quantitatively studied. The correlation/causation between two variables are measured by the sum of all paths that connect them and the contribution of a path is determined via the product of all its edge weight. This simple but effective framework allows us to estimate the magnitude of direct and indirect effects within the causal model.

Wright himself was also a population geneticist and he used path analysis extensively to study inheritance. Some of his famous work included the piebald patterns of Guinea-Pigs and factor analysis for offspring phenotype (Wright [1920], Wright [1921]). In fact, Wright had already started using the concept of path analysis in 1918 before he formally defined the term (Wright [1918]). It is safe to say the his biological research inspired

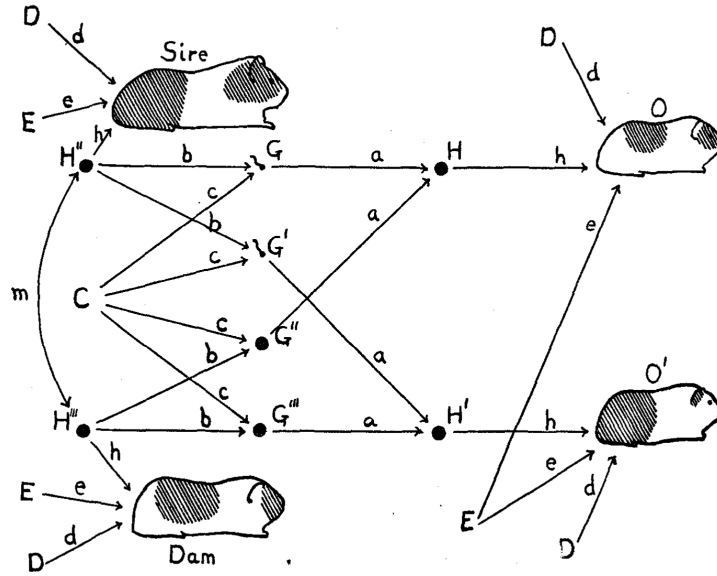


Figure 1.15. A path analysis diagram from [Wright \[1921\]](#). Wright used this diagram to visualize and analyze the relations between two mated individuals and their progeny.

the creation of this important mathematical model, yet another great victory for the interdisciplinary research between math and biology!

It is also worth noting that the Wright's work has profoundly influenced the field of PGM and computer science. Path analysis is considered a direct precursor to modern Structural Equation Modeling (SEM). In 1960s and 1970s, social scientist used path analysis and created a set of tools that are still widely used today in social science, psychology and economics ([Duncan \[1975\]](#), [Bollen \[1989\]](#)).

The inspiration of path analysis did not stop there. Fast-forwarding to the 1980s, the concept of PGM was widely recognized and used with the invention of Bayesian networks and Markov random fields, the two pillars of the realm. Continuing from Wright's work, Judea Pearl formalized Bayesian Networks (BN) in 1988 ([Pearl \[1988\]](#)). Similar to path analysis, BN uses nodes to represent variables and arrows (directed edges) to represent

direct dependence. On top of that, BN accurately captures the conditional probability using the chain decomposition. Let $X_1, X_2, \dots, X_n$ be a collection of random variables and $p(x_1, \dots, x_n)$ their joint density function. We have

$$(1.21) \quad p(x_1, \dots, x_n) = p(x_1)p(x_2|x_1) \cdots p(x_n|x_1, \dots, x_{n-1}).$$

BN uses the decomposition, but based on the graph structure. So compared to path analysis, the computation of weight in BN produces actual probability. This greatly increase the information density of the same framework. Using BNs, we are able to effectively navigate through complex dependency relationships and compute the probability of events from evidence.

BNs have been widely used in various fields. Besides biology and medical research, BNs are also used in software engineering and finance. One of the most well-known application of BNs is the email spam filter ([Sahami et al. \[1998\]](#)). The early version of email spam filters had very simple structures. It looked at the overall probability of an email being potential spam and conditional probability when certain words are present. Intuitively it worked a lot like how we detect scam calls in some way. When we pick up a phone call, if the number is unknown, then we will likely raise our suspicion (conditional scam probability goes up). Then if the caller starts to ask for personal information or request money, then the conditional probability of the call being a scam goes up even more. We then most likely hang up because there is a Bayesian network-like process that just happens in our head. Today, email spam filters have got much more sophisticated, with some including LLMs. However, LLMs are expensive to run so traditional, BN-based email filters are still the mainstream choices, keeping billions of people safe from

spam emails every day. For finance, the nature of BNs makes it the ideal tool to model potential risks ([Shenoy and Shenoy \[2003\]](#), [Gemela \[2001\]](#)). Risks and risk assessment can be generalized as attempts to answer the “what if” questions. In our daily lives, such questions can be easy. “What if I got a flat tire?” “Prepare a tire repair kit in your trunk.” Financial market and related risks have much more intertwined underlying causes. Such needs directly overlap with the specialty of BNs, making it the ideal tool for risk assessment and management til this day.

Now let us turn our attention to Markov Random Field (MRF). While Bayesian networks are very effective in representing direct dependencies and causal relationships. There are other types of links lacking clear “directionality”. As with the pixels of a picture we previously mentioned. It is clear that two adjacent pixels in a picture are likely to be correlated because they can depict the same object. However, it is impossible, or more accurately, logically unsound to say that one pixel causes the other. There are a lot more examples where the correlations are undirected or symmetric. In order to model those probabilistic dependencies, we need a different PGM other than BN. That is where MRF comes in.

The formalization of MRF is widely considered to be the work of Besag in 1974 ([Besag \[1974\]](#)). In this famous paper, Besag also presented an alternate proof for the famous Hammersley-Clifford theorem, showcasing the MRF as a powerful tool in terms of modeling dependencies based on geographic proximity. We will not go into the details of the proof, but it would be beneficial to state the theorem and compare the notion of MRF with a traditional Markov Chain.

Let $[n] = \{1, 2, \dots, n\}$ and $\mathcal{X} = (X_t)_{t \in [n]}$ be a finite dimension random vector with each X_t taking values in a finite set $\mathcal{S}_t = \{1, 2, \dots, m_t\}$. Suppose $[n]$ is the set of vertices of a graph. For each t , denote $\mathcal{N}_t$ the set of vertices s such that s is adjacent to t in the graph.

Definition 2. *The process $(X_t)_{t \in [n]}$ is said to be a Markov random field if*

(i) for each tuple $x = (x_1, x_2, \dots, x_n) \in \prod_{i=1}^n \mathcal{S}_i$, the joint probability satisfies

$$(1.22) \quad P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) > 0;$$

(ii) for each t and x ,

$$(1.23) \quad \mathbb{P}\{X_t = x_t | X_s = x_s, s \neq t\} = \mathbb{P}\{X_t = x_t | X_s = x_s, s \in \mathcal{N}_t\}.$$

We can see that the definition of MRF resembles much of the discrete-time Markov chain. Instead of the sequential Markov property, MRF has its own Markov property induced by the graph topology. A subset of vertices $A \subset [n]$ is said to be complete (or a clique) if each pair of vertices is connected with an edge in the graph. We can use this notion to define a Gibbs distribution.

Definition 3. *Let $x = (x_1, x_2, \dots, x_n) \in \prod_{i=1}^n \mathcal{S}_i$ be an arbitrary tuple from the joint distribution, and $\mathcal{C}$ the collection of all complete vertex subsets. A probability distribution Q is called a Gibbs distribution (or Gibbs random field) if it can be written in the form*

$$(1.24) \quad Q(x) = \prod_{A \in \mathcal{C}} V_A(x),$$

where $V_A(x)$ is a positive function that depends only on x_i 's with $i \in A$.

It is clear that each process $(X_t)_{t \in [n]}$ has a corresponding Gibbs distribution based on the dependencies. Now we can state the powerful Hammersley–Clifford theorem:

Theorem 3 (Hammersley–Clifford theorem). *Let Q be a probability distribution on $\prod_{i=1}^n \mathcal{S}_i$. Then Q is a Gibbs distribution if and only if it is a Markov random field with respect to the same graph structure.*

Besag’s paper is a major cornerstone in statistics and fields dealing with spatial data. Before his work, defining a valid joint probability distribution over many interacting variables was often difficult. Besag showed how one could specify the model based on conditional probability of each variable given its neighbors (using the local Markov property). Such practice is much more intuitive and actionable. On the other hand, his paper did not stop at a theoretical level. Besag also proposed practical methods for statistical analysis using pseudo-likelihood, which uses conditional probabilities to approximate maximum likelihood. Such estimations used to be intractable due to computational difficulties. Bridging theory and application, this paper was a huge achievement in the field of statistical learning that inspired countless advancements after it, such as the work by Geman and Geman ([Geman and Geman \[1984\]](#)) we mentioned in section [1.3.1](#).

The introduction of BN and MRF made researchers realize the power of PGMs. Also, graph theory and probability happen to be two of the mathematical sub-fields that are most related to computer science. People started to give PGMs much more focus, aiming to unify the models and theories so that a more comprehensive framework could serve to tackle the dependency problems emerging from complex systems and machine learning.

Image Restoration

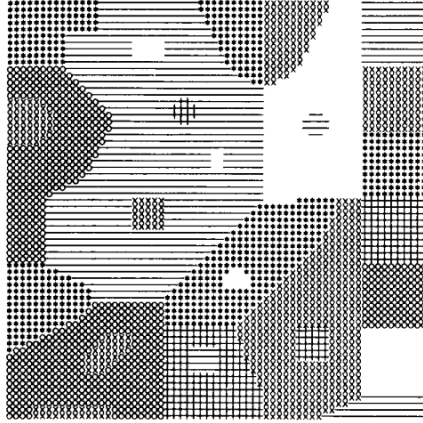
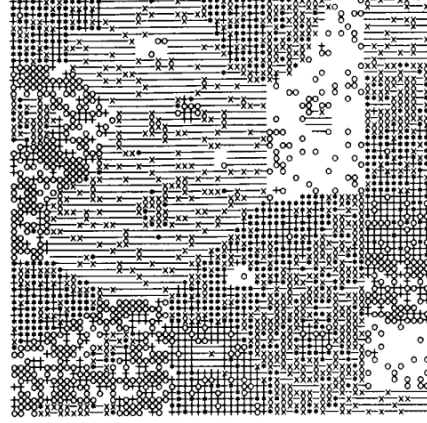
Fig. 3a. True six-colour scene: 64×64 .

Fig. 3b. Maximum likelihood classifier: 32% error rate.

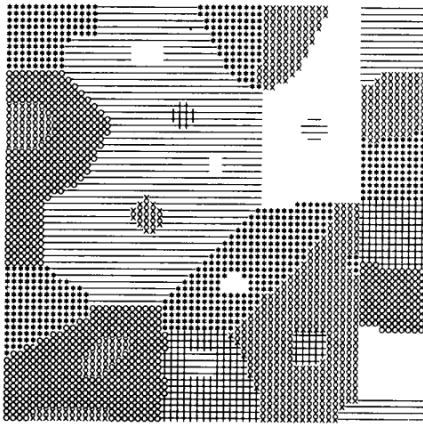
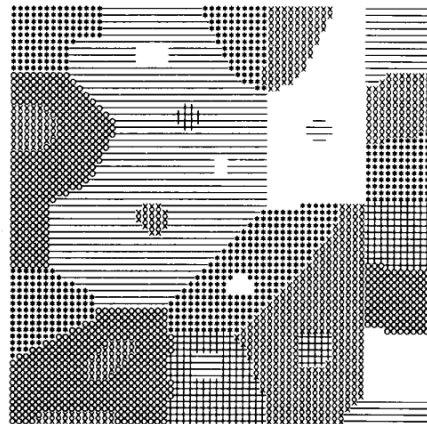
Fig. 3c. ICM reconstruction with $\beta \uparrow 1.5$: 1.2% error rate.Fig. 3d. ICM reconstruction with β estimated:
 $\hat{\beta} = 1.80$: 1.1% error rate.

Figure 1.16. Besag used MRF to develop the Iterated Condition Modes (ICM) algorithm that is able to restore dirty images. The figure is from his work in 1986 [Besag \[1986\]](#). As we can see, ICM performs much better than the old maximum-likelihood classifier.

Factor graph was created for this very purpose in 2001 ([Kschischang et al. \[2001\]](#)). As the name suggests, factor graphs provide a general representation that explicitly shows the factorization of a global function into local factors. The joint probability we discussed above for MRF is a good example. They connect widely used models like MRF and BN,

then unify the inference algorithms like belief propagation. The lineage from both MRF and BN is very clear.

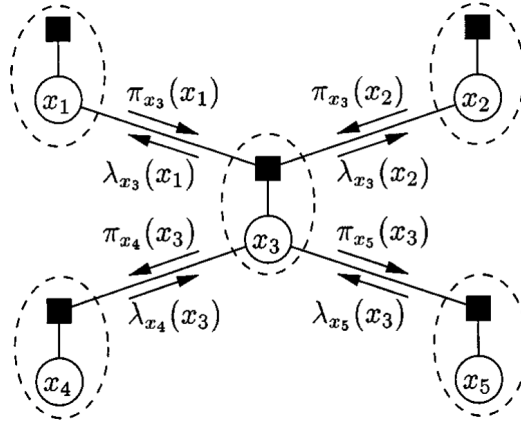


Figure 1.17. The diagram illustrating belief propagation in [Kschischang et al. \[2001\]](#). Belief propagation is a powerful model built upon BN and MRF that are broadly used in information theory and other related fields.

There were also other new ideas rising from the fertile soil of PGMs, such as Junction Trees, Variational Inference and MCMC ([Jordan et al. \[1999\]](#), [Gelfand and \[1990\]](#)). On the larger scale, the development of machine learning was also gradually shifting the fashion of the field. Instead of relying on expert knowledge, people began to use ML algorithms to learn the unknown parameters or the graph structures from complex data, building the navigation chart amidst the black sea of chaos. The effectiveness of PGMs, combined with the strong generative power of ML, expand the potentials of PGMs even further. It is exciting to see what this field will bring us for the rest of this century.

1.3.3. Bayesian Networks and Markov Random Fields

As we mentioned in section 1.3.2, Bayesian networks and Markov random fields represent the two main types of PGMs, reflecting to choice of directed or undirected graph. In chapter 3, we will provide a self-contained walkthrough on how machine learning algorithms can be used to learn the graph structure. Towards that end, in this chapter, we will dedicate some space to introduce the two main PGMs themselves, as well as the Lovász Local Lemma. This way we will have a better understanding of the background and how chapter 3 weaves different topics together to create a new algorithm.

Let us start with Bayesian network. Let $p(x_1, \dots, x_n)$ be any probability density function. Recall that we can decompose p using a chain of condition probability:

$$(1.25) \quad p(x_1, \dots, x_n) = p(x_1)p(x_2|x_1) \cdots p(x_n|x_1, \dots, x_{n-1}).$$

Such decomposition is often computationally intractable. For BN, we modify such decomposition that each x_i only depends on a small subset of ancestor variables x_{A_i} . For example, whether one takes the walk or bus depends on tomorrow's weather, and the weather depends a series of complicated natural factors. It would be an overkill if we model the choices of transportation on all these factors. Instead, we just let the weather be the only ancestor variable of the transportation choices and weather itself depends on the natural factors such as temperature, wind direction, etc. Bearing the intuition in mind, we can now give the formal definition of BN.

Definition 4. *A Bayesian network is a directed acyclic graph (DAG) $G = (V := [n], E)$ such that:*

(i) each node i represents a random variable X_i ;
(ii) for each X_i , there is a corresponding conditional probability distribution $p(x_i|x_{A_i})$
where A_i consists of all the immediate ancestors of i in the graph G .

Conversely, we say a probability distribution p factorizes over a DAG G if it can be decomposed into a product of factors induced by G .

Let us take a look at a simple example from [Koller and Friedman \[2009\]](#).

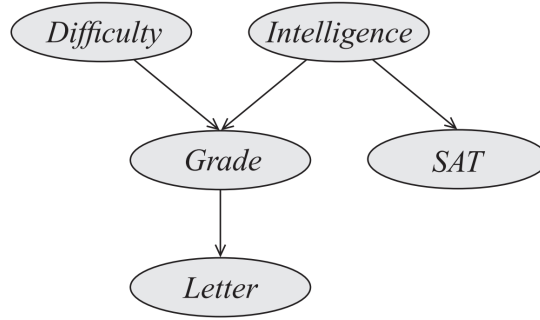


Figure 1.18. Figure 3.3 from [Koller and Friedman \[2009\]](#), a simple BN that models the letter grade and SAT scores of students.

Denote difficulties, grade, intelligence, SAT, and Letter by their first letter d, i, g, s, l respectively. We can decompose the joint probability $p(d, i, g, s, l)$ in figure 1.18 starting from end nodes s and l and moving upwards through the ancestry:

$$(1.26) \quad p(d, i, g, s, l) = p(s|i) \cdot p(l|g)p(g|i, d)p(i)p(d).$$

The reason that the example is well constructed is that not only it provides an easy-to-follow computation example, but also provides the three main patterns of conditional dependence in BN, based on whether the ancestor or intermediate node is observed or

not. For three random variables X, Y, Z , we write $X \perp Y | Z$ if X and Y are independent condition on Z . We also use arrows to denote the local graph structure. For example, $X \rightarrow Y$ means there is a directed edge from X to Y . The three common patterns are as follows.

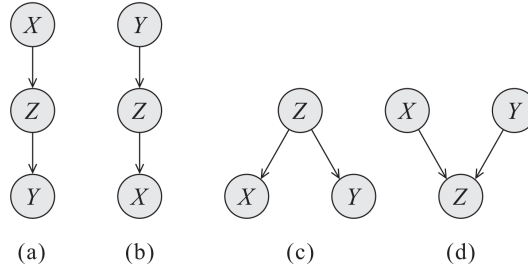


Figure 1.19. Figure 3.5 from [Koller and Friedman \[2009\]](#) that shows common connecting patterns in BN.

(i) Common cause, corresponding to (c) in the figure 1.19 above. If Z is unobserved, then obviously X and Y are not independent of each other because they share a common part of their randomness. However, if we know Z , then $X \perp Y | Z$ as the shared common part is determined and the rest of randomness is independent. To see that mathematically, first notice that we can write the joint probability as

$$(1.27) \quad p(X, Y, Z) = p(X|Z)p(Y|Z)p(Z).$$

For any x, y, z , z fixed, we have

$$(1.28) \quad \begin{aligned} \mathbb{P}(X = x, Y = y | Z = z) &= \mathbb{P}(X = x | Z = z) \mathbb{P}(Y = y | Z = z) \mathbb{P}(Z = z | Z = z) \\ &= \mathbb{P}(X = x | Z = z) \mathbb{P}(Y = y | Z = z) \end{aligned}$$

by the definition of conditional independence.

(ii) Common effect, which is (d) in the figure 1.19. If Z is unobserved, then X and Y are independent of each other. However, if we know Z , then X and Y are not independent of each other. The mathematical proof follows similarly as the example in (i) above. Moreover, it makes intuitive sense if we look at figure 1.18 for the relationship among difficulty, intelligence and grade. Of course, the difficulty of the exams and the intelligence of students are independent as they are the results from different entities (exam makers and students). However, if we know the grade of a student, say a very high grade for a hard exam, that will change the joint probability. Performing well on a hard exam is more likely to come from a smart student. Difficulty and intelligence are no longer independent in this case.

(iii) Evidential and causal effects, representing (a) and (b) in figure 1.19. Clearly for unobserved Z we have X and Y are not independent. However, if the intermediate node Z is observed, then $X \perp Y | Z$. This means that everything it takes to determine the final state is provided, so the root in the network no longer plays a role in the outcome of its children. This is also clear in figure 1.18. If the exam result is released, then the letter grade of a student will depend solely on the score, not on the smartness of the student.

The three patterns can be used to define another important concept called d-separation. As we just see, dependency can be passed through paths on the graph via a series of 3-node patterns given observed data. If there are no such way to connect two clusters of nodes, then they are essentially conditionally independent, revealing insights of the data we are trying to analyze.

Definition 5 (d-separation). *Let $\mathcal{X}$, $\mathcal{Y}$ and $\mathcal{O}$ be three sets of nodes in a Bayesian network G . We say that $\mathcal{X}$ and $\mathcal{Y}$ are d-separated given $\mathcal{O}$ (The letter $\mathcal{O}$ stands for observed) if $\mathcal{X}$ and $\mathcal{Y}$ are not connected by an active path. An undirected path in G (meaning one can go against the arrows) is called active given observed nodes $\mathcal{O}$ if for every sequence of consecutive three nodes X, Y, Z on the path satisfies one of the following conditions:*

- (i) $X \leftarrow Y \leftarrow Z, Y \notin \mathcal{O}$;
- (ii) $X \rightarrow Y \rightarrow Z, Y \notin \mathcal{O}$;
- (iii) $X \rightarrow Y \leftarrow Z, Y \in \mathcal{O}$.
- (iv) $X \leftarrow Y \rightarrow Z, Y \notin \mathcal{O}$.

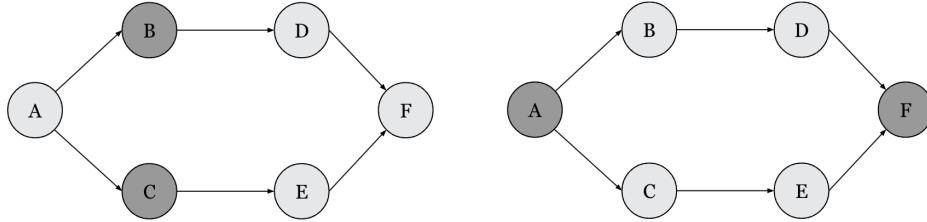


Figure 1.20. A pair BNs illustrating the concept of d-separation.

In figure 1.20, we mark the observed nodes with a darker gray color. It is easy to check that node A and F are d-separated given B and C on the left, while B and C are not d-separated given A and F on the right ($B \rightarrow D \rightarrow F \rightarrow E \rightarrow C$ is an active path). We then proceed to define

$$(1.29) \quad \mathcal{I}(G) = \{(\mathcal{X}, \mathcal{Y}, \mathcal{Z}) | \mathcal{X}, \mathcal{Y} \text{ are d-separated by } \mathcal{Z}\}.$$

This definition is where the power of graph theory starts to manifest. Recall that graph theory studies quantities about graphs that remain unchanged under various transformation and mapping, such as coloring and characteristic polynomial. The quantity $\mathcal{I}(G)$ is also one of such quantities tied to Bayesian network that helps identify different Bayesian networks with similar underlying structures. If we define two BNs G and G' as I-equivalent if $\mathcal{I}(G) = \mathcal{I}(G')$, then the following theorem holds

Theorem 4 (Theorem 3.9, [Koller and Friedman \[2009\]](#)). *Two Bayesian networks G and G' are I-equivalent if and only if there exists a sequence of network*

$$(1.30) \quad G = G_1, \dots, G_k = G'$$

such that the only difference between G_i and G_{i+1} , $1 \leq i \leq k-1$ is a single reversal of a covered edge.

This theorem, combined with an algorithm that can find $\mathcal{I}(G)$ and machine learning algorithms that can learn the graph structure, provides a capable framework for studying highly complex objects with lots of intricate dependencies. There are many other fascinating facts about BN but we should stop here to prevent ourselves from deviating too far. Now we will turn our attention to MRF.

Unlike BNs, the definition of an MRF does not derive strictly from decomposition of conditional probability. Rather, it assigns a factor(weight) to each clique(complete subgraph) in the graph.

Take figure [1.21](#) for instance, in the left graph, there are four complete subgraphs (A, B) , (B, C) , (C, D) and (A, D) . For any value assignments (a, b, c, d) of the random

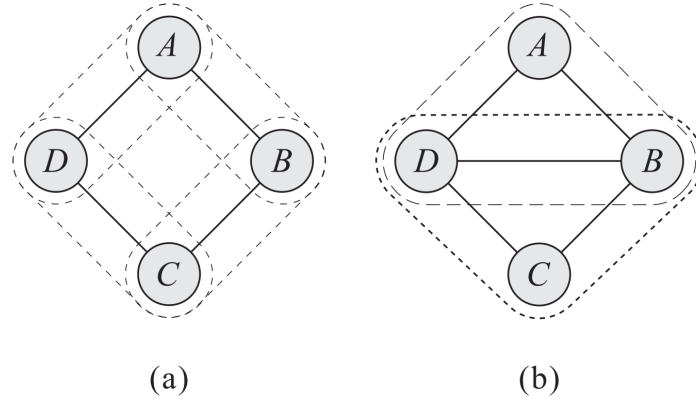


Figure 1.21. Two MRFs with different edges (and thus cliques) from figure 4.4, [Koller and Friedman \[2009\]](#).

variables (A, B, C, D) , we can write the factor of the value assignment as

$$(1.31) \quad \tilde{p}(a, b, c, d) = \phi(a, b)\phi(b, c)\phi(c, d)\phi(a, d),$$

where ϕ can be any positive functions defined by the user. Normalizing the factor will give us the joint probability distribution of the MRF:

$$(1.32) \quad p(a, b, c, d) = \frac{1}{Z} \tilde{p}(a, b, c, d),$$

and

$$(1.33) \quad Z = \sum_{(a,b,c,d)} \tilde{p}(a, b, c, d)$$

is called the partition function. We have already defined MRF in definition 2. For the graph on the right, there are only two complete subgraphs, consisting of A, B, D and

B, C, D . Then the factor score has the form

$$(1.34) \quad \tilde{p}(a, b, c, d) = \phi(a, b, d)\phi(b, c, d).$$

Now we can give an alternative and equivalent definition based on the graph structure:

Definition 6 (Alternative Definition of MRF). *Let $X_1, \dots, X_n$ be random variables. A Markov Random Field (MRF) is the joint probability distribution of $(X_i)_{i \in [n]}$ over an undirected graph $G = (V := [n], E)$ with each vertex i representing the random variable X_i . In addition, the joint probability p has the form*

$$(1.35) \quad p(x_1, \dots, x_n) = \frac{1}{Z} \prod_{C \in \mathcal{C}} \phi_C(x_C),$$

where $\mathcal{C}$ is the collection of all cliques (complete subgraphs) in G , each factor ϕ_C is a non-negative function, and

$$(1.36) \quad Z = \sum_{x_1, \dots, x_n} \prod_{C \in \mathcal{C}} \phi_C(x_C)$$

is the normalizing partition function.

While both are powerful, MRFs and BNs excel at modeling different types of dependencies. Although neither is strictly more general, any BN distribution can be represented by an MRF through a process called moralization. This involves adding an edge between any pair of parents that share a common child ('marrying parents') and then making all edges undirected. Furthermore, the concept of conditional independence is defined differently: while BNs use d-separation via active paths, two sets of nodes in an MRF are conditionally independent given a third set if that third set separates them in the graph.

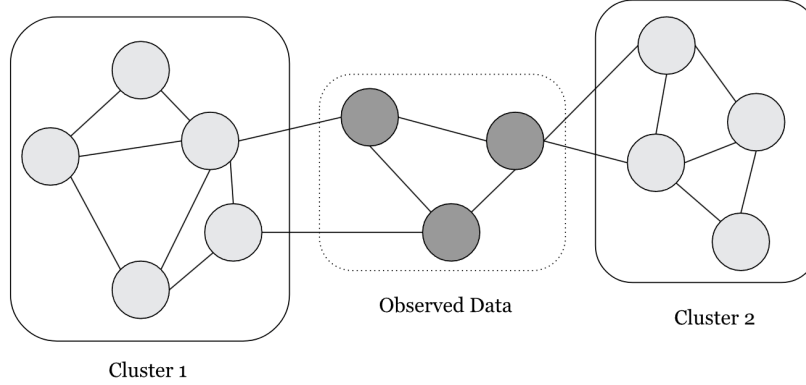


Figure 1.22. An MRF with two indicated independent clusters. The nodes with darker grey in the middle are observed, causing the clusters on the left and right to be independent.

The idea follows similarly as the evidential and causal effects in BN: a path of nodes connecting two clusters is the shared part of their randomness. If the path is fully observed, then the rest of randomness is conditionally independent. In particular, if all neighbors of a node are observed, then this node is independent from the rest of the network.

Upon this point, we can see that MRF has several advantages over BN. First, MRF is more versatile. We can apply MRF to a broader range of problems. Any BNs can be converted to MRF but not vice versa, and especially when the dependencies of the model are naturally symmetric. This gives us the second advantage, that MRFs are able to describe certain dependency structures that cannot be captured by BNs like k -CNF formulas.

However, there are reasons that we cannot just use MRF everywhere. The primary drawback of MRF is the computation of Z . To compute the partition function Z , one might need to sum over an exponential number of possible value assignments. This

summation is generally NP-hard. Moreover, since every BN is required to be acyclic and directed, it is very easy to generate data using the hierarchy of dependencies relations. Since MRF, in contrast, is undirected and thus has no “starting points” or ancestors, it is often much harder to generate data from a MRF.

Since our work in chapter 3 mainly deals with intractable computations, we will dedicate some space to discuss sampling and Lovász Local Lemma in the next section. We also encourage the reader to check out the textbook by Koller and Friedman [Koller and Friedman \[2009\]](#) for a more comprehensive introduction to PGMs, which is considered the standard textbook on the subject.

1.3.4. The Necessity of Sampling and Lovász Local Lemma

In the previous section, we mentioned that computing the partition function Z in an MRF directly is likely to be computationally intractable. Therefore, sampling algorithms are usually required so that we can avoid computations that scale exponentially. This is a clever and natural idea and approach, but it introduces a new problem.

As the theme throughout this section, dependency is everywhere in the data we are trying to analyze. It often comes in the form of constraints, or a series of “bad events”. For the machine learning algorithm to effectively learn the PGM model, we need to make sure the samples we generate are from the actual conditional distributions from the data. Therefore when we are trying to sampling with constraints, it is important for us to know the (approximate) answers of the following two questions:

- (i) Given a set of constraints, do the samples satisfying all conditions exist?

(ii) If yes, how do we sample from the distribution effectively without spending exponential time or stuck in an finite loop. (Meeting one constraint will fail another. Sometimes constant adjustment without proper strategy will indefinitely prolong the sampling process.)

Though there is no one-size-fits-all answer to the two questions, there are some general principles that can be applied to a wide range of problems. One of the most famous and widely used principle is Lovász Local Lemma (LLL), discovered and proved by Erdős and Lovász in 1975.

Lemma 1 (Erdős and László [1974]). *Let $A_1, \dots, A_k$ be a sequence of events such that each event occurs with probability at most p and such that each event is independent of all the other events except for at most d of them. Assume*

$$(1.37) \quad 4pd \leq 1.$$

Then there is a nonzero probability that none of the events occurs.

Shearer improved the results in 1985 and the updated lemma becomes the Lovász Local Lemma (LLL) we know today:

Lemma 2 (Shearer [1985]). *Assuming the same conditions as lemma 1, if we have $p \leq \frac{1}{2d}$. Assume*

$$(1.38) \quad \begin{aligned} p &< \frac{(d-1)^{(d-1)}}{d^d}, & d > 1; \\ p &< \frac{1}{2}, & d = 1. \end{aligned}$$

Then there is a nonzero probability that none of the events occurs.

Both lemma 1 and 2 answered the first existence problem. However, there is a lot of room for improvement. All the bad events have a common upper bound p , which reduces the applicability of the lemmas. Spencer in 1977 proved the asymmetric version of the lemma. To state this result, we need to the concept of dependency graphs, which are extensively used in chapter 3. To put in simple terms, a dependency graph is a graph with nodes representing a set of events such that mutually independent events are not adjacent, otherwise connected via an edge. With that being said, we have the following statement.

Lemma 3 (Spencer [1977], Asymmetric Lovász local lemma). *Let $\mathcal{A} = \{A_1, \dots, A_n\}$ be a finite set of events in the probability space Ω . For $A \in \mathcal{A}$, let $\Gamma(A)$ be the neighbors of A in the dependency graph. Assume there exists a map $f : \mathcal{A} \rightarrow [0, 1)$ such that for every $A \in \mathcal{A}$,*

$$(1.39) \quad \mathbb{P}(A) \leq f(A) \prod_{B \in \Gamma(A)} (1 - f(B)).$$

Then there exists a nonzero probability that none of the events in $\mathcal{A}$ occurs. Additionally

$$(1.40) \quad \mathbb{P}\left(\bigcap_{i=1}^n \bar{A}_i\right) \geq \prod_{A \in \mathcal{A}} (1 - f(A)).$$

As we can see, the asymmetric LLL has a stronger result, even with a positive lower bound. However, the improved existence does not lead to a actionable algorithm that helps us sample from the distribution. All proofs of the three lemmas above are non-constructive. In order to put this powerful mathematical fact into practice, there has

been many attempts seeking for constructive proofs or better, an algorithmic version of LLL. To name a few, Beck in 1991 gave an algorithmic proof of LLL under polynomial time, one of the early milestones that turned LLL into an effective sampling technique. The most well-known result is probably by Moser and Tardos ([Moser and Tardos \[2009\]](#)). Because of their improvement and novelty, Robin Moser and Gábor Tardos received the Gödel Prize in 2020.

Thus far, we have introduced most background knowledge related to our work in chapter 3. We started with the importance of the overarching dependency problem, then proceeded to give a brief history of the tools we use. Finally, we introduced LLL, which we later incorporate into a neural solver with parallelism capability. Starting from graph theory and probability, the intersection between various fields of mathematics and computer science never ceases to amaze us with its beauty and potential.

1.4. Factor Analysis and Variational Inference

1.4.1. Combining DR and PGM

In previous sections [1.2](#) and [1.3](#), we have discussed the importance and difficulties of problems in probability and data science that arise from high dimensionality and dependencies. To solve these two problems, scientists created dimension reduction and probabilistic graphic models to tackle them respectively. For our work in chapter 4, the algorithm we develop involves both dimension reduction and probabilistic graphic models. The framework we invented is called Variational inference Factor Analysis (VarFA). It is self-explanatory that this toolkit combines variational inference with factor analysis.

From the dimension reduction perspective, the variational inference part draws inspiration of classical Variational AutoEncoder (VAE), which utilizes the so-called “latent dimensions” to generate output similar to the input data set. To understand latent dimensions, let us take a look at some real-life examples, such as modeling in games. For large games (such as games with vast open worlds) it is impossible to create models for every single NPC, building and tree, etc. One could use the same object repetitively, but seeing two identical people on the street will definitely break immersion. Instead, game developers want to generate a set of objects that are similar to the in-game object they have already created, but different. If we see input image X as high-dimensional data with pixel information, then we want to find a probabilistic algorithm that can compute the probability $p(X)$, where p refers to the probability of how likely this collection of pixel can be seen as similar objects created manually by developers. If p is high, we take the image and a new in-game object is generated.

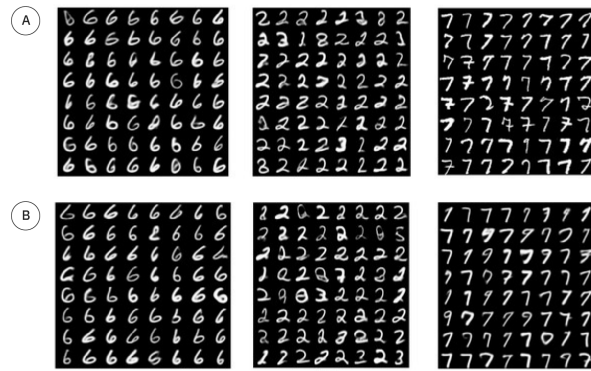


Figure 1.23. The original handwritten digits from MNIST (top row) and VAE-generated digits (bottom row), from [Mak et al. \[2023\]](#). Though some outputs are not entirely natural, all are recognizable and there is no mixing of two different digits.

But the task is not as easy as it seems. If we use brutal force approaches, we can end up with an NPC with body of a human and head of a bus, if the pool of hand-drawn objects contain both humans and buses. This will create a lot of trouble. However, if the model is designed to assign a category to each of the objects, such as human, vehicles, plants, etc, then it will know not to mix things up when generating objects. The vector (human, vehicles, plants, ...) is the latent dimension we are talking about. In practice, latent dimensions usually do not have meaningful labels, and they do not need to. Just as t-SNE, the latent dimensions are used to represent different types of data samples and make sure the output is consistent with the categorization/representing clusters.

As for PGM viewpoint, the architecture of VarFA can be represented as a neural network (directed graph), where each layers of input determines the values/distributions of the next layers. The training process of the model is essentially learning the parameters on the graph. This firmly places VarFA in the PGM family. Indeed, as we can see in chapter 4, we model the performance/grades of students based on latent skill levels. How well a student perform on a specific test question can be a combination of related skills. VarFA establishes the dependency structures between competence and exam scores, not just predicting them.

The only background we have not covered is factor analysis. In the next section, we will cover the basic concepts of factor analysis as well as its historical heritage. Afterwards, we will discuss the concepts and techniques we borrow from VAE. With all the topics covered, the readers should have a sufficient context to understand the motivation and contribution of our work in chapter 4.

1.4.2. Lineage of Factor Analysis

We have introduced the brief history of dimension reduction in section 1.2.2, including a brief reference to VAE. In fact, Factor Analysis (FA) is also a dimension reduction technique, but not in the same fashion as the other algorithms we introduced earlier. Therefore, it would be beneficial to give a short rundown of this approach before we move onto the next section.

FA was widely considered to be the brain child of Charles Spearman ([Spearman \[1904\]](#)), the famous statistician and psychologist known for the famous “Spearman Correlation”. In his famous paper, he conducted a series of experiments on the performance of students in different school subjects and found that although a lot of the subjects were seemingly unrelated, the students’ performance was consistent across the board. This led him to think that there was one single underlying factor that determines the competence of students, or colloquially speaking, “smart kids can learn any subject quickly and easily”. What he proposed was the so-called single-factor model, associating students’ performance with a single variable called “general mental ability”. Coincidentally, VarFA also focuses its application in education and the modelling of students’ performance. It is no doubt that education is more than vital to the entire human society, and that is precisely why we continue to go back to such topic, more so in the era of machine learning and AI.

After Spearman’s idea, various researchers tried to expand the model to naturally include the cases where multiple factors are required to model and predict the data. One of the most important cornerstones in this field is the work by Thurstone ([Thurstone \[1947\]](#)).

This book includes various important works starting from the 1930s. The discoveries in this period laid the foundation of the FA framework we are familiar with today.

In particular, assume there are p rounds of observations for n individuals. FA assumes that the collective behavior of the individuals can be explained by k common factors, and $k < p$. Mathematically speaking,

$$(1.41) \quad x_{i,m} - \mu_i = \sum_{j=1}^k l_{i,j} f_{j,m} + \epsilon_{i,m}.$$

Here, $x_{i,m}$ is the i -th observation of the m -th individual, μ_i is the mean of the i -th observation, $l_{i,j}$ is the loading of the j -th factor on the i -th observation, $f_{j,m}$ is the j -th factor score for the m -th individual and $\epsilon_{i,m}$ is the error term, usually with zero mean and finite variance. This definition is the formulation of basically one sentence:

For the samples we are investigating, the observations are linear combinations of the common factors plus some noise.

And it is easy to see that (1.41) can be rewritten in the form of matrix multiplication:

$$(1.42) \quad X - M = LF + \epsilon.$$

More details about factor analysis will be given in chapter 4. In this introductory chapter, it is more important to address the unaddressed statement at the beginning of this section: why is factor analysis different from other dimension reduction techniques, though it seems very basic, only using linear combinations and independent noise? The answer lies not in the mathematical formulation, but rather the philosophy of usage. **FA is the quantitative catalyst of statistical intuition.**

To explain this answer, let us point out that FA can be divided into two categories: Exploratory Factor Analysis (EFA) and Confirmatory Factor Analysis (CFA). EFA is created in 1930s and largely contributed by Thurstone ([Thurstone \[1947\]](#)). In EFA, the user does not specify the structure of observations and is required to determine the number of factors. Afterwards, the user needs to choose a rotation method to make the factors more interpretable. Unlike PCA, EFA’s goal is to help researchers interpret the latent factors, whereas PCA stops after giving the data points in lower dimensions. EFA aims to generate intuitions. Though closely related to PCA, the mind set behind EFA was already different even in early 20th century.

This unique philosophy of EFA was extended further when CFA was created in 1960s by Jöreskog ([Jöreskog \[1967\]](#), [Jöreskog \[1969\]](#)). The user now needs to specify the structure of the model, from the number of factors and the loadings for each of them. CFA was created to provide a statistically sound procedure to verify what might be considered “a good guess from experienced researchers”. To use this model, researchers are required to think carefully about the underlying structure of their measurements, and provide a plausible model to evaluate those thoughts. CFA tests intuition.

The DR techniques we introduced in section [1.2](#) are mostly automatic, input the data and the algorithm will do the rest for you. Meanwhile, FA values human input and strives to make the results explainable. Though a lot of advancement in today’s AI comes with the caveat of “black box”, FA’s lineage was well preserved and inspired a lot of subsequent discoveries in the field of Explainable Artificial Intelligence (XAI) ([Doshi-Velez and Kim \[2017\]](#), [Miller \[2018\]](#)). It is also the reason why we choose FA to combine with the generative power of VAE. Our work in chapter 4 is not a mere mix and swap

of models, but rather a concrete practice of new mindsets towards the goal of fostering human learning with machine learning.

1.4.3. The Modern Toolkit: Variational Autoencoders

Apart from FA, VAE is another core piece of the puzzle in our work about VarFA. We brought up VAE in section 1.2.2 briefly and now it is a good time to formally introduce it. Similar to algorithms like t-SNE, VAE uses the so-called latent dimensions that are often not interpretable. However, unlike t-SNE, VAE is a generative model, meaning instead of visualization, it is to generate new data points that follow the same distribution as the input data. Invented by Welling and Kingma in 2013 (Kingma and Welling [2014]), not long after CNN was coined. VAE is an important milestone in the field of artificial intelligence because the capability of generating content based on training materials back then was truly revolutionary.

That being said, VAE is not the silver bullet, far from it. As we will see very soon in the deduction of the method, VAE relies on a lot of bold assumptions and approximations, and offers little room for tuning. Unlike t-SNE and MRF, VAE is not used in chapter 4, but extremely important in the overall literature. Therefore we will give a more detailed walkthrough of this amazing algorithm, focusing on the underlying assumptions and more explicit mathematical details. (There are many other more standard materials on VAE, such as the famous tutorial by Carl Doersch Doersch [2021].)

Given a data point $\mathbf{x}$ in high dimension, VAE assumes that $\mathbf{x}$ is determined by a fewer number of latent variables, denoted as a lower dimensional vector $\mathbf{z}$. Using conditional

probability decomposition, we can write

$$(1.43) \quad p(\mathbf{x}|\mathbf{z}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} \equiv \int p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z},$$

where θ is the hyperparameter VAE is trying to learn. The exact formulation is given in (1.59) and (1.63). The true bold claim here is not that data itself depends on latent variables, but the assumption that $p(\mathbf{z})$ is (often) a multi-dimensional random variable with i.i.d., standard normal entries. Bold as it might be, the assumption itself is in fact pretty well-grounded. For instance, your performance at a certain sport is determined by a series of factors such as your endurance and strength. It would be normal that both your strength and endurance follow a Gaussian-like distribution. Your endurance might fluctuate around your “average” level and performing significantly above or below your average is getting exponentially rare. Since all Gaussian distributions are the result of a simple affine transformation of the standard normal distribution, we might as well assume that the latent variables are i.i.d. standard normal from the very beginning. Furthermore, p_θ is also mostly set to be a Gaussian distribution if the target data is continuous following the same logic.

After giving $p(\mathbf{z})$ a distribution, determining the parameters for $p_\theta(\mathbf{x}|\mathbf{z})$ directly is not feasible. That is the primal motivation for the encoder-decoder architecture. By Bayes formula, we can write

$$(1.44) \quad p_\theta(\mathbf{z}|\mathbf{x}) = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})} = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{\int p_\theta(\mathbf{x}|\mathbf{z}')p(\mathbf{z}')d\mathbf{z}'}.$$

This means we want to learn the relationship between the input data and the latent factors first before we start generating data. This way we can use the learned hyperparameter

θ and samples from $p(\mathbf{z})$ to generate new data points. This leads to the next problem: $p(\mathbf{x})$ is also intractable. Therefore, we need another parametrized Gaussian distribution $q_\phi(\mathbf{z}|\mathbf{x})$ to approximate $p_\theta(\mathbf{z}|\mathbf{x})$. This is the second bold assumption, that the conditional distribution of latent variables given observed data is (almost) Gaussian. To give an intuitive example, suppose we see a student performing exceptionally well in a math exam, we might take a high guess at the student's math skill (mean), or the related latent variable. If the student is consistent, then we might decrease the variance. Formally speaking, $q_\phi(\mathbf{z}|\mathbf{x})$ takes the following form:

$$(1.45) \quad q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))).$$

Here, $\boldsymbol{\mu}_\phi(\mathbf{x})$ is the mean vector. $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$ is the variance vector (diagonal elements of the covariance matrix). The encoder network takes $\mathbf{x}$ as input and outputs the parameters $\boldsymbol{\mu}_\phi(\mathbf{x})$ and $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$ (or often $\log \boldsymbol{\sigma}_\phi^2(\mathbf{x})$ for numerical stability). The explicit formula for the probability density function (PDF) of $q_\phi(\mathbf{z}|\mathbf{x})$, assuming a N -dimensional latent space, is:

$$(1.46) \quad q_\phi(\mathbf{z}|\mathbf{x}) = \frac{1}{(2\pi)^{N/2} |\boldsymbol{\Sigma}_\phi(\mathbf{x})|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{z} - \boldsymbol{\mu}_\phi(\mathbf{x}))^T \boldsymbol{\Sigma}_\phi(\mathbf{x})^{-1} (\mathbf{z} - \boldsymbol{\mu}_\phi(\mathbf{x})) \right),$$

where $|\boldsymbol{\Sigma}_\phi(\mathbf{x})|$ is the determinant of the covariance matrix $\boldsymbol{\Sigma}_\phi(\mathbf{x})$, which simplifies to $\prod_{j=1}^N \sigma_{j,\phi}^2(\mathbf{x})$.

Let us take a pause here and recall that we make two bold assumptions during the setup of the encoder-decoder framework. We assume that the latent variables are i.i.d. standard normal random variables, and the conditional distribution of latent variables given observed data is also Gaussian. Without these two assumptions, VAE will not be

computationally possible. So qualitatively speaking, we can say VAE roots itself in one core belief:

The data we are working on is intrinsically Gaussian.

Now let us go back to the mathematical details. We call $p_\theta(\mathbf{z}|\mathbf{x})$ the true posterior distribution, the distribution “post the observation” and $p(\mathbf{z})$ the prior, the distribution “prior to the observation”. We measure the distance between our approximation $q_\phi(\mathbf{z}|\mathbf{x})$ and $p_\theta(\mathbf{z}|\mathbf{x})$ using KL-divergence. By definition 1, we compute

$$(1.47) \quad D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) = \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z}|\mathbf{x})} d\mathbf{z}$$

$$(1.48) \quad = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p_\theta(\mathbf{z}|\mathbf{x})].$$

Substitute $p_\theta(\mathbf{z}|\mathbf{x}) = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}$:

$$(1.49) \quad \begin{aligned} & D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) \\ &= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p_\theta(\mathbf{x}|\mathbf{z}) - \log p(\mathbf{z}) + \log p(\mathbf{x})]. \end{aligned}$$

Since $\log p(\mathbf{x})$ does not depend on $\mathbf{z}$, the expectation is just $\log p(\mathbf{x})$:

$$(1.50) \quad \begin{aligned} & D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) \\ &= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})] - \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] + \log p(\mathbf{x}). \end{aligned}$$

The first term is the KL divergence between the approximate posterior and the prior:

$$(1.51) \quad D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})].$$

Substituting this back into the equation of $D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))$ gives:

$$\begin{aligned}
 & D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) \\
 (1.52) \quad & = D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) - \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] + \log p(\mathbf{x}).
 \end{aligned}$$

Rearranging for $\log p(\mathbf{x})$:

$$\begin{aligned}
 & \log p(\mathbf{x}) \\
 (1.53) \quad & = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) + D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})).
 \end{aligned}$$

Since KL divergence is always non-negative ($D_{\text{KL}}(q || p) \geq 0$), we have:

$$(1.54) \quad \log p(\mathbf{x}) \geq \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})).$$

This right-hand side is the Evidence Lower Bound (ELBO), often denoted $\mathcal{L}(\theta, \phi; \mathbf{x})$:

$$(1.55) \quad \mathcal{L}(\theta, \phi; \mathbf{x}) = \underbrace{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction Term}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))}_{\text{KL Divergence / Regularization Term}}.$$

Maximizing the ELBO with respect to parameters θ and ϕ serves two purposes:

1. It pushes $\log p(\mathbf{x})$ higher (our original goal).
2. It minimizes the KL divergence between the approximate posterior $q_\phi(\mathbf{z}|\mathbf{x})$ and the true posterior $p_\theta(\mathbf{z}|\mathbf{x})$ (because $\log p(\mathbf{x})$ is fixed for a given $\mathbf{x}$, and the ELBO is $\log p(\mathbf{x}) - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x}))$).

To making things mathematically explicit, let us take a look at the the formula of ELBO (1.55). The KL divergence term measures how much the approximate posterior

(learned by the encoder) deviates from the prior $p(\mathbf{z})$. Specifically, when

$$(1.56) \quad q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))), \quad p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}),$$

this KL divergence has a convenient analytical solution:

$$(1.57) \quad D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) = \frac{1}{2} \sum_{j=1}^N (\mu_{j,\phi}^2(\mathbf{x}) + \sigma_{j,\phi}^2(\mathbf{x}) - \log(\sigma_{j,\phi}^2(\mathbf{x})) - 1).$$

To see this, the general KL divergence between $Q = \mathcal{N}(\boldsymbol{\mu}_Q, \boldsymbol{\Sigma}_Q)$ and $P = \mathcal{N}(\boldsymbol{\mu}_P, \boldsymbol{\Sigma}_P)$ is

$$(1.58) \quad \frac{1}{2} \left(\text{Tr}(\boldsymbol{\Sigma}_P^{-1} \boldsymbol{\Sigma}_Q) + (\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q)^T \boldsymbol{\Sigma}_P^{-1} (\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q) - N + \log \frac{|\boldsymbol{\Sigma}_P|}{|\boldsymbol{\Sigma}_Q|} \right).$$

Plugging in $\boldsymbol{\mu}_Q = \boldsymbol{\mu}_\phi(\mathbf{x})$, $\boldsymbol{\Sigma}_Q = \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))$, $\boldsymbol{\mu}_P = \mathbf{0}$, and $\boldsymbol{\Sigma}_P = \mathbf{I}$ leads to the simplified formula above.

On the other hand, the reconstruction term usually depends on whether the data is binary or continuous. For binary data such as MNIST digits, we assume each pixel x_i is a Bernoulli random variable. The decoder network (parameterized by θ) takes $\mathbf{z}$ and outputs a vector $\mathbf{p}_\theta(\mathbf{z})$, where each element $p_{i,\theta}(\mathbf{z})$ is the probability that pixel x_i is 1.

$$(1.59) \quad p_\theta(\mathbf{x}|\mathbf{z}) = \prod_{i=1}^N p_{i,\theta}(\mathbf{z})^{x_i} (1 - p_{i,\theta}(\mathbf{z}))^{1-x_i},$$

where N is the dimensionality of $\mathbf{x}$. The log-likelihood is:

$$(1.60) \quad \log p_\theta(\mathbf{x}|\mathbf{z}) = \sum_{i=1}^N [x_i \log p_{i,\theta}(\mathbf{z}) + (1 - x_i) \log(1 - p_{i,\theta}(\mathbf{z}))].$$

This is the negative binary cross-entropy between the input $\mathbf{x}$ and the reconstructed probabilities $\mathbf{p}_\theta(\mathbf{z})$. The reconstruction term becomes:

$$(1.61) \quad \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\sum_{i=1}^N (x_i \log p_{i,\theta}(\mathbf{z}) + (1 - x_i) \log(1 - p_{i,\theta}(\mathbf{z}))) \right].$$

For continuous data, we assume each dimension x_i is Gaussian. The decoder network takes $\mathbf{z}$ and outputs the mean of the Gaussian $\boldsymbol{\mu}_\theta(\mathbf{z})$. We often assume a fixed variance, say $\sigma^2 \mathbf{I}$, for simplicity (e.g., $\sigma^2 = 1$), or the decoder could also output the variance. Assuming fixed variance σ^2 :

$$(1.62) \quad p_\theta(\mathbf{x}|\mathbf{z}) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x_i - \mu_{i,\theta}(\mathbf{z}))^2}{2\sigma^2} \right).$$

The log-likelihood is:

$$(1.63) \quad \log p_\theta(\mathbf{x}|\mathbf{z}) = \sum_{i=1}^N \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x_i - \mu_{i,\theta}(\mathbf{z}))^2}{2\sigma^2} \right]$$

$$(1.64) \quad = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu_{i,\theta}(\mathbf{z}))^2$$

$$(1.65) \quad = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|^2.$$

This is proportional to the negative squared error between the input $\mathbf{x}$ and the reconstructed mean $\boldsymbol{\mu}_\theta(\mathbf{z})$. The reconstruction term becomes:

$$(1.66) \quad \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[-\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|^2 \right].$$

Since the constant term doesn't affect optimization w.r.t θ, ϕ , maximizing this term is equivalent to minimizing the expected squared error: $\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|^2]$.

After clearing things up, let us go back to (1.55). There's a problem with the reconstruction term $\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})}[\dots]$. We need to compute its gradient with respect to ϕ . However, the expectation is taken over a distribution q_ϕ whose parameters $(\boldsymbol{\mu}_\phi, \boldsymbol{\sigma}_\phi^2)$ depend on ϕ . Computationally, standard backpropagation cannot handle gradients through the sampling process $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$.

The famous reparameterization trick (Kingma and Welling [2014]) solves this (which is also applicable to VarFA). For distributions like the Gaussian, we can rewrite the sampling process. Instead of sampling $\mathbf{z}$ directly from $\mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x})))$, we first sample a noise vector $\boldsymbol{\epsilon}$ from a fixed, simple distribution, typically a standard normal: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Then we can transform this noise using the parameters from the encoder:

$$(1.67) \quad \mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}$$

where $\odot$ denotes the element-wise product (Hadamard product), and $\boldsymbol{\sigma}_\phi(\mathbf{x})$ is the vector of standard deviations (square root of the variances $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$). Note that the encoder usually outputs $\log \boldsymbol{\sigma}_\phi^2(\mathbf{x})$, so $\boldsymbol{\sigma}_\phi(\mathbf{x}) = \exp(0.5 \times \log \boldsymbol{\sigma}_\phi^2(\mathbf{x}))$.

Now, $\mathbf{z}$ still follows the desired distribution $q_\phi(\mathbf{z}|\mathbf{x})$, but the stochasticity comes from $\boldsymbol{\epsilon}$, which is independent of ϕ . The dependence on ϕ is now deterministic through $\boldsymbol{\mu}_\phi(\mathbf{x})$ and $\boldsymbol{\sigma}_\phi(\mathbf{x})$.

We can rewrite the reconstruction term in (1.55) using this trick:

$$(1.68) \quad \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} [\log p_\theta(\mathbf{x}|\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon})].$$

In practice, we approximate this expectation using Monte Carlo sampling, typically with just one sample ($L = 1$) of ϵ per data point $\mathbf{x}$ per training step:

$$(1.69) \quad \log p_\theta(\mathbf{x}|\mathbf{z}^{(i,l)}) \quad \text{where} \quad \mathbf{z}^{(i,l)} = \boldsymbol{\mu}_\phi(\mathbf{x}^{(i)}) + \boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(l)} \quad \text{and} \quad \boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Here we used superscript to distinguish sampled data points from general variables and coordinates. Combining everything, the objective function we want to maximize (or minimize its negative) for a single data point $\mathbf{x}$, using the reparameterization trick (1.67) and the analytical KL divergence (1.57), is approximated as:

$$(1.70) \quad \begin{aligned} \mathcal{L}(\theta, \phi; \mathbf{x}) &\approx \log p_\theta(\mathbf{x}|\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}) \\ &\quad - \frac{1}{2} \sum_{j=1}^N \left(\mu_{j,\phi}^2(\mathbf{x}) + \sigma_{j,\phi}^2(\mathbf{x}) - \log(\sigma_{j,\phi}^2(\mathbf{x})) - 1 \right), \end{aligned}$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is sampled once.

We typically minimize the *negative* ELBO averaged over a mini-batch of M data points:

$$(1.71)$$

$$(1.72) \quad \begin{aligned} \text{Loss} &= -\frac{1}{M} \sum_{i=1}^M \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) \\ &\approx \frac{1}{M} \sum_{i=1}^M \left[\underbrace{-\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})}_{\text{Reconstruction Loss}} + \underbrace{\frac{1}{2} \sum_{j=1}^N \left(\mu_{j,\phi}^2(\mathbf{x}^{(i)}) + \sigma_{j,\phi}^2(\mathbf{x}^{(i)}) - \log(\sigma_{j,\phi}^2(\mathbf{x}^{(i)})) - 1 \right)}_{\text{KL Loss}} \right], \end{aligned}$$

where $\mathbf{z}^{(i)} = \boldsymbol{\mu}_\phi(\mathbf{x}^{(i)}) + \boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(i)}$ with $\boldsymbol{\epsilon}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

The term $-\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})$ is the reconstruction loss (e.g., binary cross-entropy or mean squared error, depending on the data type and choice of p_θ).

At this point, we have broken down the VAE algorithm into as many details as possible for mathematical clarity. Let us end this section with the formal procedure of the training process and data generation after the training is done, so that the readers know what exactly is being computed in each step.

Training Procedure of VAE

- (1) Initialize encoder parameters ϕ and decoder parameters θ .
- (2) Repeat until convergence:
 - (a) Sample a mini-batch of data $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(M)}\}$.
 - (b) For each $\mathbf{x}^{(i)}$ in the mini-batch:
 - (i) Pass $\mathbf{x}^{(i)}$ through the encoder network to get $\boldsymbol{\mu}_\phi(\mathbf{x}^{(i)})$ and $\log \boldsymbol{\sigma}_\phi^2(\mathbf{x}^{(i)})$.
 - (ii) Calculate $\boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) = \exp(0.5 \times \log \boldsymbol{\sigma}_\phi^2(\mathbf{x}^{(i)}))$.
 - (iii) Sample $\boldsymbol{\epsilon}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
 - (iv) Compute the latent variable sample: $\mathbf{z}^{(i)} = \boldsymbol{\mu}_\phi(\mathbf{x}^{(i)}) + \boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(i)}$.
 - (v) Pass $\mathbf{z}^{(i)}$ through the decoder network to get the parameters of $p_\theta(\mathbf{x}|\mathbf{z}^{(i)})$ (e.g., $\mathbf{p}_\theta(\mathbf{z}^{(i)})$ for Bernoulli, or $\boldsymbol{\mu}_\theta(\mathbf{z}^{(i)})$ for Gaussian).
 - (vi) Calculate the reconstruction loss term $-\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})$.
 - (vii) Calculate the KL divergence term $D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}^{(i)})||p(\mathbf{z}))$ using the analytical formula (1.57) with $\boldsymbol{\mu}_\phi(\mathbf{x}^{(i)})$ and $\boldsymbol{\sigma}_\phi^2(\mathbf{x}^{(i)})$.
 - (c) Compute the average loss over the mini-batch (Negative ELBO, (1.72)).

- (d) Compute gradients of the loss with respect to ϕ and θ using back propagation (enabled by the reparameterization trick).
- (e) Update ϕ and θ using an optimizer (e.g., Adam, SGD).

Data Generation Procedure after Training

Once the VAE is trained, we can generate new data points $\mathbf{x}_{new}$ that resemble the training data:

- (1) Sample a latent vector $\mathbf{z}_{new}$ from the prior distribution: $\mathbf{z}_{new} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- (2) Pass $\mathbf{z}_{new}$ through the trained decoder network to obtain the parameters of $p_{\theta}(\mathbf{x}|\mathbf{z}_{new})$.
- (3) Sample a new data point $\mathbf{x}_{new}$ from this distribution $p_{\theta}(\mathbf{x}|\mathbf{z}_{new})$. (For example, if p_{θ} is Gaussian with mean $\boldsymbol{\mu}_{\theta}(\mathbf{z}_{new})$ and fixed variance σ^2 , then $\mathbf{x}_{new} \sim \mathcal{N}(\boldsymbol{\mu}_{\theta}(\mathbf{z}_{new}), \sigma^2 \mathbf{I})$). Often, for image generation, people just use the mean output of the decoder (e.g., $\mathbf{p}_{\theta}(\mathbf{z}_{new})$ or $\boldsymbol{\mu}_{\theta}(\mathbf{z}_{new})$) directly as the generated sample, rather than sampling from the output distribution.

1.4.4. The High-Level View of VarFA based on VAE

After going through VAE thoroughly, we can end the introduction of chapter 4 with a short section highlighting the difference between VarFA and VAE. The mathematical deduction in the previous section, along with the high-level description provided in this section should prepare us well for the content in chapter 4. Generally speaking, VarFA can be applied in all scenarios where both dimension reduction and interoperability are necessary. In our case, we choose education as the main field of application because the

necessity and advantage of interpretable models and dimension reduction are more or less self-evident.

In our work, VarFA starts with two more interpretable assumptions compared to VAE:

1. The students' overall performance follow a FA model.
2. The students' latent skills follow a multi-dimensional Gaussian distribution. (Like we mentioned previously, we assume everyone performs around their average level, with over low and over high performance being exponentially rare.)

With these two assumptions, VarFA uses a similar encoder-decoder architecture as VAE, and VarFA can also be trained using the reparameterization trick efficiently based on a modified ELBO (details in chapter 4). After the training is complete, VarFA takes students' performance record and determines their latent skills during the encoder phase. This step is much more interpretable than the usual VAE and much more similar to the natural thinking process of humans. For instance, if a student performs well in math and physics, VarFA will assign a high competence factor for their quantitative reasoning skills. This is crucial for machine learning in education, as prediction is not the end goal, but the reflection and feedback to students.

In the decoder phase, given students' latent skills, questions' difficulty and their correlations to the skills, VarFA is able to predict students' performance in a natural, interpretable way. Intuitively speaking, if a student is good at reading and writing, it is very likely for them to answer an easy question in the subject of literature. We can use VarFA to track the performance of students and predict their potential areas for improvement. On the other hand, VarFA can also be used to spot possible academic dishonesty. A well-trained VarFA model can detect students who suddenly perform well in a subject

they have been struggling with. This interpretable generative model is a versatile and useful tool to use in academic environments.

VarFA marks the small but meaningful effort to transfer the power of machine learning into human learning. As machines learn faster and better, we should not forget our shared responsibility to prepare our future generations for this ever-changing world. Because the new science and technology require a longer study plan than ever, it is imperative to make sure that we use the advancements in AI to make sure that knowledge is widely accessible, and we are indeed making this world a better place.

CHAPTER 2

Convergence of t-Distributed Stochastic Neighbor Embedding with Generalized Input and Output Kernels

2.1. The Concept of Input and Output Kernels

t-distributed stochastic neighbor embedding (t-SNE) is a powerful technique for visualizing high-dimensional data in a low-dimensional space, developed in 2008 by Laurens van der Maaten and Geoffrey Hinton ([van der Maaten and Hinton \[2008\]](#)). Despite its wide application, the theoretical aspects and mathematical rigor of t-SNE have rarely been discussed thoroughly. Recently, Auffinger and Fletcher proved that if data is sampled independently from a compact probability measure μ_X , then the output of t-SNE converges to an equilibrium measure that is also compact ([Auffinger and Fletcher \[2023\]](#)). This is one of the very first results that rigorously establishes the theoretical guarantees of t-SNE.

We aim to generalize the results from [Auffinger and Fletcher \[2023\]](#). In the original version of the t-SNE algorithm, for each pair of input or output data points, a weight is assigned based on the distance between them. This weight is used to construct probability distributions that will be used in the loss function. A natural question to ask is: can we generalize the result for different weights such that the convergence still holds and the equilibrium measure still exists? A generalized formulation allows us to further reveal the theoretical nature of this algorithm, greatly enhancing our understanding of t-SNE and

dimension reduction. Our work starts by generalizing the algorithm with the concept of input and output kernels.

Let $d, s \geq 2$ with $d > s$, and $n \geq 1$. Let $X_1, \dots, X_n \in \mathbb{R}^d$ be the input data points and $Y_1, \dots, Y_n \in \mathbb{R}^s$ the output data points. We define the input and output kernels as follows:

Definition 7 (Generalized Input Kernel). *The input kernel $\mathbf{Ker_I}(x, y, \sigma) : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^+$ is a function of the input data points and parameter σ such that for any $x, y \in \mathbb{R}^d$ and $\sigma > 0$,*

$$(2.1) \quad \mathbf{Ker_I}(x, y, \sigma) = \mathbf{Ker_I}(y, x, \sigma).$$

Definition 8 (Generalized Output Kernel). *The output kernel $\mathbf{Ker_O}(z, w) : \mathbb{R}^s \times \mathbb{R}^s \rightarrow \mathbb{R}^+$ is a function of the output data points such that for any $z, w \in \mathbb{R}^s$,*

$$(2.2) \quad \mathbf{Ker_O}(z, w) = \mathbf{Ker_O}(w, z).$$

Clearly, by letting

$$(2.3) \quad \mathbf{Ker_I}(x, y, \sigma) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right), \quad \mathbf{Ker_O}(z, w) = \frac{1}{1 + \|z - w\|^2},$$

we recover the original t-SNE algorithm by using both $\mathbf{Ker_I}$ and $\mathbf{Ker_O}$ as the weights for the input and output data points. We can also say that the original t-SNE algorithm has a Gaussian input kernel and a t-distributed output kernel.

From now on, we will use the terminology **generalized t-SNE** when we use the generalized formulation for input and output kernels to distinguish it from the original algorithm.

Clearly, not all choices of $\mathbf{Ker_I}$ and $\mathbf{Ker_O}$ will yield a valid generalized t-SNE algorithm, such as $\mathbf{Ker_I}$ that diverges as $\|x - y\| \rightarrow \infty$. To prove similar results as in [Auffinger and Fletcher \[2023\]](#), we need to impose some conditions on both $\mathbf{Ker_I}$ and $\mathbf{Ker_O}$. The conditions should provide just enough constraints for the convergence to hold, while applying to a wide range of function choices. Towards this end, we use the following conditions for the rest of our work. We will provide a detailed discussion on the generality of these conditions in section [2.4](#).

Condition 1 (Input Kernel Conditions). *Let $w : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ be a fixed function. The input kernel has the form of*

$$(2.4) \quad \mathbf{Ker_I}(x, x', \sigma) = \exp(-w(\sigma \|x - x'\|^\theta))$$

Here, $\theta > 0$ is a fixed constant. w satisfies:

1. $\forall t \geq 0, w'(t) > 0. \lim_{t \rightarrow \infty} w(t) = \infty$.
2. $w'(t) + tw''(t) \geq 0$ for all $t \geq 0$.
3. The following integral converges:

$$(2.5) \quad \int_0^\infty t^{d-1} w(t^\theta) \exp(-w(t^\theta)) dt < \infty.$$

Since the all probability masses are normalized, we can assume without loss of generality that $w(0) = 0$ from now on.

Condition 2 (Output Kernel Conditions). *The output kernel depends only on the distance between two output data points y and y' . In particular, $\mathbf{Ker}_O(y, y') = k(\|y - y'\|)$ and:*

1. $k : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^+$ is decreasing and bounded.

2. $\mathbf{Ker}_O$ satisfies

$$(2.6) \quad \iint_{(0, \infty)^2} \mathbf{Ker}_O(y, y') dy dy' < \infty.$$

3. k has bounded derivative.

4. $k'(0) = 0$.

2.2. Setup and Notations

After defining the input and output kernels, we are able to provide an introduction to the generalized t-SNE algorithm incorporating the new terminology. Recall that the input of the generalized t-SNE consists of vectors $X_1, \dots, X_n \in \mathbb{R}^d$, with the purpose of finding vectors $Y_1, \dots, Y_n \in \mathbb{R}^s$ that serve as a good representation of the original vectors. The algorithm starts by defining the following probability masses:

$$(2.7) \quad p_{j|i} = \frac{\mathbf{Ker}_I(X_i, X_j, \sigma_i)}{\sum_{k \neq i} \mathbf{Ker}_I(X_i, X_k, \sigma_i)}.$$

σ_i satisfies that for each $i = 1, \dots, n$,

$$(2.8) \quad - \sum_{j=1, j \neq i}^n p_{j|i} \log p_{j|i} = \log \mathbf{Perp} =: \log(n\rho).$$

Perp is called the perplexity and $\rho \in (0, 1)$ is a given constant. The purpose of σ_i is to make sure that for each i , the probability distribution induced by $p_{j|i}$ has the same entropy across all i . Clearly, $p_{j|i}$ is not symmetric.

Remark 2. *For finite data set perplexity is usually set as a constant. Moreover, the scaling of perplexity with respect to $\log n$ is strictly necessary here. In [Murray and Pickarski \[2024\]](#), it has been proved that if we let $\text{Perp} = \log n^\gamma \rho$ for some constant parameter γ , we cannot prove convergence and the existence of equilibrium measures for neither $\gamma < 1$ nor $\gamma > 1$.*

We then define $p_{ii} = 0$ and for $1 \leq i \neq j \leq n$,

$$(2.9) \quad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n} = \frac{1}{2n} \left(\frac{\mathbf{Ker}_I(X_i, X_j, \sigma_i)}{\sum_{k \neq i} \mathbf{Ker}_I(X_i, X_k, \sigma_i)} + \frac{\mathbf{Ker}_I(X_i, X_j, \sigma_j)}{\sum_{k \neq j} \mathbf{Ker}_I(X_j, X_k, \sigma_j)} \right).$$

Obviously,

$$(2.10) \quad \sum_{\substack{j=1 \\ j \neq i}}^n p_{j|i} = 1, \quad \sum_{1 \leq i \neq j \leq n} p_{ij} = 1.$$

As for the output vectors $Y_1, \dots, Y_n \in \mathbb{R}^s$, set for $1 \leq i \neq j \leq n$,

$$(2.11) \quad q_{ij} = \frac{\mathbf{Ker}_O(Y_i, Y_j)}{\sum_{k \neq l} \mathbf{Ker}_O(Y_k, Y_l)}.$$

The goal of the generalized t-SNE is to find Y that minimizes the relative entropy

$$(2.12) \quad L_{n,\rho}(X, Y) = \sum_{1 \leq i \neq j \leq n} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right).$$

More precisely, the output of t-SNE Y^* is defined as

$$(2.13) \quad Y^* = \arg \min_{Y \in (\mathbb{R}^s)^n} L_n(X, Y).$$

In the expression of $L_n(X, Y)$, all p_{ij} are given and q_{ij} is a function of Y_i and Y_j . Therefore, Y^* can be found simply by using gradient descent. This completes the description of the algorithm itself. Now we introduce necessary notations for convergence and asymptotic analysis. Let $\mathcal{M}(\mathbb{R}^d)$ be the space of Borel probability measures on $\mathbb{R}^d$ and $\mu_X \in \mathcal{M}(\mathbb{R}^d)$ be a probability measure. Suppose all input $X = (X_1, \dots, X_n)$ are i.i.d. random variables with probability measure μ_X . Meanwhile, let $Y^* = (Y_1, \dots, Y_n)$ be the output of the t-SNE algorithm. Consider the following empirical measure:

$$(2.14) \quad \mu_n := \frac{1}{n} \sum_{i=1}^n \delta(X_i, Y_i) \in \mathcal{M}(\mathbb{R}^d \times \mathbb{R}^s).$$

We are interested in its limiting measure as $n \rightarrow \infty$. To finish the setup, there are a few more remaining quantities we would like to introduce. Let $\mu \in \mathcal{M}(\mathbb{R}^d \times \mathbb{R}^s)$. For a given kernel $\mathbf{Ker}_{\mathbf{I}}(x, y)$, define $F_{\rho, \mu}(x, \sigma) : \mathbb{R}^d \times \mathbb{R}^+ \rightarrow \mathbb{R}$ as

$$(2.15) \quad F_{\rho, \mu}(x, \sigma) = \int \frac{\mathbf{Ker}_{\mathbf{I}}(x, x', \sigma)}{\int \mathbf{Ker}_{\mathbf{I}}(x, x', \sigma) d\mu(x')} \log \left(\frac{\mathbf{Ker}_{\mathbf{I}}(x, x', \sigma)}{\int \mathbf{Ker}_{\mathbf{I}}(x, x', \sigma) d\mu(x')} \right) d\mu(x') + \log \rho.$$

Note that the last s variables in the integrands stay constant. It is not hard to see that $F_{\rho, \mu}(x, \sigma)$ is the continuous version of the entropy. Next, we extend the definition of σ_i

by defining $\sigma_{\rho,\mu}^* : \mathbb{R}^d \rightarrow \mathbb{R}$ the unique solution of the equation

$$(2.16) \quad F_{\rho,\mu}(x, \sigma) = 0.$$

The introduction of σ^* aligns with the procedure of finding σ_i that controls the entropy to be the same for all i . Of course, the existence and uniqueness of σ^* is not trivial and will be discussed in section 2.6. For the symmetric probability mass, let $\psi : \mathbb{R}^d \rightarrow \mathbb{R}$ be a given function and $p_\psi : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ be defined by

$$(2.17) \quad p_\psi(x, x') = \frac{1}{2} \left(\frac{\mathbf{Ker}_{\mathbf{I}}(x, x', \psi(x))}{\int \mathbf{Ker}_{\mathbf{I}}(x, x', \psi(x)) d\mu(x')} + \frac{\mathbf{Ker}_{\mathbf{I}}(x, x', \psi(x'))}{\int \mathbf{Ker}_{\mathbf{I}}(x, x', \psi(x')) d\mu(x)} \right).$$

For the extension on the output side, let $q : \mathbb{R}^s \times \mathbb{R}^s \rightarrow \mathbb{R}$ be given by

$$(2.18) \quad q(y, y') = \frac{\mathbf{Ker}_{\mathbf{O}}(y, y')}{\iint \mathbf{Ker}_{\mathbf{O}}(y, y') d\mu(y) d\mu(y')}.$$

Based on the construction of p and q , define

$$(2.19) \quad I_\rho(\mu) = \iint p_{\sigma_{\rho,\mu}^*}(x, x') \log \left(\frac{p_{\sigma_{\rho,\mu}^*}(x, x')}{q(y, y')} \right) d\mu(x, y) d\mu(x', y').$$

Recall $\mu_X \in \mathcal{M}(\mathbb{R}^d)$ is an arbitrary given probability measure. We set $\mathcal{P}_X$ to be the set of probability measures on $\mathbb{R}^d \times \mathbb{R}^s$ whose marginal coincides with μ_X . Formally speaking,

$$(2.20) \quad \mathcal{P}_X := \{\mu \in \mathcal{M}(\mathbb{R}^d \times \mathbb{R}^s) : \text{the } \mathbb{R}^d \text{ marginal of } \mu \text{ is } \mu_X\}.$$

Lastly, define

$$(2.21) \quad \tilde{\mathcal{P}}_X := \left\{ \mu \in \mathcal{P}_X : \int (p_{\sigma_{\rho,\mu}^*}(x, x') - q(y, y')) \frac{\frac{\partial}{\partial y} \mathbf{Ker}_O(y, y')}{\mathbf{Ker}_O(y, y')} d\mu(x', y') = 0, \mu(x, y)\text{-a.e.} \right\}.$$

2.3. Main Results

Using the established setup, we are able to prove the same results as in [Auffinger and Fletcher \[2023\]](#) with a broad class of input and output kernels. On top of that, compared to the original paper, we no longer require the sample probability measure μ_X to have C^1 density. We managed to relax the condition to C^0 .

Theorem 5. *Assume μ_X has a continuous density $f(x)$ and compact support. Let $X_i \in \mathbb{R}^d$ be i.i.d random variables with probability measure μ_X . If condition 1 and condition 2 hold, then for any $\rho \in (0, 1)$,*

$$(2.22) \quad \lim_{n \rightarrow \infty} \inf_Y L_{n,\rho}(X, Y) = \inf_{\mu \in \tilde{\mathcal{P}}_X} I_\rho(\mu).$$

Moreover, there exist a measure μ^* and a sub-sequence $\{n_k\}$ such that $\mu_{n_k} \rightarrow \mu^*$ weakly and $I_\rho(\mu^*) = \inf_{\mu \in \tilde{\mathcal{P}}_X} I_\rho(\mu)$.

Theorem 6. *Under the same assumptions of theorem 5, μ^* has compact support.*

Remark 3. In [Auffinger and Fletcher \[2023\]](#), a lot of the lemmas are proved under the assumption that the sample distribution μ_X is sub-Gaussian. This can also be done for our work, if we replace condition 1 with the following condition 3:

Condition 3 (Alternative Input Kernel Conditions). *Let $w : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ be some fixed function. The input kernel has the form of $\mathbf{Ker_I}(x, x, \sigma) = \exp(-w(\sigma \|x - x'\|^\theta))$. Here, σ is used to control entropy (thus perplexity) and $\theta > 0$ is a fixed constant. Define*

$$(2.23) \quad \mathcal{K}_w = \left\{ \tilde{C} \in \mathbb{R}_{\geq 0} \left| \exists \epsilon > 0, \text{ s.t. } \lim_{n \rightarrow \infty} n^{-1/2} \exp \left(6w \left(\left(\tilde{C} \sqrt{\log n} \right)^\theta \right) \right) = O(n^{-\epsilon}) \right. \right\}.$$

Then w satisfies:

1. $\forall t \geq 0, w'(t) > 0. \lim_{t \rightarrow \infty} w(t) = \infty;$
2. $w'(t) + tw''(t) \geq 0$ for all $t \geq 0;$
3. *The following integral converges:*

$$(2.24) \quad \int_0^\infty t^{d-1} w(t^\theta) \exp(-w(t^\theta)) dt < \infty;$$

4. $\exists M > 0$ s.t. $\forall t \geq 0, w'(t) \leq w(t),$ and $\sup \mathcal{K}_w > 0.$

We will not include proofs for lemmas assuming condition 3 and sub-Gaussian samples, because such generalization will not carry onto the main theorems. One could adapt the proofs we present assuming μ_X being sub-Gaussian and condition 3. However, it remains unknown that whether both theorem 5 and 6 hold assuming a more general data distribution.

2.4. The Generality of Conditions

Since the main goal of our work is to generalize the theoretical guarantees of the t-SNE algorithm, it is important that we do not lose comprehensiveness when assuming a concrete form of both the input and output kernels. Therefore, both condition 1 and 2 warrant an explanation.

For the formula of the input kernel in definition 7, we start by realizing that t -SNE is about the relationship between all data points. The insights we gain from the algorithm should be translation and rotation invariant with respect to the coordinate system we choose. Hence, it is only reasonable to assume that the input kernel is a function of $\|x - x'\|$ for each pair of input data points x, x' .

On the other hand, the weight we assign to each pair of data points is always positive due to the nature of probability masses. We thus lose no generality by assuming the input kernel has an exponential form because obviously, the exponential function is a smooth bijection from $\mathbb{R}$ to $\mathbb{R}^+$. The same above can be said for the output kernel.

Additionally, though it is coincidental that in the original version of the algorithm σ_i can be viewed as the variance of a Gaussian random variable, in reality σ_i 's are only used to control the entropy of each probability distribution induced by x_i . With that being said, we only need to introduce σ in such a way that it is able to control the entropy for each x . As we can see in section 2.6, simply multiplying σ by $\|x - x'\|^\theta$ is enough to achieve this goal. Moreover, we have the convergence of σ^* in section 2.7.1, providing the anchor of the following results just as in Auffinger and Fletcher [2023].

Combining all the above, we arrive at the form of the input kernel in (2.4), by adding a general function w to the product $\sigma \|x - x'\|^\theta$. This gives us a kernel we can easily work on, without losing any generality.

Besides the form of the input kernel, we also want to make sure that the constraints we impose on the kernels in condition 1 and 2 are also general enough for the significance of our results. Since we frequently deal with integrals containing the input and output

kernels, their integrability is the bare minimum we should assume. Along with integrability comes the requirement of their decay to 0 at infinity. Apart from that, we do not add any excessive constraints to the kernels and readers will see later in proofs that those assumptions are strictly necessary for the results to hold.

Specifically, notice the second assumption of condition 1 is basically requiring the input kernel to decay at least at a minimal speed by controlling its second order derivative. If we let $w(t) = \log t$, we have $w'(t) + tw''(t) = 0$. Roughly speaking, this implies that the main theorems hold as long as the input kernel decays at polynomial rate or faster, which is a big step up than proving the convergence for Gaussian kernels alone.

2.5. Summary and Structure of Proofs

Before we move onto mathematical details, we dedicate this short section to give a summary of our work and the structure of the proofs. Since t-SNE assigns a “weight” to each pair of data points using Gaussian and t -distribution, we propose the concept of generalized kernels so that the generalized t-SNE can use a wide range of functions to define the weights, thus the subsequent probability distributions associated with the input and output data points. A natural question to ask is: under which conditions can we prove the convergence of the generalized t-SNE algorithm towards an equilibrium measure? Our work here answers the questions by providing a concrete formulation of the generalized kernels as well as rigorous proofs of the convergence towards the equilibrium measure.

The generality of the kernel formulation as well as the conditions we impose are discussed in the previous section. For the proofs, we first establish the existence and uniqueness of σ^* as the asymptotic limit of σ_i 's in section 2.6. Then we state the uniform convergence of σ^* in section 2.7.1. σ^* serves as the anchor point of all subsequent convergence results. With the existence and uniqueness of σ^* as well as its convergence established, we are able to prove the convergence towards the equilibrium measure by proving inequalities from both sides, the details of which are discussed in section 2.7.2 and 2.7.3. Finally the proofs are wrapped up in section 2.7.5 and 2.7.6, where we show that the equilibrium measure also has compact support just as the input measure μ_X .

2.6. Existence and Uniqueness of σ^*

In this section we establish the existence and uniqueness of $\sigma_{\rho,\mu}^*(x)$. Recall that $\sigma_{\rho,\mu}^*(x)$ is the unique solution of the equation

$$(2.25) \quad F_{\rho,\mu}(x, \sigma) = \int \frac{\mathbf{Ker}_{\mathbf{I}}(x, x', \sigma)}{\int \mathbf{Ker}_{\mathbf{I}}(x, x', \sigma) d\mu(x')} \log \left(\frac{\mathbf{Ker}_{\mathbf{I}}(x, x', \sigma)}{\int \mathbf{Ker}_{\mathbf{I}}(x, x', \sigma) d\mu(x')} \right) d\mu(x') + \log \rho = 0$$

Intuitively speaking, σ^* is the limit function of σ_i 's that are used to control the entropy. The purpose of this section is to prove that such limit exists and is unique for each given x . The existence and uniqueness of σ^* will become the foundation of everything we will

prove afterwards. By condition 1, we can write

$$\begin{aligned}
& F_{\rho,\mu}(x, \sigma) \\
&= \int \frac{\exp(-w(\sigma \|x - x'\|^\theta))}{\int \exp(-w(\sigma \|x - x'\|^\theta)) d\mu(x')} \log \left(\frac{\exp(-w(\sigma \|x - x'\|^\theta))}{\int \exp(-w(\sigma \|x - x'\|^\theta)) d\mu(x')} \right) d\mu(x') + \log \rho \\
&= - \int \frac{w(\sigma \|x - x'\|^\theta) \exp(-w(\sigma \|x - x'\|^\theta))}{\int \exp(-w(\sigma \|x - x'\|^\theta)) d\mu(x')} d\mu(x') \\
& \quad - \log \left(\int \exp(-w(\sigma \|x - x'\|^\theta)) d\mu(x') \right) + \log \rho.
\end{aligned}
\tag{2.26}$$

We start with the lemma that gives F an increasing lower bound.

Lemma 4. *If μ has a continuous, bounded density $f(x)$, then there exists a constant $C > 0$ such that*

$$F_{\rho,\mu}(x, \sigma) \geq C \log \sigma - \log f(x) + \log \rho.$$

$$(2.27)$$

PROOF. Notice that for any integrable function h on $\mathbb{R}_{\geq 0}$, we have

$$(2.28) \quad \int_{\mathbb{R}^d} h(\|x\|) dx = \int_{\mathbb{S}^{d-1}} \int_0^\infty r^{d-1} h(r) dr d\nu = S_{d-1} \int_0^\infty r^{d-1} h(r) dr,$$

where $\mathbb{S}^{d-1}$ is the $(d-1)$ -dimensional sphere, $d\nu$ the corresponding surface measure, and S_{d-1} the volume of the sphere. By Condition 1, we know that

$$(2.29) \quad \int_0^\infty t^{d-1} w(t^\theta) \exp(-w(t^\theta)) dt < \infty.$$

Since w is increasing and $\lim_{t \rightarrow \infty} w(t) = \infty$, it is easy to see that

$$(2.30) \quad \int_0^\infty t^{d-1} \exp(-w(t^\theta)) dt < \infty.$$

Denote

$$(2.31) \quad Z_d := \int_{\mathbb{R}^d} \exp(-w(\|\tau\|^\theta)) d\tau = S_{d-1} \int_0^\infty t^{d-1} \exp(-w(t^\theta)) dt.$$

Consider

$$(2.32) \quad \begin{aligned} & \int \exp(-w(\sigma \|x - x'\|^\theta)) d\mu(x') = \int \exp(-w(\sigma \|x - x'\|^\theta)) f(x') dx' \\ & = Z_d \sigma^{-d/\theta} \int \frac{\exp(-w(\sigma \|x - x'\|^\theta))}{Z_d \sigma^{-d/\theta}} f(x') dx'. \end{aligned}$$

Because $\lim_{x \rightarrow \infty} w(x) = \infty$, the integrand of the above integral converges weakly to $\delta_x f(x')$. To prove this statement, notice that

$$(2.33) \quad \begin{aligned} & \int \frac{\exp(-w(\sigma \|x - x'\|^\theta))}{Z_d \sigma^{-d/\theta}} f(x') dx' \\ & = \frac{1}{Z_d} \int \exp\left(-w\left(\|\sigma^{1/\theta}(x - x')\|^\theta\right)\right) f(x') d\sigma^{1/\theta}(x - x') \\ & \equiv \frac{1}{Z_d} \int \exp(-w(\|\tau\|^\theta)) f(x - \tau \sigma^{-1/\theta}) d\tau. \end{aligned}$$

Since f is bounded, $\exp(-w(\|\tau\|^\theta)) \sup f$ is integrable. By Dominant Convergence Theorem and the continuity of f , we have

$$\begin{aligned}
 & \lim_{\sigma \rightarrow \infty} \frac{1}{Z_d} \int \exp(-w(\|\tau\|^\theta)) f(x - \tau \sigma^{-1/\theta}) d\tau \\
 &= \frac{1}{Z_d} \int \lim_{\sigma \rightarrow \infty} \exp(-w(\|\tau\|^\theta)) f(x - \tau \sigma^{-1/\theta}) d\tau \\
 &= f(x) \frac{1}{Z_d} \int \exp(-w(\|\tau\|^\theta)) d\tau \\
 (2.34) \quad &= f(x).
 \end{aligned}$$

There exists some $\epsilon(\sigma) \rightarrow 0$ as $\sigma \rightarrow \infty$ and a constant $C > 0$, that

$$\begin{aligned}
 -\log \left(\int \exp(-w(\sigma \|x - x'\|^\theta)) d\mu(x') \right) &= -\log(Z_d \sigma^{-d/\theta}) - \log f(x) + \epsilon(\sigma) \\
 (2.35) \quad &\geq C \log \sigma - \log f(x).
 \end{aligned}$$

Next, since f is bounded, there exists $M > 0$ such that $f \leq M$.

$$\begin{aligned}
 & \int \frac{w(\sigma \|x - x'\|^\theta) \exp(-w(\sigma \|x - x'\|^\theta))}{\int \exp(-w(\sigma \|x - x'\|^\theta)) d\mu(x')} d\mu(x') \\
 &\leq M \frac{\int w(\sigma \|x - x'\|^\theta) \exp(-w(\sigma \|x - x'\|^\theta)) dx'}{\int \exp(-w(\sigma \|x - x'\|^\theta)) f(x') dx'} \\
 &= M \frac{\sigma^{-d/\theta} \int w(\|\sigma^{1/\theta}(x - x')\|^\theta) \exp(-w(\|\sigma^{1/\theta}(x - x')\|^\theta)) d\sigma^{1/\theta}(x - x')}{\int \exp(-w(\sigma \|x - x'\|^\theta)) f(x') dx'} \\
 &= M \frac{\sigma^{-d/\theta} \int w(\|\tau\|^\theta) \exp(-w(\|\tau\|^\theta)) d\tau}{\int \exp(-w(\sigma \|x - x'\|^\theta)) f(x') dx'} \\
 (2.36) \quad &\rightarrow \frac{M}{Z_d f(x)} \int w(\|\tau\|^\theta) \exp(-w(\|\tau\|^\theta)) d\tau < \infty.
 \end{aligned}$$

The numerator converges because of condition 1. As the bound in (2.36) does not depend on σ , we can combine (2.26), (2.35) and (2.36) and deduce that

$$(2.37) \quad F_{\rho,\mu}(x, \sigma) \geq C \log \sigma - \log f(x) + \log \rho.$$

□

This lemma proves the existence of $\sigma_{\rho,\mu}^*(x)$. Indeed, by (2.26), apparently

$$(2.38) \quad \lim_{\sigma \rightarrow 0} F_{\rho,\mu}(x, \sigma) = \log \rho < 0$$

since $\rho \in (0, 1)$. Now with lemma 4, we get

$$(2.39) \quad \lim_{\sigma \rightarrow \infty} F_{\rho,\mu}(x, \sigma) = \infty.$$

By continuity of f and intermediate value theorem, $\sigma_{\rho,\mu}^*(x)$ exists. To prove the uniqueness, we are left to show $F_{\rho,\mu}$ is strictly increasing. The next lemma will do just that.

Lemma 5. *Given the same condition as in lemma 4, $F_{\rho,\mu}(x, \sigma)$ is strictly increasing in σ .*

To tackle this proof, we need one additional lemma inspired by theorem 7 in Liu et al. [2009].

Lemma 6. *Let $f(x), g(x), h(x)$ be continuous functions $\mathbb{R}^d \rightarrow \mathbb{R}^+$ such that for all $x, y \in \mathbb{R}^d$*

$$(2.40) \quad (g(x) - g(y)) \left(\frac{f(y)}{h(y)} - \frac{f(x)}{h(x)} \right) \geq 0.$$

Let μ be an arbitrary Lebesgue measure and assume all integrals in this lemma converge.

Then the inequality

$$(2.41) \quad \frac{\int_{\mathbb{R}^d} f(x) d\mu}{\int_{\mathbb{R}^d} h(x) d\mu} \geq \frac{\int_{\mathbb{R}^d} f(x)g(x) d\mu}{\int_{\mathbb{R}^d} h(x)g(x) d\mu}$$

holds. If (2.40) reverses, then (2.41) reverses.

PROOF. The proof of this lemma is similar to the one in [Liu et al. \[2009\]](#). It suffices to show that

$$(2.42) \quad \int_{\mathbb{R}^d} f(x) d\mu \int_{\mathbb{R}^d} h(x)g(x) d\mu \geq \int_{\mathbb{R}^d} h(x) d\mu \int_{\mathbb{R}^d} f(x)g(x) d\mu.$$

The above inequality is equivalent to

$$(2.43) \quad \int_{\mathbb{R}^d} f(x) d\mu(x) \int_{\mathbb{R}^d} h(y)g(y) d\mu(y) \geq \int_{\mathbb{R}^d} h(x) d\mu(x) \int_{\mathbb{R}^d} f(y)g(y) d\mu(y).$$

That means we need to show

$$(2.44) \quad \mathcal{G} := \int_{\mathbb{R}^d} \int_{\mathbb{R}^d} g(y)(f(x)h(y) - f(y)h(x)) d\mu(x) d\mu(y) \geq 0.$$

Swap x and y in the formula of $\mathcal{G}$ and we have an equivalent expression:

$$(2.45) \quad \mathcal{G} = \int_{\mathbb{R}^d} \int_{\mathbb{R}^d} g(x)(f(y)h(x) - f(x)h(y)) d\mu(x) d\mu(y).$$

Adding the last two formulas together yields

$$\begin{aligned}
 2\mathcal{G} &= \int_{\mathbb{R}^d} \int_{\mathbb{R}^d} (g(x) - g(y))(f(y)h(x) - f(x)h(y))d\mu(x)d\mu(y) \\
 (2.46) \quad &= \int_{\mathbb{R}^d} \int_{\mathbb{R}^d} h(x)h(y) (g(x) - g(y)) \left(\frac{f(y)}{h(y)} - \frac{f(x)}{h(x)} \right) d\mu(x)d\mu(y) \geq 0
 \end{aligned}$$

by the hypothesis of the lemma. This finishes the proof. $\square$

PROOF OF LEMMA 5. We compute the derivative of $F_{\rho,\mu}(x, \sigma)$ directly using (2.26).

The following simplified notations will be used in the first several steps:

$$(2.47) \quad w = w(\sigma \|x - x'\|^\theta), \quad w' = w'(\sigma \|x - x'\|^\theta), \quad \exp(\cdot) = \exp(-w(\sigma \|x - x'\|^\theta)).$$

Omitting $d\mu(x')$ in the integration for cleaner writing, and we have

$$\begin{aligned}
 &\frac{\partial}{\partial \sigma} F_{\rho,\mu}(x, \sigma) \\
 &= - \int \frac{w' \|x - x'\|^\theta \exp(\cdot)}{\int \exp(\cdot)} - \int \frac{w \exp(\cdot)'}{\int \exp(\cdot)} + \int \frac{w \exp(\cdot) (\int \exp(\cdot))'}{(\int \exp(\cdot))^2} - \frac{(\int \exp(\cdot))'}{\int \exp(\cdot)} \\
 &= - \int \frac{w \exp(\cdot)'}{\int \exp(\cdot)} + \int \frac{w \exp(\cdot) (\int \exp(\cdot))'}{(\int \exp(\cdot))^2} \quad (\text{The first and fourth terms cancel.}) \\
 (2.48) \quad &= \int \frac{w \cdot w' \cdot \|x - x'\|^\theta \cdot \exp(\cdot)}{\int \exp(\cdot)} - \frac{\int w \cdot \exp(\cdot) \int w' \cdot \|x - x'\|^\theta \exp(\cdot)}{(\int \exp(\cdot))^2}.
 \end{aligned}$$

We are able to move differentiation inside the integral because of dominant convergence theorem. To show that $\frac{\partial}{\partial \sigma} F_{\rho,\mu}(x, \sigma) \geq 0$, it suffices to show

$$(2.49) \quad \int w \cdot w' \cdot \|x - x'\|^\theta \cdot \exp(\cdot) \int \exp(\cdot) \geq \int w \cdot \exp(\cdot) \int w' \cdot \|x - x'\|^\theta \exp(\cdot).$$

For each $x, y, x' \in \mathbb{R}$ if the equality

$$(2.50) \quad \begin{aligned} & (w'(\sigma \|x - x'\|^\theta) \|x - x'\|^\theta - w'(\sigma \|y - x'\|^\theta) \|y - x'\|^\theta) \\ & \cdot \left(\frac{1}{w(\sigma \|y - x'\|^\theta)} - \frac{1}{w(\sigma \|x - x'\|^\theta)} \right) \geq 0 \end{aligned}$$

is true, then setting $f(x) = \exp(\cdot)$, $h(x) = w \cdot \exp(\cdot)$ and $g(x) = w' \cdot \|x - x'\|^\theta$ in lemma 6 shows that (2.49) holds. Let $\|x - x'\|^\theta > \|y - x'\|^\theta$ without loss of generality. Then by monotonicity of w , we have $w(\sigma \|x - x'\|^\theta) > w(\sigma \|y - x'\|^\theta)$. For (2.50) to be true, we are left to show

$$(2.51) \quad w'(\sigma \|x - x'\|^\theta) \|x - x'\|^\theta - w'(\sigma \|y - x'\|^\theta) \|y - x'\|^\theta \geq 0.$$

Since $\sigma > 0$, the above inequality is equivalent to:

$$(2.52) \quad w'(\sigma \|x - x'\|^\theta) \sigma \|x - x'\|^\theta - w'(\sigma \|y - x'\|^\theta) \sigma \|y - x'\|^\theta \geq 0.$$

Let $\tilde{w}(x) = xw'(x)$. Based on the second assumption in Condition 1 and mean value theorem, $\tilde{w}$ satisfies for all $a, b \geq 0$, $a > b$, there exists $c \in [b, a]$ that

$$(2.53) \quad \frac{aw'(a) - bw'(b)}{a - b} = w'(c) + cw''(c) \geq 0.$$

This proves (2.52) by letting $a = \sigma \|x - x'\|^\theta$ and $b = \sigma \|y - x'\|^\theta$, as desired. $\square$

We can now formally state and prove the main result of this section.

Proposition 1. *Let $\mu \in \mathcal{M}(\mathbb{R}^d)$ with C^0 density f such that f is bounded. The equation $F_{\rho,\mu}(x, \sigma) = 0$ has a unique solution $\sigma_{\rho,\mu}^*(x)$ for all $\rho \in (0, 1)$ and $x \in \mathbb{R}^d$. Moreover, $x \mapsto \sigma_{\rho,\mu}^*(x)$ is smooth.*

PROOF. The existence and uniqueness can be proven by combining (2.38), (2.39), lemma 4 and lemma 5. The smoothness of $\sigma_{\rho,\mu}^*(x)$ follows from the implicit function theorem. $\square$

2.7. Proofs of the Main Theorems

From now on, we will assume both condition 1 and condition 2 throughout this section. We will also assume that the distribution μ_X that we draw samples from has C^0 density with bounded density and gradient of density. The assumptions on the sample distribution μ_X are inherited directly from section 2.6 to avoid lengthy theorem and lemma statements.

2.7.1. Convergence Lemmas for Input Kernels

The main difficulties of proving the convergence lemmas related to the input kernel is the existence and uniqueness of $\sigma_{\rho,\mu}^*(x)$, which we have already established in section 2.6. The rest of lemmas are similar, if not easier than the corresponding results in Auffinger and Fletcher [2023]. We will list the important lemmas here without all proofs. The reader can easily adapt the proofs in Auffinger and Fletcher [2023] to our case.

Lemma 7. *Let $\delta > 0$ be constant, $K_n = [\delta, \sqrt{n}]$ and $h : \mathbb{R}^d \times \mathbb{R}^d \times \mathbb{R} \rightarrow \mathbb{R}$. Suppose the following hold:*

1. μ is sub-Gaussian;
2. $h(x, x', \sigma)$ and $\nabla_x h(x, x', \sigma)$ are bounded for all x, x' and all $\delta \in K_n$;

3. $h(x, x', \sigma) \rightarrow 0$ as $\sigma \|x - x'\| \rightarrow \infty$.

Let $X_1, \dots, X_n$ be i.i.d. random variables drawn from μ , and $\mu_n = (1/n) \sum_i \delta_{X_i}$. Then for any $\epsilon > 0$ with probability at least $1 - e^{-c\epsilon^2 n}$ we have

$$(2.54) \quad \sup_{\sigma^{-1} \in K_{n,x}} \left| \int h(x, x', \sigma) d\mu_n(x) - \int h(x, x', \sigma) d\mu(x') \right| \leq \epsilon + O\left(\sqrt{\frac{\log n}{n}}\right).$$

The lemma assumes μ to be sub-Gaussian so it automatically covers the cases where μ has compact support with C^0 density. Using lemma 7 can give us the two following important propositions about the uniform convergence of $F_{\mu_n}(x, \sigma)$ and $\sigma_{\mu_n}^*(x)$.

Proposition 2. *Let μ_X be a probability measure with compact support, and $X_1, \dots, X_n$ be i.i.d. random variables drawn from μ . $\mu_n = (1/n) \sum_i \delta_{X_i}$ is the empirical measure. Then there exists a constant $\epsilon > 0$, that almost surely in $X_1, \dots, X_n$,*

$$(2.55) \quad \sup_{\substack{\sigma^{-1} \in [\delta^\theta, n^{\theta/2}] \\ i=1, \dots, n}} |F_{\mu_n}(X_i, \sigma) - F_\mu(X_i, \sigma)| = O(n^{-\epsilon}).$$

Proposition 3. *Suppose μ_X has compact support and C^0 density. Then almost surely, as $n \rightarrow \infty$,*

$$(2.56) \quad \sup_{i=1, \dots, n} |\sigma_{\mu_n}^*(X_i) - \sigma_\mu^*(X_i)| \rightarrow 0.$$

In particular,

$$(2.57) \quad |\sigma_{\mu_n}^*(X_i) - \sigma_\mu^*(X_i)| = O(|F_{\mu_n}(X_i, \sigma) - F_\mu(X_i, \sigma)|).$$

Next two lemmas can be borrowed directly from [Auffinger and Fletcher \[2023\]](#) as they are irrelevant to the generalized kernels.

Lemma 8. *Any sequence in $\tilde{\mathcal{P}}_X$ is tight.*

Lemma 9. *There exists μ^* such that*

$$(2.58) \quad I_\rho(\mu^*) = \inf_{\mu \in \tilde{\mathcal{P}}_X} I_\rho(\mu)$$

To further simplify up the notations, define

$$(2.59) \quad \begin{aligned} \sigma_n(x) &= \sigma_{\rho, \mu_n}^*(x), \sigma(x) = \sigma_{\rho, \mu_X}^*(x), \\ f(x, x') &= f_\sigma(x, x') =: \exp(-w(\sigma \|x - x'\|^\theta)), f_n(x, x') = f_{\sigma_n}(x, x'), \\ p'_n(x, x') &= \frac{f_n(x, x')}{\int f_n(x, x') d\mu_n(x') - 1/n}, \quad p'(x, x') = \frac{f(x, x')}{\int f(x, x') d\mu(x')}, \\ p_n(x, x') &= \frac{1}{2}(p'_n(x, x') + p'_n(x', x)), \quad p(x, x') = \frac{1}{2}(p'(x, x') + p'(x', x)). \end{aligned}$$

Note that $p_{j|i} = p'_n(X_i, X_j)/n$ and $p_{ij} = p_n(X_i, X_j)/n^2$, and for $\sigma^{-1} \in [\delta^\theta, n^{\theta/2}]$ we have

$$(2.60) \quad \int f(x, x') d\mu(x'), \quad \int f_n(x, x') d\mu_n(x') \geq C$$

for some constant $C > 0$.

Lemma 10. *Assume μ_X have compact support. As $n \rightarrow \infty$,*

$$(2.61) \quad \sup_{i,j} |f_n(X_i, X_j) - f(X_i, X_j)| = O(n^{-\epsilon})$$

for some $\epsilon > 0$.

PROOF. By definition and assume $\sigma(x) \geq \sigma_n(x)$ WLOG (or we can always compute the other way around), we have

$$\begin{aligned}
 |f_n - f| &= \left| \exp(w(\sigma(x) \|x - x\|^\theta) - w(\sigma_n(x) \|x - x\|^\theta)) - 1 \right| \exp(w(-\sigma(x) \|x - x\|^\theta)) \\
 &\leq \left| \exp(w(\sigma(x) \|x - x\|^\theta) - w(\sigma_n(x) \|x - x\|^\theta)) - 1 \right| \\
 (2.62) \quad &= \left| \exp(w'(\tilde{\sigma}(x) \|x - x\|^\theta)(\sigma(x) - \sigma_n(x) \|x - x\|^\theta)) - 1 \right|.
 \end{aligned}$$

By condition 1, w is twice differentiable. Since μ_X has compact support, $\|x - x'\|$ is bounded, so is $\exp(w'(\tilde{\sigma}(x) \|x - x'\|^\theta) \|x - x'\|^\theta)$. By the simple limit $\lim_{x \rightarrow 0} (e^x - 1)/x = 1$, the difference (2.62) can be bounded as

$$(2.63) \quad |f_n - f| = O(n^{-1/2}(\log n)^\theta).$$

The proof is thus finished. □

Lemma 11. *Let μ_X have compact support. There exists $\epsilon > 0$ such that as $n \rightarrow \infty$,*

$$\sup_{i,j} |p'_n(X_i, X_j) - p'(X_i, X_j)| = O(n^{-\epsilon}),$$

$$\sup_{i,j} |p_n(X_i, X_j) - p(X_i, X_j)| = O(n^{-\epsilon}).$$

PROOF. Once again, we omit X_i and X_j for lighter notations. Using the triangular inequality, we have

$$\begin{aligned}
 |p'_n - p'| &= \left| \frac{f_n}{\int f_n d\mu_n - 1/n} - \frac{f}{\int f d\mu} \right| \\
 &\leq \left| \frac{f_n}{\int f_n d\mu_n - 1/n} - \frac{f_n}{\int f_n d\mu_n} \right| + \left| \frac{f_n}{\int f_n d\mu_n} - \frac{f_n}{\int f d\mu_n} \right| \\
 &\quad + \left| \frac{f_n}{\int f d\mu_n} - \frac{f_n}{\int f d\mu} \right| + \left| \frac{f_n}{\int f d\mu} - \frac{f}{\int f d\mu} \right|.
 \end{aligned}
 \tag{2.64}$$

In order to get the convergence, we need to check the convergence of each term. Note that $f, f_n \leq 1$. Denote The first can be bounded using (2.60):

$$\left| \frac{f_n}{\int f_n d\mu_n - 1/n} - \frac{f_n}{\int f_n d\mu_n} \right| \leq \frac{1}{n} \left| \frac{1}{\int f_n d\mu_n - 1/n} \cdot \frac{1}{\int f_n d\mu_n} \right| = O(n^{-1}).
 \tag{2.65}$$

Continue to the second term, we compute

$$\left| \frac{f_n}{\int f_n d\mu_n} - \frac{f_n}{\int f d\mu_n} \right| \leq \frac{|f_n - f|}{|\int f_n d\mu_n| |\int f d\mu_n|} = O(|f_n - f|) = O(n^{-1/2}(\log n)^\theta)
 \tag{2.66}$$

by lemma 10 and (2.60).

For the third term we can apply lemma 7, which gives us

$$\left| \frac{f_n}{\int f d\mu_n} - \frac{f_n}{\int f d\mu} \right| \leq \frac{|\int f d\mu_n - \int f d\mu|}{|\int f d\mu| |\int f d\mu_n|} = O(n^{-1/2}(\log n)^\theta),
 \tag{2.67}$$

which is also $O(n^{-\epsilon})$ based on the hypothesis of the lemma.

Finally, we have a similar inequality as the second term for the fourth term:

$$\left| \frac{f_n}{\int f d\mu} - \frac{f}{\int f d\mu} \right| = O(|f_n - f|) = O(n^{-1/2}(\log n)^\theta).
 \tag{2.68}$$

The proof is finished by putting (2.64), (2.65), (2.66), (2.67), and (2.68) together. $\square$

Lemma 12. *Assuming μ_X has compact support, then there exists $\epsilon > 0$ such that as $n \rightarrow \infty$,*

$$\sup_{i,j} |p_n(X_i, X_j) \log p_n(X_i, X_j) - p(X_i, X_j) \log p(X_i, X_j)| = O(n^{-\epsilon}).$$

PROOF. Let $\epsilon_n = n^{-1/2}(\log n)^\theta$, where we inherit from (2.66). We compute

$$\begin{aligned} p_n \log p_n &= p \log p + p \log \left(\frac{p_n}{p} \right) + (p_n - p) \log p_n \\ (2.69) \quad &= p \log p + p \log \left(\frac{p + O(\epsilon_n)}{p} \right) + O(\epsilon_n) \log p_n. \end{aligned}$$

The third term will apparently vanish. For the second term, recall that p is bounded below because μ_X is compactly supported (which is essentially (2.60)). This means

$$(2.70) \quad O(\epsilon_n)/p = O(\epsilon_n) = O(n^{-1/2}(\log n)^\theta) = O(n^{-\epsilon}),$$

as desired. $\square$

Lemma 13. *Assuming μ_X has compact support, then as $n \rightarrow \infty$,*

$$(2.71) \quad \sum_j p_{j|i} \log p_{j|i} + \log(\rho n) = o(1).$$

PROOF. Recall that $p_{j|i} = \frac{1}{n}p'_n(X_i, X_j)$. Therefore

$$\begin{aligned}
 \sum_j p_{j|i} \log p_{j|i} &= \sum_{j \neq i} \frac{1}{n} p'_n(X_i, X_j) \log \left(\frac{1}{n} p'_n(X_i, X_j) \right) + \log n + \log \rho \\
 &= \frac{1}{n} \sum_{j \neq i} p'_n(X_i, X_j) \log p'_n(X_i, X_j) + \log \rho + \log n \left(1 - \frac{1}{n} \sum_{j \neq i} p'_n(X_i, X_j) \right) \\
 (2.72) \quad &= \frac{1}{n} \sum_{j \neq i} p'_n(X_i, X_j) \log p'_n(X_i, X_j) + \log \rho
 \end{aligned}$$

because by definition $n - \sum_{j \neq i} p'_n(X_i, X_j) = 0$. We proceed to write the first summation as an integral over the measure μ_n , with the difference being the diagonal terms.

$$\begin{aligned}
 &\frac{1}{n} \sum_{j \neq i} p'_n(X_i, X_j) \log p'_n(X_i, X_j) - \int p'_n(X_i, x') \log p'_n(X_i, x') d\mu_n(x') \\
 (2.73) \quad &= -\frac{1}{n} \frac{1}{\int f(X_i, x') d\mu_n(x') - \frac{1}{n}} \log \left(\frac{1}{\int f(X_i, x') d\mu_n(x') - \frac{1}{n}} \right).
 \end{aligned}$$

Since μ_X has compact support, then the above diagonal term is $O(n^{-1})$. This finishes the proof. $\square$

2.7.2. Lower Bound for $\inf_{Y \in (\mathbb{R}^s)^n} L_{n,\rho}(X, Y)$

We write $g(y, y') = \mathbf{Ker}_O(y, y')$ from now on for simpler notations.

Lemma 14. *There exists a set of points $y^*(X)$ such that*

$$(2.74) \quad \inf_{Y \in (\mathbb{R}^s)^n} L_{n,\rho}(X, Y) = L_{n,\rho}(X, y^*).$$

PROOF. Let $k \in \{1, \dots, n\}$. We are going to show that

$$(2.75) \quad \lim_{|y_k| \rightarrow \infty} L_{n,\rho}(X, y) = \infty.$$

By definition, L is the relative entropy and

$$(2.76) \quad L_{n,\rho}(X, y) = \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right) = \sum_{i \neq j} p_{ij} \log p_{ij} - \sum_{i \neq j} p_{ij} \log q_{ij}.$$

For n given and fixed, it suffices to show that

$$(2.77) \quad \sum_{i \neq j} p_{ij} \log(q_{ij}) = \sum_{i \neq j} p_{ij} \log \left(\frac{g(y_i, y_j)}{\sum_{m \neq l} g(y_m, y_l)} \right) \rightarrow -\infty.$$

Recall that $g(y, y')$ is bounded by condition 2. We have

$$(2.78) \quad \begin{aligned} & - \sum_{i \neq j} p_{ij} \log \left(\sum_{m \neq l} g(y_m, y_l) \right) = - \log \left(\sum_{m \neq l} g(y_m, y_l) \right) \\ & \leq - \sum_{i \neq j} p_{ij} \log \left(\sum_{m \neq k, l \neq k, m \neq l} g(y_m, y_l) \right) < \infty. \end{aligned}$$

On the other hand, we have $g(y_i, y_k) = k(\|y_i - y_k\|) \rightarrow 0$ as $|y_k| \rightarrow \infty$ because $\|y_i - y_k\| \rightarrow \infty$ and k satisfies

$$(2.79) \quad \iint_{(0,\infty)^2} k(\|x - x'\|) dx dx' < \infty.$$

This means

$$(2.80) \quad \lim_{|y_k| \rightarrow \infty} \sum_{i \neq j} p_{ij} \log(g(y_i, y_j)) \rightarrow -\infty.$$

Hence, the minimum of the $L_{n,\rho}(X, y)$ is achieved on a bounded set, the lemma follows by the continuity of $L_{n,\rho}(X, y)$. $\square$

Lemma 15. *As $n \rightarrow \infty$, we have*

$$(2.81) \quad \sum_{i \neq j} p_{ij} \log p_{ij} = \iint p(x, x') \log p(x, x') d\mu d\mu - 2 \log n + o(1).$$

PROOF. First recall that $p_{ij} = p_n(X_i, X_j)/n^2$. Therefore we have

$$\begin{aligned} \sum_{i \neq j} p_{ij} \log p_{ij} &= \sum_{i \neq j} \frac{1}{n^2} p_n(X_i, X_j) \log \left(\frac{1}{n^2} p_n(X_i, X_j) \right) \\ &= \frac{1}{n^2} \sum_{i \neq j} p_n(X_i, X_j) \log(p_n(X_i, X_j)) - 2 \log n \sum_{i \neq j} \frac{1}{n^2} p_n(X_i, X_j) \\ (2.82) \quad &= \frac{1}{n^2} \sum_{i \neq j} p_n(X_i, X_j) \log(p_n(X_i, X_j)) - 2 \log n, \end{aligned}$$

because the second summation is just the sum of all probability mass. Now we get the $2 \log n$ term and we are left to deal with the first summation. We begin by showing that the diagonal terms do not matter in our estimation. Consider

$$\begin{aligned} &\frac{1}{n^2} \sum_{i \neq j} p_n(X_i, X_j) \log(p_n(X_i, X_j)) - \iint p_n(x, x') \log p_n(x, x') d\mu_n(x) d\mu_n(x') \\ (2.83) \quad &= -\frac{1}{n^2} \sum_{t=1}^n \frac{1}{\int f(X_t, x') d\mu_n(x') - \frac{1}{n}} \log \left(\frac{1}{\int f(X_t, x') d\mu_n(x') - \frac{1}{n}} \right). \end{aligned}$$

If we refer to the proof of lemma 14 in [Auffinger and Fletcher \[2023\]](#), we can see that the difference of diagonal term vanishes and it does not depend on the specific X_i . Therefore, in our proof, when we sum over all the X_i and normalize by dividing n^2 , we reach the exact same conclusion. The diagonal term is $O(n^{-2})$ which vanishes as $n \rightarrow \infty$.

Then we apply lemma 12 and since $d\mu_n \times d\mu_n$ is a probability measure, we have

$$(2.84) \quad \iint p_n(x, x') \log p_n(x, x') d\mu_n(x) d\mu_n(x') = \iint p(x, x') \log p(x, x') d\mu_n(x) d\mu_n(x') + o(1),$$

which immediately implies

$$(2.85) \quad \sum_{i \neq j} p_{ij} \log p_{ij} = \iint p(x, x') \log p(x, x') d\mu_n(x) d\mu_n(x') - 2 \log n + o(1).$$

The proof will be finished if we are able to replace the measure in the last integral with $d\mu$. We verify that

$$(2.86) \quad h(x, x', \sigma) := p(x, x') \log p(x, x')$$

satisfies all the requirements in lemma 7. First μ has compact support with bounded density, automatically making it sub-Gaussian. Then it is easy to verify that h is C^1 for all x, x' and σ . Moreover, by calculus we have $p(x, x') \in (0, 1]$ and $\lim_{a \rightarrow 0} a \log a = 0$. Thus $p \rightarrow 0$ as $\|x - x'\| \sigma \rightarrow \infty$. It is also obvious that h is bounded and $h \rightarrow 0$ as $\|x - x'\| \sigma \rightarrow \infty$. The rest of the conditions are also very easy to prove. We can finally conclude that

$$(2.87) \quad \iint p(x, x') \log p(x, x') d\mu_n(x) d\mu_n(x') = \iint p(x, x') \log p(x, x') d\mu(x) d\mu(x') + o(1).$$

Combine this estimation with (2.85) and we can conclude

$$(2.88) \quad \sum_{i \neq j} p_{ij} \log p_{ij} = \iint p(x, x') \log p(x, x') d\mu(x) d\mu(x') - 2 \log n + o(1),$$

as desired. $\square$

Before we move on, we would like to take a small deviation and compute the partial derivative of the relative entropy. This is important because it will impact the property of the set $\tilde{\mathcal{P}}_X$.

Lemma 16. *Let $X \in (\mathbb{R}^d)^n$ and $Y = (y_1, \dots, y_n) \in (\mathbb{R}^s)^n$ be the input and output of the t -SNE algorithm, then we have*

$$(2.89) \quad \frac{\partial L_{n,\rho}(X, Y)}{\partial y_i} = - \sum_{j=1, j \neq i}^n (p_{ij} - q_{ij}) \frac{\frac{\partial}{\partial y_i} g(y_i, y_j)}{g(y_i, y_j)}.$$

PROOF. By definition,

$$(2.90) \quad \begin{aligned} L_{n,\rho}(X, Y) &= \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right) \\ &= \sum_{i \neq j} p_{ij} \log p_{ij} - \sum_{i \neq j} p_{ij} \log q_{ij}. \end{aligned}$$

The first summation does not depend on y_i and will vanish when taking the partial derivative. Moreover, we can further breakdown the second summation:

$$(2.91) \quad \begin{aligned} - \sum_{i \neq j} p_{ij} \log q_{ij} &= - \sum_{i \neq j} p_{ij} \log g(y_i, y_j) + \sum_{i \neq j} p_{ij} \log \left(\sum_{k \neq l} g(y_k, y_l) \right) \\ &= - \sum_{i \neq j} p_{ij} \log g(y_i, y_j) + \log \left(\sum_{k \neq l} g(y_k, y_l) \right). \end{aligned}$$

Taking derivative of the last expression and note that only terms that contain y_i remain.

This means

$$\begin{aligned}
 \frac{\partial L_{n,\rho}(X, Y)}{\partial y_i} &= \sum_{j \neq i, j=1}^n \left(-p_{ij} \frac{\frac{\partial}{\partial y_i} g(y_i, y_j)}{g(y_i, y_j)} + \frac{1}{\sum_{k \neq l} g(y_k, y_l)} \frac{\partial}{\partial y_i} g(y_i, y_j) \right) \\
 &= \sum_{j \neq i, j=1}^n \left(-p_{ij} \frac{\frac{\partial}{\partial y_i} g(y_i, y_j)}{g(y_i, y_j)} + \frac{g(y_i, y_j)}{\sum_{k \neq l} g(y_k, y_l)} \frac{\frac{\partial}{\partial y_i} g(y_i, y_j)}{g(y_i, y_j)} \right) \\
 (2.92) \quad &= - \sum_{j=1, j \neq i}^n (p_{ij} - q_{ij}) \frac{\frac{\partial}{\partial y_i} g(y_i, y_j)}{g(y_i, y_j)}.
 \end{aligned}$$

□

We can easily do a sanity test by letting $g(y_i, y_j) = (1 + \|y_i - y_j\|^2)^{-1}$ and get formula (A.1) in [Auffinger and Fletcher \[2023\]](#). The derivative in lemma 16 explains why $\tilde{\mathcal{P}}_X$ has that specific form.

Lemma 17. *Assume μ_X has compact support. The following holds:*

$$(2.93) \quad \liminf_{n \rightarrow \infty} \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right) \geq \iint p(x, x') \log \frac{p(x, x')}{q(y, y')} d\mu d\mu.$$

PROOF. Just as in lemma 16, we first break down the summation:

$$(2.94) \quad \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right) = \sum_{i \neq j} p_{ij} \log p_{ij} - \sum_{i \neq j} p_{ij} \log q_{ij}.$$

The convergence of the first summation has been proved in lemma 15. We just need to deal with the second summation. Note that we now have

$$(2.95) \quad \mu_n = \frac{1}{n} \sum \delta_{X_i} \times \delta_{y_i}$$

if y_i is involved. By the definition of q_{ij} , we have

$$\begin{aligned}
 & - \sum_{i \neq j} p_{ij} \log(q_{ij}) \\
 &= \frac{1}{n^2} \sum_{i \neq j} p_n(X_i, X_j) \log g(y_i, y_j) + \log \left(\frac{1}{n^2} \sum_{i \neq j} g(y_i, y_j) \right) + 2 \log n \\
 (2.96) \quad &= \iint p_n(x, x') g(y, y') d\mu_n d\mu_n + \log \left(\iint g(y, y') d\mu_n d\mu_n - \frac{1}{n} \right) + 2 \log n.
 \end{aligned}$$

By Fatou's lemma and lemma 11, we have

$$(2.97) \quad \liminf_{n \rightarrow \infty} \iint p_n(x, x') \log g(y, y') d\mu_n d\mu_n \geq \iint p(x, x') \log g(y, y') d\mu d\mu.$$

Similarly, using Fatou lemma again and we have

$$(2.98) \quad \liminf_{n \rightarrow \infty} \iint g(y, y') d\mu_n d\mu_n - \frac{1}{n} \geq \iint g(y, y') d\mu d\mu.$$

The $1/n$ is the duplicated diagonal term in the empirical measure and will obviously vanish as we take $n \rightarrow \infty$. Moreover, because of condition 2, the integrals on the RHS of the last two inequalities converge. In addition, since $\log$ is a continuous and increasing function, (2.98) implies

$$(2.99) \quad \liminf_{n \rightarrow \infty} \log \left(\iint g(y, y') d\mu_n d\mu_n - \frac{1}{n} \right) \geq \log \left(\iint g(y, y') d\mu d\mu \right).$$

The proof is finished by combining (2.96), (2.97), (2.99) and lemma 15. The $2 \log n$ cancelled out because of (2.85). $\square$

Proposition 4. *There exists a sub-sequence $n(k)$ such that*

$$(2.100) \quad \liminf_{k \rightarrow \infty} \inf_{Y \in (\mathbb{R}^s)^{n(k)}} L_{n(k), \rho}(X, Y) \geq I_\rho(\mu).$$

PROOF. Let y^* be given in the lemma and define

$$(2.101) \quad \mu_n = \frac{1}{n} \sum \delta_{X_i} \times \delta_{y_i}.$$

We can easily show that μ_n converges weakly to a measure μ . By lemma 22 in section 2.7.4, we know that the limiting measure $\mu \in \tilde{\mathcal{P}}_X$. The proof is complete by using lemma 14 and lemma 17. $\square$

2.7.3. Upper Bound for $\inf_{Y \in (\mathbb{R}^s)^n} L_{n, \rho}(X, Y)$

For convenience, define

$$(2.102) \quad q_n(y, y') = \frac{g(y, y')}{\iint g(y, y') d\mu_n(y) d\mu_n(y') - \frac{1}{n} k(0)}.$$

Here $k(0) = g(y, y)$ is the value of the output kernel when both inputs are equal. Since multiplying a constant to g does not change anything, we can just treat $k(0) = 1$ without loss of generality as everything is normalized. By lemma 9 there exists μ^* such that $I(\mu^*) = \inf_{\mu} I_\rho(\mu)$. Let $(X_1, Y_1), \dots, (X_n, Y_n)$ be i.i.d. random variables drawn from $m\mu^*$ and $\mu_n = \frac{1}{n} \sum_{i=1}^n \delta_{(X_i, Y_i)}$ their empirical measure. μ_n converges to μ^* weakly.

Lemma 18. *As $n \rightarrow \infty$, we have*

$$(2.103) \quad \sum_{i \neq j} \log \left(\frac{p_{ij}}{q_{ij}} \right) = \iint p_n(x, x') \log \left(\frac{p_n(x, x')}{q_n(y, y')} \right) d\mu_n d\mu_n + o(1).$$

PROOF. We begin with the familiar computation

$$\begin{aligned}
 \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right) &= \frac{1}{n^2} \sum_{i \neq j} p_n(X_i, X_j) \log \left(\frac{p_n(X_i, X_j)}{q_n(X_i, X_j)} \right) \\
 (2.104) \qquad &= \iint p_n(x, x') \log \left(\frac{p_n(X_i, X_j)}{q_n(X_i, X_j)} \right) d\mu_n d\mu_n + \text{Err}_n,
 \end{aligned}$$

where

$$\begin{aligned}
 &\text{Err}_n \\
 &= -\frac{1}{n^2} \sum_{i=1}^n \frac{1}{\frac{1}{n} \sum_{k \neq i} f_n(X_i, X_k)} \left(\log \left(\frac{1}{\frac{1}{n} \sum_{k \neq i} f_n(X_i, X_k)} \right) - \log \left(\frac{1}{\frac{1}{n} \sum_{k \neq l} g(Y_k, Y_l)} \right) \right) \\
 &= -\frac{1}{n^2} \sum_{i=1}^n \frac{1}{\frac{1}{n} \sum_{k \neq i} f_n(X_i, X_k)} \log \left(\frac{1}{\frac{1}{n} \sum_{k \neq i} f_n(X_i, X_k)} \right) \\
 (2.105) \qquad &+ \frac{1}{n^2} \sum_{i=1}^n \frac{1}{\frac{1}{n} \sum_{k \neq i} f_n(X_i, X_k)} \log \left(\frac{1}{\frac{1}{n} \sum_{k \neq l} g(Y_k, Y_l)} \right).
 \end{aligned}$$

By bounds in (2.60), it is easy to see that the first summation in the error term is $o(1)$

and we can write

$$(2.106) \qquad \text{Err}_n = o(1) + \frac{1}{n^2} \sum_{i=1}^n \frac{1}{\frac{1}{n} \sum_{k \neq i} f_n(X_i, X_k)} \log \left(\frac{1}{\frac{1}{n} \sum_{k \neq l} g(Y_k, Y_l)} \right).$$

For the second term, it is already clear that

$$(2.107) \qquad \frac{1}{n^2} \sum_{i=1}^n \frac{1}{\frac{1}{n} \sum_{k \neq i} f_n(X_i, X_k)} = o(1).$$

On the other hand, we have by Fatou's lemma:

$$(2.108) \quad \liminf_{n \rightarrow \infty} \frac{1}{n^2} \sum_{k \neq l} g(Y_k, Y_l) = \liminf_{n \rightarrow \infty} \iint g(y, y') d\mu_n d\mu_n - \frac{1}{n} \geq \iint g(y, y') d\mu^* d\mu^* < \infty,$$

by condition 2. We can see that the second term in Err_n is also $o(1)$, which immediately implies $\text{Err}_n = o(1)$ and completes the proof. $\square$

The next lemma is needed to close up the lower bound part of the main theorem.

Lemma 19. *As $n \rightarrow \infty$, we have*

$$(2.109) \quad \iint g(y, y') d\mu_n(y) d\mu_n(y) - \frac{1}{n} = \iint g(y, y') d\mu^* d\mu^* + o(1).$$

PROOF. Since $\mu_n \rightarrow \mu$ weakly, it is tight by Prokhorov's theorem. For arbitrary $\epsilon > 0$, there exists a compact set K such that for all n , $\mu^*(K^c), \mu_n(K^c) \leq \epsilon$. Since $g(y, y')$ is bounded, there exists a constant C such that

$$(2.110) \quad \begin{aligned} & \left| \iint g(y, y') d\mu_n(y) d\mu_n(y) - \iint g(y, y') d\mu^* d\mu^* \right| \\ &= \left| \iint g(y, y') \mathbb{1}_{K \times K} (d\mu_n d\mu_n - d\mu^* d\mu^*) \right| + C\epsilon. \end{aligned}$$

Define

$$(2.111) \quad G_n(y) = \int g(y, y') \mathbb{1}_K d\mu_n(y'), \quad G(y) = \int g(y, y') \mathbb{1}_K d\mu^*(y').$$

We now require the kernel g to have bounded derivative, and we have the following claims:

Claim 1. *$G_n(y)$ is uniformly equicontinuous on K .*

PROOF OF CLAIM. By mean value theorem and condition 2, we get

$$\begin{aligned}
 |G_n(y) - G_n(\tilde{y})| &= \left| \int k(\|y - y'\|) - k(\|\tilde{y} - y'\|) \mathbb{1}_K d\mu_n(y') \right| \\
 &\leq \int \|y - \tilde{y}\| \left| \frac{k(\|y - y'\|) - k(\|\tilde{y} - y'\|)}{\|(y - y') - (\tilde{y} - y')\|} \right| \mathbb{1}_K d\mu_n(y') \\
 (2.112) \quad &= O(\|y - \tilde{y}\|).
 \end{aligned}$$

□

Claim 2. $\sup_{y \in K} |G_n(y) - G(y)| = o(1)$.

PROOF OF CLAIM. Since K is compact, it suffices to show that for each $y \in K$, there exists $\delta > 0$ such that

$$(2.113) \quad \sup_{\tilde{y} \in B(y, \delta)} |G_n(\tilde{y}) - G(\tilde{y})| \leq \epsilon.$$

Towards this end, by triangular inequality,

$$(2.114) \quad |G_n(\tilde{y}) - G(\tilde{y})| \leq |G_n(\tilde{y}) - G_n(y)| + |G_n(y) - G(y)| + |G(y) - G(\tilde{y})|.$$

By the previous claim, the first term on the RHS vanishes because of equicontinuity. The second term vanishes because of weak convergence and the boundedness of the integrand $g(y, y')$. Finally, the last term vanishes because we can prove the equicontinuity of G using a similar argument like the claim 1. □

By claim 2 we have

$$(2.115) \quad \iint g(y, y') \mathbb{1}_{K \times K} d\mu_n d\mu_n = \iint g(y, y') \mathbb{1}_{K \times K} d\mu^* d\mu_n + o(1) = \int G(y) \mathbb{1}_K d\mu_n(y) + o(1).$$

Since G is bounded and continuous by condition 2, using weak convergence of μ_n and we have

$$(2.116) \quad \int G(y) \mathbb{1}_K d\mu_n(y) = \int G(y) \mathbb{1}_K d\mu^{*n}(y) + o(1) = \iint g(y, y') d\mu^* d\mu^* + o(1),$$

as desired. $\square$

Lemma 20. *If μ^* has compact support in Y , then*

$$(2.117) \quad \iint p_n(x, x') \log(g(y, y')) d\mu_n d\mu_n = \iint p_n(x, x') \log(g(y, y')) d\mu^* d\mu^* + o(1).$$

PROOF. By compactness, $\|y - y'\|$ is bounded from above and $g(y, y')$ does not decay to zero. In that case, by lemma 11, we can replace the p_n in the integral by p :

$$(2.118) \quad \iint p_n(x, x') \log(g(y, y')) d\mu_n d\mu_n = \iint p(x, x') \log(g(y, y')) d\mu_n d\mu_n + o(1).$$

mimicking the proof of lemma 15 we can show that

$$(2.119) \quad \iint p(x, x') \log(g(y, y')) d\mu_n d\mu_n = \iint p(x, x') \log(g(y, y')) d\mu^* d\mu^* + o(1).$$

This finishes the proof. $\square$

Lemma 21. *If μ^* has compact support in Y , then*

$$(2.120) \quad \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right) = \iint p(x, x') \log \left(\frac{p(x, x')}{q(y, y')} \right) d\mu^* d\mu^* + o(1)$$

PROOF. This lemma can be proved by combining the last 3 lemmas and lemma 15. $\square$

Using the previous lemma and we can immediately get

Proposition 5. *If μ^* has compact support in Y , then*

$$(2.121) \quad \limsup_{n \rightarrow \infty} \inf_{Y \in (\mathbb{R}^s)^n} L_{n,\rho}(X, Y) \leq I_\rho(\mu^*).$$

2.7.4. Auxiliary Lemmas

This section contains lemmas that are necessary but are not a part of the flow in the proofs of convergence. Recall that $\mu_n = \frac{1}{n} \sum_{i=1}^n \delta_{X_i, Y_i}$ is the empirical measure of both the input and output data points. We just ignore the first or the second part based on the integration variable because it is a probability measure. Let $Y = Y^* = (Y_1, \dots, Y_n)$ be a minimizer of L . Recall from (2.92) that

$$(2.122) \quad \frac{\partial L_{n,\rho}(X, Y)}{\partial y_i} = - \sum_{j=1, j \neq i}^n (p_{ij} - q_{ij}) \frac{\frac{\partial}{\partial y_i} g(y_i, y_j)}{g(y_i, y_j)} = 0.$$

Lemma 22. *Suppose that μ_X has either compact support and a C^0 density $f(x)$. Then for μ -a.e. x and y we have*

$$(2.123) \quad \int (p(x, x') - q(y, y')) \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu(x', y') = 0.$$

PROOF. Let $\alpha(x, y)$ be a test function. It suffices to show that

$$(2.124) \quad \iint \alpha(x, y)(p(x, x') - q(y, y')) \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu(x', y') = 0.$$

Recalling the definition of p_n and q_n , we see from (2.92) that for all i ,

$$(2.125) \quad \frac{1}{n^2} \sum_{j=1, j \neq i}^n (p_n(X_i, X_j) - q_n(X_i, X_j)) \frac{\frac{\partial}{\partial Y_i} g(Y_i, Y_j)}{g(Y_i, Y_j)} = 0.$$

Multiply by $\alpha(X_i, Y_i)$ for each i and the both sides are still zero. Afterwards, take the summation over i and we get

$$(2.126) \quad \begin{aligned} & \iint \alpha(x, y)(p_n(x, x') - q_n(y, y')) \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu_n d\mu_n \\ &= \frac{1}{n^2} \sum_{1 \leq i, j \leq n} (p_n(X_i, X_j) - q_n(X_i, X_j)) \frac{\frac{\partial}{\partial Y_i} g(Y_i, Y_j)}{g(Y_i, Y_j)} = 0 \end{aligned}$$

if we require $k'(0) = 0$. By lemma 11 and lemma 19, we have

$$(2.127) \quad p_n(X_i, X_j) - q_n(Y_i, Y_j) = p(X_i, X_j) - q(Y_i, Y_j) + o(1).$$

Since both α and $\frac{\frac{\partial}{\partial y_i} g(y_i, y_j)}{g(y_i, y_j)}$ are bounded based on our assumptions, we have

$$(2.128) \quad \begin{aligned} & \iint \alpha(x, y)(p_n(x, x') - q_n(y, y')) \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu_n d\mu_n \\ &= \iint \alpha(x, y)(p(x, x') - q(y, y')) \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu_n d\mu_n + o(1). \end{aligned}$$

By lemma 23 and lemma 24 below we have

$$\begin{aligned}
 & \iint \alpha(x, y)(p(x, x') - q(y, y')) \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu_n d\mu_n \\
 (2.129) \quad &= \iint \alpha(x, y)(p(x, x') - q(y, y')) \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu d\mu + o(1).
 \end{aligned}$$

Since α is an arbitrary test function,

$$(2.130) \quad \int (p(x, x') - q(y, y')) \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu(x', y') = 0.$$

□

Lemma 23. *The following holds as $n \rightarrow \infty$:*

$$(2.131) \quad \iint \alpha(x, y)p(x, x') \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu_n d\mu_n = \iint \alpha(x, y)p(x, x') \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu d\mu + o(1)$$

This lemma can be proved by mimicking lemma 15

Lemma 24. *The following holds as $n \rightarrow \infty$:*

$$(2.132) \quad \iint \alpha(x, y)q(y, y') \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu_n d\mu_n = \iint \alpha(x, y)q(y, y') \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu d\mu + o(1)$$

This lemma can be proved by mimicking lemma 18.

2.7.5. Proof of Theorem 5

Write

$$(2.133) \quad d_{n,\rho}(X) = \inf_{Y \in (\mathbb{R}^s)^n} L_{n,\rho}(X, Y).$$

By proposition 5, 4 and the fact that $\mu^* = \arg \min_{\mu \in \tilde{\mathcal{P}}_X} I_\rho(\mu)$, there exists $\mu \in \tilde{\mathcal{P}}_X$ and a subsequence $n(k)$ such that

$$(2.134) \quad I_\rho(\mu^*) \geq \limsup_{n \rightarrow \infty} d_{n,\rho}(X) \geq \limsup_{n \rightarrow \infty} d_{n(k),\rho}(X) \geq I_\rho(\mu) \geq I_\rho(\mu^*).$$

Therefore the limit

$$(2.135) \quad \lim_{n \rightarrow \infty} d_{n(k),\rho}(X) = I_\rho(\mu) = I_\rho(\mu^*)$$

holds when we look at the sub-sequence $n(k)$ with the empirical measure μ converging to μ weakly. For the general limit, we can repeat the reasoning above by looking at any convergent sub-sequences of $d_{n,\rho}(X)$. We are able to find a sub-sequence inside each convergent sub-sequence and they all converges to $I_\rho(\mu^*)$. Therefore each convergent sub-sequence of $d_{n,\rho}(X)$ has the same limit and the theorem is proved. $\square$

2.7.6. Proof of Theorem 6

We need another lemma before proving this theorem.

Lemma 25. *For every $\mu \in \tilde{\mathcal{P}}_X$, there exists a function $h_\mu(x)$ that does not depend on y such that*

$$(2.136) \quad \int p(x, x') \log(1 + \|y - y'\|) d\mu = h_{\mu(x)} - \frac{\int g(y, y') d\mu}{\iint g(y, y') d\mu d\mu}.$$

Moreover, if μ_X has compact support, then h_μ is continuous on the support of μ_X .

PROOF. Recall from (16), we have

$$\begin{aligned}
 0 &= \int (p(x, x') - q(y, y')) \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu(x', y') \\
 &= \int p(x, x') \frac{\frac{\partial}{\partial y} g(y, y')}{g(y, y')} d\mu - \frac{\int \frac{\partial}{\partial y} g(y, y') d\mu}{\iint g(y, y') d\mu d\mu} \\
 (2.137) \quad &= \nabla_y \left(\int p(x, x') \log(g(y, y')^{-1}) d\mu - \frac{\int g(y, y') d\mu}{\iint g(y, y') d\mu d\mu} \right).
 \end{aligned}$$

This means that there exists $h_\mu : \mathbb{R}^d \rightarrow \mathbb{R}$ such that

$$(2.138) \quad h_{\mu(x)} = \int p(x, x') \log(g(y, y')^{-1}) d\mu + \frac{\int g(y, y') d\mu}{\iint g(y, y') d\mu d\mu}.$$

Since $h_{\mu(x)}$ does not change with y , it suffices to show that $\int p(x, x') \log(g(y, y')^{-1}) d\mu$ is continuous in x . Let $\epsilon > 0$ and suppose $x_n \rightarrow x$. Since p is continuous, and μ_X has compact support, we have for some constant C ,

$$(2.139) \quad \iint \log(g(y, y')^{-1}) d\mu d\mu \leq C \int p(x, x') \log(g(y, y')^{-1}) d\mu < \infty.$$

In particular, for almost every y , $\int \log(g(y, y')^{-1}) d\mu < \infty$. Since $p(x, x')$ is continuous and μ_X has compact support, $p(x, x')$ is uniformly continuous. Specifically, there exists $\delta = \delta(y, \mu)$ such that if $\|x_n - x\| \leq \delta$,

$$(2.140) \quad \sup_x |p_n(x, x') - p(x, x')| \leq \frac{\epsilon}{\log(g(y, y')^{-1}) d\mu}.$$

This means

$$(2.141) \quad \left| \int (p_n(x, x') - p(x, x')) \log(g(y, y')^{-1}) d\mu \right| < \epsilon,$$

which is the definition of continuity. The proof is finished. $\square$

PROOF OF THEOREM 6. We prove by contradiction. Define $D_n = \{\|y\| \geq n\}$. If μ^* does not have compact support in Y then $\mu(D_n) > 0$ for all n . By lemma 25, there exists continuous $h(x)$ such that

$$(2.142) \quad \int p(x, x') \log(g(y, y')^{-1}) d\mu^* = h(x) - \frac{\int g(y, y') d\mu^*}{\iint g(y, y') d\mu^* d\mu^*}.$$

Multiply both sides by $\mathbb{1}_{D_n(x, y)}/\mu^*(D_n)$ and integrate with respect to the other variable:

$$(2.143) \quad \begin{aligned} & \iint p(x, x') \log(g(y, y')^{-1}) \frac{\mathbb{1}_{D_n(x, y)}}{\mu^*(D_n)} d\mu^* d\mu^* \\ &= \int h(x) \frac{\mathbb{1}_{D_n(x, y)}}{\mu^*(D_n)} d\mu^* - \frac{\iint g(y, y') \frac{\mathbb{1}_{D_n(x, y)}}{\mu^*(D_n)} d\mu^* d\mu^*}{\iint g(y, y') d\mu^* d\mu^*}. \end{aligned}$$

Since h is continuous and μ_X has compact support, the first term of the RHS is bounded uniformly in n . Moreover, $g(y, y')$ is bounded so the second term is also bounded uniformly in n . It follows that the LHS is bounded uniformly in n . With this being said, because $p(x, x') > 0$ and μ_X has compact support, there exists a constant c such that $p(x, x') \geq c$ for all x, x' in the support of μ_X .

Meanwhile, since the output kernel $g(y, y') = k(\|y - y'\|)$ is integrable, it has a decaying tail and there exists $C(n)$ such that $C(n) \rightarrow \infty$ and if $\|x - x'\| \geq n/2$, $g(y, y')^{-1} =$

$k(\|y - y'\|)^{-1} \geq \log(1 + C(n))$. We compute

$$\begin{aligned}
 & \frac{1}{\mu^*(D_n)} \iint p(x, x') \log(g(y, y')^{-1}) \mathbb{1}_{\|y\| \geq n} d\mu^* d\mu^* \\
 & \geq \frac{1}{\mu^*(D_n)} \iint p(x, x') \log(g(y, y')^{-1}) \mathbb{1}_{\|y\| \geq n} \mathbb{1}_{\|y - y'\| \geq n/2} d\mu^* d\mu^* \\
 & \geq c \frac{\log(1 + C(n))}{\mu^*(D_n)} \iint \mathbb{1}_{\|y\| \geq n} \mathbb{1}_{\|y'\| \leq n/2} d\mu^* d\mu^* \\
 (2.144) \quad & = c \log(1 + C(n)) \mu^*(\|Y'\| \leq n/2).
 \end{aligned}$$

Clearly,

$$(2.145) \quad \lim_{n \rightarrow \infty} c \log(1 + C(n)) \mu^*(\|Y'\| \leq n/2) = \infty.$$

This contradicts with the boundedness. We must have that $\mu^*(D_n) = 0$ for n sufficiently large. That is, $\mu^*(\|Y\| \geq n) = 0$, so μ^* has compact support in Y . $\square$

CHAPTER 3

Learning Markov Random Fields for Combinatorial Structures via Sampling through Lovász Local Lemma

Overview

Learning to generate complex combinatorial structures satisfying constraints will have transformative impacts in many applications. However, it is beyond the capabilities of existing approaches due to the highly intractable nature of the embedded probabilistic inference. Prior works spend most of the training time learning to separate valid from invalid structures but not valid structures inside and outside of the training set. We develop NEural Lovász Sampler (NELSON), which embeds the sampler through Lovász Local Lemma (LLL) as a fully differentiable neural network layer. The NELSON sampler allows us to obtain valid samples from the current model distribution. Contrastive divergence is then applied to separate these samples from those in the training set. NELSON is implemented as a fully differentiable neural net, taking advantage of the parallelism of GPUs. Experimental results on several real-world domains reveal that NELSON learns to generate 100% valid structures, while baselines either time out or cannot ensure validity. NELSON also outperforms other approaches in running time, log-likelihood, and MAP scores.

3.1. Introduction

In recent years, tremendous progress has been made in generative modeling [Hinton, 2002; Tsochantaridis et al., 2005; Goodfellow et al., 2014; Kingma and Welling, 2014; Germain et al., 2015; Larochelle and Murray, 2011; van den Oord et al., 2016; Arjovsky et al., 2017; Song and Ermon, 2019; Song et al., 2021; Murphy et al., 1999; Yedidia et al., 2000; Wainwright and Jordan, 2006]. In the meantime, tremendous progress has been made in automated reasoning [Braunstein et al., 2005; Andersen et al., 2007; Chavira et al., 2006; Van Hentenryck, 1989; Gogate and Dechter, 2012; Sang et al., 2005]. Nevertheless, reasoning and learning have been growing independently for a long time. Only recently do ideas emerge exploring the role of reasoning in learning [Kusner et al., 2017; Jin et al., 2018; Dai et al., 2018; Hu et al., 2017; Lowd and Domingos, 2008; Ding et al., 2021].

Learning a generative model involves increasing the divergence in likelihood scores between the structures in the training set and those structures sampled from the current generative model distribution. While current approaches have achieved successes in *un-structured* domains such as vision or speech, their performance is degraded in the structured domain, because it is already computationally intractable to search for a valid structure in a combinatorial space subject to constraints, not to mention sampling, which has a higher complexity. In fact, when applied in a constrained domain, existing approaches spend most of their training time manipulating the likelihood of invalid structures, but not learning the difference between valid structures inside and outside of the training set.

The Lovász Local Lemma (LLL) [Erdős and Lovász, 1973] is a classic gem in combinatorics, which states that there exists a positive probability that none of *bad* events occur,

as long as these events are *mostly* independent from one another and are not too likely individually. Recently, [Moser and Tardos, 2010] came up with an algorithm (noted as algorithmic-LLL), in which find such a instance by executing the algorithm. [Guo et al., 2019] further proved that the algorithmic-LLL is an unbiased sampler if those bad events satisfy the so-called “extreme” condition. The expected running time of the sampler is also shown to be polynomial. As one contribution of this piece of research, we offer proofs of the two aforementioned results using precise mathematical notations, clarifying a few descriptions not precisely defined in the original proof. While this line of research clearly demonstrates the potential of LLL in generative learning (generating samples that satisfy all hard constraints), it is not clear how to embed LLL-based samplers into learning and no empirical studies have been performed to evaluate LLL-based samplers in machine learning.

In this chapter, we develop NEural Lovász Sampler (NELSON), which implements the LLL-based sampler as a fully differentiable neural network module. Our NELSON-CD embeds NELSON into the contrastive divergence learning process of Markov Random Fields (MRFs). Embedding an LLL-based sampler allows the contrastive learning algorithm to focus on learning the difference between the training data and the valid structures drawn from the current model distribution. Baseline approaches, on the other hand, spend most of their training time learning to generate valid structures. In addition, NELSON is fully differentiable, hence allowing for efficient learning harnessing the parallelism of GPUs.

We conduct experiments NELSON-CD on learning preferences towards (i) random K -satisfiability solutions (ii) sink-free orientations of un-directed graphs and (iii) vehicle delivery routes. In all these applications, NELSON-CD (i) has the fastest training time due

to seamless integration into the learning framework (shown in Tables 3.1(a), 3.3(a)). (ii) NELSON generates samples 100% satisfying constraints (shown in Tables 3.1(b), 3.3(b)), which facilitates effective contrastive divergence learning. Other baselines either cannot satisfy constraints or time out. (iii) The fast and valid sample generation allows NELSON to obtain the best learning performance (shown in Table 3.1(c), 3.2(a,b), 3.3(c,d)).

Our contributions can be summarized as follows: **(a)** We present NELSON-CD, a contrastive divergence learning algorithm for constrained MRFs driven by sampling through the Lovász Local Lemma (LLL). **(b)** Our LLL-based sampler (NELSON) is implemented as a fully differentiable multi-layer neural net, allowing for end-to-end training on GPUs. **(c)** We offer a mathematically sound proof of the sample distribution and the expected running time of the NELSON algorithm. **(d)** Experimental results reveal the effectiveness of NELSON in (i) learning models with high likelihoods (ii) generating samples 100% satisfying constraints and (iii) having high efficiency in training¹.

3.1.1. Related Work

Related to our NELSON are neural-based approaches to solve combinatorial optimization problems [Selsam et al., 2019; Duan et al., 2022; Li et al., 2018]. Machine learning is also used to discover better heuristics [Yolcu and Póczos, 2019; Chen and Tian, 2019]. Reinforcement learning [Karalias and Loukas, 2020; Bengio et al., 2021] as well as approaches integrating search with neural nets [Mandi et al., 2020] are found to be effective in solving combinatorial optimization problems as well. Regarding probabilistic inference, there are a rich line of research on MCMC-type sampling [Neal, 1993; Dagum and Chavez, 1993; Ge

¹Code is at: <https://github.com/jiangnanhugo/nelson-cd>

et al., 2018] and various versions of belief propagation [Murphy et al., 1999; Ihler et al., 2005; Coja-Oghlan et al., 2020; Ding and Xue, 2020]. SampleSearch [Gogate and Dechter, 2011] integrates importance sampling with constraint-driven search. Probabilistic inference based on hashing and randomization obtains probabilistic guarantees for marginal queries and sampling via querying optimization oracles subject to randomized constraints [Gomes et al., 2006; Ermon et al., 2013; Achlioptas and Theodoropoulos, 2017; Chakraborty et al., 2013].

3.2. Preliminaries

3.2.1. Markov Random Fields (MRF)

represent a Boltzmann distribution of the discrete variables $X = \{X_i\}_{i=1}^n$ over a Boolean hypercube $\mathcal{X} = \{0, 1\}^n$. For $x \in \mathcal{X}$, we have:

$$(3.1) \quad P_\theta(X = x) = \frac{\exp(\phi_\theta(x))}{Z(\theta)}.$$

Here, $Z(\theta) = \sum_{x' \in \mathcal{X}} \exp(\phi_\theta(x'))$ is the partition function that normalizes the total probability to 1. The potential function is $\phi_\theta(x) = \sum_{j=1}^m \phi_{\theta,j}(x_j)$. Each $\phi_{\theta,j}$ is a *factor potential*, which maps a value assignment over a subset of variables $X_j \subseteq X$ to a real number. We use upper case letters, such as X_j to represent (a set of) random variables, and use lower case letters, such as x_j , to represent its value assignment. We also use $\text{var}(\phi_{\theta,j})$ to represent the domain of $\phi_{\theta,j}$, i.e., $\text{var}(\phi_{\theta,j}) = X_j$. θ are the parameters to learn.

3.2.2. Constrained MRF

is the MRF model subject to a set of hard constraints $\mathcal{C} = \{c_j\}_{j=1}^L$. Here, each constraint c_j limits the value assignments of a subset of variables $\text{var}(c_j) \subseteq X$. We write $c_j(x) = 1$ if the assignment x satisfies the constraint c_j and 0 otherwise. Note that x is an assignment to all random variables, but c_j only depends on variables $\text{var}(c_j)$. We denote $C(x) = \prod_{j=1}^L c_j(x)$ as the indicator function. Clearly, $C(x) = 1$ if all constraints are satisfied and 0 otherwise. The constrained MRF is:

$$(3.2) \quad P_\theta(X = x|\mathcal{C}) = \frac{\exp(\phi_\theta(x)) C(x)}{Z_{\mathcal{C}}(\theta)},$$

where $Z_{\mathcal{C}}(\theta) = \sum_{x' \in \mathcal{X}} \exp(\phi_\theta(x')) C(x')$ sums over only valid variable assignments.

3.2.3. Learn Constrained MRF

Given a data set $\mathcal{D} = \{x^k\}_{k=1}^N$, where each x^k is a valid assignment that satisfies all constraints, learning can be achieved via maximal likelihood estimation. The optimal parameters θ^* is obtained by minimizing the negative log-likelihood $\ell_{\mathcal{C}}(\theta)$:

$$(3.3) \quad \begin{aligned} \ell_{\mathcal{C}}(\theta) &= -\frac{1}{N} \sum_{k=1}^N \log P_\theta(X = x^k|\mathcal{C}) \\ &= -\frac{1}{N} \sum_{k=1}^N \phi_\theta(x^k) + \log Z_{\mathcal{C}}(\theta). \end{aligned}$$

The parameters θ can be trained using gradient-based optimization: $\theta^{t+1} = \theta^t - \eta \nabla \ell_{\mathcal{C}}(\theta)$, where η is the learning rate. Let $\nabla \ell_{\mathcal{C}}(\theta)$ denotes the gradient of the objective $\ell_{\mathcal{C}}(\theta)$, that

is calculated as:

$$\begin{aligned}
 \nabla \ell_{\mathcal{C}}(\theta) &= -\frac{1}{N} \sum_{k=1}^N \nabla \phi_{\theta}(x^k) + \nabla \log Z_{\mathcal{C}}(\theta) \\
 (3.4) \qquad &= -\mathbb{E}_{x \sim \mathcal{D}} (\nabla \phi_{\theta}(x)) + \mathbb{E}_{\tilde{x} \sim P_{\theta}(x|\mathcal{C})} (\nabla \phi_{\theta}(\tilde{x})).
 \end{aligned}$$

The first term is the expectation over all data in training set $\mathcal{D}$. During training, this is approximated using a mini-batch of data randomly drawn from the training set $\mathcal{D}$. The second term is the expectation over the current model distribution $P_{\theta}(X = x|\mathcal{C})$ (detailed in section 3.9.2). Because learning is achieved following the directions given by the divergence of two expectations, this type of learning is commonly known as contrastive divergence (CD) [Hinton, 2002]. Estimating the second expectation is the bottleneck of training because it is computationally intractable to sample from this distribution subject to combinatorial constraints. Our approach, NELSON, leverages the sampling through Lovász Local Lemma to approximate the second term.

3.2.4. Factor Potential in Single Variable Form

Our method requires each factor potential $\phi_{\theta,j}(x_j)$ in Eq. (3.1) to involve only one variable. This is NOT an issue as *all constrained MRF models can be re-written in single variable form* by introducing additional variables and constraints. Our transformation follows the idea in [Sang et al., 2005]. We illustrate the idea by transforming one-factor potential $\phi_{\theta,j}(x_j)$ into the single variable form. First, notice all functions including $\phi_{\theta,j}(x_j)$ over a

Boolean hypercube $\{0, 1\}^n$ have a (unique) discrete Fourier expansion:

$$(3.5) \quad \phi_{\theta,j}(x_j) = \sum_{S \in [\mathbf{var}(\phi_{\theta,j})]} \hat{\phi}_{\theta,j,S} \chi_S(x).$$

Here, $\chi_S(x) = \prod_{X_i \in S} X_i$ is the basis function and $\hat{\phi}_{\theta,j,S}$ are Fourier coefficients. $[\mathbf{var}(\phi_{\theta,j})]$ denotes the power set of $\mathbf{var}(\phi_{\theta,j})$. For example, if $\mathbf{var}(\phi_{\theta,j}) = \{X_1, X_2\}$, then we have $[\mathbf{var}(\phi_{\theta,j})] = \{\emptyset, \{X_1\}, \{X_2\}, \{X_1, X_2\}\}$. See [Mansour, 1994] for details of Fourier transformation. To transform $\phi_{\theta,j}(x_j)$ into single variable form, we introduce a new Boolean variable $\hat{\chi}_S$ for every $\chi_S(x)$. Because $\hat{\chi}_S$ and all X_i 's are Boolean, we can use combinatorial constraints to guarantee $\hat{\chi}_S = \prod_{X_i \in S} X_i$. These constraints are incorporated into $\mathcal{C}$. Afterward, $\phi_{\theta,j}(x_j)$ is represented as the sum of several single-variable factors. Notice this transformation is only possible when the MRF is subject to constraints. We offer a detailed example in section 3.9.1 for further explanation. Equipped with this transformation, we assume all $\phi_{\theta,j}(x_j)$ are single variable factors for the rest of the chapter.

3.2.5. Extreme Condition

The set of constraints $\mathcal{C}$ is called “extremal” if no variable assignment violates two constraints sharing variables, according to [Guo et al., 2019].

Condition 4. *A set of constraints $\mathcal{C}$ is called extremal if and only if for each pair of constraints $c_i, c_j \in \mathcal{C}$, (i) either their domain variables do not intersect, i.e., $\mathbf{var}(c_i) \cap \mathbf{var}(c_j) = \emptyset$. (ii) or for all $x \in \mathcal{X}$, $c_i(x) = 1$ or $c_j(x) = 1$.*

3.3. Sampling Through Lovász Local Lemma

Lovász Local Lemma (LLL) [Erdős and Lovász, 1973] is a fundamental method in combinatorics to show the existence of a valid instance that avoids all the bad events, if the occurrences of these events are “mostly” independent and are not very likely to happen individually. Since the occurrence of a bad event is equivalent to the violation of a constraint, we can use the LLL-based sampler to sample from the space of constrained MRFs. To illustrate the idea of LLL-based sampling, we assume the constrained MRF model is given in the single variable form (as discussed in the previous section):

$$(3.6) \quad P_\theta(X = x|\mathcal{C}) = \frac{\exp(\sum_{i=1}^n \theta_i x_i) C(x)}{Z_{\mathcal{C}}(\theta)},$$

where $Z_{\mathcal{C}}(\theta) = \sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) C(x')$.

As shown in Algorithm 1, the LLL-based sampler [Guo et al., 2019] takes the random variables $X = \{X_i\}_{i=1}^n$, the parameters of constrained MRF θ , and constraints $\mathcal{C} = \{c_j\}_{j=1}^L$ that satisfy Condition 4 as the inputs. In Line 1 of Algorithm 1, the sampler gives an initial random assignment of each variable following its marginal probability: $x_i \sim \frac{\exp(\theta_i x_i)}{\sum_{x_i \in \{0,1\}} \exp(\theta_i x_i)}$, for $1 \leq i \leq n$. Here we mean that x_i is chosen with probability mass $\frac{\exp(\theta_i x_i)}{\sum_{x_i \in \{0,1\}} \exp(\theta_i x_i)}$. Line 2 of Algorithm 1 checks if the current assignment satisfies all constraints in $\mathcal{C}$. If so, the algorithm terminates. Otherwise, the algorithm finds the set of violated constraints $S = \{c_j | c_j(x) = 0, c_j \in \mathcal{C}\}$ and re-samples related variables $X_k \in \text{var}(S)$ using the same marginal probability, *i.e.*, $x_k \sim \frac{\exp(\theta_k x_k)}{\sum_{x_k \in \{0,1\}} \exp(\theta_k x_k)}$. Here $\text{var}(S) = \cup_{c_j \in S} \text{var}(c_j)$. The algorithm repeatedly samples all those random variables violating constraints until all the constraints are satisfied.

Algorithm 1 Sampling Through Lovász Local Lemma.

Input: Random variables $X = \{X_i\}_{i=1}^n$; Constraints $\mathcal{C} = \{c_j\}_{j=1}^L$; Parameters of the constrained MRF θ .

- 1: $x_i \sim \frac{\exp(\theta_i x_i)}{\sum_{x_i \in \{0,1\}} \exp(\theta_i x_i)}$, for $1 \leq i \leq n$. $\triangleright$ initialize
 - 2: **while** $C(x) = 0$ **do**
 - 3: Find all violated constraints $S \subseteq \mathcal{C}$ in x .
 - 4: $x_k \sim \frac{\exp(\theta_k x_k)}{\sum_{x_k \in \{0,1\}} \exp(\theta_k x_k)}$, for $x_k \in \text{var}(S)$. $\triangleright$ resample
 - 5: **end while return** A valid sample x drawn from $P_\theta(X = x|\mathcal{C})$.
-

Under Condition 4, Algorithm 1 guarantees each sample is from the constrained MRFs' distribution $P_\theta(X = x|\mathcal{C})$ (shown in Theorem 7). In section 3.7, we present the detailed proof and clarify the difference to the original descriptive proof [Guo et al., 2019].

Theorem 7 (Probability Distribution). *Given random variables $X = \{X_i\}_{i=1}^n$, constraints $\mathcal{C} = \{c_j\}_{j=1}^L$ that satisfy Condition 4, and the parameters of the constrained MRF in the single variable form θ . Upon termination, Algorithm 1 outputs an assignment x that is randomly drawn from the constrained MRF distribution: $x \sim P_\theta(X = x|\mathcal{C})$.*

SKETCH OF PROOF. We first show that in the last round, the probability of obtaining two possible assignments conditioning on all previous rounds in Algorithm 1 has the same ratio as the probability of those two assignments under distribution $P_\theta(X = x|\mathcal{C})$. Then we show when Algorithm 1 ends, the set of all possible outputs is equal to the domain of non-zero probabilities of $P_\theta(X = x|\mathcal{C})$. Thus we conclude the execution of Algorithm 1 produces a sample from $P_\theta(X = x|\mathcal{C})$ because of the identical domain and the match of probability ratios of any two valid assignments. $\square$

The expected running time of Algorithm 1 is determined by the number of rounds of re-sampling. In the uniform case that $\theta_1 = \dots = \theta_n$, the running time is linear in the size

of the constraints $\mathcal{O}(L)$. The running time for the weighted case has a closed form. We leave the details in section 3.8.

3.4. Neural Lovász Sampler

We first present the proposed Neural Lovász Sampler (NELSON) that implements the LLL-based sampler as a neural network, allowing us to draw multiple samples in parallel on GPUs. We then demonstrate how NELSON is embedded in CD-based learning for constrained MRFs.

3.4.1. NELSON: Neural Lovász Sampler

Represent Constraints as CNF. NELSON obtains samples from the constrained MRF model in single variable form (Eq. 3.6). To simplify notations, we denote $P_\theta(X_i = x_i) = \frac{\exp(\theta_i x_i)}{\sum_{x_i \in \{0,1\}} \exp(\theta_i x_i)}$. Since our constrained MRF model is defined on the Boolean hyper-cube $\{0,1\}^n$, we assume all constraints $\{c_j\}_{j=1}^L$ are given in the Conjunctive Normal Form (CNF). Note that all propositional logic can be reformulated in CNF format with at most a polynomial-size increase. A formula represented in CNF is a conjunction ($\wedge$) of clauses. A clause is a disjunction ($\vee$) of literals, and a literal is either a variable or its negation ($\neg$). Mathematically, we use c_j to denote a clause and use $l_{j,k}$ to denote a literal. In this case, a CNF formula would be:

$$(3.7) \quad c_1 \wedge \dots \wedge c_L, \quad \text{where } c_j = l_{j,1} \vee \dots \vee l_{j,K}$$

A clause is true if and only if at least one of the literals in the clause is true. The whole CNF is true if all clauses are true.

We transform each step of Algorithm 1 into arithmetic operations, hence encoding it as a multi-layer neural network. To do that, we first need to define a few notations:

- Vector of assignment $x^t = (x_1^t, \dots, x_n^t)$, where x_i^t is the assignment of variable X_i in the t -th round of Algorithm 1. $x_i^t = 1$ denotes variable X_i takes value 1 (or true).
- Vector of marginal probabilities $P = (P_1, \dots, P_n)$, where P_i is the probability of variable X_i taking value 0 (false): $P_i = P_\theta(X_i = 0) = \exp(0)/(\exp(0) + \exp(\theta_i))$.
- Tensor $W \in \{-1, 0, 1\}^{L \times K \times n}$ and matrix $b \in \{0, 1\}^{L \times n}$, that are used for checking constraint satisfaction:

$$(3.8) \quad W_{jki} = \begin{cases} 1 & \text{if } k\text{-th literal of clause } c_j \text{ is } X_i, \\ -1 & \text{if } k\text{-th literal of clause } c_j \text{ is } \neg X_i, \\ 0 & \text{otherwise.} \end{cases}$$

$$(3.9) \quad b_{jk} = \begin{cases} 1 & \text{if } k\text{-th literal of clause } c_j \text{ is negated,} \\ 0 & \text{otherwise.} \end{cases}$$

- Matrix $V \in \{0, 1\}^{L \times n}$, denoting the mapping from clauses to variables in the CNF form for constraints $\mathcal{C}$:

$$(3.10) \quad V_{ji} = \begin{cases} 1 & \text{if clause } c_j \text{ contains a literal involving } X_i \\ 0 & \text{otherwise.} \end{cases}$$

- Vector of resampling indicators A^t , where $A_i^t = 1$ indicates variable X_i needs to be resampled at round t .

Given these defined variables, we represent each step of Algorithm 1 using arithmetic operations as follows:

Initialization. To complete line 1 of Algorithm 1, given the marginal probability vector P , the first step is sampling an initial assignment of X , $x^1 = (x_1^1, \dots, x_n^1)$. It is accomplished by: for $1 \leq i \leq n$,

$$(3.11) \quad x_i^1 = \begin{cases} 1 & \text{if } u_i > P_i, \\ 0 & \text{otherwise.} \end{cases}$$

Here u_i is sampled from the uniform distribution in $[0, 1]$.

Check Constraint Satisfaction. To complete line 2 of Algorithm 1, given an assignment x^t at round $t \geq 1$, tensor W and matrix b , we compute Z^t as follows:

$$(3.12) \quad Z^t = W \circledast x^t + b,$$

where $\circledast$ represents a special multiplication between tensor and vector: $(W \circledast x)_{jk} = \sum_{i=1}^n W_{jki} x_i^t$. Note that $Z_{jk}^t = 1$ indicates the k -th literal of j -th clause is true (takes value 1). Hence, we compute S_j^t as:

$$(3.13) \quad S_j^t = 1 - \max_{1 \leq k \leq K} Z_{jk}^t, \quad \text{for } 1 \leq j \leq L.$$

Here $S_j^t = 1$ indicates x^t violates j -th clause. We check $\sum_{j=1}^L S_j^t \neq 0$ to see if any clause is violated, which corresponds to $C(x) = 0$ and is the continuation criteria of the while loop.

Extract Variables in Violated Clauses. To complete line 3 of Algorithm 1, we extract all the variables that require resampling based on vector S^t computed from the last step. The vector of resampling indicator A^t can be computed as:

$$(3.14) \quad A_i^t = \mathbf{1} \left(\sum_{j=1}^L S_j^t V_{ji} \geq 1 \right), \quad \text{for } 1 \leq i \leq n$$

where $\sum_{j=1}^L S_j^t V_{ji} \geq 1$ implies X_i requires resampling.

Resample. To complete line 4 of Algorithm 1, given the marginal probability vector P , resample indicator vector A^t and assignment x^t , we draw a new random sample x^{t+1} . This can be done using this update rule: for $1 \leq i \leq n$,

$$(3.15) \quad x_i^{t+1} = \begin{cases} (1 - A_i^t)x_i^t + A_i^t & \text{if } u_i > P_i, \\ (1 - A_i^t)x_i^t & \text{otherwise.} \end{cases}$$

Again, u_i is drawn from the uniform distribution in $[0, 1]$. Drawing multiple assignments in parallel is attained by extending x^t with a new dimension (See implementation in section 3.10.1). Example 1 show the detailed steps of NELSON (See more examples in section 3.7.5).

Example 1. Assume we have random variables X_1, X_2, X_3 with $n = 3$, Constraints $\mathcal{C} = (X_1 \vee X_2) \wedge (\neg X_1 \vee X_3)$ in the CNF form with $L = 2, K = 2$. Tensor W is:

$$W = \begin{bmatrix} w_{11}=[w_{111}, w_{112}, w_{113}], & w_{12}=[w_{121}, w_{122}, w_{123}] \\ w_{21}=[w_{211}, w_{212}, w_{213}], & w_{22}=[w_{221}, w_{222}, w_{223}] \end{bmatrix},$$

$$w_{11} = [1, 0, 0], w_{12} = [0, 1, 0], w_{21} = [-1, 0, 0], w_{22} = [0, 0, 1].$$

Algorithm 2 Learn Constrained MRFs via NELSON-CD.

Input: Dataset $\mathcal{D}$; Constraints $\mathcal{C}$; #Samples m ; Learning Iterations $T_{\max}$; Parameters of Constrained MRFs θ .

- 1: $\text{NELSON}(W, b, V) \leftarrow \text{build}(X, \mathcal{C})$. $\triangleright$ in Sec. 3.4
 - 2: **for** $t = 1$ **to** $T_{\max}$ **do**
 - 3: $\{x^j\}_{j=1}^m \sim \mathcal{D}$. $\triangleright$ from data
 - 4: $\{\tilde{x}^j\}_{j=1}^m \leftarrow \text{NELSON}(\theta^t, m)$. $\triangleright$ from model
 - 5: $g^t \leftarrow \frac{1}{m} \sum_{j=1}^m \nabla \phi(x^j) - \nabla \phi(\tilde{x}^j)$ $\triangleright$ divergence
 - 6: $\theta^{t+1} \leftarrow \theta^t - \eta g^t$. $\triangleright$ update parameters
 - 7: **end for** **return** The converged MRF model $\theta^{T_{\max}}$.
-

Note that $w_{111} = 1$ means X_1 is the 1st literal in the 1st clause and $w_{211} = -1$ means $\neg X_1$ is the 1st literal in the 2nd clause. Matrix b and the mapping matrix V are:

$$b = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix},$$

$b_{21} = 1$ indicates the 1st literal in the 2nd clause is negated. For the mapping matrix, $V_{11} = V_{12} = 1$ implies the 1st clause contains X_1 and X_2 . For $t = 1$, suppose we have an initialized assignment $x^1 = [0 \ 0 \ 1]^\top$, meaning $X_1 = X_2 = 0, X_3 = 1$. The intermediate results of Z^1, S^1, A^1 become:

$$Z^1 = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}, \quad S^1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad A^1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix},$$

where $S_1^1 = 1$ implies the 1st clause is violated. $A_1^1 = A_2^1 = 1$ denotes variables X_1, X_2 require resampling.

3.4.2. Contrastive Divergence-based Learning

The whole learning procedure is shown in Algorithm 2. At every learning iteration, we call NELSON to draw assignments $\{\tilde{x}^j\}_{j=1}^m$ from constrained MRF's distribution $P_\theta(X|\mathcal{C})$. Then we pick m data points at random from the training set $\{x^j\}_{j=1}^m \sim \mathcal{D}$. The divergence g^t in line 5 of Algorithm 2 is an estimation of $\nabla \ell_{\mathcal{C}}(\theta)$ in Eq. (3.4). Afterward, the MRFs' parameters are updated, according to line 6 of Algorithm 2. After $T_{\max}$ learning iterations, the algorithm outputs the constrained MRF model with parameters $\theta^{T_{\max}}$.

3.5. Experiments

We show the efficiency of the proposed NELSON on learning MRFs defined on the solutions of three combinatorial problems. Over all the tasks, we demonstrate that NELSON outperforms baselines on learning performance, i.e., generating structures with high likelihoods and MAP@10 scores (Table 3.1(c), 3.2(a,b), 3.3(c,d)). NELSON also generates samples which 100% satisfy constraints (Tables 3.1(b), 3.3(b)). Finally, NELSON is the most efficient sampler. Baselines either time out or cannot generate valid structures (Tables 3.1(a), 3.3(a)).

3.5.1. Experimental Settings

Baselines. We compare NELSON with other contrastive divergence learning algorithms equipped with other sampling approaches. In terms of baseline samplers, we consider:

- Gibbs sampler [Carter and Kohn, 1994], which is a special case of MCMC that is widely used in training MRF models.

- Weighted SAT samplers, including WAPS [Gupta et al., 2019], WeightGen [Chakraborty et al., 2014] and XOR sampler [Ermon et al., 2013; Ding and Xue, 2021].
- Uniform SAT samplers, including CMSGen [Golia et al., 2021], QuickSampler [Dutra et al., 2018], UniGen [Soos et al., 2020] and KUS [Sharma et al., 2018]. Notice these samplers cannot sample SAT solutions from a non-uniform distribution. We include them in the learning experiments as a comparison, and exclude them in the weighted sampling experiment (in Fig. 3.2).

Metrics. In terms of evaluation metrics, we consider:

- Training time per iteration, which computes the average time for every learning method to finish one iteration.
- Validness, that is the percentage of generated solutions that satisfy the given constraints $\mathcal{C}$.
- Mean Averaged Precision (MAP@10), which is the percentage that the solutions in the training set $\mathcal{D}$ reside among the top-10 *w.r.t.* likelihood score. The higher the MAP@10 scores, the better the model generates structures closely resembling those in the training set.
- log-likelihood of the solutions in the training set $\mathcal{D}$ (in Eq. 3.3). The model that attains the highest log-likelihood learns the closest distribution to the training set.
- Approximation error of $\nabla \log Z_{\mathcal{C}}(\theta)$, which is the L_1 distance between the exact value $\nabla \log Z_{\mathcal{C}}(\theta)$ and the approximated value given by the sampler.

See section 3.10 for detailed settings of baselines and evaluation metrics, as well as the following task definition, dataset construction, and potential function definition.

Table 3.1. Sampling efficiency and accuracy for learning K -SAT solutions with preferences. The proposed NELSON is the most efficient (see “Training Time Per Epoch”) and always generates valid assignments (see “Validness”) with a small approximation error (see “Approximation Error of Gradient”) against all baselines. T.O. means time out.

Problem size	(a) Training time per iteration (Mins) ($\downarrow$)								
	NELSON	XOR	WAPS	WeightGen	CMSGGen	KUS	QuickSampler	Unigen	Gibbs
10	0.13	26.30	1.75	0.64	0.22	0.72	0.40	0.66	0.86
20	0.15	134.50	3.04	T.O.	0.26	0.90	0.30	2.12	1.72
30	0.19	1102.95	6.62	T.O.	0.28	2.24	0.32	4.72	2.77
40	0.23	T.O.	33.70	T.O.	0.31	19.77	0.39	9.38	3.93
50	0.24	T.O.	909.18	T.O.	0.33	1532.22	0.37	13.29	5.27
500	5.99	T.O.	T.O.	T.O.	34.17	T.O.	T.O.	T.O.	221.83
1000	34.01	T.O.	T.O.	T.O.	177.39	T.O.	T.O.	T.O.	854.59
(b) Validness of generated solutions (%) ($\uparrow$)									
10 – 50	100	100	100	100	100	100	82.65	100	90.58
500	100	T.O.	T.O.	T.O.	100	T.O.	7.42	100	54.27
1000	100	T.O.	T.O.	T.O.	100	T.O.	0.00	100	33.91
(c) Approximation error of $\nabla \log Z_C(\theta)$ ($\downarrow$)									
10	0.10	0.21	0.12	3.58	3.96	4.08	3.93	4.16	0.69
12	0.14	0.19	0.16	5.58	5.50	5.49	5.55	5.48	0.75
14	0.15	0.25	0.19	T.O.	6.55	6.24	7.79	6.34	1.30
16	0.16	0.25	0.15	T.O.	9.08	9.05	9.35	9.03	1.67
18	0.18	0.30	0.23	T.O.	10.44	10.30	11.73	10.20	1.90

3.5.2. Random K -SAT Solutions with Preference

Task Definition & Dataset. This task is to learn to generate solutions to a K -SAT problem. We are given a training set $\mathcal{D}$ containing solutions to a corresponding CNF formula $c_1 \wedge \dots \wedge c_L$. Note that not all solutions are equally likely to be presented in $\mathcal{D}$. The *learning* task is to maximize the log-likelihood of the assignments seen in the training set $\mathcal{D}$. Once learning is completed, the *inference* task is to generate valid solutions that closely resemble those in $\mathcal{D}$ [Dodaro and Previti, 2019]. To generate the training set $\mathcal{D}$, we use CNFGen [Lauria et al., 2017] to generate the random K -SAT problem and use Glucose4 solver to generate random valid solutions [Ignatiev et al., 2018].

Sampler’s Efficiency and Accuracy. Table 3.1 shows the proposed NELSON is an efficient sampler that generates valid assignments, in terms of the training time for learning

Table 3.2. The quality of learning outcomes for learning random K -SAT solutions with preferences. NELSON achieves the best likelihood and MAP@10 scores. T.O. is time out.

	(a) log-likelihood ($\uparrow$)			
Problem size	NELSON	Gibbs	CMSGen	Quicksampler WeightGen,KUS XOR, WAPS
100	-49.16	- 36.36	-60.12	T.O.
300	- 52.61	-53.11	-128.39	
500	- 196.47	-197.21	-272.49	
700	- 238.60	-238.75	-389.44	
1000	- 294.22	-296.33	-532.85	
	(b) MAP@10 (%) ($\uparrow$)			
100	82.13	83.32	86.34	T.O.
300	66.37	64.42	64.50	
500	90.03	73.14	70.67	
700	69.74	69.74	48.10	
1000	91.70	77.56	78.72	

constrained MRF, approximation error for the gradient and validness of the generated assignments. In Table 3.1(a), NELSON takes much less time for sampling against all the samplers and can train the model with the dataset of problem size 1000 within an hour. In Table 3.1(b), NELSON always generates valid samples. The performance of QuickSampler and Gibbs methods decreases when the problem size becomes larger. In Table 3.1(c), NELSON, XOR and WAPS are the three algorithms that can effectively estimate the gradient while the other algorithms incur huge estimation errors. Also, the rest methods are much slower than NELSON.

Learning Quality Table 3.2 demonstrates NELSON-CD learns a more accurate constrained MRF model by measuring the log-likelihood and MAP@10 scores. Note that baselines including Quicksampler, Weightgen, KUS, XOR and WAPS timed out for the problem sizes we considered. Compared with the remaining baselines, NELSON attains the best log-likelihood and MAP@10 metric.

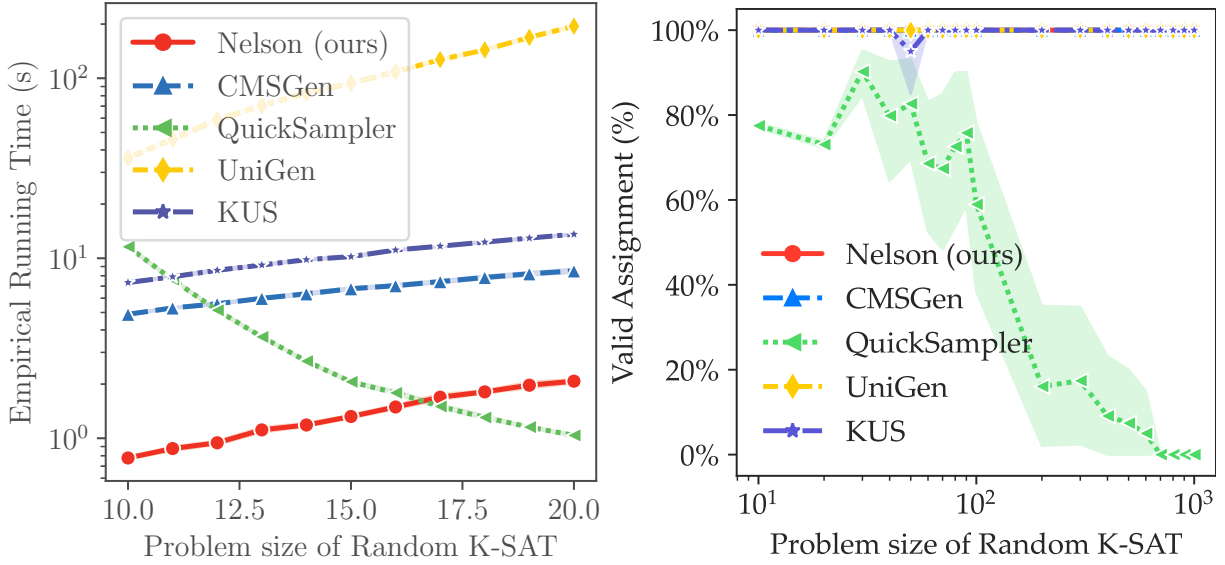


Figure 3.1. Running time and the percentage of valid structures sampled uniformly at random from solutions of K-SAT problems. Among all the problem sizes, NELSON always generate valid solutions and is the most efficient sampler.

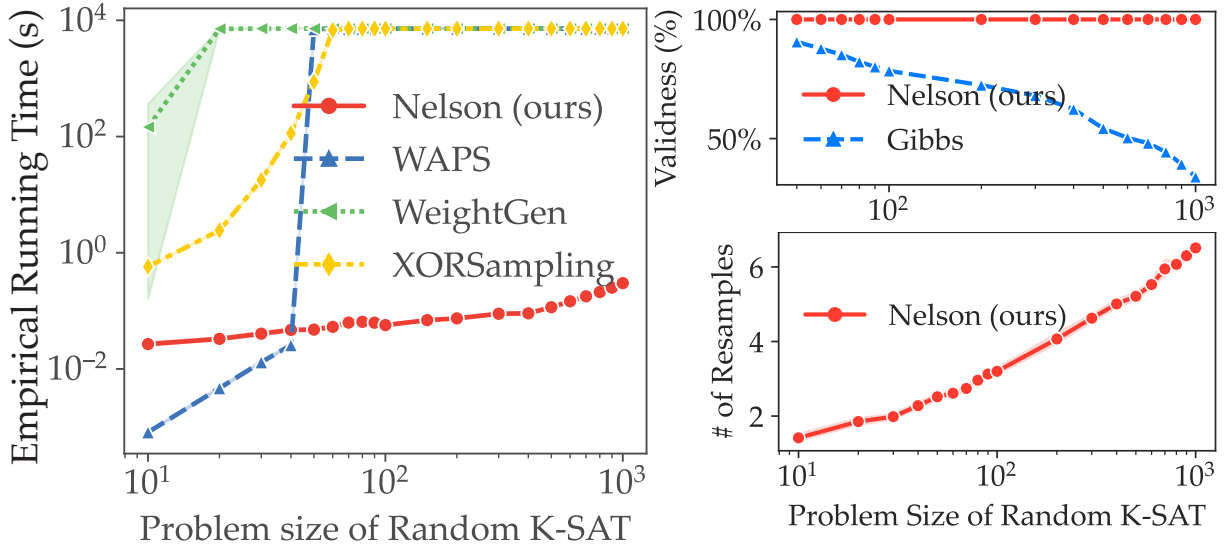


Figure 3.2. Running time, the percentage of valid solutions generated, and rounds of resampling for weighted sample generation of K-SAT solutions. Among all the problem sizes, NELSON scales the best among all approaches and always generates valid solutions.

Abalation Study. We also evaluated the samplers’ efficiency in isolation (not embedded in learning). The sampling cases we considered are uniform and weighted (mainly following the experiment setting in [Chakraborty and Meel, 2019]). In weighted sampling, the weights are specified by fixed values to the single factors in Eq. (3.6). In the uniform sampling case in Fig. 3.1, NELSON and Quicksampler require much less time to draw samples compared to other approaches. However, the solutions generated by Quicksampler rarely satisfy constraints. In the weighted sampling case in Fig. 3.2, NELSON scales better than all the competing samplers as the sizes of the K -SAT problems increase.

3.5.3. Sink-Free Orientation in Undirected Graphs

Task Definition & Dataset A *sink-free* orientation of an undirected graph is a choice of orientation for each arc such that every vertex has at least one outgoing arc [Cohn et al., 2002]. This task has wide applications in robotics routing and IoT network configuration [Takahashi et al., 2009]. Even though finding a sink-free orientation is tractable, sampling a sink-free orientation from the space of all orientations is still $\#P$ -hard. Given a training set of preferred orientations $\mathcal{D}$ for the graph, the *learning* task is to maximize the log-likelihood of the orientations seen in the training set. The *inference* task is to generate valid orientations that resemble those in the training set. To generate the training set, we use the Erdős-Rényi random graph from the NetworkX library. The problem size is characterized by the number of vertices in the graph. The baselines we consider are CD-based learning with Gibbs sampling and CMSGen.

Learning Quality. In Table 3.3(a), we show the proposed NELSON method takes much less time to train MRF for one epoch than the competing approaches. Furthermore, in

Table 3.3. Sample efficiency and learning performance of the sink-free orientation task. NELSON is the most efficient (see Training Time Per Epoch) and always generates valid assignments (see Validness), has the smallest error approximating gradients, and has the best learning performance (see MAP@10) among all baselines.

Problem size	(a) Training Time Per Epoch (Mins) ($\downarrow$)		
	NELSON	Gibbs	CMSGen
10	0.53	9.85	0.69
20	0.53	80.12	1.93
30	0.72	256.38	3.65
40	0.93	777.01	5.99
50	1.17	T.O.	9.08
	(b) Validness of Orientations (%) ($\uparrow$)		
7	100	50.16	100
8	100	64.63	100
9	100	47.20	100
10	100	62.60	100
11	100	84.95	100
	(c) Approximation Error of $\nabla \log Z_C(\theta)$ ($\downarrow$)		
5	0.01	0.09	0.21
7	0.05	0.08	2.37
9	0.03	0.11	2.37
11	0.04	0.17	8.62
13	0.05	0.28	11.27
	(d) MAP@10 (%) ($\uparrow$)		
10	61.14	60.01	64.56
20	55.26	55.20	47.79
30	100.00	96.29	100.00
40	40.01	39.88	38.90
50	46.12	T.O.	42.11

Table 3.3(b), NELSON and CMSGen generate 100% valid orientations of the graph while the Gibbs-based model does not. Note the constraints for this task satisfy Condition 4, hence NELSON sampler’s performance is guaranteed by Theorem 7. In Table 3.3(c), NELSON attains the smallest approximation error for the gradient (in Eq. 3.4) compared to baselines. Finally, NELSON learns a higher MAP@10 than CMSGen. The Gibbs-based

approach times out for problem sizes larger than 40. In summary, our NELSON is the best-performing algorithm for this task.

3.5.4. Learn Vehicle Delivery Routes

Task Definition & Dataset. Given a set of locations to visit, the task is to generate a sequence to visit these locations in which each location is visited once and only once and the sequence closely resembles the trend presented in the training data. The training data are such routes collected in the past. The dataset is constructed from TSPLIB, which consists of 29 cities in Bavaria, Germany. The constraints for this problem do not satisfy Condition 4. We still apply the proposed method to evaluate if the NELSON algorithm can handle those general hard constraints.

In Fig. 3.3, we see NELSON can obtain samples of this delivery problem efficiently. We measure the number of resamples taken as well as the corresponding time used by the NELSON method. NELSON takes roughly 50 times of resamples with an average time of 0.3 seconds to draw a batch (the batch size is 100) of valid visiting sequences.

3.6. Chapter Recap

Before we move onto the more technical part, let us recap the research problems this chapter wants to address. In this chapter, we present NELSON, which embeds a sampler based on Lovász Local Lemma into the contrastive divergence learning of Markov random fields. The embedding is fully differentiable. This approach allows us to learn generative models over constrained domains, which presents significant challenges to other state-of-the-art models. We also give sound proofs of the performance of the LLL-based sampler.

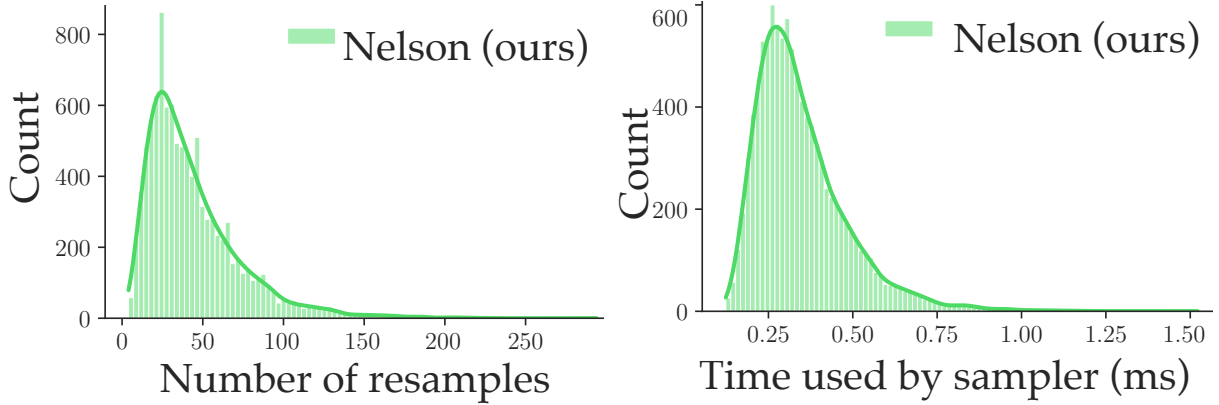


Figure 3.3. Frequency histograms for the number of resample and the total time of NELSON method for uniformly sampling visiting paths for vehicle routing problem.

Experimental results on several real-world domains reveal that NELSON learns to generate 100% valid structures, while baselines either time out or cannot generate valid structures. NELSON also outperforms other approaches in the running times and in various learning metrics.

3.7. Probability Distribution of Algorithm 1

3.7.1. Definitions and Notations

This section is for the proofs related to the probability distribution in the proposed Algorithm 1. For convenience, commonly used notations are listed in Table 3.4. We make some slight changes to some notations that appear in this chapter to make sure they are consistent and well-defined in this proof.

Similar to the previous analysis [Guo et al., 2019; Jerrum, 2021], we begin by introducing the concept “dependency graph” (in Definition 9) for the constraints $\mathcal{C}$.

Definition 9 (Dependency Graph). *The dependency graph $G = (\mathcal{C}, E)$, where the vertex set is the set of constraints $\mathcal{C}$. Two vertices c_i and c_j are connected with an edge $(c_i, c_j) \in E$ if and only if they are defined on at least one common random variable, i.e., $\text{var}(c_i) \cap \text{var}(c_j) \neq \emptyset$.*

For keeping track of the whole sampling procedure, we need the concept “sampling record” (in Definition 10) [Guo et al., 2019], which are the broken constraints at every round in Algorithm 1. It is also known as the witness tree in [Moser and Tardos, 2010]. This allows us to check the constraint satisfaction of the assignment at every round.

Under Condition 4, for any edge in the dependency graph $(c_i, c_j) \in E$, either $\mathbf{1}(x, c_i) = 0$ or $\mathbf{1}(x, c_j) = 0$ for all $x \in \mathcal{X}$. In other words, two constraints with shared related variables, representing two *adjacent vertices* in the dependency graph G , are not broken simultaneously. Thus, the constraints in record S_t form an *independent set*² over the dependency graph under Condition 4.

Definition 10 (Sampling Record). *Given dependency graph $G(\mathcal{C}, E)$, let $X_t = x$ be one possible assignment obtained at round t of Algorithm 1. Let $S_t \subseteq \mathcal{C}$ be the set of vertices in graph G (subset of constraints) that x violates*

$$(3.16) \quad S_t = \{c_i | c_i \in \mathcal{C} \text{ and } \mathbf{1}(x, c_i) = 0\},$$

where indicator function $\mathbf{1}(x, c_i) = 0$ implies x violates constraint c_i at round t . Define the sampling record as the sequence of violated constraints $S_1, \dots, S_t$ throughout the execution.

²A set of vertices with no two adjacent vertices in the graph.

At round t ($t \geq 1$) of Algorithm 1, suppose the violated constraints is $S_t \subseteq \mathcal{C}$. The constraints that are not adjacent to S_t in the dependency graph are still satisfied after re-sample. The only possible constraints that might be broken after the re-sample operation are among S_t itself, or those constraints directly connected to S_t in the dependency graph. Therefore,

$$S_{t+1} \subset \Gamma(S_t), \quad \text{for all } t \geq 1.$$

where $\Gamma(S_t)$ is the set of vertices of S_t and its adjacent neighbors in the dependency graph G (see Table 3.4). When Algorithm 1 terminates at round $T + 1$, no constraints are violated anymore, i.e., $s_{T+1} = \emptyset$. To summarize the above discussion on a sampling record by Algorithm 1, we have the following Claim 3.

Claim 3. *Under Condition 4, a potential sampling record of length $T + 1$ by the Algorithm 1 is a sequence of independent sets: $S_1, S_2, \dots, S_T, \emptyset$ with*

- (1) $S_{t+1} \subseteq \Gamma(S_t)$ and $S_t \neq \emptyset$, for $1 \leq t \leq T$;
- (2) $s_{T+1} = \emptyset$.

Extra Notations related to Constrained MRF. The constrained MRF model over constraints set $\mathcal{C}$ is defined as:

$$P_\theta(X = x|\mathcal{C}) = \frac{\exp(\sum_{i=1}^n \theta_i x_i) \mathbf{1}(x, \mathcal{C})}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C})}$$

where the partition function only sums over valid assignments in $\mathcal{X}$. Note that $C(x)$ in Equation (3.6) is the same as $\mathbf{1}(x', \mathcal{C})$ in the above equation. We slightly change the notations for consistency in this proof. Also notice that the output distribution can no

Table 3.4. Summary of all the notations used in the theoretical analysis of Algorithm 1.

Notation	Definition
$X = \{X_i\}_{i=1}^n$	set of discrete random variables
$x \in \mathcal{X}$	possible assignments for variables X
$x_i \in \mathcal{X}_i$	variable X_i can take all values in $\mathcal{X}_i$
$\mathcal{C} = \{c_j\}_{j=1}^m$	given constraints
$S_t \subseteq \mathcal{C}$	subset of constraints violated at round t of Algorithm 1
$G(\mathcal{C}, E)$	the dependency graph (in Definition 9)
$\text{var}(c_j)$	the indices of domain variables that are related to constraint c_i
$\text{var}(S_t)$	the indices for domain variables that are related to constraints S_t
$\Gamma(c_j)$	c_j and its direct neighbors in the dependency graph
$\Gamma(S_t)$	S_t and direct neighbors of S_t in the dependency graph
$\mathcal{C} \setminus \Gamma(S_t)$	all constraints in $\mathcal{C}$ but not in $\Gamma(S_t)$
$S_1, \dots, S_T, \emptyset$	a sampling record of Algorithm 1 (in Definition 10)
$\mathbf{1}(x, c_i)$	indicator function that evaluates if assignment x satisfies constraint c_i
$\mathbf{1}(x, S_t)$	indicator function that evaluates if assignment x satisfies constraints in S_t
$\mathbf{1}(x, \mathcal{C})$	indicator function that evaluates if assignment x satisfies all constraints $\mathcal{C}$
$P_\theta(X \mathcal{C} \setminus \Gamma(S_t))$	see Definition 11
$\mathbb{P}(X S_t)$	see Definition 12

longer be factorized after constraints are enforced, since the partition function cannot be factorized. Our task is to draw samples from this distribution.

To analyze the intermediate steps in Algorithm 1, we further need to define the following notations.

Definition 11. *The constrained MRF distribution for constraints $\mathcal{C} \setminus \Gamma(S_t)$ is*

$$P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t)) = \frac{\exp(\sum_{i=1}^n \theta_i x_i) \mathbf{1}(x, \mathcal{C} \setminus \Gamma(S_t))}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C} \setminus \Gamma(S_t))}$$

Definition 12. *At round t of Algorithm 1, assume $S_t \subseteq \mathcal{C}$ are the set of broken constraints, Define $\mathbb{P}(X_{t+1} = x | S_1, \dots, S_t)$ to be the probability of obtaining a new assignment x after we re-sample random variables indexed by $\text{var}(S_t)$.*

3.7.2. Ratio Property Lemma

Lemma 26 (Ratio Property). *Under Condition 4, assume Algorithm 1 is at round t . Conditioning on observing one possible sampling record $S_1, \dots, S_t$, Algorithm 1 step 4 will re-sample variables in $\text{var}(S_t)$ at round $t + 1$. Let $x, x' \in \mathcal{X}$ be two possible assignments after this re-sample. The probability ratio of obtaining these two results equals that under constrained MRF $P_\theta(x|\mathcal{C} \setminus \Gamma(S_t))$:*

$$(3.17) \quad \frac{\mathbb{P}(X_{t+1} = x | S_1, \dots, S_t)}{\mathbb{P}(X_{t+1} = x' | S_1, \dots, S_t)} = \frac{P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t))}{P_\theta(X = x' | \mathcal{C} \setminus \Gamma(S_t))},$$

where $\mathbb{P}(X_{t+1} = x | S_1, \dots, S_t)$ is the probability of Algorithm 1 step 4 produces assignment x at round $t + 1$, conditioning on the observed record $S_1, \dots, S_t$ and re-sample $\text{var}(S_t)$. $P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t))$ is the constrained MRF (for the constraints $\mathcal{C} \setminus \Gamma(S_t)$) probability on assignment x .

PROOF. During the intermediate step of the algorithm, assume the set of constraints S_t are violated. We want to re-sample variables indexed by $\text{var}(S_t)$, so variables indexed by $\text{var}(\mathcal{C} \setminus \Gamma(S_t))$ won't change assignments. Also, because $\Gamma(S_t)$ is the largest possible set of constraints that can be infected by the re-sample, constraints $\mathcal{C} \setminus \Gamma(S_t)$ are still satisfied after the re-sample.

At round t , we re-sample variables in $\text{var}(S_t)$ according to step 4 in Algorithm 1, we thus have:

$$\mathbb{P}(X_{\text{var}(S_t)}^{t+1} = x_{\text{var}(S_t)} | S_1 \dots S_t) = \prod_{i \in \text{var}(S_t)} \frac{\exp(\theta_i x_i)}{\sum_{x'_i \in \mathcal{X}_i} \exp(\theta_i x'_i)}.$$

Here the notation $X_{\text{var}(S_t)}^{t+1} = x_{\text{var}(S_t)}$ means $X_i = x_i$ for $i \in \text{var}(S_t)$ at round t . For any two possible assignments x, x' after the re-sample,

$$x_i = x'_i, \quad \text{for } i \in \{1, \dots, n\} \setminus \text{var}(S_t)$$

since the rest variable's assignments are kept the same after re-sample. Thus, we can have ratio:

$$(3.18) \quad \frac{\mathbb{P}(X_{t+1} = x | S_1, \dots, S_t)}{\mathbb{P}(X_{t+1} = x' | S_1, \dots, S_t)} = \frac{\exp(\sum_{i \in \text{var}(S_t)} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(S_t)} \theta_i x'_i)} = \frac{\exp(\sum_{i \in \text{var}(\Gamma(S_t))} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(\Gamma(S_t))} \theta_i x'_i)}.$$

For the last step, since every assignment outside $\text{var}(S_t)$ is not changed, we can enlarge the index set of summation to $\Gamma(S_t)$ by multiplying

$$1 = \frac{\exp(\sum_{i \in \text{var}(\Gamma(S_t)) \setminus \text{var}(S_t)} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(\Gamma(S_t)) \setminus \text{var}(S_t)} \theta_i x'_i)}.$$

After re-sample, we know x must satisfy the constraints $\mathcal{C} \setminus \Gamma(S_t)$. Thus, the probability of this x conditioned on constraints $\mathcal{C} \setminus \Gamma(S_t)$ holding in the constrained MRF model is:

$$\begin{aligned} P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t)) &= \frac{\exp(\sum_{i=1}^n \theta_i x_i) \mathbf{1}(x, \mathcal{C} \setminus \Gamma(S_t))}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C} \setminus \Gamma(S_t))} \\ &= \frac{\exp(\sum_{i=1}^n \theta_i x_i)}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C} \setminus \Gamma(S_t))}. \end{aligned}$$

In the constrained MRF model (for constraints $\mathcal{C} \setminus \Gamma(S_t)$), the ratio of these two probabilistic assignments x, x' is:

$$(3.19) \quad \frac{P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t))}{P_\theta(X = x' | \mathcal{C} \setminus \Gamma(S_t))} = \frac{\exp(\sum_{i \in \text{var}(\Gamma(S_t))} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(\Gamma(S_t))} \theta_i x'_i)},$$

because the x_i outside $\text{var}(\Gamma(S_t))$ remains the same. Note that x, x' are two possible assignments produced according to step 4 in Algorithm 1 at round t . Combining Equation (3.18) and Equation (3.19), we conclude that:

$$\frac{\mathbb{P}(X_{t+1} = x | S_1, \dots, S_t)}{\mathbb{P}(X_{t+1} = x' | S_1, \dots, S_t)} = \frac{P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t))}{P_\theta(X = x' | \mathcal{C} \setminus \Gamma(S_t))}.$$

The proof is finished. $\square$

3.7.3. Proof of Theorem 7

Suppose the re-sampling process terminates at round $T + 1$ and we obtain a valid sample x . Upon the termination of Algorithm 1, all the constraints are satisfied. So we have: $S_{T+1} = \emptyset$. In other words, $\mathbf{1}(x, \mathcal{C}) = 1$.

Let x, x' be two possible valid assignments produced at round $T + 1$ by the Algorithm 1. Using the analysis in Lemma 26, we can still have:

$$\frac{\mathbb{P}(X_{T+1} = x | S_1, \dots, S_T)}{\mathbb{P}(X_{T+1} = x' | S_1, \dots, S_T)} = \frac{\exp(\sum_{i \in \text{var}(S_T)} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(S_T)} \theta_i x'_i)}.$$

The probability of this x in the constrained MRF model (for constraints $\mathcal{C}$) is:

$$P_\theta(X = x | \mathcal{C}) = \frac{\exp(\sum_{i=1}^n \theta_i x_i) \mathbf{1}(x, \mathcal{C})}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C})} = \frac{\exp(\sum_{i=1}^n \theta_i x_i)}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C})}.$$

Then we conclude that:

$$\frac{\mathbb{P}(X_{T+1} = x | S_1, \dots, S_T)}{\mathbb{P}(X_{T+1} = x' | S_1, \dots, S_T)} = \frac{P_\theta(X = x | \mathcal{C})}{P_\theta(X = x' | \mathcal{C})}.$$

Note that this ratio property holds for all the possible sampling records $S_1, \dots, S_T, \emptyset$.

Summation of All Possible Sampling Records. Define $\mathbb{P}(S_1, \dots, S_T)$ to be the probability of observing record $S_1, \dots, S_T$ by Algorithm 1. For any possible sampling record $S_1, \dots, S_T, \emptyset$, the ratio property still holds:

$$\frac{\mathbb{P}(X_{T+1} = x | S_1, \dots, S_T) \mathbb{P}(S_1, \dots, S_T)}{\mathbb{P}(X_{T+1} = x' | S_1, \dots, S_T) \mathbb{P}(S_1, \dots, S_T)} = \frac{P_\theta(X = x | \mathcal{C})}{P_\theta(X = x' | \mathcal{C})}$$

where the term $\mathbb{P}(S_1, \dots, S_T)$ on the Left-hand-side (LHS) is actually the same. After we summarize over all possible sampling records $S_1, \dots, S_T, \emptyset$, the ratio property still holds. Let $\mathbb{P}(X_{T+1} = x)$ be the probability of obtaining one valid assignment x by the execution of Algorithm 1.

(3.20)

$$\frac{\mathbb{P}(X_{T+1} = x)}{\mathbb{P}(X_{T+1} = x')} = \frac{\sum_{S_1, \dots, S_T} \mathbb{P}(X_{T+1} = x | S_1, \dots, S_T) \mathbb{P}(S_1, \dots, S_T)}{\sum_{S_1, \dots, S_T} \mathbb{P}(X_{T+1} = x' | S_1, \dots, S_T) \mathbb{P}(S_1, \dots, S_T)} = \frac{P_\theta(X = x | \mathcal{C})}{P_\theta(X = x' | \mathcal{C})}$$

Sample Space Analysis At Termination. We need one more statement to show Theorem 7 holds. Let $\mathcal{X}_{\text{LLL}}$ be the set of all possible assignments x that can be generated by Algorithm 1:

$$\mathcal{X}_{\text{LLL}} = \bigcup_{S_1 \dots S_T} \{x | \mathbb{P}(X_{T+1} = x | S_1 \dots S_T) \neq 0 \text{ and } \mathbb{P}(S_1 \dots S_T) \neq 0\}.$$

where $\mathbb{P}(S_1 \dots S_T) \neq 0$ means $S_1, \dots, S_T$ is a possible record. $\mathbb{P}(X_{T+1} = x | S_1 \dots S_T) \neq 0$ means it is possible to obtain x given the record $S_1, \dots, S_T$.

Let $\mathcal{X}_{\mathcal{C}}$ be the set of assignments x that satisfy all the constraints in the constrained MRF (for constraints $\mathcal{C}$):

$$\mathcal{X}_{\mathcal{C}} = \{x | P_\theta(X = x | \mathcal{C}) \neq 0, \text{ for all } x \in \mathcal{X}\}.$$

Lemma 27. $\mathcal{X}_{LLL} \subseteq \mathcal{X}_C$ and $\mathcal{X}_C \subseteq \mathcal{X}_{LLL}$, thus $\mathcal{X}_{LLL} = \mathcal{X}_C$.

PROOF. When Algorithm 1 terminates, it only produces valid assignments; thus, we must have: $\mathcal{X}_{LLL} \subseteq \mathcal{X}_C$. On the other hand, there is always a non-zero probability that Algorithm 1 will generate every valid assignment $x \in \mathcal{X}_C$, which implies that $\mathcal{X}_C \subseteq \mathcal{X}_{LLL}$. Therefore we can conclude that $\mathcal{X}_{LLL} = \mathcal{X}_C$. $\square$

Lemma 27 show that the two distributions have the same sample space when Algorithm 1 terminates. What's more, Equation (3.20) shows they have the same probability ratio for any possible valid assignments x, x' . This shows that the execution of the Algorithm 1 is a random draw from the constrained MRF distribution $P_\theta(X = x|\mathcal{C})$. The proof of Theorem 7 is finished.

3.7.4. Difference to the Original Proof

The main difference in the above proof to the existing proof in [Guo et al., 2019, Lemma 7] is that: We show Lemma 26 that characterizes the proportional ratio of getting different assignments of variables, which is more general than the descriptive proof for [Guo et al., 2019, Lemma 7].

3.7.5. A Running Example in View of Markov Chain

We dedicate this section to demonstrate the execution of Algorithm 1 with Example 1. Algorithm 1 can be viewed as a Markov chain, so we will show the probability of obtaining valid samples is unbiased by running thousands of steps of the Markov chain. The constraints are $\mathcal{C} = \{c_1 = (X_1 \vee X_2), c_2 = (\neg X_1 \vee X_3)\}$. We use the assignment of all the

variables as the states $s_1, \dots, s_8$ in the rounds of Algorithm 1.

$$\begin{aligned}
 s_1 &= (X_0 = 0, X_1 = 0, X_2 = 0) \\
 s_2 &= (X_0 = 0, X_1 = 0, X_2 = 1) \\
 s_3 &= (X_0 = 0, X_1 = 1, X_2 = 0) \\
 s_4 &= (X_0 = 0, X_1 = 1, X_2 = 1) \\
 s_5 &= (X_0 = 1, X_1 = 0, X_2 = 0) \\
 s_6 &= (X_0 = 1, X_1 = 0, X_2 = 1) \\
 s_7 &= (X_0 = 1, X_1 = 1, X_2 = 0) \\
 s_8 &= (X_0 = 1, X_1 = 1, X_2 = 1)
 \end{aligned}
 \tag{3.21}$$

Here s_1, s_2, s_3, s_4 correspond to *valid* assignments of variables with respect to the constraints $\mathcal{C}$ and s_5, s_6, s_7, s_8 correspond to *invalid* assignments of variables, that requires resampling.

For simplicity, we consider the uniform setting where $\theta_1 = \theta_2 = \theta_3$. The goal is to sample every valid assignment with equal probability. Therefore, the probability for every variable is:

$$P(X_i) = \begin{cases} \frac{1}{2} & \text{for variable } X_i \text{ taking value 1} \\ \frac{1}{2} & \text{for variable } X_i \text{ taking value 0} \end{cases}$$

for $i = 1, 2, 3$. Based on Algorithm 1, we know the probability of transferring from s_i to s_j ($1 \leq i, j \leq 8$). Thus we can construct the transition matrix between every state:

$$(3.22) \quad T = \begin{bmatrix} & s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 & s_8 \\ s_1 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ s_2 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\ s_3 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\ s_4 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\ s_5 & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} & 0 & 0 & 0 \\ s_6 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} \\ s_7 & \frac{1}{4} & 0 & \frac{1}{4} & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 \\ s_8 & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{bmatrix}$$

where $T_{ij} = T(s_i, s_j)$ denotes the transition probability from state s_i to state s_j .

Taking state s_5 as an example, it violates constraint C_2 thus X_2, X_3 will be resampled. There are 4 possible assignments of X_2, X_3 , which corresponds to states $\{s_1, s_2, s_3, s_5\}$. Since each variable is resampled uniformly at random, the probability of transition from state s_5 to the states $\{s_1, s_2, s_3, s_5\}$ are $1/4$. The Algorithm 1 will terminate once it reaches states $\{s_1, s_2, s_3, s_4\}$, which corresponds to the (valid) state only transit to itself with probability 1. Thus we find $T(s_i, s_i) = 1$ for $i = 1, 2, 3, 4$.

For a randomly initialized assignment:

[illegible]

that has an equal probability of being any state. After executing Algorithm 1 for 2000 steps, we have:

$$(3.24) \quad T^{2000} = \begin{bmatrix} & s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 & s_8 \\ s_1 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ s_2 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\ s_3 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\ s_4 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\ s_5 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 & 0 & 0 \\ s_6 & \frac{1}{6} & \frac{1}{2} & \frac{1}{6} & \frac{1}{6} & 0 & 0 & 0 & \frac{1}{4} \\ s_7 & \frac{1}{3} & 0 & \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 & 0 \\ s_8 & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{2} & 0 & 0 & 0 & 0 \end{bmatrix},$$

$$(3.25) \quad xT^{2000} = \begin{bmatrix} s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 & s_8 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 & 0 & 0 & 0 \end{bmatrix}.$$

This implies Algorithm 1 outputs every valid assignment with the same probability in the uniform setting, which follows the result in Theorem 7.

3.8. Running Time Analysis of Algorithm 1

We dedicate this section to showing the running time of Algorithm 1 on a general weighted case. The expected running time of Algorithm 1 is determined by the number of rounds of re-sampling. Algorithm 1 re-sample all the related random variables simultaneously in every single round. However, it is hard to get an estimation of the exact

total running time over the *random variables*. Instead, we can only have a loose upper bound of the expected running time over the *sequence of sampling record* (the sequence of violated constraints).

The overall structure of the proof is similar to the proof in [Guo et al., 2019, Theorem 13]. We show the difference in our proof at the end of this section.

3.8.1. Definitions and Notations

We define the following terms to simplify our notations.

Definition 13. Let S_t be a subset of vertices in a dependency graph. 1) Define p_{S_t} as the probability of constraints in S_t being violated:

$$(3.26) \quad p_{S_t} = \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \right)$$

where we use $\neg c_i$ to indicate the constraint c_i is violated. 2) Define q_{S_t} as the probability that only the constraints in S_t are violated and nothing else.

$$(3.27) \quad q_{S_t} = \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \right)$$

where $\bigwedge_{c_i \in S_t} \neg c_i$ corresponds to only the constraints in S_t are violated and $\bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j$ corresponds to all the rest constraints are satisfied. So $q_{\{c_i\}}$ is the probability that only constraint c_i is broken and all the rest still hold. Similarly, $q_{\emptyset}$ denotes the probability that all the constraints are satisfied.

Lemma 28. *Given Definition 13, we can further expand q_{S_t} under Condition 4:*

$$q_{S_t} = p_{S_t} \mathbb{P} \left(\bigwedge_{c_j \in \mathcal{C} \setminus \Gamma(S_t)} c_j \right)$$

PROOF. We can split q_{S_t} into the probability of two independent events:

$$\begin{aligned} q_{S_t} &= \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \right) && \text{By definition of } q_{S_t} \text{ in Equation (3.27)} \\ &= \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus \Gamma(S_t)} c_j \right) \\ &= \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \right) \mathbb{P} \left(\bigwedge_{c_n \in \mathcal{C} \setminus \Gamma(S_t)} c_n \right) \\ &= p_{S_t} \mathbb{P} \left(\bigwedge_{c_j \in \mathcal{C} \setminus \Gamma(S_t)} c_j \right). && \text{By definition of } p_{S_t} \text{ in Equation (3.26)} \end{aligned}$$

The second equality holds because under Condition 4, adjacent vertices have zero probability. In other words, when we observe that constraints in S_t are violated, constraints in $\Gamma(S_t) \setminus S_t$ cannot be violated. The third equality holds because the random variables in $\mathbf{var}(S_t)$ are *independent to* those variables in $\mathbf{var}(\mathcal{C} \setminus \Gamma(S_t))$. So we can apply $P(AB) = P(A)P(B)$ when the events A, B are independent to each other. $\square$

Remark 4 (Equivalence of Record). *At round t of Algorithm 1, it finds all the constraints S_t that are broken ($\bigwedge_{c_i \in S_t} \neg c_i$), which implies the rest of the constraints $\mathcal{C} \setminus \Gamma(S_t)$ are satisfied ($\bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j$). Thus the probability of observing S_t in the record is equivalent*

to the following:

$$(3.28) \quad \mathbb{P}(S_t) = \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \right)$$

Lemma 29. *Given a possible sampling record $S_1 \dots S_{t-1}, S_t$ by Algorithm 1, the following equality holds for the pair (S_{t-1}, S_t) :*

$$\sum_{S_t} q_{S_t} = \mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)$$

PROOF. By Definition 10 of the sampling record, we have $S_t \subset \Gamma(S_{t-1})$. The relationship of its complement would be:

$$\mathcal{C} \setminus \Gamma(S_{t-1}) \subset \mathcal{C} \setminus S_t.$$

Using the above result, we have:

$$(3.29) \quad \mathbb{P} \left(\bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \wedge \bigwedge_{c_k \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_k \right) = \mathbb{P} \left(\bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \right)$$

Based on Remark 4 and Baye's theorem, we have:

$$\begin{aligned}
& \mathbb{P}(S_t \mid \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i) \\
&= \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \mid \bigwedge_{c_k \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_k \right) && \text{By Equation (3.28)} \\
&= \frac{\mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \wedge \bigwedge_{c_k \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_k \right)}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)} && \text{By Bayes's formula} \\
&= \frac{\mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_k \in \mathcal{C} \setminus S_t} c_k \right)}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)} && \text{By Equation (3.29)} \\
&= \frac{q_{S_t}}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)}. && \text{By definition of } p_{S_t} \text{ in Equation (3.27)}
\end{aligned}$$

Since LHS of Equation (3.30) sums over all possible S_t is one: $\sum_{S_t} \mathbb{P}(S_t \mid \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i) = 1$. Thus, summing over S_t for the RHS of Equation (3.30), we have:

$$(3.30) \quad 1 = \sum_{S_t} \frac{q_{S_t}}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)} = \frac{\sum_{S_t} q_{S_t}}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)}$$

In the second equality, the reason that we can move the summation operator to the numerator is that the denominator is a constant *w.r.t.* all possible S_t . To be specific, given $S_t \subseteq \Gamma(S_{t-1})$, we have S_t is independent to $\mathcal{C} \setminus \Gamma(S_{t-1})$. Based on Equation (3.30), we finally obtain:

$$\sum_{S_t} q_{S_t} = \mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i).$$

The proof is finished. □

Lemma 30. *The probability of observing the sampling record $S_1, \dots, S_T$ by Algorithm 1 under Condition 4 is:*

$$(3.31) \quad \mathbb{P}(S_1, \dots, S_T) = q_{S_T} \prod_{t=1}^{T-1} p_{S_t}$$

PROOF. Given sampling record $S_1, \dots, S_{t-1}$, the conditional probability of observing the next record S_t, S'_t can be expanded based on Lemma 26,

$$\frac{\mathbb{P}(S_t|S_1, \dots, S_{t-1})}{\mathbb{P}(S'_t|S_1, \dots, S_{t-1})} = \frac{\mathbb{P}(S_t| \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)}{\mathbb{P}(S'_t| \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)}$$

Based on Equation (3.30), we can simplify the RHS of the above ratio equality and obtain:

$$\frac{\mathbb{P}(S_t|S_1, \dots, S_{t-1})}{\mathbb{P}(S'_t|S_1, \dots, S_{t-1})} = \frac{q_{S_t}}{q_{S'_t}}$$

Because of $\sum_{S_t} \mathbb{P}(S_t|S_1, \dots, S_{t-1}) = 1$ and Equation (3.30), we can get:

$$(3.32) \quad \mathbb{P}(S_t|S_1, \dots, S_{t-1}) = \frac{q_{S_t}}{\mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)}$$

We finally compute the probability of observing the sampling record $S_1 \dots, S_T$ by:

$$\begin{aligned} \mathbb{P}(S_1 \dots, S_T) &= \mathbb{P}(S_1) \prod_{t=2}^T \mathbb{P}(S_t|S_1, \dots, S_{t-1}) && \text{By Chain rule} \\ &= q_{S_1} \prod_{t=2}^T \frac{q_{S_t}}{\mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)} && \text{By Equation (3.32)} \\ &= q_{S_T} \prod_{t=2}^T \frac{q_{S_{t-1}}}{\mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)} && \text{Left shift the numerator from } S_t \text{ to } S_{t-1} \\ &= q_{S_T} \prod_{t=1}^{T-1} p_{S_t} && \text{Plugin Lemma 28} \end{aligned}$$

The proof is finished. $\square$

3.8.2. An Upper Bound on Expected Running Time

Suppose the expected number of samplings of constraints c_i is $\mathbb{E}(T_i)$, then the total running time will be:

$$\mathbb{E}(T) \leq \sum_{i=1}^n \mathbb{E}(T_i)$$

Since each random variable has equal status, then the question comes down to the computation of individual T_i 's expectation. Let $S_1, \dots, S_T$ be any record of the algorithm that successfully terminates, and $T_i(S_1, \dots, S_T)$ be the total number of sampling related to constraint c_i throughout this record. Based on Lemma 30, we have:

$$\mathbb{E}(T_i) = \sum_{S_1, \dots, S_T} \mathbb{P}(S_1, \dots, S_T) T_i(S_1, \dots, S_T)$$

By far, we have shown the original proof of our work. We leave the difference between our proof with the existing one in section 3.8.3.

The rest of the computation can be done in the same way as the proof in [Guo et al., 2019]. Thus we cite the necessary intermediate steps in the existing work and finish the proof logic for the coherence of the whole running time analysis.

Lemma ([Guo et al., 2019] Lemma 12). *Let $q_\emptyset$ be a non-zero probability of all the constraints are satisfied. Let $q_{\{c_j\}}$ denote the probability that only constraint c_j is broken and the rest all hold. If $q_\emptyset > 0$, then $\mathbb{E}(T_i) = q_{\{c_j\}}/q_\emptyset$.*

After incorporating our fix, we can conclude the upper bound on the expected running time in Theorem 8.

Theorem 8 ([Guo et al., 2019] Theorem 13). *Under Condition 4, the total number of re-samplings throughout the algorithm is then $\frac{1}{q_0} \sum_{j=1}^L q_{\{c_j\}}$.*

3.8.3. Difference to the Existing Proof

The main difference in the above proof to the existing proof in [Guo et al., 2019, Theorem 13] is that: based on Equation (3.32) and (3.30), we show

$$\mathbb{P}(S_t | S_1, \dots, S_{t-1}) = \mathbb{P}(S_t | \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)$$

In [Guo et al., 2019]’s Equation (9), the first step cannot hold without the above equality.

The original paper uses this result directly without providing enough justification.

3.9. Constrained MRF Model

3.9.1. Single Variable Form of Constrained MRF

Here we provide an example of transforming MRF with pairwise and single potential functions into a single potential form by introducing extra variables. Given random variables X_1, X_2, X_3 , we have the following example MRF model:

$$\begin{aligned} \phi_\theta(x_1, x_2, x_3) &= \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 \\ P_\theta(x) &= \frac{\exp(\phi_\theta(x_1, x_2, x_3))}{Z(\theta)} \end{aligned}$$

In the above formula, we have a cross term $x_1 x_2$. Two Boolean variables can have 4 different assignments in total. Therefore we can construct 4 extra Boolean variables to encode all these assignments. To illustrate, we introduce extra random variables $\hat{X}_{00}$,

X_1	X_2	$\hat{X}_{00}, \hat{X}_{01}, \hat{X}_{10}, \hat{X}_{11}$
0	0	1, 0, 0, 0
0	1	0, 1, 0, 0
1	0	0, 0, 1, 0
1	1	0, 0, 0, 1

Table 3.5. 4 constraints for converting pairwise terms in the potential function into single variable form.

$\hat{X}_{01}, \hat{X}_{10}, \hat{X}_{11}$. We further introduce extra constraints: When $X_1 = 0, X_2 = 0$, the extra variable must take values: $\hat{X}_{00} = 1, \hat{X}_{01} = 0, \hat{X}_{10} = 0, \hat{X}_{11} = 0$. See the rest constraints in Table 3.5.

Then the new potential function, including extended variables and pairwise to single variable constraints $\mathcal{C}$, is reformulated as:

$$\begin{aligned}\hat{\phi}_\theta(x_1, x_2, x_3, \hat{x}_{00}, \hat{x}_{01}, \hat{x}_{10}, \hat{x}_{11}) &= \theta_1 x_1 + \theta_2 x_2 + \theta_3 \hat{x}_{00} + \theta_3 \hat{x}_{01} + \theta_3 \hat{x}_{10} + \theta_3 \hat{x}_{11} \\ P_\theta(x|\mathcal{C}) &= \frac{\exp(\hat{\phi}_\theta(x_1, x_2, x_3, \hat{x}_{00}, \hat{x}_{01}, \hat{x}_{10}, \hat{x}_{11}))}{Z_{\mathcal{C}}(\theta)}\end{aligned}$$

For clarity, the newly added constraints do not impact Condition 4. Since the single variable transformation in the MRFs model originates from [Sang et al., 2005], thus is not considered as our contribution.

3.9.2. Gradient of log-Partition Function $\nabla \log Z_{\mathcal{C}}(\theta)$

We use the Chain rule of the gradient to give a detailed deduction of Equation (3.4).

$$\begin{aligned}\nabla \log Z_{\mathcal{C}}(\theta) &= \frac{\nabla Z_{\mathcal{C}}(\theta)}{Z_{\mathcal{C}}(\theta)} = \frac{1}{Z_{\mathcal{C}}(\theta)} \nabla \sum_{x \in \mathcal{X}} \exp(\phi_\theta(x)) C(x) \\ &= \sum_{x \in \mathcal{X}} \frac{\exp(\phi_\theta(x)) C(x)}{Z_{\mathcal{C}}(\theta)} \nabla \phi_\theta(x) = \sum_{x \in \mathcal{X}} P_\theta(x|\mathcal{C}) \nabla \phi_\theta(x) = \mathbb{E}_{x \sim P_\theta(\tilde{x}|\mathcal{C})} (\nabla \phi_\theta(x).)\end{aligned}$$

The above result shows the gradient of the constrained partition function is equivalent to the expectation of the gradient of the potential function $\nabla\phi_\theta$ over the model’s distribution (*i.e.*, $P_\theta(\tilde{x}|\mathcal{C})$). Therefore, we transform the gradient estimation problem into the problem of sampling from the current MRF model.

3.10. Experiment Settings and Configurations

3.10.1. Implementation Details

Implementation of NELSON. The proposed sampler can be implemented with Numpy, Pytorch, or Jax. We further offer a “batch version” implementation, that draws a batch of samples in parallel on GPU. The batched sampler is useful for those tasks that require a huge number of samples to estimate the gradient with a small approximation error.

In Section 3.4.1, we define vector of assignment $x^t = (x_1^t, \dots, x_n^t)$, where x_i^t is the assignment of variable X_i in the t -th round of Algorithm 1. $x_i^t = 1$ denotes variable X_i takes value 1 (or true). In the batch version, we define the matrix for a batch of assignments. Let b be the batch size, we have

$$x^t = \begin{bmatrix} x_{11}^t & \dots & x_{1n}^t \\ \vdots & \ddots & \vdots \\ x_{b1}^t & \dots & x_{bn}^t \end{bmatrix}$$

In the following part, we provide the detailed computation pipeline for the batch version of the proposed algorithm.

Initialization. The first step is to sample an initial assignment of X from the given the marginal probability vector P :

$$(3.33) \quad x_{li}^1 = \begin{cases} 1 & \text{if } u_{li} > P_i, \\ 0 & \text{otherwise.} \end{cases}, \quad \text{if } 1 \leq i \leq n, 1 \leq l \leq b$$

Here u_{li} is sampled from the uniform distribution over $[0, 1]$.

Check Constraint Satisfaction. The second step extract which constraint is violated. Given an assignment x^t at round $t \geq 1$, tensor W and matrix b , the computation of tensor Z^t is:

$$Z_{lik}^t = \sum_{i=1}^n W_{ikj} x_{lj}^t + b_{lik},$$

The special multiplication between tensor and matrix can be efficiently implemented with the Einstein summation³. Note that $Z_{ljk}^t = 1$ indicates for l -th batched assignment x_l , the k -th literal of j -th clause is true (takes value 1). Next, we compute S_{lj}^t as:

$$S_{lj}^t = 1 - \max_{1 \leq k \leq K} Z_{ljk}^t, \quad \text{for } 1 \leq j \leq L, 1 \leq l \leq b$$

Here $S_{lj}^t = 1$ indicates x_l^t violates j -th clause. We can check $\sum_{l=1}^b \sum_{j=1}^L S_{lj}^t \neq 0$ to see if any clause is violated for the current batch of assignments, which corresponds to $\sum_{i=1}^b C(x_l) = 0$.

Extract Variables in Violated Clauses. We extract all the variables that require resampling based on vector S^t computed from the last step. The vector of the resampling

³https://github.com/dgasmath/opt_einsum

indicator matrix A^t can be computed as:

$$A_{li}^t = \mathbf{1} \left(\sum_{j=1}^L S_{lj}^t V_{ji} \geq 1 \right), \quad \text{for } 1 \leq i \leq n, 1 \leq l \leq b$$

where $\sum_{j=1}^L S_{lj}^t V_{ji} \geq 1$ implies X_{li} requires resampling.

Resample. Given the marginal probability vector P , resample indicator matrix A^t and assignment matrix x^t , we draw a new random sample x^{t+1} .

$$x_{li}^{t+1} = \begin{cases} (1 - A_{li}^t)x_{li}^t + A_{li}^t & \text{if } u_{li} > P_i, \\ (1 - A_{li}^t)x_{li}^t & \text{otherwise.} \end{cases} \quad \text{for } 1 \leq i \leq n, 1 \leq l \leq b$$

where u_{li} is drawn from the uniform distribution in $[0, 1]$.

Since GPUs are more efficient at computing tensor, matrix, and vector operations but are slow at processing for loops. Drawing a batch of samples using the above extended computational pipeline is much faster than using a for loop over the computational pipeline in Section 3.4.1.

The sampler is involved with one hyper-parameter T_{tryout} . NELSON would terminate when it reaches T_{tryout} number of re-samples. This practice is commonly used to handle randomized programs that might run forever in rare cases.

Implementation of Algorithm 2. We first use the Constraints $\mathcal{C}$ and parameters θ^t to build the current NELSON module. Then we draw m samples from NELSON module $\{\tilde{x}^j\}_{j=1}^m$ and draw from dataset randomly m samples $\{x^j\}_{j=1}^m$. Continuing from that point, we compute the potential value from the two sets of inputs, i.e., $\{\phi_\theta(\tilde{x}^j)\}_{j=1}^m$ and $\{\phi_\theta(x^j)\}_{j=1}^m$. Pytorch would be slow if we compute each potential's gradient using a

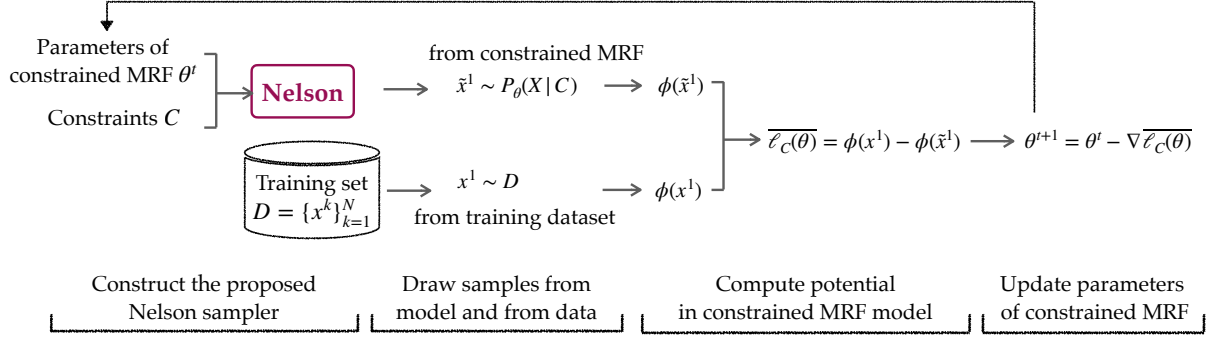


Figure 3.4. Implementation pipeline of the NELSON-CD algorithm with $m = 1$. The proposed NELSON can be efficiently adapted to a Pytorch-based machine learning library and enforces constraint satisfaction during learning.

for-loop. To bypass this problem, we instead compute the following:

$$(3.34) \quad \overline{\ell_C(\theta)} = \frac{1}{m} \sum_{j=1}^m \phi_\theta(x^j) - \frac{1}{m} \sum_{j=1}^m \phi_\theta(\tilde{x}^j).$$

Following that, we call the PyTorch library's gradient function, which computes exactly

$$\nabla \overline{\ell_C(\theta)} = \nabla \left(\frac{1}{m} \sum_{j=1}^m \phi_\theta(x^j) - \frac{1}{m} \sum_{j=1}^m \phi_\theta(\tilde{x}^j) \right) = \frac{1}{m} \sum_{j=1}^m \nabla \phi_\theta(x^j) - \frac{1}{m} \sum_{j=1}^m \nabla \phi_\theta(\tilde{x}^j)$$

Note that $\nabla \overline{\ell_C(\theta)}$ recovers the result in Equation (3.4). Finally, we update the parameters θ . The proposed NELSON module and the neural network are computed on the same GPU device. This allows us to exploit the parallel computing power of modern GPUs and remove time for the data transfer from CPU to GPU or vice versa. See Figure 3.4 for a visualized overview of the implementation with Pytorch.

3.10.2. Learn Random K-SAT Solutions with Preference

Task Definition. We are given a training set $\mathcal{D}$ containing some preferred assignments $\mathcal{D} = \{x^j\}_{j=1}^N$ for the corresponding CNF formula $c_1 \wedge \dots \wedge c_L$. We require the CNF formula to be true. This means, by the definition of CNF formulas, that every clause has to be satisfied. These clauses become our set of constraints. Under the constrained MRF model, the learning task is to maximize the log-likelihood of the assignments seen in the training set $\mathcal{D}$. The inference task is to generate valid solutions from the learned model’s distribution [Dodaro and Previti, 2019; Rosa et al., 2011].

Dataset. We denote the Boolean variables’ size in K -SAT as the “problem size”. We consider several datasets of different problem sizes generated from CNFGen⁴ [Lauria et al., 2017] random K -SAT functions. K is fixed as 5; the number of variables and clauses are kept the same, ranging from 10 to 1500. We generate 100 different CNF formulas for every problem size. To generate the training set $\mathcal{D}$, we use the Glucose4 solver from PySAT⁵ library [Ignatiev et al., 2018] to generate 200 assignments randomly as the preferred assignments for every formula.

It should be pointed out that we don’t consider datasets like SATLIB and SAT competitions. It is mainly because these datasets are hard instances with a much larger input space but a limited number of solutions. NELSON would generally take exponential time to find these solutions, just like finding needles in a haystack. The other reasons are that using neural networks to learn these limited assignments is straightforward since we can simply hard-wire the network to memorize all the valid assignments. The main purpose

⁴<https://github.com/MassimoLauria/cnfgen>

⁵<https://pysathq.github.io/>

of this work is to let a constrained MRF learn a representation for the underlying preference pattern, not create a neural solver that can generate valid assignments for any CNF formula. Thus, we conform to the settings of the easy formula where obtaining valid solutions is easy.

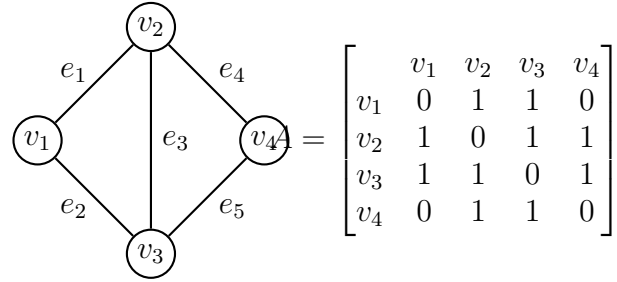
3.10.3. Learn Sink-Free Orientation in Undirected Graphs

Task Definition In graph theory, a *sink-free* orientation of an undirected graph is a choice of orientation for each edge such that every vertex has at least one outgoing edge [Cohn et al., 2002]. It has wide applications in robotics routing and IoT network configuration [Takahashi et al., 2009]. The Constraints for this problem are that every vertex has at least one outgoing edge after orientation. As stated in [Guo et al., 2019], these constraints satisfy Condition 4.

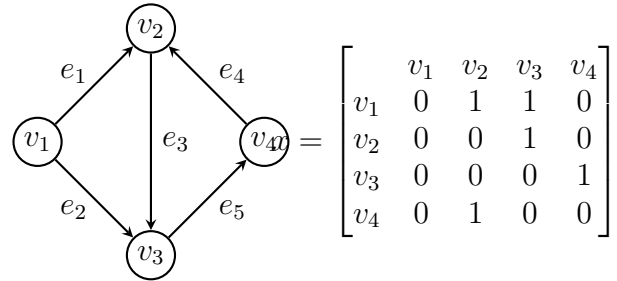
See Figure 3.5 for an example graph and one possible sink-free edge orientation. We define binary variables $X_1, \dots, X_m$, and associate variable X_i to edge e_i for $1 \leq i \leq m$. Variable X_i takes value 1 if the edge orientation is $v_i \rightarrow v_j$ where $i < j$. Otherwise, X_i takes value 0. The constraints are:

$$\mathcal{C} = (X_1 \vee X_2) \wedge (\neg X_1 \vee X_3 \vee \neg X_4) \wedge (\neg X_2 \vee \neg X_3 \vee X_5) \wedge (X_4 \vee \neg X_5)$$

where the single constraint $c_1 = (X_1 \vee X_2)$ corresponds to vertex v_1 , constraint $c_2 = (\neg X_1 \vee X_3 \vee \neg X_4)$ corresponds to vertex v_2 , constraint $c_3 = (\neg X_2 \vee \neg X_3 \vee X_5)$ corresponds to vertex v_3 , and constraint $c_4 = (X_4 \vee \neg X_5)$ corresponds to vertex v_4 . The orientation assignment matrix x shown in Figure 3.5(b) implies: $X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 0, X_5 = 1$.



(a) An undirected graph G with its adjacency matrix A



(b) An orientation of the edges and the orientation matrix x

Figure 3.5. **(a)** An un-directed graph $G(V, E)$ where the vertices are $V = \{v_1, v_2, v_3, v_4\}$ and the un-directed edges are $E = \{e_1 = (v_1, v_2), e_2 = (v_1, v_3), e_3 = (v_2, v_3), e_4 = (v_2, v_4), e_5 = (v_3, v_4)\}$. **(b)** A possible sink-free orientation of the edges in the graph and its matrix representation x , where every vertex has at least one outgoing edge.

Notations. Let graph $G(V, E)$ be an un-directed graph; its adjacency matrix A that represents graph connectivity is:

$$(3.35) \quad A_{ij} = \begin{cases} 1 & \text{If } (v_i, v_j) \in E \\ 0 & \text{otherwise} \end{cases}$$

A possible assignment for the orientation of every edge can be represented as a matrix $x \in \{0, 1\}^{|V| \times |V|}$:

$$(3.36) \quad x_{ij} = \begin{cases} 1 & \text{if the edge orientation is } v_i \rightarrow v_j \\ 0 & \text{otherwise} \end{cases}$$

In the constrained MRF model defined in Eq. (3.6), the potential function of one orientation of all edges is

$$\phi_\theta(x) = \sum_{i=1}^{|V|} \sum_{j=1}^{|V|} \theta_{ij} A_{ij} x_{ij}$$

A single constraint for vertex v_k is $c_k(x) = \mathbf{1} \left(\sum_{j=1}^n A_{k,j} x_{k,j} = 1 \right)$. If there is no ongoing edge of vertex v_k . The constraint function $C(x)$ is defined as: $\prod_{i=1}^n c_k(x)$. In Algorithm 1 step 1, edge (v_i, v_j) will pick the orientation $v_i \rightarrow v_j$ with probability:

$$\frac{\exp(\theta_{ij} A_{ij} x_{ij})}{\exp(\theta_{ji} A_{ji} x_{ji}) + \exp(\theta_{ij} A_{ij} x_{ij})}$$

Dataset. We use the NetworkX⁶ package to generate random Erdos Renyi graph with edge probability 0.55. The problem size refers to the number of vertices in the graph, we range the problem size from 10 to 100. For each problem size, we generate 100 different random undirected graphs. We then convert the graph into CNF form using the above edge-variable conversion rule. Afterward, we follow the same processing steps as the previous problem that learn preferential solution distribution for random K-SAT.

⁶<https://networkx.org/>

3.10.4. Learn Vehicle Delivery Routes

Given a set of locations to visit, the task is to generate a sequence to visit these locations in which each location is visited once and only once and the sequence closely resembles the trend presented in the training data. The training data are such routes collected in the past. The dataset is constructed from TSPLIB, which consists of 29 cities in Bavaria, Germany. In Figure 3.3, we see NELSON can obtain samples of this delivery problem highly efficiently.

A possible travel plan can be represented as a matrix $x \in \{0, 1\}^{|V| \times |V|}$:

$$(3.37) \quad x_{ij} = \begin{cases} 1 & \text{if edge } v_i \rightarrow v_j \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

The constraints are that every routing plan should visit every location once and only once.

Similarly, in the constrained MRF model defined in Eq. (3.6), the potential function of the vehicle routing plan is

$$\phi_{\theta}(x) = \sum_{i=1}^{|V|} \sum_{j=1}^{|V|} \theta_{ij} A_{ij} x_{ij}$$

3.10.5. Detailed Baselines Configuration

In terms of sampling-based methods, we consider:

- Gibbs sampler [Carter and Kohn, 1994], a special case of MCMC that is widely used in training MRF models. In each step, the Gibbs algorithm samples one

dimension based on a conditional marginal distribution. We follow this implementation⁷.

- Weighted SAT samplers, including WAPS⁸ [Gupta et al., 2019], WeightGen⁹, [Chakraborty et al., 2014] and XOR sampler¹⁰ [Ermon et al., 2013; Ding and Xue, 2021].
- Uniform SAT samplers, including UniGen¹¹, [Soos et al., 2020], QuickSampler¹², [Dutra et al., 2018], CMSGen¹³ [Golia et al., 2021] and KUS¹⁴ [Sharma et al., 2018].

Currently, there are only GPU-based SAT solvers [Prevot et al., 2021; Mahmoud, 2022] and model counters [Fichte et al., 2019], GPU-based SAT samplers are not available by far.

3.10.6. Detailed Definition of Evaluation Metrics

In terms of evaluation metrics, we consider

- Training time per epoch. The average time for the whole learning method to finish one epoch with each sampler.
- Validness. The learned model is adopted to generate assignments and we evaluate the percentage of generated assignments that satisfy the constraints.

⁷<https://github.com/Fading0924/BPChain-CD/blob/master/mrf.py>

⁸<https://github.com/meelgroup/waps>

⁹<https://bitbucket.org/kuldeepmeel/weightgen/src/master/>

¹⁰<https://cs.stanford.edu/~ermon/code/srcPAWS.zip>

¹¹<https://github.com/meelgroup/unigen>

¹²<https://github.com/RafaelTupynamba/quicksampler>

¹³<https://github.com/meelgroup/cmsgen>

¹⁴<https://github.com/meelgroup/KUS>

- Mean Averaged Precision (MAP@10). This is a ranking-oriented metric that can evaluate the closeness of the learned MRF distribution to the goal distribution. If the model learns the goal distribution in the training set, then it would assign a higher potential value to those assignments in the training set than all the rest unseen assignments. Based on this principle, we randomly pick two sets of inputs in those valid assignments: seen assignments from the training set and unseen assignments that are randomly generated. We use the value of factor potential $\phi(x)$ to rank those assignments in ascending order. Next, we check how many preferred solutions can fall into the Top-10 by computing the following

$$\text{MAP@10} = \sum_{k=1}^{10} \frac{\#\text{preferred assignments among top-}k}{k}$$

- log-likelihood of assignments in the training set $\mathcal{D}$. The model that attains the highest log-likelihood learns the closest distribution to the training set. Specifically, given a training set $\mathcal{D} = \{x^k\}_{k=1}^N$ and parameters θ , the log-likelihood value is:

$$(3.38) \quad \frac{1}{N} \sum_{k=1}^N \log P_{\theta}(X = x^k | \mathcal{C}) = \frac{1}{N} \sum_{k=1}^N \phi_{\theta}(x^k) - \log Z_{\mathcal{C}}(\theta)$$

We use the ACE algorithm to compute the approximated value of $\log Z_{\mathcal{C}}(\theta)$ ¹⁵.

- Approximation Error of $\nabla \log Z_{\mathcal{C}}(\theta)$, that is the L_1 distance between the exact gradient of $\log Z_{\mathcal{C}}(\theta)$ in Eq. (3.4) and the empirical gradient from the sampler. For small problem sizes, we enumerate all $x \in \mathcal{X}$ to get the exact gradient and

¹⁵<http://reasoning.cs.ucla.edu/ace/moreInformation.html>

draw samples $\{\tilde{x}^j\}_{j=1}^m$ with $m = 2000$ from every sampler for approximation.

$$\left| \underbrace{\sum_{x \in \mathcal{X}} \frac{\exp\left(\sum_{j=1}^n \theta_j x_j\right) C(x)}{Z_C(\theta)} x_i}_{\text{Exact gradient term}} - \underbrace{\sum_{j=1}^m \tilde{x}_i^j}_{\text{Estimated gradient with sampler}} \right|$$

For fixed parameter θ , the best sampler would attain the smallest approximation error.

3.10.7. Hyper-parameter Settings

In the implementation of NELSON, we set the maximum tryout of resampling as $T_{tryout} = 1000$ for all the experiments and all the datasets.

For the hyper-parameters used in learning the constrained MRF, we set the number of samples from the model to be $m = 200$, the learning rate η is configured as 0.1 and the total learning iterations are $T_{\max} = 1000$.

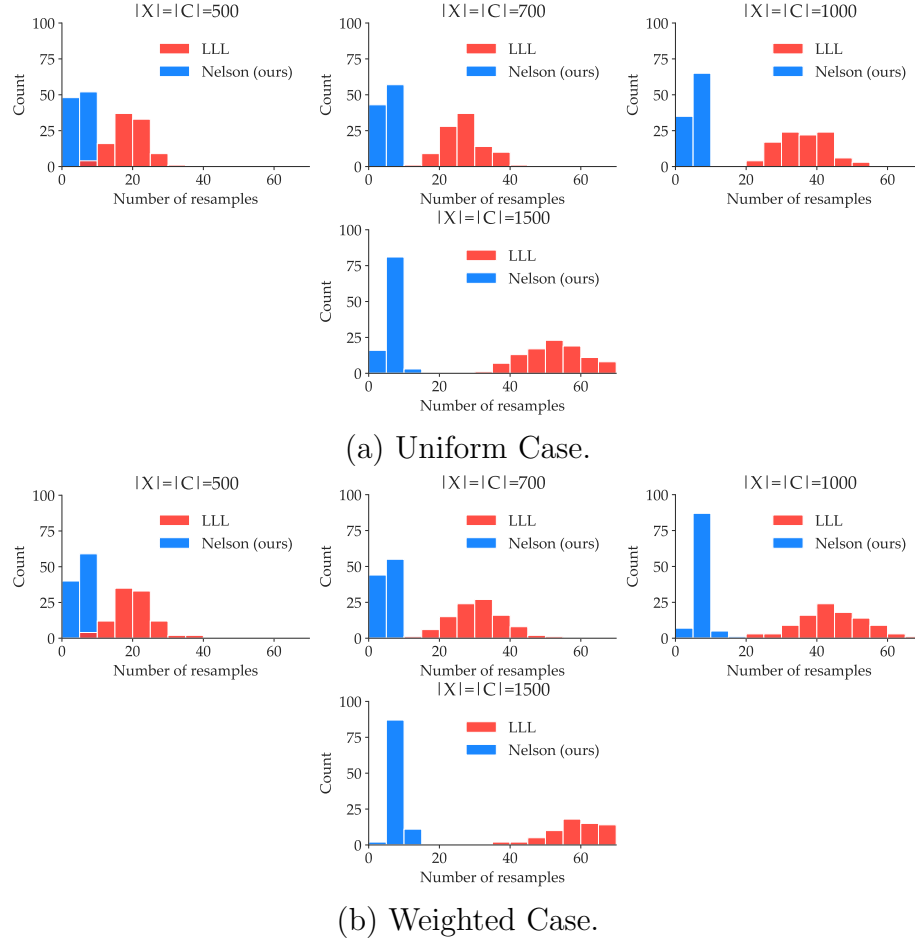


Figure 3.6. The distribution of resampling steps in the NELSON and Algorithmic-LLL [Moser and Tardos, 2010]. Both of them get a valid sample within $T_{tryouts}$. NELSON takes much fewer resamples than Algorithmic-LLL because it resamples all the violated clauses at every iteration while Algorithmic-LLL only resamples one of them.

CHAPTER 4

VarFA: A Variational Factor Analysis Framework for Efficient Bayesian Learning Analytics

Overview

In the last chapter of this thesis, we propose VarFA, a variational inference factor analysis framework that extends existing factor analysis models for educational data mining to efficiently output uncertainty estimation in the model’s estimated factors. Such uncertainty information is useful, for example, for an adaptive testing scenario, where additional tests can be administered if the model is not quite certain about a students’ skill level estimation. Traditional Bayesian inference methods that produce such uncertainty information are computationally expensive and do not scale to large data sets. VarFA utilizes variational inference which makes it possible to efficiently perform Bayesian inference even on very large data sets. We use the sparse factor analysis model as a case study and demonstrate the efficacy of VarFA on both synthetic and real data sets. VarFA is also very general and can be applied to a wide array of factor analysis models. Code and instructions to reproduce results in this chapter are available from <https://tinyurl.com/tvm4332>.

4.1. Introduction

A core task for many practical educational systems is *student modeling*, i.e., estimating students' mastery level on a set of skills or knowledge components (KC) [VanLehn \[1988\]](#); [Chrysafiadi and Virvou \[2013\]](#). Such estimates allow in-depth understanding of students' learning status and form the foundation for automatic, intelligent learning interventions. For example, intelligent tutoring systems (ITSs) [Shute and Psotka \[1994\]](#); [Graf and Kinshuk \[2012\]](#); [Anderson et al. \[1995\]](#); [Nye et al. \[2014\]](#); [Heffernan and Heffernan \[2014\]](#); [Rafferty et al. \[2015\]](#) rely on knowing the students' skill levels in order to effectively recommend individualized learning curriculum and improve students' learning outcomes. In the big data era, student modeling is usually formulated as an educational data mining (EDM) problem [Romero and Ventura \[2010\]](#); [Baker and Yacef \[2009\]](#); [Merceron and Yacef \[2005\]](#) where an underlying machine learning (ML) model estimates students' skill mastery levels from students' learning records, i.e., their answers to assessment questions.

Many student modeling methods have been proposed in prior literature. A fruitful line of research for student modeling follows the *factor analysis (FA)* approach. FA models usually assume that an unknown, potentially multi-dimensional student parameter, in which each dimension is associated with a certain skill, explains how a student answers questions and is to be estimated. Popular and successful FA models include item response theory (IRT) [van der Linden and Hambleton \[2013\]](#), multi-dimensional IRT [Ackerman \[1994\]](#), learning factor analysis (LFA) [Cen et al. \[2006\]](#), performance factor analysis (PFA) [Pavlik et al. \[2009\]](#), DASH (short for difficulty, ability, and student history) [Lindsey et al. \[2014\]](#); [Mozer and Lindsey \[2016\]](#), DAS3H (short for difficulty, ability, skill and student skill history) [Choffin et al. \[2019\]](#), knowledge tracing machines (KTM) [Vie and](#)

Kashima [2019], and so on. Recently, more complex student models based on deep neural networks (DNN) have also been proposed Piech et al. [2015]; Zhang et al. [2017]; Minn et al. [2018]. Nevertheless, thanks to their simplicity, effectiveness and robustness, FA models remain widely adopted and investigated for practical EDM tasks. Moreover, there is evidence that simple FA models could even outperform DNN models for student modeling in terms of predicting students' answers Wilson et al. [2016]. Because of FA models' competitive performance and its elegant mathematical form, we focus on FA-based student models in this chapter.

Most of the aforementioned FA models compute a single point estimate of skill levels for each student. Often, however, it is not enough to obtain mere point estimates of students' skill levels; knowing the model's uncertainty in its estimation is crucial because it potentially helps improve the model's performance and improve both students' and instructors' experience with educational systems. For example, in ITS, an underlying model can use the uncertainty information in its estimation of students' skill level to automatically decide that its recommendations based on highly uncertain estimations are unreliable and instead notify a human instructor to evaluate the students' performance. This enables collaboration between ITS and instructors to create a more effective learning environment. In adaptive testing systems Chang et al. [2009]; Yan et al. [2016], knowing the uncertainty in model's estimation could help the model intelligently pick the next test items to most effectively reduce its uncertainty about estimated students' skill levels. This will help to potentially reduce the number of items needed to have a confident, accurate estimation of the students' skill mastery level, saving time for both students to take the test and instructors to have a good assessment of the student's skills.

All the above applications require the model to “know what it does not know,” i.e., to quantify the uncertainty of its estimation. Achieving this does not necessarily change the model. rather, we need a different inference algorithm for inferring not only a point estimate of student’s skill level from observed data (i.e., students’ answer records to questions) but also uncertainty in the estimations.

Fortunately, there exist methods that both compute point estimates of students’ skill levels and quantifies the uncertainty of those estimates. These methods usually follow the Bayesian inference paradigm, where each student’s unknown skill levels are treated as random variables drawn from a *posterior distribution*. Thus, one can use the *credible interval* to quantify the model’s uncertainty on the student’s estimated skill level. A classical method for Bayesian inference is Monte Carlo Markov Chain (MCMC) sampling [Gelman et al. \[2013\]](#) which has been widely used in many other disciplines [Gilks et al. \[1995\]](#) other than EDM. In the context of student modeling with FA models, existing works such as [Lan et al. \[2014\]](#); [Fox \[2010\]](#); [Pardos et al. \[2010\]](#); [Merkle \[2011\]](#) have applied MCMC methods to obtain credible intervals.

Unfortunately, classic Bayesian inference methods suffer from extensive computational complexity. For example, each computation step in MCMC involves a time-consuming evaluation of the posterior distribution. Making matters worse, MCMC typically takes many more steps to converge than non-Bayesian inference methods. As a concrete illustration, in [Lan et al. \[2014\]](#), the Bayesian inference method (10 minutes) is about 100 times slower than the other non-Bayesian inference method based on stochastic gradient descent (6 seconds). The high computational cost prevents Bayesian inference methods

from mass-deployment in large-scale, real-time educational systems where timely feedback is critical for learning Driscoll [1994] and tens of thousands of data points need to be processed in seconds or less instead of minutes or hours. It is thus highly desirable to accelerate Bayesian inference so that one can quantify model’s uncertainty in its estimation as efficiently as non-Bayesian inference methods.

Contributions. In this work, we propose VarFA, a novel framework based on *variational inference* (VI) to perform efficient, scalable Bayesian inference for FA models. The key idea is to approximate the true posterior distribution, whose costly computation slows down Bayesian inference, with a *variational distribution*. We will see in Section 4.3 that, with this approximation, we turn Bayesian inference into an optimization problem where we can use the same efficient inference algorithms as in non-Bayesian inference methods. Moreover, the variational distribution is very flexible and we have full control specifying it, allowing us to freely use the latest development in machine learning, e.g., deep neural networks (DNNs), to design the variational distribution that closely approximates the true posterior. Thus, we also regard our work as a first step in applying DNNs to FA models for student modeling, achieving efficient Bayesian inference (enabled by DNNs) without losing interpretability (brought by FA models). We demonstrate the efficacy of our framework on both synthetic and real data sets, showcasing that VarFA substantially accelerates classic Bayesian inference for FA models with no compromise on performance.

The remainder of this chapter is organized as follows. Section 4.2 introduces the problem setup in FA and reviews the important FA models and their inference methods, in particular Bayesian inference. Section 4.3 explains our VarFA framework in detail. Section 4.4 presents extensive experimental results that substantiate the claimed advantages

of our framework. Section 4.5 concludes this chapter and discusses possible extensions to VarFA.

4.2. Background and Related Work

We first set up the problem and review related work. Assume we have a data set $\mathbf{Y} \in \mathbb{R}^{N \times Q}$ organized in matrix format where N is the total number of students and Q is the number of questions. This is a binary students' answer record matrix where each entry y_{ij} represents whether student i correctly answered question j . Usually, not all students answer all questions. Thus, $\mathbf{Y}$ contains missing values. We use $\{i, j\} \in \Omega_{\text{obs}}$ to denote entries in $\mathbf{Y}$, i.e., the i -th student's answer record to the j -th question, that are observed.

We are interested in models capable of inferring each i -th student's skill mastery level that can accurately predict the student's answers given the above data. These models are often evaluated on the prediction accuracy and whether the inferred student skill mastery levels are easily interpretable and educationally meaningful. We now review factor analysis models (FA), one of the most widely adopted and successful methodologies for the student modeling task.

4.2.1. Factor Analysis For Student Modeling

One of the earliest FA model for student modeling is based on the item response theory (IRT) [van der Linden and Hambleton \[2013\]](#). It usually has the following form

$$(4.1) \quad \mathbb{P}(y_{ij} = 1) = \sigma(c_i + \mu_j),$$

which assumes that each student's answer y_{ij} is independently Bernoulli distributed. The above formula says that the students' answers can be explained by an unknown scalar student skill level factor c_i for each student i and an unknown scalar question difficulty level factor μ_j for each question j . $\sigma(\cdot)$ is a Sigmoid activation function, i.e., $\sigma(x) = 1/(1 + \exp(-x))$. The multi-dimensional IRT model (MIRT) [Ackerman \[1994\]](#) extends IRT by using a multi-dimensional vector to represent the student skill levels. More recently, [Lan et al. \[2014\]](#) proposed sparse factor analysis model (SPARFA) which extends MIRT by imposing additional assumptions, resulting in improved interpretations of the inferred factors.

Other FA models seek to improve student modeling performance by cleverly incorporate additional auxiliary information. For example, the additive factor model (AFM) [Cen et al. \[2006\]](#) incorporates students' accumulative correct answers for a question and the skill tags associated with each question:

$$(4.2) \quad \mathbb{P}(y_{ij} = 1) = \sigma \left(c_i + \sum_{k=1}^{\kappa} (q_{kj}\beta_k + q_{kj}\rho_k b_{ik}) \right),$$

where κ is the total number of skills in the data, b_{ik} is the i -th student's total number of correct answers for skill k and $q_{kj} \in \{0, 1\}$ indicates whether the skill k is associated with question j . In particular, q_{kj} 's form matrix $\mathbf{Q} \in \mathbb{R}^{K \times Q}$ which is commonly known as the Q-matrix in literature [Tatsuoka \[1983\]](#). The unknown, to-be-inferred factors are β_k , a scalar difficulty factor for each skill k , and ρ_k , a scalar learning rate of skill k . The performance factor model (PFM) [Pavlik et al. \[2009\]](#) builds upon AFM that additionally incorporate the total number of a student's incorrect answers to questions associated with a skill. The instructor factor model (IFM) [Chi et al. \[2011\]](#) further builds on PFM

to incorporate the prior knowledge of whether a student has already mastered a skill. More recently, [Vie and Kashima \[2019\]](#) introduces knowledge tracing machines (KTM) that could flexibly incorporate a number of auxiliary information mentioned above, thus generalizing AFM, PFM and IFM.

Another way to improve student modeling performance is to utilize the so-called *memory*, i.e., using students' historic action data over time. For example, [Lindsey et al. \[2014\]](#); [Mozer and Lindsey \[2016\]](#) proposed DASH (short for difficulty, ability, and student history) that incorporates a student's total number of correct answers and total number of attempts for a question in a given time window

$$(4.3) \quad \mathbb{P}(y_{ij} = 1) = \sigma \left(c_i - \mu_j + \sum_{w=0}^{W-1} \left(\theta_{2w+1} \log(1 + \rho_{ijw}) - \theta_{2w+2} \log(1 + \gamma_{ijw}) \right) \right)$$

where W is the length of the time windows (e.g., number of days), ρ_{ijw} and γ_{ijw} are the i -th student's total number of correct answers and total number of attempts for question j at time w , respectively. θ 's are parameters that captures the effect of correct answers and total attempts. More recently, DAS3H [Choffin et al. \[2019\]](#) extends DASH to further include skill tags associated with each question which essentially amounts to adding, within the last summation term in Eq. 4.3, another summation over the number of skills.

4.2.1.1. A general formulation for FA models. Although the above FA models differ in their formulae, modeling assumptions and the available auxiliary data used, we argue that the aforementioned FA models can be unified into a canonical formulation below

$$(4.4) \quad \mathbb{P}(y_{ij} = 1) = \sigma(\mathbf{c}_i^\top \mathbf{m}_j + \mu_j),$$

where $\mathbf{c}_i \in \mathbb{R}^K$, $\mathbf{m}_j \in \mathbb{R}^K$ and $\mu_j \in \mathbb{R}$ are factors whose dimension, interpretations and subscript indices depend on the specific instantiations of the FA model. To illustrate that Eq. 4.4 subsumes the FA models mentioned above, we demonstrate, as examples, how to turn SPARFA and AFM into the form in Eq. 4.4 and how to reinterpret the parameters in the reformulation. The equivalence between Eq. 4.4 and the original SPARFA formula is immediate and we can interpret the parameters in Eq. 4.4 as follows to recover SPARFA: K is the number of *latent skills* that group the actual skill tags in the data set into meaningful coarse clusters; $\mathbf{c}_i$ represents the i -th student's skill level on the latent skills; $\mathbf{m}_j$ represents the strength of association of the j -th question with the latent skills which is assumed to be nonnegative and sparse for improved interpretation; and μ_j represents the difficulty of question j . All three factors in SPARFA are assumed to be unknown.

Similarly, for AFM, we can perform the following change of variables and indices to obtain Eq. 4.4. We first change the indices $\mathbf{m}_j$ to $\mathbf{m}_{ij}$ and μ_j to μ_i , and remove the index in $\mathbf{c}_i$ to $\mathbf{c}$. Then, we simply set $\mathbf{c} = [\beta_1, \dots, \beta_\kappa, \rho_1, \dots, \rho_\kappa]^\top$, $\mathbf{m}_{ij} = [q_{1j}, \dots, q_{\kappa j}, q_{1j}b_{ik}, \dots, q_{\kappa j}b_{ik}]^\top$ and $\mu_i = \alpha_i$ to obtain Eq. 4.4. We can interpret the parameters in the reformulation as follows to recover AFM: $\kappa = K/2$ is the total number of skill tags in the data; $\mathbf{c}$ represents the unknown skill information including skill difficulty and learning rate; $\mathbf{m}_{ij}$ summarizes known auxiliary information for the i -th student and j -th question; μ_i represents the unknown i -th student's skill mastery level. The other FA models can be cast into Eq. 4.4 in a similar fashion. Note that, if the observed data $\mathbf{Y}$ is not binary but rather continuous or categorical, we can simply use a Gaussian or categorical distribution for the observed data and change the Sigmoid activation $\sigma(\cdot)$ to some other activation functions accordingly. In

this way, we still retain the general FA model in Eq. 4.4. We will use this canonical form to facilitate discussions in the rest of this chapter.

4.2.2. Inference Methods for FA Models

We first introduce the inference objective and briefly review maximum log likelihood estimation method. Then, we review existing works that uses Bayesian inference for FA and highlight their high computational complexity, paving the way to VarFA, our proposed efficient Bayesian inference framework based on variational inference.

The factors in Eq. 4.4 may contain known factors that need no inference. Therefore, for convenience of notation, let $[\nu, \theta, \psi]$ be a partition of the factors $\mathbf{c}_i$, $\mathbf{m}_j$ and μ_j where ν contains the unknown students' skill level factors to be estimated, θ contains the remaining unknown factors and ψ contains the known factors. For example, for AFM, $\nu = \{\mu_1, \dots, \mu_N\}$, $\theta = \mathbf{c}$ and $\psi = \{\mathbf{m}_{11}, \dots, \mathbf{m}_{NQ}\}$. For SPARFA, $\nu = \{\mathbf{c}_1, \dots, \mathbf{c}_N\}$, $\theta = \{\mathbf{m}_1, \dots, \mathbf{m}_Q, \mu_1, \dots, \mu_Q\}$ and $\psi = \emptyset$. Further, let the subscripts for these partitions indicate the corresponding latent factors in FA; i.e., for SPARFA, $\theta_i = [\mathbf{m}_i, \mu_i]$ and $\nu_i = \mathbf{c}_i$. Since ψ summarizes known factors, we will omit it from the mathematical expositions in the remainder of the chapter.

The inference objective in FA is then to obtain a good estimate of the unknown factors, represented by θ and ν , with respect to some loss function $\mathcal{L}$, usually the marginal data log likelihood. There are two ways to infer the unknown factors.

4.2.2.1. Maximum Likelihood Estimation. The first way is through the maximum likelihood estimate (MLE) which obtains a point estimate of the unknown factors

$$(4.5) \quad \hat{\theta}, \hat{\nu} = \underset{\theta, \nu}{\operatorname{argmin}} \left(- \sum_{i, j \in \Omega_{\text{obs}}} \log p(y_{ij}; \theta, \nu) \right) + \lambda \mathcal{R}(\theta, \nu).$$

Recall that Ω_{obs} indicates which student i has provided an answer to question j . $\mathcal{R}(\theta, \nu)$ is a regularization term, e.g., ℓ_2 regularization on the factors and λ is a hyper-parameter that controls the strength of the regularization. Most of the works in FA use this method of inference [Cen et al. \[2006\]](#); [Pavlik et al. \[2009\]](#); [Chi et al. \[2011\]](#); [Vie and Kashima \[2019\]](#); [Choffin et al. \[2019\]](#); [Lindsey et al. \[2014\]](#); [Mozer and Lindsey \[2016\]](#). The advantage of MLE is that the above optimization objective allows the use of fast inference algorithms, in particular stochastic gradient descent (SGD) and its many variants, making the inference highly efficient.

4.2.2.2. Maximum A Posteriori Estimation. The second way is through maximum a posteriori (MAP) estimation through Bayesian inference. This is the focus of this chapter. Recall that we wish to obtain not only a point estimate but also credible interval, which MAP allows while MLE does not.

Bayesian inference methods treat the unknown parameters ν and θ as random variables drawn from some distribution. We are thus interested in inferring the *posterior* distribution of ν and θ . Using the Bayes rule, we have that

$$(4.6) \quad p(\nu | \mathbf{Y}, \theta) = \frac{p(\mathbf{Y}, \nu, \theta)}{p(\mathbf{Y})}$$

$$(4.7) \quad = \frac{p(\mathbf{Y} | \nu, \theta) p(\nu) p(\theta)}{\iint p(\mathbf{Y} | \nu, \theta) p(\nu) p(\theta) d\nu d\theta}.$$

We can similarly derive the posterior for the other unknown factor θ , although in this chapter we focus on Bayesian inference for the unknown student skill level parameter ν . In Eq. 4.6 to Eq. 4.7 we have applied standard Bayes rule. Note that, because ψ contains known factors and thus does not enter the Bayes rule equation, we have omitted it from the above equations. Once we estimate the posterior distribution for ν from data, we can obtain both point estimates and credible intervals by respectively taking the mean and the standard deviation of the posterior distributions. A number of prior works have proposed to use Bayesian inference for factor analysis, mostly developed for the IRT model Fox and Glas [2001]; Mislevy [1986]. More recently, Bayesian methods are developed for more complex models. For example, Lan et al. [2014] proposed SPARFA-B, a specialized MCMC algorithm that accounts for SPARFA's sparsity and nonnegativity assumptions.

However, Bayesian inference suffers from high computational cost despite its desired capability to quantify uncertainty. The challenge stems from the difficulty to evaluate the denominator in the posterior distribution in Eq. 4.6 and 4.7 which involves an integration over a potentially multi-dimensional variable. This term usually cannot be computed in close form, thus we usually cannot get an analytical formula for the posterior distribution. Monte Carlo Markov Chain (MCMC) method gets around this difficulty by using analytically tractable proposal distributions to approximate the true posterior and sequentially updating the proposal distribution in a manner reminiscent of gradient descent. MCMC methods also enjoy the theoretical advantage that, when left running for long enough, the proposal distribution is guaranteed to converge to the true posterior distribution Gelman et al. [2013]. However, in practice, it might take very long time for MCMC to converge

to the point that running MCMC is no longer practical when data set is very large. The high computational complexity of MCMC is apparently impractical for educational applications where timely feedback is of critical importance [Driscoll \[1994\]](#).

4.3. VarFA: A Variational Inference Factor Analysis Framework

We are ready to introduce VarFA, our variational inference (VI) factor analysis framework for efficient Bayesian learning analytics. The core idea follows the variational principle, i.e., we use a parametric variational distribution to approximate the true posterior distribution. VarFA is highly flexible and efficient, making it suitable for large scale Bayesian inference for FA models in the context of educational data mining.

In this current work, we focus on obtaining credible interval for the student skill mastery factor ν as a first step of VarFA because, recall from [Section 4.1](#), this factor is often of more practical interest than the other unknown factors. Therefore, currently VarFA is a hybrid MAP and MLE inference method: we perform VI on the unknown student factor ν and MLE on all other unknown factors θ . We consider this hybrid nature of VarFA in its current formulation as a novel feature because classic MLE or MAP estimation are not capable of performing Bayesian inference on a subset of the unknown factors. Extension to VarFA to full Bayesian inference for all unknown factors is part of an ongoing research; see [4.5](#) for more discussions.

4.3.1. VarFA Details

Now, we explain in detail how to apply variational inference for FA models for efficient Bayesian inference. Because the posterior distribution is intractable to compute (recall

Eq. 4.6 and 4.7 and the related discussion), we approximate the true posterior distribution for ν with a parametric variational distribution

$$(4.8) \quad p(\nu|\mathbf{Y}, \theta) \approx q_\phi(\nu|\mathbf{Y}) = \prod_{i=1}^N q_\phi(\nu_i|\mathbf{y}_i),$$

where ϕ is a collection of learnable parameters that parametrize the variational distribution and $\mathbf{y}_i$ is all the answer records by student i . Notably, we have removed the dependency of the variational distribution on ψ and θ so that the variational distribution is solely controlled by the variational parameter ϕ . Thus, the design of the variational distribution is highly flexible. All we need to do is to specify a class of distributions and design a function parametrized by ϕ to output the parameters of q_ϕ . Common in prior literature is to use a Gaussian with diagonal covariance for q_ϕ :

$$(4.9) \quad q_\phi(\nu|\mathbf{y}_i) = \mathcal{N}(\mathbf{u}_j, \text{diag}(\mathbf{v}_j)),$$

where its mean and variance $[\mathbf{u}_j^\top, \mathbf{v}_j^\top]^\top = f_\phi(\mathbf{y}_i)$. We can use arbitrarily complex functions such as a deep neural network for f_ϕ as long as they are differentiable; more details in Section 4.3.2. With the above approximation, Bayesian inference turns into an optimization problem under the variational principle, where we now optimize a lower bound, known as the evidence lower bound (ELBO) Bishop [2006], of the marginal data log likelihood. We derive a novel ELBO objective for variational inference applied to FA models in the EDM setting, summarized in the proposition below.

Proposition 1. *In VarFA, the ELBO objective for an FA model in the form of Eq. 4.4 is*

$$(4.10) \quad \mathcal{L}_{\text{ELBO}}(\phi, \theta) = \sum_{i,j \in \Omega_{\text{obs}}} \left(-D_{\text{KL}}[q_{\phi}(\nu_i | \mathbf{y}_i) \| p(\nu_i)] + \mathbb{E}_{\nu_i \sim q_{\phi}(\nu_i | \mathbf{y}_i)} [\log p_{\theta_j}(y_{ij} | \nu_i)] \right)$$

where D_{KL} is the Kullback–Leibler (KL) divergence [MacKay and Mac Kay \[2003\]](#) between two distributions.

PROOF. We start with the marginal data log likelihood which is the objective we want to maximize and introduce the random variables ν_i ’s:

$$\begin{aligned} \log p(\mathbf{Y}) &= \sum_{i,j \in \Omega_{\text{obs}}} \log p_{\theta_j}(y_{ij}) \\ &= \sum_{i,j \in \Omega_{\text{obs}}} \log \int p_{\theta_j}(y_{ij}, \nu_j) d\nu_i \\ &= \sum_{i,j \in \Omega_{\text{obs}}} \log \int q_{\phi}(\nu_i | \mathbf{y}_i) \frac{p_{\theta_j}(y_{ij}, \nu_j)}{q_{\phi}(\nu_i | \mathbf{y}_i)} d\nu_i \\ &\stackrel{(i)}{\geq} \sum_{i,j \in \Omega_{\text{obs}}} \mathbb{E}_{\nu_i \sim q_{\phi}(\nu_i | \mathbf{y}_i)} \left[\log \frac{p_{\theta_j}(y_{ij}, \nu_j)}{q_{\phi}(\nu_i | \mathbf{y}_i)} \right] \\ &\geq \sum_{i,j \in \Omega_{\text{obs}}} -\mathbb{E}_{\nu_i} \left[\log \frac{q_{\phi}(\nu_i | \mathbf{y}_i)}{p(\nu_j)} \right] + \mathbb{E}_{\nu_i} [\log p_{\theta_j}(y_{ij} | \nu_j)] \\ &= \mathcal{L}_{\text{ELBO}}(\phi, \theta) \end{aligned}$$

where step (i) follows from Jensen’s inequality [Jensen et al. \[1906\]](#). □

The ELBO objective differs from those used in existing literature in that, in our case, because the data matrix is only partially observed, the summation is only over

$\{i, j\} \in \Omega_{\text{obs}}$, i.e., the observed entries in the data matrix. In contrast, in prior literature, the summation in the ELBO objective is over all all entries in the data matrix.

We form the following optimization objective to estimate ϕ and θ :

$$(4.11) \quad \hat{\theta}, \hat{\phi} = \underset{\theta, \phi}{\operatorname{argmin}} -\mathcal{L}_{\text{ELBO}}(\phi, \theta) + \lambda \mathcal{R}(\theta),$$

where $\mathcal{R}(\theta)$ is a regularization term. That is, we perform VI on the student factor ν and MLE inference on the remaining factors θ . More explicitly, to perform VI on ν , we simply compute the mean and standard deviation of q_ϕ using f_ϕ with the learnt variational parameters ϕ .

4.3.2. Why is VarFA efficient?

VarFA supports efficient stochastic optimization. By formulating Bayesian inference as an optimization problem via the ELBO objective, VarFA allows efficient optimization algorithms such as SGD, with a few additional tricks. In particular, we require all terms in $\mathcal{L}(\theta, \phi)$ to be differentiable with respect to θ and ϕ which enable gradient computation necessary for SGD. This requirement is easily satisfied with a few moderate assumptions. Specifically, we let the variational distribution q_ϕ and the prior distribution $p(\nu_i)$ for each i to be Gaussian (See Eq. 4.9; we use a standard Gaussian for the prior distribution $p(\nu_i)$, i.e., $p(\nu_i) = \mathcal{N}(0, \mathbf{I})$) following existing literature [Kingma and Welling \[2014\]](#); [Rezende et al. \[2014\]](#). These two assumptions are not particularly limiting especially given the vast number of successful applications of VI that rely on the same assumptions [Zhao et al. \[2019\]](#); [Mathieu et al. \[2019\]](#); [Siddharth et al. \[2017\]](#); [Creager et al. \[2019\]](#); [Chen et al. \[2018\]](#); [Higgins et al. \[2017\]](#); [Kim and Mnih \[2018\]](#). Thanks to

the Gaussian assumption, for the first term in $\mathcal{L}(\theta, \phi)$, we can easily compute the KL divergence term analytically in closed form. For the second term in $\mathcal{L}(\theta, \phi)$, we can use the so-called reparametrization trick, i.e., $\nu_i = \mathbf{u}_i + \mathbf{v}_i \odot \boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, which allows a low variance estimation of the gradient of the second term in $\mathcal{L}(\theta, \phi)$ with respect to ϕ . See Section 2.3 in Kingma and Welling [2014], Section 2.3 in Kingma et al. [2019] and Section 3 in Rezende et al. [2014] for more details on stochastic gradient computation in VI. As a result, our framework can be efficiently implemented using a number of open source, automatic differentiation packages such as Tensorflow Abadi et al. [2015] and PyTorch Paszke et al. [2019].

VarFA supports amortized inference. Classic Bayesian inference methods such as MCMC infer each parameter ν_i for each student. As the number of students increases, the number of parameters that MCMC needs to infer also increases, which may not be scalable in large-scale data settings. In contrast, unlike classic Bayesian inference methods, VI is *amortized*. It does *not* infer each parameter ν_i . Rather, it estimates a *single* set of variational parameter ϕ responsible for inferring *all* ν_i 's. In this way, once we have trained FA models using VarFA and obtain the variational parameter ϕ , we can easily infer ν_i by simply computing q_ϕ , even for new students, without invoking any additional optimization or inference procedures.

4.3.3. Dealing with missing entries

Note from Eq. 4.10 that the variational distribution q_ϕ for each ν_i is conditioned on $\mathbf{y}_i \in \mathbb{R}^Q$, i.e., an entire row in $\mathbf{Y}$. However, in practice, the data matrix $\mathbf{Y}$ is often only partially observed, i.e., some students only answer a subset of questions. Then,

y_i 's will likely contain missing values which computation cannot be performed on. To work around this issue, we use “zero imputation”, a simple strategy that transforms the missing values to 0, following prior work on applying VI in the context of recommender systems [Liang et al. \[2018\]](#); [Salakhutdinov et al. \[2007\]](#); [Sedhain et al. \[2015\]](#); [Nazabal et al. \[2018\]](#) that demonstrated the effectiveness of this strategy despite its simplicity. We note that there exist more elaborate ways to deal with missing entries, such as designing specialized function f_ϕ for the variational distribution [Ma et al. \[2019\]](#); [Gong et al. \[2019\]](#); [Ma et al. \[2018\]](#). We leave the investigation of more effectively dealing with missing entries to future work.

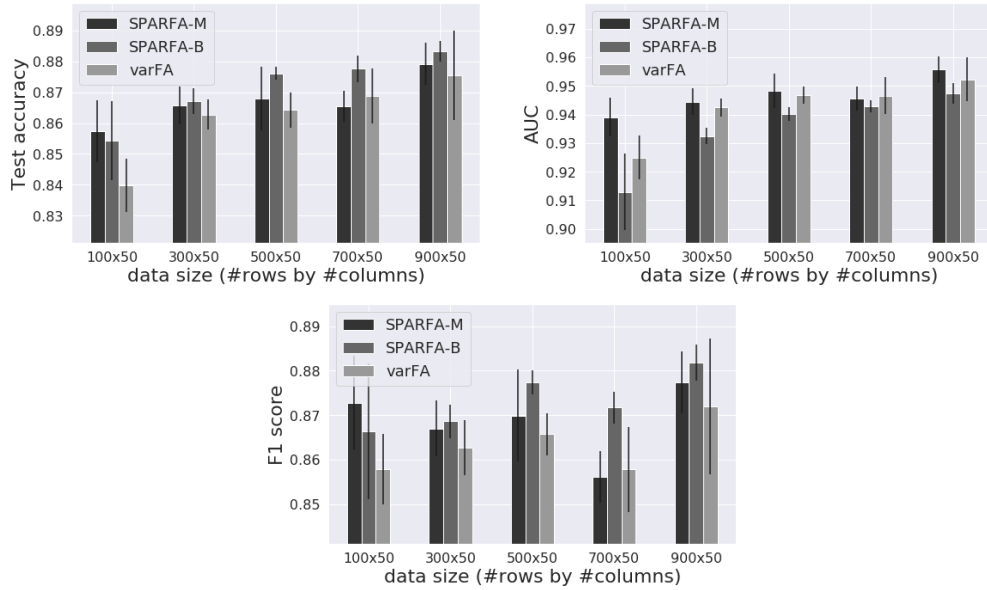


Figure 4.1. Performance of the SPARFA-M, SPARFA-B, and VarFA algorithms on the synthetic data set with different data sizes. Plots from left to right show comparison on accuracy (ACC), area under curve (AUC) and F1 metrics, respectively. Higher is better for all metrics. VarFA performs similarly to SPARFA-M and SPARFA-B.

4.3.4. Remarks

Applicability of VarFA. The VarFA framework is general and flexible and can be applied to a wide array of FA models. The recipe for applying VarFA to an FA model of choice is as follows: 1) formulate the FA model into the canonical formulation as in Eq. 4.4; 2) partition the factors into student factor ν , the remaining unknown factors θ and known factors ψ ; 3) perform VI on ν and MLE estimation on θ , following Section. 4.3.1.

Relation to variational auto-encoders. Our proposed framework can be regarded as a standard variational auto-encoder (VAE) but with the decoder implemented not as a neural network but by the FA model. Because the decoder is constraint to a FA model, it is more interpretable than a neural network.

Relation to other efficient Bayesian factor analysis methods. We acknowledge that VI applied to general FA models have been proposed in existing literature Ghahramani and Beal [2000]; Zhao and Philip [2009]. However, to our knowledge, little prior work have applied VI to educational FA models nor investigated its effectiveness. One concurrent work applied VI to IRT Wu et al. [2020]. Our VarFA framework applies generally to a number of other FA models by following the recipe in the preceding paragraph, including IRT. Thus, our work complements existing literature in providing promising results in applying VI for FA models in the context of educational data mining.

4.4. Experiments

We demonstrate the efficacy of VarFA variational inference framework using the sparse factor analysis model (SPARFA) as the underlying FA model. This choice of SPARFA as the FA model to investigate is motivated by its mathematical generality: it assumes

no auxiliary information is available and all three factors $\mathbf{c}_i$'s, $\mathbf{m}_j$'s and μ_j 's need to be estimated, which makes the inference problem more challenging. Lan et al. [2014] provides two inference algorithms including SPARFA-M (based on MLE) and SPARFA-B (based on MAP). We conduct experiments on both synthetic and real data sets. Using synthetic data sets, we compare VarFA to both SPARFA-M and SPARFA-B. We demonstrate that 1) VarFA predicts students' answers as accurately as SPARFA-M and SPARFA-B; 2) VarFA is almost $100\times$ faster than SPARFA-B. Using real data sets, we compare VarFA to SPARFA-M. We demonstrate that 1) VarFA predicts students' answers more accurately than SPARFA-M; 2) VarFA can output the same insights as SPARFA-M, including point estimate of students' skill levels and questions' associations with skill tags; 3) VarFA can additionally output meaningful uncertainty quantification for student skill levels, which SPARFA-M is incapable of, without sacrifice to computational efficiency. Note that SPARFA-B can also compute uncertainty for small data sets but fails for large data sets due to scalability issues and thus we do not compare to SPARFA-B for real data sets. Specifically for SPARFA, using the notation convention in Section 4.2.2, the question-skill association factors $\mathbf{m}_j$'s and the question difficulty factors μ_j are collected in $\theta = \{\mathbf{m}_1, \dots, \mathbf{m}_Q, \mu_1, \dots, \mu_Q\}$. The student skill level factors $\mathbf{c}_j$'s are collected in $\nu = \{\mathbf{c}_1, \dots, \mathbf{c}_N\}$. Because the factors $\mathbf{m}_j$'s are unknown, the "skills" in SPARFA are *latent* (referred to as "latent skills" in subsequent discussions) and are not attached to any specific interpretation. However, as we will see in Section 4.4.2.4, assuming the skill tags are available as auxiliary information, we can associate the estimated latent skills with the provided skills tags using the same approach proposed in Lan et al. [2014]. For the regularization term in Eq. 4.11, because the factors $\mathbf{m}_j$'s are assumed to be sparse and

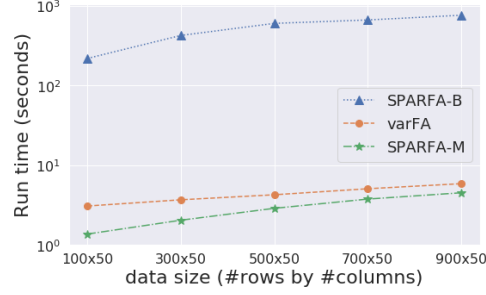


Figure 4.2. Training run time comparison among VarFA, SPARFA-M, and SPARFA-B on synthetic data sets of varying sizes. VarFA performs approximate Bayesian inference almost 100x faster than SPARFA-B and is close to the run time of SPARFA-M. Thus, VarFA enables practical and scalable Bayesian inference for very large data sets.

nonnegative, we use ℓ_1 regularization for $\mathbf{m}_j$'s. When performing MLE for $\mathbf{m}_j$'s, we use the proximal gradient algorithm for optimization problem with nonnegative and sparsity requirements; see Section 3.2 in Lan et al. [2014] for more details. For the other unknown factor μ_j 's in θ , we apply standard ℓ_2 regularization. The code along with instructions to reproduce our experiments can be downloaded from <https://tinyurl.com/tvm4332>.

4.4.1. Synthetic Data Experiments

4.4.1.1. Setup.

Data set generation. We generate data set according to the canonical FA model specified in Eq. 4.4, taking into consideration the additional sparsity and nonnegativity assumptions in SPARFA. Specifically, We sample the factors μ_j 's and $\mathbf{c}_i$'s from i.i.d. standard isotropic Gaussian distributions with variance 1 (or identity matrix for $\mathbf{c}_i$'s) and mean sampled from a uniform distribution in range $[-1, 1]$. To simulate sparse and nonnegative factors $\mathbf{m}_j$'s, we first sample an auxiliary variable $s_{kj} \stackrel{\text{i.i.d.}}{\sim} \text{Bernoulli}(\pi)$, where π controls the sparsity, and then sample $m_{kj} \stackrel{\text{i.i.d.}}{\sim} \text{Exponential}(1)$ when $s_{kj} = 1$ or setting $m_{kj} = 0$

Table 4.1. Summary statistics of pre-processed real data sets.

data set	#students	#questions	%observed
assistent	392	747	12.93%
algebra	697	782	13.02%
bridge	913	1242	11.42%

when $s_{kj} = 0$ for each j and k . In all synthetic data experiments, we use $\pi = 0.3$ and set the true number of latent skills to $K = 5$.

Experimental settings. We vary the size of the data set and use 5 different data matrix sizes: 100×50 , 300×50 , 500×50 , 700×50 and 900×50 . We select a data missing rate of 50%, i.e., we randomly choose 50% of all entries in the data matrix as training set and the rest as test set. Experimental results for each of the above data sizes are averaged over 5 runs where we randomize over the train/test data split. We train SPARFA with both SPARFA-M and VarFA using the Adam optimizer [Kingma and Ba \[2015\]](#) with learning rate = 0.05 for 100 epochs. Regularization hyper-parameters are chosen by grid search for each experiment. For VarFA, we use a simple 3-layer neural network for the function f_ϕ in the variational distribution. We use the hyper-parameter settings for SPARFA-B following Section 4.2 in [Lan et al. \[2014\]](#).

Evaluation metrics. We evaluate the inference algorithms on their ability to recover (predict) the missing entries in the data matrix given the observed entries. We term this criterion “student answer prediction”. Since our data matrix is binary, using prediction accuracy (ACC) alone does not accurately reflect the performance of the inference algorithms under comparison. Thus, we use area under the receiver operating characteristic curve (AUC) and F1 score in addition to ACC, as standard in evaluating binary predictions [Hastie et al. \[2009\]](#).

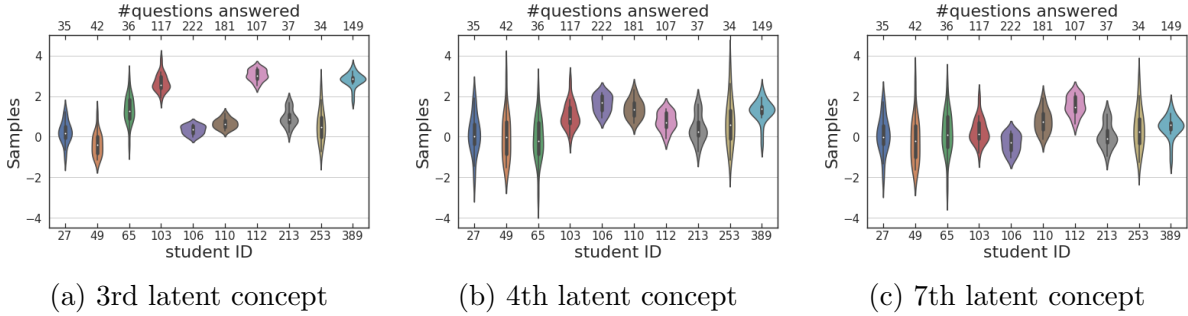


Figure 4.3. Violin plot showing the mean and standard deviation of the estimated skill mastery levels on 10 selected students on the 3rd, 4th and 7th latent skills that VarFA computes. In each sub-figure, bottom and top axes respectively shows student IDs and top axis shows the number of questions each student answered. The more questions a student answers, the tighter the credible interval. (Best viewed in color.)

4.4.1.2. Results. Fig. 4.1 shows bar plots that compare the student answer prediction performance of VarFA with SPARFA-M and SPARFA-B on all three evaluation metrics. We can observe that all three methods perform similarly and that SPARFA-M and SPARFA-B show no statistically significant advantage over VarFA. We further showcase VarFA’s scalability by comparing its training run-time with SPARFA-M and SPARFA-B. The results are shown in Fig. 4.2. VarFA is significantly faster than SPARFA-B and is almost as fast as SPARFA-M.

In summary, with VarFA, we obtain posteriors that allow uncertainty quantification with roughly the *same* computation complexity of computing point estimates and *without* compromising prediction performance.

4.4.2. Real Data Experiments

4.4.2.1. Setup.

Table 4.2. Student answer prediction performance comparing VarFA to SPARFA-M on Assistment, Algebra and Bridge data sets. $\uparrow$ and $\downarrow$ denote higher and lower is better, respectively. VarFA performs better than SPARFA-M on all three data sets and evaluation metrics most of the time. Additionally, VarFA’s run time is very close to SPARFA-M.

(a) Assistment		
Metric	Algorithm	
	SPARFA-M	VarFA
ACC $\uparrow$	0.7074 $\pm$ 0.0044	0.7101 $\pm$ 0.0048
AUC $\uparrow$	0.756 $\pm$ 0.048	0.7635 $\pm$ 0.0036
F1 $\uparrow$	0.7746 $\pm$ 0.0029	0.7765 $\pm$ 0.0014
Run time (s) $\downarrow$	5.3319 $\pm$ 0.2774	6.9167 $\pm$ 0.1074
(b) Algebra		
Metric	Algorithm	
	SPARFA-M	VarFA
ACC $\uparrow$	0.7735 $\pm$ 0.0037	0.7774 $\pm$ 0.0031
AUC $\uparrow$	0.8137 $\pm$ 0.003	0.8245 $\pm$ 0.002
F1 $\uparrow$	0.8465 $\pm$ 0.0021	0.8486 $\pm$ 0.001
Run time (s) $\downarrow$	8.464 $\pm$ 0.4568	10.3335 $\pm$ 0.4435
(c) Bridge		
Metric	Algorithm	
	SPARFA-M	VarFA
ACC $\uparrow$	0.8492 $\pm$ 0.0016	0.8468 $\pm$ 0.0016
AUC $\uparrow$	0.837 $\pm$ 0.0024	0.8419 $\pm$ 0.0028
F1 $\uparrow$	0.9121 $\pm$ 0.0005	0.912 $\pm$ 0.0009
Run time (s) $\downarrow$	15.6048 $\pm$ 0.7314	15.8558 $\pm$ 1.046

Data sets and pre-processing steps. We perform experiments on three large-scale, publicly available, real educational data sets including ASSISTments 2009-2010 (Assistment) [Heffernan and Heffernan \[2014\]](#), Algebra I 2006-2007 (algebra) [Stamper et al. \[2010\]](#) and Bridge to Algebra 2006-2007 (bridge) [Stamper et al. \[2010\]](#); [Stamper and Pardos](#)

[2016]. The details of the data sets, including data format and data collection procedure can be found in the preceding references. We remove students and questions that have too few answer records from the data sets to reduce the sparsity of the data sets. Specifically, we keep students and questions that have no less than 30, 35 and 40 answer records for Assistment, Algebra and Bridge data sets, respectively. In each data set, each student may provide more than one answer record for each question. Therefore, we also remove student-question answer records except for the first one. Table 4.1 presents the summary statistics of the resulting pre-processed data matrix for each data set.

Experimental settings and evaluation metrics. Optimizer, learning rate, number of epochs, neural network architecture and evaluation metrics are the same as in synthetic data experiments. For real data sets, we use 8 latent skills instead of 5. Other choices of the number of latent skills might result in better performance. However, since we are comparing different inference algorithms for the *same* model, it is a fair to compare the inference algorithms on the same model with the same number of latent dimensions. We use a 80:20 data split, i.e., we randomly sample 80% and 20% of the observed entries in each data set, without replacement, as training and test sets, respectively. Regularization hyper-parameters are selected by grid search and are different for each data set. We only compare VarFA with SPARFA-M because SPARFA-B does not scale to such large data sets.

4.4.2.2. Results: Performance Comparison. Table 4.2 shows the average performance on the test set of each data set comparing VarFA and SPARFA-M for all three data sets and additionally run time. We can see that VarFA achieves slightly better student answer prediction on most data sets and on most metrics. Interestingly, recall

that, in the preceding synthetic data set experiment, SPARFA-M performed better than VarFA most of the time. A possible explanation is that, for synthetic data sets, the underlying data generation process matches the SPARFA model, whereas for real data sets, the underlying data generation process is unknown. VarFA’s better performance than SPARFA-M for real data sets implies that VarFA is more robust to the unknown underlying data generation process. As a result, VarFA may be more applicable than SPARFA-M in real-world situations.

Table 4.2 also shows the run time comparison between VarFA and SPARFA-M; see the last row in each sub-table. We see that both inference algorithms have very similar run time, showing that VarFA is applicable for very large data sets. Notably, VarFA achieves this efficiency while also performing Bayesian inference on the student knowledge level factor.

4.4.2.3. Results: Bayesian Inference With VarFA. We now illustrate VarFA’s capability of outputting credible intervals using the Assistment data set. Fig. 4.3 presents violin plots that show the sampled student latent skill levels for a random subset of 10 students. Plots 4.3a, 4.3b and 4.3c shows the inferred students ability for the 3rd, 4th and 7th latent skill dimension. In each plot, the bottom axis shows the student ID and the top axis shows the total number of questions answered by the corresponding student. For each student, the horizontal width of the violin represents the density of the samples; the skinnier the violin, the more widespread the samples are, implying the model’s less certainty on its estimations.

Results in Fig. 4.3 confirms our intuition that the more questions a student answers, the more certain the model is about its estimation. For example, students with ID 106,

Table 4.3. Illustration of the estimated latent skills with the their top 3 most strongly associated skill tags in the Assistment data set. The percentage in the parenthesis shows the association probability (summed to 1 for each latent skill). We see that the tagged skills associated with each estimated latent skill form intuitive and interpretable groups.

Latent Skill 1	Latent Skill 3
Division Fractions (29.1%)	Conversion of Fraction Decimals Percents (7.3%)
Least Common Multiple (18.1%)	Addition and Subtraction Positive Decimals (6.8%)
Write Linear Equation from Ordered Pairs (17.8%)	Probability of a Single Event (5.7%)
Latent Skill 4	Latent Skill 7
Pattern Finding (17.4%)	Volume Sphere (13.4%)
Histogram as Table or Graph (11.3%)	Volume Cylinder (10.4%)
Percent Of (10.5%)	Surface Area Rectangular Prism (10.2%)

110 and 389 answered 222, 181 and 149 questions, respectively, and the credible intervals of their ability estimation is quite small. In contrast, students with ID 27, 49 and 65 answered far less questions and the credible intervals of their ability estimation is quite large. This result implies that VarFA outputs sensible and interpretable credible intervals. As mentioned in Section 4.1, such uncertainty quantification may benefit a number of educational applications such as improving adaptive testing algorithms. Note that VarFA is able to compute such credible interval as fast as SPARFA-M, making VarFA potentially useful for even real-time educational systems. In contrast, SPARFA-M is not capable of computing credible intervals. Although we may still obtain confidence interval as uncertainty quantification with SPARFA-M via bootstrapping, i.e., train with SPARFA-M multiple times with random subsets of the data set and then use the point estimates from different random runs to compute confidence intervals. However, this method suffers from two immediate drawbacks: 1) it is slow because we need to run the model multiple times and 2) different runs result in permutation in the estimated factors and thus the averaged estimations loss meaning. Given that VarFA is as efficient as SPARFA-M and

the previous drawbacks of SPARFA-M, we recommend VarFA over bootstrapping with SPARFA-M for quantifying model’s uncertainty in practice.

4.4.2.4. Results: Post-Processing for Improved Interpretability. SPARFA assumes that each student factor ν_i identifies a multi-dimensional skill level on a number of “latent” skills (recall that we use 8 latent skills in our experiments). As mentioned earlier, these latent skills are not interpretable without the aid of additional information. To improve interpretability, [Lan et al. \[2014\]](#) proposed that, when the skill tags for each question is available in the data set, we can associate each latent skill with skill tags via a simple matrix factorization. Then, we can compute each students’ mastery levels on the actual skill tags. We refer readers to Sections 5.1 and 5.2 in [Lan et al. \[2014\]](#) for more technical details.

Although the above method to improve interpretability has already been proposed, [Lan et al. \[2014\]](#) only presented results on private data sets, whereas here we presents results on publicly available data set with code, making our results more transparent and reproducible.

We again use the Assistment data set for illustration. We compute the association of skill tags in the data set with each of the latent skills and show 4 of the latent skills with their top 3 most strongly associated skill tags. We can see that each latent skill roughly identify the same group of skill tags. For example, latent skill 4 clusters skill tags on statistics and probability while latent skill 7 clusters skill tags on geometry. Thus, by simple post-processing, we obtain an interpretation of the latent skills by associating them with known skill tags in the data.

We can similarly obtain VarFA’s estimations of the students’ mastery levels on each skill tags through the above process. In Fig. 4.4, we compare the predicted mastery level for each skill tag (only for the questions this student answered) with the percent of correct answers for that skill tag. Blue curve shows the empirical student’s mastery level on a skill tag by computing the percentage of correctly answered questions belonging to a particular skill tag. Orange curve shows VarFA’s estimated student mastery level on a skill tag, normalized to range $[0, 1]$. We can see that, when the student gave more correct answers to questions of a particular skill tag, such as skill tag ID 2, 10, 14 and 30, VarFA also predicts a higher ability score for these skill tags. When the student gave more incorrect answers to questions of a particular skill tag, such as skill tag ID 0, 4 and 27, VarFA also predicts a lower ability score for those skill tags. Although the correspondence is not perfect, VarFA’s predicted student’s mastery levels match our intuition about student’s abilities (using the observed correct answer ratio as an empirical estimate) reasonably well. Thus, by post-processing, we can interpret VarFA’s estimated students’ latent skill mastery levels using the easily understandable skill tags.

4.5. Chapter Conclusion and Future Work

In this chapter, we have presented VarFA, a variational inference factor analysis framework to perform efficient Bayesian inference for learning analytics. VarFA is general and can be applied to a wide array of FA models. We have demonstrated the effectiveness of our VarFA using the sparse factor analysis (SPARFA) model as a case study. We have shown that VarFA can very efficiently output interpretable, educationally meaningful information, in particular credible intervals, much faster than classic Bayesian inference

methods. Thus, VarFA has potential application in many educational data mining scenarios where efficient credible interval computation is desired, i.e., in adaptive testing and adaptive learning systems. We have also provided open-source code to reproduce our results and facilitate further research efforts.

We outline three possible future research directions and extensions. First, VarFA currently performs Bayesian inference on the student skill mastery level factor. We are working on extending the framework to perform full Bayesian inference for all unknown factors. Extensions to some models such as IRT is straightforward, as studied in [Wu et al. \[2020\]](#), because all factors can be reasonably assumed to be Gaussian.

However, some other models involve additional modeling assumptions, making it challenging to design distributions that satisfy these assumptions. For example, in SPARFA, one of the factors is assumed to be nonnegative and sparse [Lan et al. \[2014\]](#). No standard distribution fulfill these requirements. We are exploring *mixture* distributions, in particular spike and slab models [Ishwaran et al. \[2005\]](#) for Bayesian variable selection, and methods to combine them with variational inference, following [Titsias and Lázaro-Gredilla \[2011\]](#); [Tonolini et al. \[2019\]](#).

Second, other methods exist that accelerate classic Bayesian inference. Recent works in large-scale Bayesian inference proposed *approximate* MCMC methods that scale to large data sets [Zhang et al. \[2020\]](#); [Ma et al. \[2015\]](#); [Song et al. \[2017\]](#). Some of these methods apply variational inference to perform the approximation [De Freitas et al. \[2001\]](#); [Habib and Barber \[2019\]](#). We are investigating extending VarFA to support approximate MCMC methods.

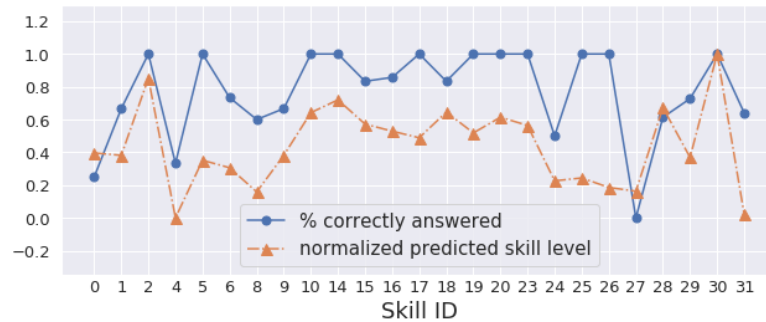


Figure 4.4. Comparison between the estimated skill mastery levels using VarFA’s predictions and using empirical observations for student with ID 110. Even though the two curves show different numeric values, they nevertheless demonstrate similar trends, showing that the predictions reasonably match our intuition about student’s skill mastery levels.

Finally, the goal of VarFA is to improve real-world educational systems and ultimately improve learning. Therefore, it is necessary to evaluate VarFA beyond synthetic and benchmark data sets. We plan to integrate VarFA into an existing educational system and conduct a case study where students interact with the system in real-time to understand the VarFA’s educational implications in the wild.

Bibliography

- [1] Oliver Stegle, Sarah Teichmann, and John Marioni. Computational and analytical challenges in single-cell transcriptomics. *Nature reviews. Genetics*, 16, 01 2015.
- [2] Dmitry Kobak, George Linderman, Stefan Steinerberger, Yuval Kluger, and Philipp Berens. *Heavy-Tailed Kernels Reveal a Finer Cluster Structure in t-SNE Visualisations*, page 124–139. Springer International Publishing, 2020. ISBN 9783030461508. URL http://dx.doi.org/10.1007/978-3-030-46150-8_8.
- [3] Sewall Wright. Systems of mating. i. the biometric relations between parent and offspring. *Genetics*, 6(2): 111–123, 03 1921, <https://academic.oup.com/genetics/article-pdf/6/2/111/35397881/genetics0111.pdf>. ISSN 1943-2631. URL <https://doi.org/10.1093/genetics/6.2.111>.
- [4] Julian Besag. On the statistical analysis of dirty pictures. *Journal of the Royal Statistical Society: Series B (Methodological)*, 48(3):259–279, 07 1986, https://academic.oup.com/jrsssb/article-pdf/48/3/259/49097842/jrsssb_48.3_259.pdf. ISSN 0035-9246. URL <https://doi.org/10.1111/j.2517-6161.1986.tb01412.x>.
- [5] F.R. Kschischang, B.J. Frey, and H.-A. Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.
- [6] D. Koller and N. Friedman. *Probabilistic Graphical Models: Principles and Techniques*. Adaptive computation and machine learning. MIT Press, 2009. ISBN 9780262013192. URL <https://books.google.co.in/books?id=7dzpHCHzNQ4C>.
- [7] Hugo Mak, Runze Han, and Hoover Yin. Application of variational autoencoder (vae) model and image processing approaches in game design, 02 2023.
- [8] Robin A. Moser and Gábor Tardos. A constructive proof of the general lovász local lemma. *J. ACM*, 57(2):11:1–11:15, 2010.
- [9] Tony Hey, Stewart Tansley, Kristin Tolle, and Jim Gray. *The Fourth Paradigm: Data-Intensive Scientific Discovery*. Microsoft Research, October 2009. ISBN 978-0-9825442-0-4. URL <https://www.microsoft.com/en-us/research/publication/fourth-paradigm-data-intensive-scientific-discovery/>.
- [10] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc.,

2012. URL https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
 - [12] David Donoho. High-dimensional data analysis: The curses and blessings of dimensionality. *AMS Math Challenges Lecture*, pages 1–32, 01 2000.
 - [13] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996, https://academic.oup.com/jrsssb/article-pdf/58/1/267/49098631/jrsssb_58.1_267.pdf. URL <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.
 - [14] Shaobing Chen and D. Donoho. Basis pursuit. In *Proceedings of 1994 28th Asilomar Conference on Signals, Systems and Computers*, volume 1, pages 41–44 vol.1, 1994.
 - [15] Jianqing Fan and Jinchi Lv. A selective overview of variable selection in high dimensional feature space. *Statistica Sinica*, 20:101–148, 01 2010.
 - [16] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *ICLR*, 2014.
 - [17] Richard Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961. ISBN 9780691079011. URL <http://www.jstor.org/stable/j.ctt183ph6v>.
 - [18] G. Hughes. On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory*, 14(1):55–63, 1968.
 - [19] Vladimir Kiselev, Tallulah Andrews, and Martin Hemberg. Challenges in unsupervised clustering of single-cell rna-seq data. *Nature Reviews Genetics*, 20:1, 01 2019.
 - [20] Peter Visscher, Naomi Wray, Qian Zhang, Pamela Sklar, Mark McCarthy, Matthew Brown, and Jian Yang. 10 years of gwas discovery: Biology, function, and translation. *The American Journal of Human Genetics*, 101:5–22, 07 2017.
 - [21] F.R.S Karl Pearson. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572, 1901, <https://doi.org/10.1080/14786440109462720>. URL <https://doi.org/10.1080/14786440109462720>.
 - [22] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *J. Ed. Psych.*, 24:417–441, 1933.
 - [23] W. S. Torgerson. Multidimensional scaling: I. theory and method. *Psychometrika*, 17:401–419, 1952.

- [24] W.S. Torgerson. *Theory and methods of scaling*. Wiley, 1958. URL <https://books.google.com/books?id=FWN-AAAAMAAJ>.
- [25] Roger N. Shepard. The analysis of proximities: Multidimensional scaling with an unknown distance function. i. and ii. *Psychometrika*, 27(2):125–140, 1962.
- [26] J. B. Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 29(1):1–27, 1964.
- [27] Sam T. Roweis and Lawrence K. Saul. Nonlinear Dimensionality Reduction by Locally Linear Embedding. *Science*, 290(5500):2323–2326, 2000, <http://www.sciencemag.org/cgi/reprint/290/5500/2323.pdf>. URL <http://www.sciencemag.org/cgi/content/abstract/290/5500/2323>.
- [28] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319, 2000.
- [29] M.L. Fredman and R.E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. In *25th Annual Symposium on Foundations of Computer Science, 1984.*, pages 338–346, 1984.
- [30] Geoffrey E Hinton and Sam Roweis. Stochastic neighbor embedding. In S. Becker, S. Thrun, and K. Obermayer, editors, *Advances in Neural Information Processing Systems*, volume 15. MIT Press, 2002. URL https://proceedings.neurips.cc/paper_files/paper/2002/file/6150ccc6069bea6b5716254057a194ef-Paper.pdf.
- [31] Antonio Auffinger and Daniel Fletcher. Equilibrium distributions for t-distributed stochastic neighbour embedding, 2023, 2304.03727. URL <https://arxiv.org/abs/2304.03727>.
- [32] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(2579-2605), 2008.
- [33] Ben Weinkove. Dimension reduction and the gradient flow of relative entropy, 2024, 2409.16963. URL <https://arxiv.org/abs/2409.16963>.
- [34] Stuart Geman and Donald Geman. Stochastic relaxation, gibbs distributions, and the bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-6(6):721–741, 1984.
- [35] Alex Zihao Zhu, Jieru Mei, Siyuan Qiao, Hang Yan, Yukun Zhu, Liang-Chieh Chen, and Henrik Kretzschmar. Superpixel transformers for efficient semantic segmentation, 2023, 2309.16889. URL <https://arxiv.org/abs/2309.16889>.
- [36] Y.Y. Boykov and M.-P. Jolly. Interactive graph cuts for optimal boundary and region segmentation of objects in n-d images. In *Proceedings Eighth IEEE International Conference on Computer Vision. ICCV 2001*, volume 1, pages 105–112 vol.1, 2001.
- [37] V. Kolmogorov and R. Zabini. What energy functions can be minimized via graph cuts? *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(2):147–159, 2004.

- [38] Y. Boykov and V. Kolmogorov. An experimental comparison of min-cut/max- flow algorithms for energy minimization in vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(9):1124–1137, 2004.
- [39] Magali Naville and Daniel Gautheret. Transcription attenuation in bacteria: theme and variations. *Briefings in Functional Genomics*, 9:178–189, 03 2010.
- [40] François Jacob and Jacques Monod. Genetic regulatory mechanisms in the synthesis of proteins. *Journal of Molecular Biology*, 3(3):318–356, 1961. ISSN 0022-2836. URL <https://www.sciencedirect.com/science/article/pii/S0022283661800727>.
- [41] Jonna Frasor, Fabio Stossi, Jeanne M. Danes, Barry Komm, C. Richard Lyttle, and Benita S. Katzenellenbogen. Selective estrogen receptor modulators: Discrimination of agonistic versus antagonistic activities by gene expression profiling in breast cancer cells. *Cancer Research*, 64(4):1522–1533, 02 2004, <https://aacrjournals.org/cancerres/article-pdf/64/4/1522/2523009/zch00404001522.pdf>. ISSN 0008-5472. URL <https://doi.org/10.1158/0008-5472.CAN-03-3326>.
- [42] Nina Heldring, Ashley Pike, Sandra Andersson, Jason Matthews, Guojun Cheng, Johan Hartman, Michel Tujague, Anders Ström, Eckardt Treuter, Margaret Warner, and Jan-Åke Gustafsson. Estrogen receptors: How do they signal and what are their targets. *Physiological Reviews*, 87(3):905–931, 2007, <https://doi.org/10.1152/physrev.00026.2006>. URL <https://doi.org/10.1152/physrev.00026.2006>. PMID: 17615392.
- [43] Nir Friedman, Michal Linial, Iftach Nachman, and Dana Pe’er. Using bayesian networks to analyze expression data. *Journal of Computational Biology*, 7(3-4): 601–620, 2000, <https://doi.org/10.1089/106652700750050961>. URL <https://doi.org/10.1089/106652700750050961>. PMID: 11108481.
- [44] Karen Sachs, Omar Perez, Dana Pe’er, Douglas Lauffenburger, and Garry Nolan. Causal protein-signaling networks derived from multiparameter single-cell data. *Science (New York, N.Y.)*, 308:523–9, 05 2005.
- [45] A Krogh. Hidden markov models in computational biology. applications to protein modeling. *Journal of molecular biology.*, 235(5), 1994-01-31. ISSN 0022-2836.
- [46] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, 1998.
- [47] Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*, volume B. Springer Science and Business Media, 01 2003.
- [48] JAMES E. LIGHTNER. A brief look at the history of probability and statistics. *The Mathematics Teacher*, 84(8):623–630, 1991. ISSN 00255769. URL <http://www.jstor.org/stable/27967334>.
- [49] Leonhard Euler. Solutio problematis ad geometriam situs pertinentis. *Commentarii Academiae Scientiarum Imperialis Petropolitanae*, 8:128–140, 1736.

- [50] Ernst Ising. Beitrag zur theorie des ferromagnetismus. *Zeitschrift für Physik*, 31: 253–258, 1925. URL <https://api.semanticscholar.org/CorpusID:122157319>.
- [51] Lars Onsager. Crystal statistics. i. a two-dimensional model with an order-disorder transition. *Phys. Rev.*, 65:117–149, Feb 1944. URL <https://link.aps.org/doi/10.1103/PhysRev.65.117>.
- [52] Sewall Wright. Correlation and causation. *Journal of agricultural research*, 20(7): 557–585, 1921.
- [53] Sewall Wright. The method of path coefficients. *The Annals of Mathematical Statistics*, 5(3):161–215, 1934. ISSN 00034851, 21688990. URL <http://www.jstor.org/stable/2957502>.
- [54] Sewall Wright. The relative importance of heredity and environment in determining the piebald pattern of guinea-pigs. *Proceedings of the National Academy of Sciences of the United States of America*, 6 6:320–32, 1920. URL <https://api.semanticscholar.org/CorpusID:12577660>.
- [55] Sewall Wright. On the nature of size factors. *Genetics*, 3 (4):367–374, 07 1918, <https://academic.oup.com/genetics/article-pdf/3/4/367/35081662/genetics0367.pdf>. ISSN 1943-2631. URL <https://doi.org/10.1093/genetics/3.4.367>.
- [56] Otis Dudley Duncan. *Introduction to Structural Equation Models*. Academic Press, 1975.
- [57] Kenneth A. Bollen. *Structural Equations with Latent Variables*. Wiley, 1989. ISBN 9781118619179.
- [58] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1988. ISBN 1558604790.
- [59] Mehran Sahami, Susan Dumais, David Heckerman, and Eric Horvitz. A bayesian approach to filtering junk e-mail. *AAAI’98 Workshop on Learning for Text Categorization*, 62, 06 1998.
- [60] Catherine Shenoy and Prakash Shenoy. *Bayesian Network Models of Portfolio Risk and Return*, pages 87–. The MIT Press, 06 2003.
- [61] Jozef Gemela. Financial analysis using bayesian networks. *Applied Stochastic Models in Business and Industry*, 17:57 – 67, 02 2001.
- [62] Julian Besag. Spatial interaction and the statistical analysis of lattice systems. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):192–225, 1 1974, https://academic.oup.com/jrsssb/article-pdf/36/2/192/49096627/jrsssb_36.2_192.pdf. ISSN 0035-9246. URL <https://doi.org/10.1111/j.2517-6161.1974.tb00999.x>.
- [63] Michael Jordan, Zoubin Ghahramani, Tommi Jaakkola, and Lawrence Saul. An introduction to variational methods for graphical models. *Machine Learning*, 37: 183–233, 01 1999.

- [64] Alan E. Gelfand and Adrian F. M. Smith and. Sampling-based approaches to calculating marginal densities. *Journal of the American Statistical Association*, 85(410):398–409, 1990, <https://www.tandfonline.com/doi/pdf/10.1080/01621459.1990.10476213>. URL <https://www.tandfonline.com/doi/abs/10.1080/01621459.1990.10476213>.
- [65] Paul Erdős and Lovász László. Problems and results on 3-chromatic hypergraphs and some related questions. *Coll Math Soc J Bolyai*, 10, 01 1974.
- [66] James B Shearer. On a problem of spencer. *Combinatorica*, 5(3):241–245, July 1985. ISSN 0209-9683. URL <https://doi.org/10.1007/BF02579368>.
- [67] Joel Spencer. Asymptotic lower bounds for ramsey functions. *Discrete Mathematics*, 20:69–76, 1977. ISSN 0012-365X. URL <https://www.sciencedirect.com/science/article/pii/0012365X77900449>.
- [68] Robin A. Moser and Gábor Tardos. A constructive proof of the general lovasz local lemma, 2009, 0903.0544. URL <https://arxiv.org/abs/0903.0544>.
- [69] C. Spearman. General intelligence, objectively determined and measured. *The American Journal of Psychology*, 15(2):201–292, 1904. ISSN 00029556. URL <http://www.jstor.org/stable/1412107>.
- [70] L.L. Thurstone. *Multiple-factor Analysis: A Development and Expansion of The Vectors of the Mind*. University of Chicago Committee on Publications in Biology and Medicine. Publications. University of Chicago Press, 1947. ISBN 9780608111896. URL <https://books.google.com/books?id=c0wAAAAAMAAJ>.
- [71] K. G. Jöreskog. Some contributions to maximum likelihood factor analysis. *Psychometrika*, 32(4):443–482, 1967.
- [72] Karl Jöreskog. A general approach to confirmatory factor analysis. *Psychometrika*, 34:183–202, 02 1969.
- [73] Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning, 2017, 1702.08608. URL <https://arxiv.org/abs/1702.08608>.
- [74] Tim Miller. Explanation in artificial intelligence: Insights from the social sciences, 2018, 1706.07269. URL <https://arxiv.org/abs/1706.07269>.
- [75] Carl Doersch. Tutorial on variational autoencoders, 2021, 1606.05908. URL <https://arxiv.org/abs/1606.05908>.
- [76] Ryan Murray and Adam Pickarski. Large data limits and scaling laws for tsne, 2024, 2410.13063. URL <https://arxiv.org/abs/2410.13063>.
- [77] Wenjun Liu, Quoc Ngo, and Vu Huy. Several interesting integral inequalities. *Journal of Mathematical Inequalities*, 3, 06 2009.
- [78] Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Comput.*, 14(8):1771–1800, 2002.
- [79] Ioannis Tsochantaridis, Thorsten Joachims, Thomas Hofmann, and Yasemin Altun. Large margin methods for structured and interdependent output variables. *J. Mach. Learn. Res.*, 6:1453–1484, December 2005.

- [80] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron C. Courville, and Yoshua Bengio. Generative adversarial nets. In *NIPS*, pages 2672–2680, 2014.
- [81] Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. MADE: masked autoencoder for distribution estimation. In *ICML*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 881–889. JMLR.org, 2015.
- [82] Hugo Larochelle and Iain Murray. The neural autoregressive distribution estimator. In *AISTATS*, volume 15 of *JMLR Proceedings*, pages 29–37. JMLR.org, 2011.
- [83] Aäron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *ICML*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1747–1756. JMLR.org, 2016.
- [84] Martín Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *ICML*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223. PMLR, 2017.
- [85] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *NeurIPS*, pages 11895–11907, 2019.
- [86] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*. OpenReview.net, 2021.
- [87] Kevin P. Murphy, Yair Weiss, and Michael I. Jordan. Loopy belief propagation for approximate inference: An empirical study. In *UAI*, pages 467–475. Morgan Kaufmann, 1999.
- [88] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. Generalized belief propagation. In *NIPS*, pages 689–695. MIT Press, 2000.
- [89] Martin J. Wainwright and Michael I. Jordan. Log-determinant relaxation for approximate inference in discrete markov random fields. *IEEE Trans. Signal Process.*, 54(6-1):2099–2109, 2006.
- [90] A. Braunstein, M. Mézard, and R. Zecchina. Survey propagation: an algorithm for satisfiability. *Random Struct. Algorithms*, 27:201–226, 2005.
- [91] Henrik Reif Andersen, Tarik Hadzic, John N. Hooker, and Peter Tiedemann. A constraint store based on multivalued decision diagrams. In *CP*, volume 4741 of *Lecture Notes in Computer Science*, pages 118–132. Springer, 2007.
- [92] Mark Chavira, Adnan Darwiche, and Manfred Jaeger. Compiling relational bayesian networks for exact inference. *Int. J. Approx. Reason.*, 42(1-2):4–20, 2006.
- [93] Pascal Van Hentenryck. *Constraint Satisfaction in Logic Programming*. MIT Press, Cambridge, MA, USA, 1989. ISBN 0-262-08181-4.
- [94] Vibhav Gogate and Rina Dechter. Importance sampling-based estimation over and/or search spaces for graphical models. *Artificial Intelligence*, 184-185:38 – 77, 2012.
- [95] Tian Sang, Paul Bearne, and Henry Kautz. Performing bayesian inference by weighted model counting. In *AAAI*, AAAI’05, pages 475–481, 2005.

- [96] Matt J. Kusner, Brooks Paige, and José Miguel Hernández-Lobato. Grammar variational autoencoder. In *ICML*, volume 70, pages 1945–1954. PMLR, 2017.
- [97] Wengong Jin, Regina Barzilay, and Tommi S. Jaakkola. Junction tree variational autoencoder for molecular graph generation. In *ICML*, volume 80 of *Proceedings of Machine Learning Research*, pages 2328–2337. PMLR, 2018.
- [98] Hanjun Dai, Yingtao Tian, Bo Dai, Steven Skiena, and Le Song. Syntax-directed variational autoencoder for structured data. In *ICLR (Poster)*. OpenReview.net, 2018.
- [99] Zhiting Hu, Zichao Yang, Xiaodan Liang, Ruslan Salakhutdinov, and Eric P. Xing. Toward controlled generation of text. In *ICML*, volume 70 of *Proceedings of Machine Learning Research*, pages 1587–1596. PMLR, 2017.
- [100] Daniel Lowd and Pedro M. Domingos. Learning arithmetic circuits. In *UAI*, pages 383–392. AUAI Press, 2008.
- [101] Fan Ding, Jianzhu Ma, Jinbo Xu, and Yexiang Xue. XOR-CD: linearly convergent constrained structure generation. In *ICML*, volume 139 of *Proceedings of Machine Learning Research*, pages 2728–2738. PMLR, 2021.
- [102] Paul Erdős and László Lovász. Problems and results on 3-chromatic hypergraphs and some related questions. In *Colloquia Mathematica Societatis Janos Bolyai 10. Infinite and Finite Sets, Keszthely (Hungary)*. Citeseer, 1973.
- [103] Heng Guo, Mark Jerrum, and Jingcheng Liu. Uniform sampling through the lovász local lemma. *J. ACM*, 66(3):18:1–18:31, 2019.
- [104] Daniel Selsam, Matthew Lamm, Benedikt Bünz, Percy Liang, Leonardo de Moura, and David L. Dill. Learning a SAT solver from single-bit supervision. In *ICLR (Poster)*. OpenReview.net, 2019.
- [105] Haonan Duan, Pashootan Vaezipoor, Max B. Paulus, Yangjun Ruan, and Chris J. Maddison. Augment with care: Contrastive learning for combinatorial problems. In *ICML*, volume 162 of *Proceedings of Machine Learning Research*, pages 5627–5642. PMLR, 2022.
- [106] Zhuwen Li, Qifeng Chen, and Vladlen Koltun. Combinatorial optimization with graph convolutional networks and guided tree search. In *NeurIPS*, pages 537–546, 2018.
- [107] Emre Yolcu and Barnabás Póczos. Learning local search heuristics for boolean satisfiability. In *NeurIPS*, pages 7990–8001, 2019.
- [108] Xinyun Chen and Yuandong Tian. Learning to perform local rewriting for combinatorial optimization. In *NeurIPS*, pages 6278–6289, 2019.
- [109] Nikolaos Karalias and Andreas Loukas. Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. In *NeurIPS*, pages 6659–6672, 2020.
- [110] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *Eur. J. Oper. Res.*, 290(2): 405–421, 2021.

- [111] Jayanta Mandi, Emir Demirovic, Peter J. Stuckey, and Tias Guns. Smart predict-and-optimize for hard combinatorial optimization problems. In *AAAI*, pages 1603–1610. AAAI Press, 2020.
- [112] Radford M Neal. *Probabilistic inference using Markov chain Monte Carlo methods*. Department of Computer Science, University of Toronto Toronto, ON, Canada, 1993.
- [113] Paul Dagum and R. Martin Chavez. Approximating probabilistic inference in bayesian belief networks. *IEEE Trans. Pattern Anal. Mach. Intell.*, 15(3):246–255, 1993.
- [114] Hong Ge, Kai Xu, and Zoubin Ghahramani. Turing: A language for flexible probabilistic inference. In *AISTATS*, volume 84, pages 1682–1690. PMLR, 09–11 Apr 2018.
- [115] Alexander T. Ihler, John W. Fisher III, and Alan S. Willsky. Loopy belief propagation: Convergence and effects of message errors. *J. Mach. Learn. Res.*, 6:905–936, 2005.
- [116] Amin Coja-Oghlan, Noëla Müller, and Jean B. Ravelomanana. Belief propagation on the random k-sat model, 2020.
- [117] Fan Ding and Yexiang Xue. Contrastive divergence learning with chained belief propagation. In *PGM*, volume 138 of *Proceedings of Machine Learning Research*, pages 161–172. PMLR, 2020.
- [118] Vibhav Gogate and Rina Dechter. Samplesearch: Importance sampling in presence of determinism. *Artif. Intell.*, 175(2):694–729, 2011.
- [119] Carla P. Gomes, Ashish Sabharwal, and Bart Selman. Near-uniform sampling of combinatorial spaces using XOR constraints. In *NIPS*, pages 481–488. MIT Press, 2006.
- [120] Stefano Ermon, Carla P. Gomes, Ashish Sabharwal, and Bart Selman. Taming the curse of dimensionality: Discrete integration by hashing and optimization. In *ICML (2)*, volume 28 of *JMLR Workshop and Conference Proceedings*, pages 334–342. JMLR.org, 2013.
- [121] Dimitris Achlioptas and Panos Theodoropoulos. Probabilistic model counting with short xors. In *SAT*, volume 10491 of *Lecture Notes in Computer Science*, pages 3–19. Springer, 2017.
- [122] Supratik Chakraborty, Kuldeep S. Meel, and Moshe Y. Vardi. A scalable and nearly uniform generator of SAT witnesses. In *CAV*, volume 8044, pages 608–623. Springer, 2013.
- [123] Yishay Mansour. Learning boolean functions via the fourier transform. In *Theoretical advances in neural computation and learning*, pages 391–424. Springer, 1994.
- [124] Chris K Carter and Robert Kohn. On gibbs sampling for state space models. *Biometrika*, 81(3):541–553, 1994.
- [125] Rahul Gupta, Shubham Sharma, Subhajit Roy, and Kuldeep S. Meel. WAPS: weighted and projected sampling. In *TACAS*, volume 11427, pages 59–76, 2019.

- [126] Supratik Chakraborty, Daniel J. Fremont, Kuldeep S. Meel, Sanjit A. Seshia, and Moshe Y. Vardi. Distribution-aware sampling and weighted model counting for SAT. In *AAAI*, pages 1722–1730. AAAI Press, 2014.
- [127] Stefano Ermon, Carla P. Gomes, Ashish Sabharwal, and Bart Selman. Embed and project: Discrete sampling with universal hashing. In *NIPS*, pages 2085–2093, 2013.
- [128] Fan Ding and Yexiang Xue. XOR-SGD: provable convex stochastic optimization for decision-making under uncertainty. In *UAI*, volume 161 of *Proceedings of Machine Learning Research*, pages 151–160. AUAI Press, 2021.
- [129] Priyanka Golia, Mate Soos, Sourav Chakraborty, and Kuldeep S. Meel. Designing samplers is easy: The boon of testers. In *FMCAD*, pages 222–230. IEEE, 2021.
- [130] Rafael Dutra, Kevin Laeufer, Jonathan Bachrach, and Koushik Sen. Efficient sampling of SAT solutions for testing. In *ICSE*, pages 549–559. ACM, 2018.
- [131] Mate Soos, Stephan Gocht, and Kuldeep S. Meel. Tinted, detached, and lazy CNF-XOR solving and its applications to counting and sampling. In *CAV (1)*, volume 12224 of *Lecture Notes in Computer Science*, pages 463–484. Springer, 2020.
- [132] Shubham Sharma, Rahul Gupta, Subhajit Roy, and Kuldeep S. Meel. Knowledge compilation meets uniform sampling. In *LPAR*, volume 57 of *EPiC Series in Computing*, pages 620–636, 2018.
- [133] Carmine Dodaro and Alessandro Previti. Minipref: A tool for preferences in SAT (short paper). In *RCRA + RiCeRcA*, volume 2538 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2019.
- [134] Massimo Lauria, Jan Elffers, Jakob Nordström, and Marc Vinyals. Cnfgcn: A generator of crafted benchmarks. In *SAT*, volume 10491, pages 464–473. Springer, 2017.
- [135] Alexey Ignatiev, António Morgado, and João Marques-Silva. Pysat: A python toolkit for prototyping with SAT oracles. In *SAT*, volume 10929 of *Lecture Notes in Computer Science*, pages 428–437. Springer, 2018.
- [136] Sourav Chakraborty and Kuldeep S. Meel. On testing of uniform samplers. In *AAAI*, pages 7777–7784, 2019.
- [137] Henry Cohn, Robin Pemantle, and James Gary Propp. Generating a random sink-free orientation in quadratic time. *Electron. J. Comb.*, 9(1), 2002.
- [138] Junji Takahashi, Takuya Yamaguchi, Kosuke Sekiyama, and Toshio Fukuda. Communication timing control and topology reconfiguration of a sink-free meshed sensor network with mobile robots. *IEEE/ASME transactions on mechatronics*, 14(2):187–197, 2009.
- [139] Mark Jerrum. Fundamentals of partial rejection sampling, 2021.
- [140] Emanuele Di Rosa, Enrico Giunchiglia, and Barry O’Sullivan. Optimal stopping methods for finding high quality solutions to satisfiability problems with preferences. In *SAC*, pages 901–906. ACM, 2011.
- [141] Nicolas Prevot, Mate Soos, and Kuldeep S. Meel. Leveraging gpus for effective clause sharing in parallel SAT solving. In *SAT*, pages 471–487, 2021.

- [142] Muhammad Osama Mahmoud. *GPU Enabled Automated Reasoning*. PhD thesis, Mathematics and Computer Science, March 2022.
- [143] Johannes Klaus Fichte, Markus Hecher, and Markus Zisser. An improved gpu-based SAT model counter. In *CP*, pages 491–509, 2019.
- [144] Kurt VanLehn. Student modeling. *Foundations of Intelligent Tutoring Systems*, 55: 78, 1988.
- [145] Konstantina Chrysafiadi and Maria Virvou. Student modeling approaches: A literature review for the last decade. *Expert Systems with Applications*, 40(11):4715–4729, 2013.
- [146] Valerie J Shute and Joseph Psotka. Intelligent tutoring systems: Past, present, and future., 1994.
- [147] Sabine Graf and Kinshuk. *Personalized Learning Systems*, pages 2594–2596. Springer US, Boston, MA, 2012. ISBN 978-1-4419-1428-6. URL https://doi.org/10.1007/978-1-4419-1428-6_152.
- [148] John R Anderson, Albert T Corbett, Kenneth R Koedinger, and Ray Pelletier. Cognitive tutors: Lessons learned. *Journal of the Learning Sciences*, 4(2):167–207, 1995.
- [149] Benjamin D Nye, Arthur C Graesser, and Xiangen Hu. Autotutor and family: A review of 17 years of natural language tutoring. *International Journal of Artificial Intelligence in Education*, 24(4):427–469, 2014.
- [150] Neil T Heffernan and Cristina Lindquist Heffernan. The assistments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. *International Journal of Artificial Intelligence in Education*, 24(4):470–497, 2014.
- [151] Anna N Rafferty, Michelle M LaMar, and Thomas L Griffiths. Inferring learners’ knowledge from their actions. *Cognitive Science*, 39(3):584–618, 2015.
- [152] Cristóbal Romero and Sebastián Ventura. Educational data mining: a review of the state of the art. *IEEE Transactions on Systems, Man, and Cybernetics*, 40(6): 601–618, 2010.
- [153] Ryan SJD Baker and Kalina Yacef. The state of educational data mining in 2009: A review and future visions. *Journal of Educational Data Mining*, 1(1):3–17, 2009.
- [154] Agathe Merceron and Kalina Yacef. Educational data mining: a case study. In *Proc. Artificial Intelligence in Education*, pages 467–474, 2005.
- [155] Wim J van der Linden and Ronald K Hambleton. *Handbook of modern item response theory*. Springer Science & Business Media, 2013.
- [156] Terry A Ackerman. Using multidimensional item response theory to understand what items and tests are measuring. *Applied Measurement in Education*, 7(4):255–278, 1994.
- [157] Hao Cen, Kenneth Koedinger, and Brian Junker. Learning factors analysis – a general method for cognitive model evaluation and improvement. In Mitsuru Ikeda, Kevin D. Ashley, and Tak-Wai Chan, editors, *Proc. Intelligent Tutoring Systems*,

- pages 164–175, 2006. ISBN 978-3-540-35160-3.
- [158] Philip I. Pavlik, Hao Cen, and Kenneth R. Koedinger. Performance factors analysis – a new alternative to knowledge tracing. In *Proc. Artificial Intelligence in Education*, page 531–538, NLD, 2009. IOS Press. ISBN 9781607500285.
 - [159] Robert V Lindsey, Jeffery D Shroyer, Harold Pashler, and Michael C Mozer. Improving students’ long-term knowledge retention through personalized review. *Psychological Science*, 25(3):639–647, 2014.
 - [160] Michael C Mozer and Robert V Lindsey. Predicting and improving memory retention: Psychological theory matters in the big data era. In *Big Data in Cognitive Science*, pages 43–73. Psychology Press, 2016.
 - [161] Benoît Choffin, Fabrice Popineau, Yolaine Bourda, and Jill-Jênn Vie. Das3h: Modeling student learning and forgetting for optimally scheduling distributed practice of skills. In *Proc. Educational Data Mining*, 2019.
 - [162] Jill-Jênn Vie and Hisashi Kashima. Knowledge tracing machines: Factorization machines for knowledge tracing. In *Proc. AAAI Conference on Artificial Intelligence*, volume 33, pages 750–757, 2019.
 - [163] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Proc. Advances in Neural Information Processing Systems*, pages 505–513. Curran Associates, Inc., 2015.
 - [164] Jiani Zhang, Xingjian Shi, Irwin King, and Dit-Yan Yeung. Dynamic key-value memory networks for knowledge tracing. In *Proc. International Conference on World Wide Web*, pages 765–774, 2017.
 - [165] Sein Minn, Yi Yu, Michel C Desmarais, Feida Zhu, and Jill-Jênn Vie. Deep knowledge tracing and dynamic student classification for knowledge tracing. In *Proc. IEEE International Conference on Data Mining*, pages 1182–1187. IEEE, 2018.
 - [166] Kevin H Wilson, Yan Karklin, Bojian Han, and Chaitanya Ekanadham. Back to the basics: Bayesian extensions of irt outperform neural networks for proficiency estimation. In *Proc. Educational Data Mining*, 2016.
 - [167] Hua-Hua Chang, Zhiliang Ying, et al. Nonlinear sequential designs for logistic item response theory models with applications to computerized adaptive tests. *The Annals of Statistics*, 37(3):1466–1488, 2009.
 - [168] Duanli Yan, Alina A Von Davier, and Charles Lewis. *Computerized multistage testing: Theory and applications*. CRC Press, 2016.
 - [169] Andrew Gelman, John B Carlin, Hal S Stern, David B Dunson, Aki Vehtari, and Donald B Rubin. *Bayesian data analysis*. Chapman and Hall/CRC, 2013.
 - [170] Walter R Gilks, Sylvia Richardson, and David Spiegelhalter. *Markov chain Monte Carlo in practice*. Chapman and Hall/CRC, 1995.
 - [171] Andrew S. Lan, Andrew E. Waters, Christoph Studer, and Richard G. Baraniuk. Sparse factor analysis for learning and content analytics. *Journal of Machine Learning Research*, 15:1959–2008, 2014.

- [172] Jean-Paul Fox. *Bayesian item response modeling: Theory and applications*. Springer Science & Business Media, 2010.
- [173] Zachary A Pardos, Neil T Heffernan, Brigham Anderson, Cristina L Heffernan, and Worcester Public Schools. Using fine-grained skill models to fit student performance with bayesian networks. *Handbook of Educational Data Mining*, 417, 2010.
- [174] Edgar C Merkle. A comparison of imputation methods for bayesian factor analysis models. *Journal of Educational and Behavioral Statistics*, 36(2):257–276, 2011.
- [175] Marcy Perkins Driscoll. *Psychology of learning for instruction*. Allyn & Bacon, 1994.
- [176] Kikumi K. Tatsuoaka. Rule space: An approach for dealing with misconceptions based on item response theory. *Journal of Educational Measurement*, 20(4):345–354, 1983. ISSN 00220655, 17453984. URL <http://www.jstor.org/stable/1434951>.
- [177] Min Chi, Kenneth R Koedinger, Geoffrey J Gordon, Pamela Jordon, and Kurt VanLahn. Instructional factors analysis: A cognitive model for multiple instructional interventions. *Educational Data Mining*, 2011.
- [178] Jean-Paul Fox and Cees AW Glas. Bayesian estimation of a multilevel irt model using gibbs sampling. *Psychometrika*, 66(2):271–288, 2001.
- [179] Robert J Mislevy. Bayes modal estimation in item response models. *Psychometrika*, 51(2):177–195, 1986.
- [180] Christopher M Bishop. *Pattern recognition and machine learning*. springer, 2006.
- [181] David JC MacKay and David JC Mac Kay. *Information theory, inference and learning algorithms*. Cambridge University Press, 2003.
- [182] Johan Ludwig William Valdemar Jensen et al. Sur les fonctions convexes et les inégalités entre les valeurs moyennes. *Acta Mathematica*, 30:175–193, 1906.
- [183] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic back-propagation and approximate inference in deep generative models. In *Proc. International Conference on Machine Learning, ICML’14*, page II–1278–II–1286. JMLR.org, 2014.
- [184] Shengjia Zhao, Jiaming Song, and Stefano Ermon. Infovae: Balancing learning and inference in variational autoencoders. In *Proc. AAAI Conference on Artificial Intelligence*, volume 33, pages 5885–5892, Feb. 2019.
- [185] Emile Mathieu, Tom Rainforth, N Siddharth, and Yee Whye Teh. Disentangling disentanglement in variational autoencoders. In *Proc. International Conference on Machine Learning*, pages 4402–4412, Jun 2019.
- [186] Narayanaswamy Siddharth, Brooks Paige, Jan-Willem Van de Meent, Alban Desmaison, Noah Goodman, Pushmeet Kohli, Frank Wood, and Philip Torr. Learning disentangled representations with semi-supervised deep generative models. In *Proc. Advances in Neural Information Processing Systems*, pages 5925–5935, Dec. 2017.
- [187] Elliot Creager, David Madras, Joern-Henrik Jacobsen, Marissa Weis, Kevin Swersky, Toniann Pitassi, and Richard Zemel. Flexibly fair representation learning by

- disentanglement. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proc. Machine Learning Research*, pages 1436–1445, Long Beach, California, USA, 09–15 Jun 2019. PMLR.
- [188] Tian Qi Chen, Xuechen Li, Roger B Grosse, and David K Duvenaud. Isolating sources of disentanglement in variational autoencoders. In *Proc. Advances in Neural Information Processing Systems*, pages 2610–2620, 2018.
 - [189] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-vae: Learning basic visual concepts with a constrained variational framework. In *Proc. International Conference on Learning Representations*, 2017.
 - [190] Hyunjik Kim and Andriy Mnih. Disentangling by factorising. In *Proc. International Conference on Machine Learning*, volume 80, pages 2649–2658, 2018.
 - [191] Diederik P Kingma, Max Welling, et al. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, 2019.
 - [192] Martín Abadi et al. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL <https://www.tensorflow.org/>. Software available from [tensorflow.org](https://www.tensorflow.org/).
 - [193] Adam Paszke et al. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Proc. Advances in Neural Information Processing Systems*, pages 8024–8035. Curran Associates, Inc., 2019.
 - [194] Dawen Liang, Rahul G Krishnan, Matthew D Hoffman, and Tony Jebara. Variational autoencoders for collaborative filtering. In *Proc. International Conference on World Wide Web*, pages 689–698, 2018.
 - [195] Ruslan Salakhutdinov, Andriy Mnih, and Geoffrey Hinton. Restricted boltzmann machines for collaborative filtering. In *Proc. International Conference on Machine Learning*, pages 791–798, 2007.
 - [196] Suvash Sedhain, Aditya Krishna Menon, Scott Sanner, and Lexing Xie. Autorec: Autoencoders meet collaborative filtering. In *Proc. International Conference on World Wide Web*, pages 111–112, 2015.
 - [197] Alfredo Nazabal, Pablo M Olmos, Zoubin Ghahramani, and Isabel Valera. Handling incomplete heterogeneous data using vaes. *arXiv preprint arXiv:1807.03653*, 2018.
 - [198] Chao Ma, Sebastian Tschiatschek, Konstantina Palla, José Miguel Hernández-Lobato, Sebastian Nowozin, and Cheng Zhang. Eddi: Efficient dynamic discovery of high-value information with partial vae. In *Proc. International Conference on Machine Learning*, 2019.
 - [199] Wenbo Gong, Sebastian Tschiatschek, Richard Turner, Sebastian Nowozin, and José Miguel Hernández-Lobato. Icebreaker: Element-wise active information acquisition with bayesian deep latent gaussian model. In *Proc. Advances in Neural Information Processing Systems*, 2019.

- [200] Chao Ma, Wenbo Gong, José Miguel Hernández-Lobato, Noam Koenigstein, Sebastian Nowozin, and Cheng Zhang. Partial vae for hybrid recommender system. In *Proc. NeurIPS Workshop on Bayesian Deep Learning*, 2018.
- [201] Zoubin Ghahramani and Matthew J Beal. Variational inference for bayesian mixtures of factor analysers. In *Proc. Advances in Neural Information Processing Systems*, pages 449–455, 2000.
- [202] Jian-hua Zhao and LH Philip. A note on variational bayesian factor analysis. *Neural Networks*, 22(7):988–997, 2009.
- [203] Mike Wu, Richard L Davis, Benjamin W Domingue, Chris Piech, and Noah Goodman. Variational item response theory: Fast, accurate, and expressive. *arXiv preprint arXiv:2002.00276*, 2020.
- [204] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. International Conference on Learning Representations*, 2015.
- [205] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2009.
- [206] Neil T. Heffernan and Cristina Heffernan. The assistments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. *International Journal of Artificial Intelligence in Education*, 24:470–497, 2014.
- [207] J. Stamper, A. Niculescu-Mizil, S. Ritter, G.J. Gordon, and K.R. Koedinger. Algebra i 2006-2007. *Development data set from KDD Cup 2010 Educational Data Mining Challenge*, 2010.
- [208] J. Stamper, A. Niculescu-Mizil, S. Ritter, G.J. Gordon, and K.R. Koedinger. Bridge to algebra 2006-2007. *Development data set from KDD Cup 2010 Educational Data Mining Challenge*, 2010.
- [209] John C. Stamper and Zachary A. Pardos. The 2010 kdd cup competition dataset: Engaging the machine learning community in predictive learning analytics. In *Journal of Learning Analytics*, 2016.
- [210] Hemant Ishwaran, J Sunil Rao, et al. Spike and slab variable selection: frequentist and bayesian strategies. *The Annals of Statistics*, 33(2):730–773, 2005.
- [211] Michalis K Titsias and Miguel Lázaro-Gredilla. Spike and slab variational inference for multi-task and multiple kernel learning. In *Proc. Advances in neural information processing systems*, pages 2339–2347, 2011.
- [212] Francesco Tonolini, Bjorn Sand Jensen, and Roderick Murray-Smith. Variational sparse coding, July 2019. URL <http://eprints.gla.ac.uk/191553/>.
- [213] Ruqi Zhang, Chunyuan Li, Jianyi Zhang, Changyou Chen, and Andrew Gordon Wilson. Cyclical stochastic gradient mcmc for bayesian deep learning. *Proc. International Conference on Learning Representations*, 2020.

- [214] Yi-An Ma, Tianqi Chen, and Emily Fox. A complete recipe for stochastic gradient mcmc. In *Proc. Advances in Neural Information Processing Systems*, pages 2917–2925, 2015.
- [215] Jiaming Song, Shengjia Zhao, and Stefano Ermon. A-nice-mc: Adversarial training for mcmc. In *Proc. Advances in Neural Information Processing Systems*, pages 5140–5150, 2017.
- [216] Nando De Freitas, Pedro Højen-Sørensen, Michael I Jordan, and Stuart Russell. Variational mcmc. In *Proc. conference on Uncertainty in Artificial Intelligence*, pages 120–127. Morgan Kaufmann Publishers Inc., 2001.
- [217] Raza Habib and David Barber. Auxiliary variational mcmc. *Proc. International Conference on Learning Representations*, 2019.
- [218] Aditya Ravuri and Neil D. Lawrence. Towards one model for classical dimensionality reduction: A probabilistic perspective on umap and t-sne, 2024, 2405.17412. URL <https://arxiv.org/abs/2405.17412>.
- [219] Pavlin G. Poličar and Blaž Zupan. Visualizing high-dimensional temporal data using direction-aware t-sne, 2024, 2403.19040. URL <https://arxiv.org/abs/2403.19040>.
- [220] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.
- [221] Dmitry Kobak, George Linderman, Stefan Steinerberger, Yuval Kluger, and Philipp Berens. Heavy-tailed kernels reveal a finer cluster structure in t-sne visualisations. In Ulf Brefeld, Elisa Fromont, Andreas Hotho, Arno Knobbe, Marloes Maathuis, and Céline Robardet, editors, *Machine Learning and Knowledge Discovery in Databases*, pages 124–139, Cham, 2020. Springer International Publishing. ISBN 978-3-030-46150-8.
- [222] Nan Jiang, Yi Gu, and Yexiang Xue. Learning markov random fields for combinatorial structures with sampling through lovász local lemma, 2022.
- [223] T. Yang, C. Studer, R. Baker, N. Hefferenan, and A. S. Lan. Active learning for student affect detection. In *Proc. Educational Data Mining*, 2019.
- [224] Gregory Aist, Barry Kort, Rob Reilly, Jack Mostow, and Rosalind Picard. Experimentally augmenting an intelligent tutoring system with human-supplied capabilities: adding human-provided emotional scaffolding to an automated reading tutor that listens. In *Proc. IEEE International Conference on Multimodal Interfaces*, pages 483–490. IEEE, 2002.
- [225] Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *Proc. International Conference on Machine Learning - Volume, ICML’15*, page 1530–1538. JMLR.org, 2015.
- [226] Durk P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. In *Proc. Advances in Neural Information Processing Systems*, pages 4743–4751, 2016.

- [227] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction, 2020, 1802.03426. URL <https://arxiv.org/abs/1802.03426>.
- [228] József Beck. An algorithmic approach to the lovász local lemma. i. *Random Structures & Algorithms*, 2(4):343–365, 1991, <https://onlinelibrary.wiley.com/doi/pdf/10.1002/rsa.3240020402>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/rsa.3240020402>.
- [229] George C. Linderman and Stefan Steinerberger. Clustering with t-sne, provably, 2017, 1706.02582. URL <https://arxiv.org/abs/1706.02582>.